série livros didáticos informática ufrgs

linguagens formais e autômatos

6ª edição

paulo blauth menezes

bookman

o autor

Paulo Blauth Menezes é doutor em matemática pelo IST/Universidade Técnica de Lisboa, Portugal, mestre em ciência da computação e licenciado em matemática pela UFRGS. É professor pesquisador do Departamento de Informática Teórica do Instituto de Informática da UFRGS, tendo mais de 30 anos de experiência no meio acadêmico.

CÓPIA NÃO AUTORIZADA É CRIME
ABDR
ASSOCIAÇÃO BRASILEIRA DE DIREITOS REPROGRÁFICOS
RESPEITE O DIREITO AUTORAL

M543l Menezes, Paulo Blauth.
Linguagens formais e autômatos [recurso eletrônico] / Paulo Blauth Menezes. – 6. ed. – Dados eletrônicos. – Porto Alegre : Bookman, 2011.

Editado também como livro impresso em 2011.
ISBN 978-85-7780-799-4

1. Ciência da computação – Teoria da computação. 2. Linguagens formais. 3. Autômatos. I. Título.

CDU 004

Catalogação na publicação: Ana Paula M. Magnus – CRB-10/Prov-009/10

paulo blauth menezes

linguagens formais e autômatos

6ª edição

Versão impressa
desta obra: 2011

bookman

2011

© Artmed Editora SA, 2010

Capa e projeto gráfico interno: *Tatiana Sperhacke*

Imagem da capa: *iStockphoto.com/enot-poloskun*

Leitura final: *Renata Ramisch*

Editora Sênior: *Arysinha Jacques Affonso*

Editora responsável por esta obra: *Júlia Angst Coelho*

Editoração eletrônica: *Techbooks*

Reservados todos os direitos de publicação, em língua portuguesa, à
ARTMED® EDITORA S. A.
(BOOKMAN® COMPANHIA EDITORA é uma divisão da ARTMED® EDITORA S.A.)
Av. Jerônimo de Ornelas, 670 - Santana
90040-340 Porto Alegre RS
Fone (51) 3027-7000 Fax (51) 3027-7070

É proibida a duplicação ou reprodução deste volume, no todo ou em parte, sob quaisquer formas ou por quaisquer meios (eletrônico, mecânico, gravação, fotocópia, distribuição na Web e outros), sem permissão expressa da Editora.

SÃO PAULO
Av. Embaixador Macedo Soares, 10.735 - Pavilhão 5 - Cond. Espace Center
Vila Anastácio 05095-035 São Paulo SP
Fone (11) 3665-1100 Fax (11) 3667-1333

SAC 0800 703-3444

IMPRESSO NO BRASIL
PRINTED IN BRAZIL

Este livro é dedicado a:

Maria Fernanda,
Maria Lúcia e
Maria Luiza

apresentação

A Série *Livros didáticos* do Instituto de Informática da Universidade Federal do Rio Grande do Sul tem como objetivo a publicação de material didático para disciplinas ministradas em cursos de graduação em computação e informática, ou seja, para os cursos de bacharelado em ciência da computação, de bacharelado em sistemas de informação, de engenharia da computação e de licenciatura em informática. A série é desenvolvida tendo em vista as diretrizes curriculares do MEC e é resultante da experiência dos professores do Instituto de Informática e dos colaboradores externos no ensino e na pesquisa.

Os primeiros títulos, *Fundamentos da matemática intervalar* e *Programando em Pascal XSC* (esgotados), foram publicados em 1997 no âmbito do Projeto Aritmética Intervalar Paralela (ArInPar), financiados pelo ProTeM - CC CNPq/Fase II. Essas primeiras experiências serviram de base para os volumes subsequentes, os quais se caracterizam como livros-texto para disciplinas dos cursos de computação e informática.

Em seus títulos mais recentes, a série *Livros didáticos* tem contado com a colaboração de professores externos que, em parceria com professores do Instituto, estão desenvolvendo livros de alta qualidade e valor didático. Hoje a série está aberta a qualquer autor de reconhecida capacidade.

O sucesso da experiência com esses livros, aliado à responsabilidade que cabe ao Instituto na formação de professores e pesquisadores em Computação e Informática, conduziu à ampliação da abrangência e à institucionalização da série.

Em 2008, um importante passo foi dado para a consolidação e ampliação de todo o trabalho: a publicação dos livros pelo Grupo A, por meio da editora Bookman. Hoje são 17 os títulos publicados – uma lista completa encontra-se nas orelhas desta obra – ampliando a oferta aos leitores da série. Sempre com a preocupação em manter nível compatível com a elevada qualidade do ensino e da pesquisa desenvolvidos no âmbito do Instituto de Informática da UFRGS e no Brasil.

Prof. Paulo Blauth Menezes
Comissão Editorial da Série Livros Didáticos
Instituto de Informática da UFRGS

prefácio

Como o título indica, este livro apresenta os principais conceitos e resultados de linguagens formais e autômatos, usando uma linguagem simples e acessível a qualquer aluno de graduação, mas sem descuidar do desenvolvimento do raciocínio, nem dos aspectos matemático-formais. Sempre que possível, as construções apresentadas são instanciadas em casos aplicados a computação e informática. Este é um livro-texto para disciplinas dos cursos de graduação em computação e informática, de acordo com as Diretrizes Curriculares do MEC, bem como um livro de referência para diversos aspectos da computação em geral.

Trata-se de um trabalho baseado em experiências letivas desenvolvidas no curso de bacharelado em ciência da computação da Universidade Federal do Rio Grande do Sul. O livro é autocontido, possui um texto simples, exemplos detalhados e exercícios em níveis crescentes de raciocínio.

A primeira edição tinha como principal objetivo suprir a necessidade de um livro-texto didático voltado para o curso de bacharelado em ciência da computação da UFRGS. Entretanto, foi uma grata surpresa verificar que o livro foi adotado em diversas instituições em todo o Brasil, esgotando-se rapidamente e fazendo com que uma segunda edição fosse antecipada em cerca de dois anos. A segunda edição seguiu a mesma estrutura da primeira e aprimorou pequenos, mas importantes detalhes. A aceitação foi muito boa e a edição também esgotou-se rapidamente. Com o objetivo de manter o custo do livro acessível (fator importante para muitos estudantes), optou-se, para a terceira e quarta edições, fazer apenas pequenas revisões.

Entretanto, para a quinta edição, o livro foi praticamente todo revisto, incluindo uma nova estruturação do conteúdo, redistribuição dos capítulos e atualizações tecnológicas, como o tratamento de hipertextos, hipermídias e animações no contexto das linguagens formais e autômatos. Além disso, novos exemplos e exercícios foram incluídos.

Nesta sexta edição, houve uma revisão em todos os capítulos, com o intuito de preparar o lançamento de um novo livro, complementar a este:

Aprendendo linguagens formais e autômatos com exercícios

Trata-se de um livro com muito mais do que simples soluções de exercícios e que atende a um grande conjunto de solicitações recebidas nesta linha.

Para os que conhecem ou usam a edição anterior, esta sexta edição contém uma importante modificação conceitual no que diz respeito ao autômato finito não determinístico. Sugere-se especial atenção à nova definição, que simplifica construções decorrentes e soluções de exercícios.

A carga horária recomendada é de **60** a **90** horas, dependendo da formação dos alunos, do detalhamento dos exemplos e dos formalismos e dos tópicos a serem desenvolvidos. Recomenda-se fortemente o desenvolvimento dos exercícios de implementação sugeridos. Experiências demonstram que o entendimento dos alunos é significativamente reforçado e ampliado nesse caso.

Nesta edição, o autor agradece:

- à Maria Lúcia Menezes pelas bem-humoradas ilustrações de autômatos usadas ao longo de todo o texto;
- ao professor Fernando Accorsi pelos trabalhos conjuntos e resultados atingidos, os quais foram parcialmente incorporados neste livro;
- ao Marnes Augusto Hoff pelas revisões sugeridas no texto;
- aos colegas prof. Tiarajú Diverio e profa. Laira Vieira Toscani pelo apoio e incentivo recebidos para a viabilização deste e de outros trabalhos didáticos, bem como em toda a minha carreira universitária.
- a todos os alunos e ex-alunos das turmas de graduação do Instituto de Informática da UFRGS. Todos devem se considerar citados nominalmente neste espaço;
- aos amigos do Classic Car Club – RS (http://www.classiccarclub-rs.com.br), pelos momentos de descontração;
- ao CNPq, à FINEP, à CAPES e à FAPERGS, pelos auxílios financeiros aos projetos de pesquisa que viabilizaram a realização deste e de outros trabalhos.

Para as diversas edições anteriores, o autor agradece: Júlio Pereira Machado, Karina Girardi Roggia, Simão Toscani, Carlos Tadeu Queiroz de Morais, Simone Costa, Leonardo Penczek, Gustavo Link Federizzi, Aline Malanovicz, Guilherme de Campos Magalhães, Lúcio Dorneles Moser e Renato Fernandes.

Um agradecimento especial ao Instituto de Informática da UFRGS, que tem apoiado esta série de *Livros didáticos*.

Paulo Blauth Menezes
blauth@inf.ufrgs.br

sumário

1 introdução e conceitos básicos 17

1.1 introdução ... 18

1.1.1 sintaxe e semântica ... 18
1.1.2 abordagem ... 19

1.2 conjuntos, relações e funções ... 20

1.2.1 conjuntos e operações sobre conjuntos ... 20
1.2.2 relações ... 25
1.2.3 funções parciais ... 29
1.2.4 funções ... 30
1.2.5 cardinalidade de conjuntos ... 32

1.3 noções de lógica ... 33

1.4 técnicas de demonstração ... 37

1.4.1 prova direta ... 39
1.4.2 prova por contraposição ... 40
1.4.3 prova por redução ao absurdo ... 40

1.5 indução ... 41

1.5.1 princípio da indução matemática ... 41
1.5.2 prova indutiva ... 42
1.5.3 segundo princípio da indução matemática ... 43
1.5.4 definição indutiva ... 44

1.6 exercícios ... 45

2 linguagens e gramáticas 53

2.1 alfabeto ... 54
2.2 palavra ... 55
2.3 linguagem formal ... 57
2.4 gramática ... 58
2.5 exercícios ... 62

3 linguagens regulares 65

3.1 sistema de estados finitos ... 67
3.2 composição sequencial, concorrente e não determinista ... 68
3.3 autômato finito ... 69
3.4 autômato finito não determinístico ... 79
3.5 autômato finito com movimentos vazios ... 87
3.6 expressão regular ... 93
3.7 gramática regular ... 100
3.8 exercícios ... 106

4 propriedades das linguagens regulares 113

4.1 bombeamento para as linguagens regulares ... 115
4.2 investigação se é linguagem regular ... 117
4.3 operações fechadas sobre as linguagens regulares ... 118
4.4 investigação se uma linguagem regular é vazia, finita ou infinita ... 120

4.5 igualdade de linguagens regulares 122

4.6 minimização de um autômato finito 122

4.7 exercícios 128

5 autômato finito com saída 133

5.1 máquina de Mealy 135

5.2 máquina de Moore 136

5.3 equivalência das máquinas de Moore e de Mealy 138

5.4 hipertexto e hipermídia como autômato finito com saída 141

5.5 animação como autômato finito com saída 145

5.6 exercícios 148

6 linguagens livres do contexto 153

6.1 gramática livre do contexto 155

6.2 árvore de derivação 157

6.3 gramática livre do contexto ambígua 160

6.4 simplificação de gramática livre do contexto 161

6.4.1 símbolos inúteis 162
6.4.2 produções vazias 164
6.4.3 produções que substituem variáveis 166
6.4.4 simplificações combinadas 167

6.5 formas normais 167

6.5.1 forma normal de Chomsky 168
6.5.2 forma normal de Greibach 170

6.6 recursão à esquerda 174

6.7 **autômato com pilha** **175**

6.7.1 definição do autômato com pilha 176
6.7.2 autômato com pilha e linguagens livres do contexto 181
6.7.3 número de pilhas e o poder computacional 184

6.8 **exercícios** **185**

propriedades e reconhecimento das linguagens livres do contexto 193

7.1 **propriedades das linguagens livres do contexto** **195**

7.1.1 investigação se é linguagem livre do contexto 195
7.1.2 operações fechadas sobre as linguagens livres do contexto 197
7.1.3 investigação se é vazia, finita ou infinita 200

7.2 **algoritmos de reconhecimento** **201**

7.2.1 autômato com pilha como reconhecedor 201
7.2.2 algoritmo de Cocke-Younger-Kasami 203
7.2.3 algoritmo de Early 205

7.3 **exercícios** **207**

linguagens recursivamente enumeráveis e sensíveis ao contexto 211

8.1 **máquina de Turing** **214**

8.1.1 noção intuitiva 214
8.1.2 modelo 215

8.2 **modelos equivalentes à máquina de Turing** **220**

8.3 **hipótese de Church** **222**

8.4 **máquina de Turing como reconhecedor** **222**

8.4.1 linguagem recursivamente enumerável 223
8.4.2 linguagem recursiva 225

8.5 **gramática irrestrita e a linguagem recursivamente enumerável** 225

8.6 **linguagem sensível ao contexto** 226
8.6.1 gramática sensível ao contexto 226
8.6.2 máquina de Turing com fita limitada 227

8.7 **propriedades das linguagens recursivamente enumeráveis e recursivas** 230

8.8 **exercícios** 232

hierarquia de classes de linguagens e conclusões 237

9.1 **hierarquia de Chomsky** 238

9.2 **conclusões** 240

9.3 **leitura complementar: gramática de grafos** 240

9.4 **exercícios** 243

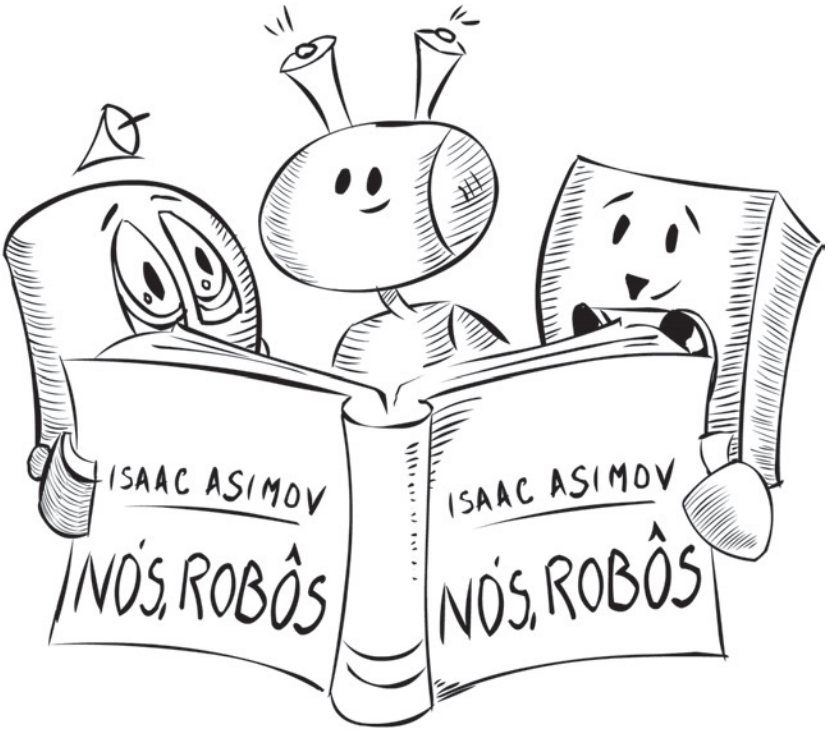
leituras sugeridas 245

índice 249

capítulo 1

introdução e conceitos básicos

A teoria das linguagens formais nasceu na década de 1950 com o objetivo de tratar linguagens naturais. Logo foi verificada a sua importância para o estudo de linguagens artificiais e, em particular, para as linguagens computacionais, com ênfase em análise léxica e sintática. Mais modernamente, destacam-se aplicações em hipertextos, hipermídias e linguagens não lineares. Este capítulo apresenta linguagens formais e autômatos e faz uma revisão dos principais pré-requisitos de teoria dos conjuntos, lógica e indução e técnicas de demonstração. Além disso, introduz e caracteriza as noções de sintaxe e semântica.



1.1 introdução

A *teoria das linguagens formais* foi originariamente desenvolvida na década de 1950 com o objetivo de desenvolver teorias relacionadas com as linguagens naturais. Entretanto, logo foi verificado que esta teoria era importante para o estudo de *linguagens artificiais* e, em especial, para as linguagens originárias da computação e informática. Desde então, o estudo das linguagens formais desenvolveu-se significativamente e com diversos enfoques, com destaque para aplicações em *análise léxica* e *análise sintática* de linguagens de programação, modelagem de *circuitos lógicos* ou *redes lógicas*, de *sistemas biológicos*, entre outros. Mais recentemente, destacam-se aplicações relacionadas com *sistemas de animação*, *hipertextos* e *hipermídias*, bem como o tratamento de *linguagens não lineares*, como *linguagens planares*, *linguagens espaciais* e *linguagens n-dimensionais*.

1.1.1 sintaxe e semântica

Linguagens formais preocupa-se com os problemas *sintáticos* das linguagens. Assim, inicialmente, é importante introduzir os conceitos de sintaxe e de semântica de linguagens.

Historicamente, no estudo do entendimento das linguagens de programação, o problema sintático foi reconhecido antes do problema semântico e foi o primeiro a receber um tratamento adequado. Adicionalmente, os problemas sintáticos são de tratamento mais simples que os semânticos. Como consequência, foi dada uma grande ênfase à sintaxe, ao ponto de levar à ideia de que as questões das linguagens de programação resumiam-se às questões da sintaxe. Atualmente, a teoria da sintaxe possui construções matemáticas bem definidas e universalmente reconhecidas como, por exemplo, as *Gramáticas de Chomsky*.

Uma linguagem de programação (bem como qualquer modelo matemático) pode ser vista de duas formas:

- como uma entidade livre, ou seja, sem qualquer significado associado;
- como uma entidade juntamente com uma interpretação do seu significado.

A *sintaxe* trata das propriedades livres da linguagem como, por exemplo, a verificação gramatical de programas. A *semântica* objetiva dar uma interpretação para a linguagem como, por exemplo, um significado ou valor para um determinado programa. Consequentemente, a sintaxe basicamente manipula símbolos sem considerar os seus correspondentes significados. Note-se que, para resolver qualquer problema real, é necessário dar uma interpretação semântica aos símbolos como, por exemplo:

estes símbolos representam os inteiros.

Sintaticamente falando, não existe uma noção de programa "errado": neste caso, simplesmente não é um programa da linguagem em questão. Por outro lado, um programa sintaticamente válido ("correto"), pode não ser o programa que o programador esperava escrever. Assim, a questão de considerar um programa "correto" ou "errado" deve considerar se o mesmo modela adequadamente o comportamento desejado.

Nem sempre os limites entre a sintaxe e a semântica são claros. Um exemplo é a ocorrência de um nome em um programa o qual pode ser tratado de forma igualmente fácil como um problema sintático ou semântico. Entretanto, a distinção entre sintaxe e semântica em linguagens artificiais é, em geral, óbvia para a maioria dos problemas relevantes.

Observe que *análise léxica* pode ser vista como um tipo especial de análise sintática, a qual é centrada nas componentes básicas da linguagem. Portanto, *linguagens formais* também se preocupa com os problemas léxicos.

1.1.2 abordagem

A abordagem desta publicação é centrada no tratamento sintático de linguagens lineares abstratas com fácil associação às linguagens típicas da computação e informática. Os formalismos usados podem ser classificados nos seguintes tipos:

a *Operacional*. Define-se um autômato ou uma máquina abstrata, baseada em estados, em instruções primitivas e na especificação de como cada instrução modifica cada estado.

Uma máquina abstrata deve ser suficientemente simples para não permitir dúvidas sobre a execução de seu código. Um formalismo operacional também é dito um formalismo *reconhecedor*, no sentido em que permite a análise de uma dada entrada para verificar se é "reconhecida" pela máquina. As principais máquinas definidas nesta publicação são *autômato finito*, *autômato com pilha* e *máquina de Turing*;

b *Axiomático*. Associam-se regras às componentes da linguagem. As regras permitem afirmar o que será verdadeiro após a ocorrência de cada cláusula, considerando-se o que era verdadeiro antes da ocorrência. A abordagem axiomática que segue é sobre *gramáticas* (*regulares*, *livres do contexto*, *sensíveis ao contexto* e *irrestritas*). Uma gramática também é dita um formalismo *gerador* no sentido em que permite verificar se um determinado elemento da linguagem é "gerado";

c *Denotacional*. Também é denominado formalismo *funcional*. Define-se um domínio que permite a caracterização do conjunto de palavras admissíveis na linguagem. Trata-se de funções, as quais são, em geral, *composicionais* (*horizontalmente*) no sentido em que o valor denotado por uma construção é especificado em termos dos valores denotados por suas subcomponentes. Nesta publicação, a abordagem denotacional é restrita às *expressões regulares*. Como, a partir de uma expressão regular, é simples inferir ("gerar") as palavras da linguagem denotada, este formalismo também é frequentemente denominado, de forma não muito precisa, formalismo *gerador*.

1.2 conjuntos, relações e funções

No texto que segue, supõe-se que o leitor está familiarizado com os conceitos básicos relativos à *teoria dos conjuntos*. Caso o leitor domine esses conceitos, sugere-se uma passagem rápida por esta seção para verificar a nomenclatura e as convenções adotadas ao longo do livro.

1.2.1 conjuntos e operações sobre conjuntos

O conceito de conjunto é fundamental no estudo de linguagens formais e autômatos, pois todos os conceitos desenvolvidos, bem como os correspondentes resultados, são baseados em conjuntos ou em construções sobre conjuntos.

Conjunto é uma estrutura que agrupa objetos e constitui uma base para construir estruturas mais complexas. Assim, informalmente, um conjunto é uma coleção, sem repetições e sem qualquer ordenação, de objetos denominados *elementos*. O termo "elemento" é usado de forma ampla e pode designar um objeto concreto ou abstrato. Neste contexto, um elemento é uma entidade básica a qual não é definida formalmente.

definição 1.1 – Conjunto

Um *conjunto* é uma coleção de zero ou mais objetos distintos, chamados *elementos* do conjunto, os quais não possuem qualquer ordem associada. ❑

Relativamente ao relacionamento entre elementos e conjuntos, vale que:

a *Pertinência*. Se um determinado elemento **a** é elemento de um conjunto **A**, tal fato é denotado por:

$$a \in A$$

o qual é interpretado como segue:

a pertence *ao conjunto* ***A***

Caso contrário, se afirma que **a** *não* pertence ao conjunto **A**. Tal fato é denotado por:

$$a \notin A$$

b *Continência e subconjunto*. Se todos os elementos de um conjunto **A** também são elementos de um conjunto **B**, então afirma-se que **A** está *contido* em **B** e denota-se por:

$$A \subseteq B$$

ou, alternativamente, que **B** *contém* **A**, e denota-se por:

$$B \supseteq A$$

Nesse caso ($A \subseteq B$ ou $B \supseteq A$), afirma-se que **A** é *subconjunto* de **B**.
Adicionalmente, se $A \subseteq B$, mas existe $b \in B$ tal que $b \notin A$, então se afirma que **A** está *contido propriamente* em **B**, ou que **A** é *subconjunto próprio* de **B**, e denota-se por:

$$A \subset B$$

Ou, alternativamente, que **B** *contém propriamente* **A** e denota-se por:

$$B \supset A$$

Quando não é fato que $A \subseteq B$ (respectivamente, $A \subset B$), é usual denotar como segue:

$$A \not\subseteq B \text{ (respectivamente, } A \not\subset B\text{)}$$

c *Igualdade de conjuntos*. Os conjuntos **A** e **B** são ditos *conjuntos iguais*, o que é denotado por:

$$A = B$$

se e somente se possuem exatamente os mesmos elementos. Formalmente, afirma-se que:

$$A = B \quad \text{se e somente se} \quad A \subseteq B \text{ e } B \subseteq A$$

Um conjunto especialmente importante é o *conjunto vazio*, ou seja, o conjunto sem elementos **{ }**, o qual é usualmente representado pelo seguinte símbolo:

$$\varnothing$$

A definição de um conjunto pela listagem de todos os seus elementos é denominada *denotação por extensão* e é dada pela lista de todos os seus elementos, em qualquer ordem (separados por vírgulas) e entre chaves como, por exemplo:

Vogais = { a, e, i, o, u }

Neste caso, **Vogais** denota o conjunto **{ a, e, i, o, u }**.

A definição de um conjunto por propriedades é denominada *denotação por compreensão* como, por exemplo (suponha que **N** denota o conjunto dos números naturais):

$$\textbf{Pares} = \{ n \in \mathbf{N} \mid n \text{ é número par} \}$$

a qual é interpretada como:

> *conjunto de todos os elementos **n** pertencentes ao conjunto **N***
> *tal que **n** é número par.*

Assim, a forma geral de definição de um conjunto por propriedades é como segue:

$$\{ x \mid x \in A \text{ e } p(x) \} \quad \text{ou} \quad \{ x \in A \mid p(x) \}$$

e é tal que um determinado elemento **a** é elemento deste conjunto se a propriedade **p** é verdadeira para **a**, ou seja, se **p(a)** é verdadeira. Quando é claro que $x \in A$, pode-se denotar esse conjunto simplesmente na forma:

$$\{ x \mid p(x) \}$$

Muitas vezes é conveniente especificar um conjunto de outra forma que não por compreensão, como, por exemplo:

Dígitos = { 0, 1, 2, 3,..., 9 }
Pares = { 0, 2, 4, 6,... }

nas quais os elementos omitidos podem ser facilmente deduzidos do contexto.

Um conjunto pode possuir um número finito ou infinito de elementos. A definição formal de conjunto finito e infinito será apresentada adiante. Informalmente, um conjunto é dito:

a *Conjunto finito* se pode ser denotado por extensão, ou seja, listando exaustivamente todos os seus elementos;

b *Conjunto infinito*, caso contrário.

exemplo 1.1 – Conjuntos, elementos

a $a \in \{ b, a \}$
$c \notin \{ b, a \}$

b $\{ a, b \} \subseteq \{ b, a \}$
$\{ a, b \} \subseteq \{ a, b, c \}$
$\{ 1, 2, 3 \} = \{ 3, 3, 3, 2, 2, 1 \}$

Este último item ilustra claramente a definição de igualdade. De fato, é fácil verificar que:

$$\{ 1, 2, 3 \} \subseteq \{ 3, 3, 3, 2, 2, 1 \} \quad \text{e} \quad \{ 3, 3, 3, 2, 2, 1 \} \subseteq \{ 1, 2, 3 \}$$

Observe que este exemplo também ilustra por que as repetições de elementos podem ser desconsideradas (se o leitor tiver alguma dúvida, revise a definição de continência);

c Os seguintes conjuntos são infinitos:

N	*conjunto dos números naturais*
Z	*conjunto dos números inteiros*
Q	*conjunto dos números racionais*
I	*conjunto dos números irracionais*
R	*conjunto dos números reais*

d $\{ 1, 2, 3 \} = \{ x \in N \mid x > 0 \text{ e } x < 4 \}$

$N = \{ x \in Z \mid x \geq 0 \}$

$\text{Pares} = \{ y \mid y = 2x \text{ e } x \in N \}$ ❑

As principais operações sobre conjuntos são as seguintes.

definição 1.2 – União, intersecção, diferença, complemento, conjunto das partes, produto cartesiano

Sejam **A** e **B** conjuntos. Então:

a *União*.

$A \cup B = \{ x \mid x \in A \text{ ou } x \in B \}$

b *Intersecção*.

$A \cap B = \{ x \mid x \in A \text{ e } x \in B \}$

c *Complemento*. A operação de complemento é definida em relação a um conjunto fixo **U** denominado *conjunto universo*, como segue:

$\sim A = A' = \{ x \mid x \in U \text{ e } x \notin A \}$

d *Diferença*.

$A - B = \{ x \mid x \in A \text{ e } x \notin B \}$

Ou, alternativamente:

$A - B = A \cap \sim B$

e *Conjunto das partes*.

$2^A = P(A) = \{ S \mid S \subseteq A \}$

f *Produto cartesiano*.

$A \times B = \{ (a, b) \mid a \in A \text{ e } b \in B \}$ ❑

Dados dois conjuntos **A** e **B**, sendo ambos não vazios, se $A \cap B = \varnothing$, então **A** e **B** são ditos *conjuntos disjuntos*, *conjuntos independentes* ou *conjuntos mutuamente exclusivos*.

É usual denotar um produto cartesiano de um conjunto com ele mesmo como um expoente. Por exemplo:

$$A \times A = A^2$$

Um elemento de um produto cartesiano denotado na forma:

$$(a, b)$$

é denominado *par ordenado* e não deve ser confundido com o conjunto $\{ a, b \}$:

- a ordem é importante, pois são distinguidas as duas componentes;
- as componentes podem ser repetidas.

O conceito de par ordenado pode ser generalizado para **n**-upla ordenada $(x_1, x_2, x_3, \ldots, x_n)$, ou seja, para $n \in \mathbf{N}$ componentes.

exemplo 1.2 – Operações sobre conjuntos

Suponha o universo **N**, e sejam $A = \{ 0, 1, 2 \}$ e $B = \{ 2, 3 \}$. Então:

a $A \cup B = \{ 0, 1, 2, 3 \}$
b $A \cap B = \{ 2 \}$
c $A - B = \{ 0, 1 \}$
d $\sim A = \{ x \in \mathbf{N} \mid x > 2 \}$
e $2^B = \mathbf{P}(B) = \{ \varnothing, \{ 2 \}, \{ 3 \}, \{ 2, 3 \} \}$
f $A \times B = \{ (0, 2), (0, 3), (1, 2), (1, 3), (2, 2), (2, 3) \}$ ❑

As seguintes propriedades das operações sobre conjuntos podem ser facilmente verificadas (suponha o universo **U** e os conjuntos **A**, **B** e **C**):

a *Idempotência*.
$A \cup A = A$
$A \cap A = A$

b *Comutativa*.
$A \cup B = B \cup A$
$A \cap B = B \cap A$

c *Associativa*.
$A \cup (B \cup C) = (A \cup B) \cup C$
$A \cap (B \cap C) = (A \cap B) \cap C$
Observe que, a partir da associatividade, pode-se concluir que a sequência de realização da operação em questão pode ser qualquer. Portanto, os parênteses não são importantes e, consequentemente, podem ser omitidos. Por exemplo, a expressão $A \cup (B \cup C)$ pode ser denotada simplesmente como segue:

$$A \cup B \cup C$$

d *Distributiva*.
$A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$
$A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$

e *Duplo complemento*.
$\sim(\sim A) = A$

f *DeMorgan*.
$\sim(A \cup B) = \sim A \cap \sim B$
$\sim(A \cap B) = \sim A \cup \sim B$
Esta propriedade permite concluir que o complemento da união (respectivamente, intersecção) pode ser calculado em termos da intersecção (respectivamente, união). Por exemplo:

$$A \cup B = \sim(\sim(A \cup B)) = \sim(\sim A \cap \sim B)$$

Este resultado é importante no texto que segue:

g *Universo e Vazio*.

$\mathbf{A} \cup \sim\mathbf{A} = \mathbf{U}$

$\mathbf{A} \cap \sim\mathbf{A} = \varnothing$

1.2.2 relações

O conceito de relação é usado com frequência ao longo de todo o texto, além de ser a base para um correto entendimento de algumas importantes construções matemáticas como, por exemplo, o conceito de função. Relativamente ao estudo das linguagens formais e autômatos, os conceitos de relação de ordem, relação de equivalência e fecho de uma relação são especialmente importantes.

definição 1.3 – Relação

Suponha **A** e **B** conjuntos. Uma *relação* (binária) **R** de **A** em **B** é um subconjunto de um produto cartesiano $\mathbf{A} \times \mathbf{B}$, ou seja:

$$\mathbf{R} \subseteq \mathbf{A} \times \mathbf{B}$$

sendo que:

A é denominado *domínio*, *origem* ou *conjunto de partida* de **R**
B é denominado *contradomínio*, *codomínio*, *destino* ou *conjunto de chegada* de **R** ❑

É importante observar que uma relação $\mathbf{R} \subseteq \mathbf{A} \times \mathbf{B}$ é constituída de três partes: a origem **A**, o destino **B** e o conjunto de pares **R**. Qualquer alteração em uma destas três partes define uma outra relação.

Uma relação $\mathbf{R} \subseteq \mathbf{A} \times \mathbf{B}$ também é denotada como segue:

$$\mathbf{R}: \mathbf{A} \rightarrow \mathbf{B}$$

e um elemento $(\mathbf{a}, \mathbf{b}) \in \mathbf{R}$ é frequentemente denotado de forma infixada, como segue:

$$\mathbf{a}\ \mathbf{R}\ \mathbf{b}$$

definição 1.4 – Endorrelação, autorrelação

Suponha **A** um conjunto. Então uma relação $\mathbf{R}: \mathbf{A} \rightarrow \mathbf{A}$ (origem e destino no mesmo conjunto) é dita uma *endorrelação* ou *autorrelação*. Nesse caso, afirma-se que **R** é uma *relação em* **A**. ❑

Uma endorrelação $\mathbf{R}: \mathbf{A} \rightarrow \mathbf{A}$ é frequentemente denotada por:

$$(\mathbf{A}, \mathbf{R})$$

As endorrelações são especialmente importantes, razão pela qual uma série de estudos é desenvolvida especialmente para este tipo de relações, como os apresentados no restante desta seção.

definição 1.5 – Relação conexa, reflexiva, simétrica, antissimétrica, transitiva

Sejam **A** um conjunto e **R** uma endorrelação em **A**. Então **R** é uma relação:

a *Conexa*, se, para todo **a**, **b** ∈ **A**, vale que **a R b** ou **b R a** ou **a = b**
b *Reflexiva*, se, para todo **a** ∈ **A**, vale que **a R a**
c *Simétrica*, se, para todo **a**, **b** ∈ **A**, caso **a R b**, então **b R a**
d *Antissimétrica*, se, para todo **a**, **b** ∈ **A**, caso **a R b** e **b R a**, então **a = b**
e *Transitiva*, se, para todo **a**, **b**, **c** ∈ **A**, caso **a R b** e **b R c**, então **a R c** ❑

Antissimetria estabelece que, para uma relação **R** e um par **(a, b)** ∈ **R**, não é possível inverter a ordem dos elementos (ou seja, o par **(b, a)** ∉ **R**), excetuando-se o caso em que **a** e **b** são iguais.

Note-se que uma relação pode não ser simétrica, nem antissimétrica, ou seja, simetria e antissimetria não são noções complementares. Adicionalmente, uma relação pode ser simultaneamente simétrica e antissimétrica. Tais casos são ilustrados a seguir.

exemplo 1.3 – Relação conexa, reflexiva, simétrica, antissimétrica, transitiva

Considere um conjunto não vazio **A**. Então, as seguintes endorrelações são:

a Conexa, reflexiva, antissimétrica e transitiva: **(N, ≤)**
b Reflexiva, antissimétrica e transitiva (em que condições é conexa?): **(2^A, ⊆)**
c Conexa, antissimétrica e transitiva: **(Z, <)**
d Antissimétrica e transitiva (em que condições é conexa?): **(2^A, ⊂)**
e Não conexa, reflexiva, simétrica, antissimétrica e transitiva: **(Q, =)**
f Não conexa, nem reflexiva, nem simétrica, nem antissimétrica, e nem transitiva: **({ 1, 2, 3, 4 }, R)** tal que **R = { (1, 2), (2, 1), (2, 3) }** ❑

Frequentemente é desejável estender uma relação de forma a garantir que ela satisfaz determinado conjunto de propriedades. Por exemplo, garantir que determinada relação **R** satisfaz à propriedade reflexiva. Assim, se **R** não é reflexiva, são introduzidos os pares (e somente estes) que garantem a reflexividade de **R**.

definição 1.6 – Fecho de uma relação

Sejam **R: A → A** uma endorrelação e **P** um conjunto de propriedades. Então o *fecho de* **R** *em relação a* **P** é a menor endorrelação em **A** que contém **R** e que satisfaz as propriedades de **P** e é denotado por:

$$\text{FECHO-P(R)}$$ ❑

Portanto, para qualquer conjunto de propriedades considerado, a relação sempre será subconjunto de seu fecho, ou seja:

$$R \subseteq \text{FECHO-P(R)}$$

Em que condições uma relação é igual ao seu fecho, ou seja, **R = FECHO-P(R)**?

Dois tipos de fecho de uma relação são especialmente importantes no trabalho que segue.

definição 1.7 – Fecho transitivo, fecho transitivo e reflexivo

Seja **R** uma endorrelação em **A**. Então:

a O fecho de **R** em relação ao conjunto de propriedades **{ transitiva }**, denominado *fecho transitivo* de **R** e denotado por $\mathbf{R}^+$, é definido como segue:

a.1) Se $(a, b) \in R$, então $(a, b) \in R^+$

a.2) Se $(a, b) \in R^+$ e $(b, c) \in R^+$, então $(a, c) \in R^+$

a.3) Os únicos elementos de R^+ são os construídos como acima;

b O fecho de **R** em relação ao conjunto de propriedades **{ transitiva, reflexiva }**, denominado *fecho transitivo e reflexivo* de **R** e denotado por **R***, é tal que:

$$R^* = R^+ \cup \{ (a, a) \mid a \in A \}$$ ❑

A definição de fecho transitivo acima é de um tipo especial denominada *definição indutiva* ou *definição recursiva* a qual é estudada na seção 1.5 – Indução. Por enquanto, basta o entendimento intuitivo da construção.

exemplo 1.4 – Fecho de um grafo visto como uma relação

Um *grafo* (direto ou dirigido) pode ser definido como uma endorrelação **A** (de *arcos*, *arestas* ou *setas*) em um conjunto **V** (de *nodos* ou *vértices*). Assim:

$$A = \{ (1, 2), (2, 3), (3, 4), (1, 5) \}$$

é um grafo em $V = \{ 1, 2, 3, 4, 5 \}$ como ilustrado na figura 1.1 (esquerda), na qual o par ordenado que define um arco é representado por uma seta cujos nodos de origem e de destino, representados como circunferências, denotam a primeira e a segunda componente do par, respectivamente. O fecho transitivo e reflexivo:

$$A^* = \{ (1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (2, 2), (2, 3), (2, 4), (3, 3), (3, 4), (4, 4), (5, 5) \}$$

é ilustrado na figura 1.1 (direita), na qual os arcos adicionados pelo fecho são representados com um traço diferente.

Essa definição de grafo (como uma relação) permite a existência de arcos paralelos, ou seja, dois ou mais arcos com o mesmo nodo origem e destino? ❑

definição 1.8 – Relação de ordem parcial, relação de ordem conexa, cadeia

Sejam **A** um conjunto e **R** uma endorrelação em **A**. Então **R** é uma:

a *Relação de ordem parcial* se **R** é uma relação:

- reflexiva;
- antissimétrica;
- transitiva;

b *Relação de ordem conexa* ou *cadeia* se **R** é uma relação:

- de ordem parcial;
- conexa. ❑

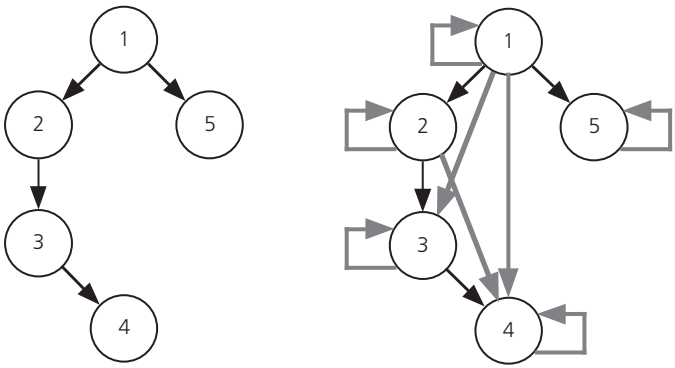


figura 1.1 Grafo (esquerda) e o correspondente fecho transitivo e reflexivo (direita).

exemplo 1.5 – Relações de ordem parcial, cadeia

Considere um conjunto não vazio **A**. Então, as seguintes relações são de:

a Ordem parcial: $(\mathbf{N}, \leq)$, $(2^{\mathbf{A}}, \subseteq)$ e $(\mathbf{Q}, =)$

b Cadeia: $(\mathbf{N}, \leq)$ ❑

definição 1.9 – Relação de equivalência

Sejam **A** um conjunto e **R** uma relação em **A**. Então **R** é uma *Relação de Equivalência* se for reflexiva, simétrica e transitiva. ❑

Um importante resultado é que cada relação de equivalência $\mathbf{R}: \mathbf{A} \rightarrow \mathbf{A}$ induz uma *partição do conjunto* **A** em subconjuntos disjuntos e não vazios denominados de *classes de equivalência* ou *blocos da partição*, tal que a união de todos os blocos resulta em **A**. Observe que **A** pode ser vazio. Então, quais são os blocos da partição do vazio?

Suponha que, para dado conjunto **A**, $\{ A_1, A_2, \ldots, A_n \}$ é uma partição de **A**. Então, é usual denotar uma classe de equivalência por um elemento representativo desta, entre colchetes. Assim, se $a_1 \in A_1$, $a_2 \in A_2, \ldots, a_n \in A_n$, vale que:

$$[a_1] = A_1, [a_2] = A_2, \ldots, [a_n] = A_n$$

exemplo 1.6 – Relação de equivalência, classes de equivalência

Considere a seguinte relação:

$$\mathbf{R} = \{ (a, b) \in \mathbf{N}^2 \mid (a \text{ MOD } 2) = (b \text{ MOD } 2) \}$$

onde **MOD** é a operação que resulta no resto da divisão inteira. É fácil verificar que **R** é uma relação de equivalência.

Intuitivamente, a relação **R** induz uma partição do conjunto **N**, como segue:

[**0**], a classe de equivalência dos números pares (resto **zero**);
[**1**], a classe de equivalência dos números ímpares (resto **um**). ❑

1.2.3 funções parciais

Normalmente, no estudo da teoria dos conjuntos, o conceito de função parcial é marginalmente desenvolvido, comparativamente com o conceito de função (total). Entretanto, para o estudo da computação e informática em geral, e das linguagens formais em particular, o conceito de função parcial é tão ou mais importante que o de função (total). De fato, a maioria dos formalismos desenvolvidos ao longo do texto é baseada em funções parciais.

definição 1.10 – Função parcial

Uma *função parcial* é uma relação $\mathbf{f} \subseteq \mathbf{A} \times \mathbf{B}$ tal que:

$$\text{se } (\mathbf{a}, \mathbf{b}) \in \mathbf{f} \text{ e } (\mathbf{a}, \mathbf{c}) \in \mathbf{f}, \text{ então } \mathbf{b} = \mathbf{c}$$ ❑

Portanto, uma função parcial é uma relação na qual cada elemento do domínio está relacionado com, no máximo, um elemento do codomínio. Uma função parcial $\mathbf{f} \subseteq \mathbf{A} \times \mathbf{B}$ normalmente é denotada por:

$$\mathbf{f}: \mathbf{A} \nrightarrow \mathbf{B}$$

ou simplesmente por $\mathbf{f}: \mathbf{A} \rightarrow \mathbf{B}$, quando é claro que se trata de uma função parcial. Adicionalmente, $(\mathbf{a}, \mathbf{b}) \in \mathbf{f}$ é usualmente denotado por $\mathbf{f}(\mathbf{a}) = \mathbf{b}$.

definição 1.11 – Imagem, conjunto imagem

Seja $\mathbf{f}: \mathbf{A} \rightarrow \mathbf{B}$ uma função parcial. Então:

a Se, para $\mathbf{a} \in \mathbf{A}$, existe $\mathbf{b} \in \mathbf{B}$ tal que $\mathbf{f}(\mathbf{a}) = \mathbf{b}$, então afirma-se que **f** está *definida* para **a**, e que **b** é *Imagem* de **a**; caso contrário (*não* existe $\mathbf{b} \in \mathbf{B}$), se afirma que **f** é *não definida* para **a**;

b O *conjunto imagem* de **f**, denotado por $\mathbf{f}(\mathbf{A})$ ou $\mathbf{Img}(\mathbf{f})$, é tal que:

$$\mathbf{f}(\mathbf{A}) = \mathbf{Img}(\mathbf{f}) = \{ \mathbf{b} \in \mathbf{B} \mid \text{existe } \mathbf{a} \in \mathbf{A} \text{ tal que } \mathbf{f}(\mathbf{a}) = \mathbf{b} \}$$ ❑

exemplo 1.7 – Função parcial, conjunto imagem

a *Adição nos naturais*. A operação $\mathbf{ad}: \mathbf{N} \times \mathbf{N} \rightarrow \mathbf{N}$ definida como abaixo

$$\mathbf{ad}(\mathbf{a}, \mathbf{b}) = \mathbf{a} + \mathbf{b}$$

é uma função parcial na qual o conjunto imagem é **N**, ou seja, coincide com o codomínio;

b *Divisão nos reais*. A operação $\mathbf{div}: \mathbf{R} \times \mathbf{R} \rightarrow \mathbf{R}$ tal que:

$$\mathbf{div}(\mathbf{x}, \mathbf{y}) = \mathbf{x} / \mathbf{y}$$

é uma função parcial (qual o conjunto imagem?). ❑

definição 1.12 – Composição de funções parciais

Sejam **f: A → B** e **g: B → C** funções parciais. A *composição* de **f** e **g** é a função **g o f: A → C** tal que, para todo **a ∈ A**:

(g o f)(a) = g(f(a)), se **f(a)** e **g(f(a))** são definidas
(g o f)(a) é indefinida, caso contrário ❑

exemplo 1.8 – Composição de funções parciais

Considere a figura 1.2. A composição das funções parciais **f: A → B** e **g: B → C** é **g o f: A → C** e é tal que:

(g o f)(a) = g(f(a)) = g(1) = x
(g o f)(b) é indefinida, pois **g** é indefinida para **f(b) = 3**
(g o f)(c) é indefinida, pois **f(c)** é indefinida
(g o f)(d) = g(f(d)) = g(5) = z ❑

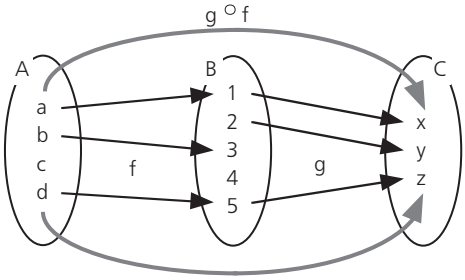


figura 1.2 Composição de funções parciais.

1.2.4 funções

Função total, ou simplesmente função, é um caso particular de função parcial.

definição 1.13 – Função, aplicação

Uma *aplicação*, *função total* ou simplesmente *função* é uma função parcial **f: A → B** a qual é *total*, ou seja, para todo **a ∈ A**, existe **b ∈ B** tal que **f(a) = b**. ❑

Portanto, uma função (total) é uma função parcial definida para todos os elementos do domínio.

exemplo 1.9 – Função

Compare o que segue com o exemplo 1.7 – Função parcial, conjunto imagem:

a *Adição nos naturais*. A operação **ad: N × N → N** definida como abaixo é uma função:

$$ad(a, b) = a + b$$

b *Divisão nos reais*. A operação **div: R × R → R** definida como abaixo *não* é uma função, pois não é definida para **(x, 0)**, qualquer que seja **x ∈ R**:

$$div(x, y) = x/y$$ ❑

definição 1.14 – Composição de funções

Sejam **f: A → B** e **g: B → C** funções. A *composição* de **f** e **g** é a função:

$$g \circ f: A \to C$$

tal que, para todo **a ∈ A**, **(g ∘ f)(a) = g(f(a))**. ❑

Portanto, a composição de duas funções **f: A → B** e **g: B → C** é a função **g ∘ f: A → C** onde **(g ∘ f)(a) = g(f(a))** é a aplicação de **f** ao elemento **a** e, na sequência, de **g** à imagem **f(a)**.

exemplo 1.10 – Composição de funções

A composição das funções **ad: N × N → N** (exemplo 1.9 – Função) e **quadrado: N → N** é a função **quadrado ∘ ad: N × N → N** sendo que, para **(3, 1) ∈ N × N**, vale que:

$$(\text{quadrado} \circ \text{ad})(3, 1) = \text{quadrado}(\text{ad}(3, 1)) = \text{quadrado}(4) = 16$$

Ou seja, a composição das funções resulta no quadrado da adição. ❑

definição 1.15 – Função injetora, função sobrejetora, função bijetora

Uma função **f: A → B** é dita:

a *Injetora* ou *monomorfismo* se, para todo **b ∈ B**, existe no máximo um **a ∈ A** tal que **f(a) = b**;

b *Sobrejetora* ou *epimorfismos* se, para todo **b ∈ B**, existe pelo menos um **a ∈ A** tal que **f(a) = b**;

c *Bijetora* ou *isomorfismo* se é simultaneamente injetora e sobrejetora. ❑

Portanto, uma função é:

a Injetora se cada elemento do codomínio é imagem de, no máximo, um elemento do domínio. Uma função injetora **f: A → B** frequentemente é denotada por:

$$f: A \rightarrowtail B$$

b Sobrejetora se todo elemento do codomínio é imagem de pelo menos um elemento do domínio. Uma função sobrejetora **f: A → B** frequentemente é denotada por:

$$f: A \twoheadrightarrow B$$

c Bijetora se todo elemento do codomínio é imagem de exatamente um elemento do domínio. Uma função bijetora **f: A → B** frequentemente é denotada por:

$$f: A \leftrightarrow B$$

É comum usar os termos *injeção*, *sobrejeção* e *bijeção* em vez de função injetora, função sobrejetora e função bijetora, respectivamente. Adicionalmente, se existe um isomorfismo (função bijetora) entre dois conjuntos, estes são ditos *conjuntos isomorfos*.

exemplo 1.11 – Função injetora, função sobrejetora, função bijetora, conjuntos isomorfos

a *Função identidade*. É uma função **f: A → A** tal que, para todo **a ∈ A**, **f(a) = a**, e é usualmente denotada por:

$$id_A: A \to A$$

Uma função identidade é sempre uma bijeção, ou seja, $id_A: A \leftrightarrow A$

b *Função inclusão.* É uma função $f: A \rightarrow B$ tal que $A \subseteq B$ e, para todo $a \in A$, $f(a) = a$, e é usualmente denotada por:

$$inc_{A,B}: A \hookrightarrow B \text{ ou simplesmente } inc: A \hookrightarrow B$$

Uma função inclusão é sempre uma injeção, ou seja, $inc_{A,B}: A \rightarrowtail B$

c A função **módulo:** $Z \rightarrow N$ tal que **módulo(a) =** $|a|$ é sobrejetora, ou seja, **módulo:** $Z \twoheadrightarrow N$

d A função $f: Z \rightarrow N$ tal que:

$f(a) = 2a$ se $a \geq 0$
$f(a) = |2a| - 1$ se $a < 0$

é bijetora, ou seja, é um isomorfismo. Portanto, os conjuntos **Z** e **N** são conjuntos isomorfos. Note-se que os inteiros não negativos são associados aos naturais pares, e os inteiros negativos, aos naturais ímpares. ❑

1.2.5 cardinalidade de conjuntos

A cardinalidade de um conjunto é uma medida de seu tamanho e é definida usando funções bijetoras. Uma importante conclusão do estudo da cardinalidade de conjuntos é que nem todos os conjuntos infinitos possuem a mesma cardinalidade, o que contradiz a noção intuitiva da maioria das pessoas. De fato, como será visto ao longo do livro, o cardinal (infinito) dos *problemas solucionáveis* (um problema é solucionável se existe pelo menos um algoritmo computacional capaz de solucioná-lo) é menor que o cardinal (infinito) dos *problemas não solucionáveis*. Isto significa que existem mais problemas não solucionáveis do que problemas solucionáveis.

definição 1.16 – Cardinalidade finita, cardinalidade infinita

A *cardinalidade* de um conjunto **A**, representada por **#A**, é:

a *Finita* se existe uma bijeção entre **A** e o conjunto $\{ 1, 2, 3, \ldots, n \}$, para algum $n \in N$. Nesse caso, afirma-se que (como fica o caso em que $n = 0$?):

$$\#A = n$$

b *Infinita* se existe uma bijeção entre **A** e um subconjunto próprio de **A**. ❑

Portanto, um conjunto **A** é um:

- *conjunto finito* (ou seja, possui uma cardinalidade finita) se for possível representá-lo por extensão, como já comentado;
- *conjunto infinito* se for possível retirar alguns elementos de **A** e, mesmo assim, estabelecer uma bijeção com **A**.

exemplo 1.12 – Cardinalidade do conjunto dos números inteiros

Já foi exemplificado (exemplo 1.11 – Função injetora, função sobrejetora, função bijetora, conjuntos isomorfos) que a função $f: Z \rightarrow N$ tal que:

se $a \geq 0$, então $f(a) = 2a$
se $a < 0$, então $f(a) = |2a| - 1$

é bijetora. Claramente, **N** é subconjunto próprio de **Z**. Então **Z** é infinito. ❑

De especial interesse para computação e informática é o cardinal do conjunto dos números naturais **N**, denotado por $\aleph_0$. O símbolo $\aleph$ (lê-se "álef") é a primeira letra do alfabeto hebraico. De fato, $\aleph_0$ é o menor cardinal dos conjuntos infinitos.

definição 1.17 – Conjunto contável, conjunto não contável, conjunto enumerável

Um conjunto infinito **A** é dito:

a *Infinitamente contável* ou *enumerável* se existe uma bijeção entre **A** e um subconjunto infinito de **N**. Neste caso, a bijeção é denominada *enumeração* de **A**;

b *Não contável*, caso contrário. ❑

Portanto, um conjunto é contável se for possível enumerar seus elementos como uma sequência na forma $a_0, a_1, a_2, \ldots$. O cardinal de qualquer conjunto contável é $\aleph_0$.

exemplo 1.13 – Conjunto enumerável, conjunto não contável

a *Enumeráveis*: os conjuntos **Z** e **Q**

b *Não contáveis*: os conjuntos **I** e **R**. De fato, prova-se que o cardinal de **I** e **R** é $2^{\aleph_0}$. ❑

1.3 → noções de lógica

Lógica matemática é básica para qualquer estudo em computação e informática e, em particular, para estudo de linguagens formais. O principal objetivo do texto que segue é apresentar ou revisar os conceitos e terminologia usados e, portanto, não é uma abordagem detalhada. Caso o leitor domine esses conceitos, sugere-se uma passagem rápida por esta seção para verificar a nomenclatura e as convenções adotadas ao longo do livro.

Entende-se por *lógica booleana* ou *lógica de Boole* o estudo dos princípios e métodos usados para distinguir sentenças verdadeiras de sentenças falsas. George Boole (inglês, 1815-1864) foi um dos precursores da lógica.

definição 1.18 – Proposição

a Uma *proposição* é uma construção (sentença, frase, pensamento) a que se pode atribuir juízo. No caso da *lógica matemática*, o tipo de juízo é o verdadeiro-falso, ou seja, o interesse é na "verdade" das proposições;

b Considere um conjunto universo **U**. Uma *Proposição Sobre* **U** é uma proposição cujo valor lógico depende de um elemento $x \in U$ considerado. ❑

Em lógica matemática, a forma tradicional de tratar com a "verdade" é considerar dois *valores verdade* **V** e **F** (*verdadeiro* e *falso*, respectivamente) e estipular que proposições só podem assumir esses valores. Uma proposição **p** a qual descreve alguma propriedade de um elemento

$x \in U$ é usualmente denotada por $p(x)$. Toda proposição p sobre U induz uma partição de U em duas classes de equivalência, como segue:

a $\{ x \mid p(x)$ é verdadeira $\}$, denominado *conjunto verdade* de p
b $\{ x \mid p(x)$ é falsa $\}$, denominado *conjunto falsidade* de p

definição 1.19 – Tautologia, contradição

Seja p uma proposição sobre o conjunto universo U. Então:

a p é dita uma *tautologia* se $p(x)$ é verdadeira para qualquer $x \in U$
b p é dita uma *contradição* se $p(x)$ é falsa para qualquer $x \in U$ ❑

exemplo 1.14 – Proposição, tautologia, contradição

a A sentença $3 + 4 > 5$ é uma proposição;
b Para a proposição $n! < 10$ sobre N, vale que:
$\{ 0, 1, 2, 3 \}$ é o conjunto verdade;
$\{ n \in N \mid n > 3 \}$ é o conjunto falsidade;
c A proposição $n + 1 > n$ sobre N é uma tautologia;
d A proposição "$2n$ é ímpar" sobre N é uma contradição. ❑

As proposições introduzidas até o momento são ditas *proposições atômicas* ou simplesmente *átomos*, no sentido em que não podem ser decompostas em proposições mais simples.

Entretanto, é possível construir proposições mais complexas, usualmente denominadas *fórmulas lógicas* ou simplesmente *fórmulas*, compondo proposições usando *operadores lógicos*, também denominados de *conetivos (lógicos)*.

Neste contexto, *tabela-verdade* é uma tabela que descreve os valores lógicos de uma proposição em termos das possíveis combinações dos valores lógicos das proposições componentes e dos conetivos usados. Assim, para cada combinação de valores-verdade e de conetivos, a tabela-verdade fornece o valor-verdade da expressão resultante.

definição 1.20 – Operadores lógicos

Para proposições lógicas p e q, a semântica dos seguintes *operadores lógicos* ou *conetivos (lógicos)* é dada pela tabela-verdade ilustrada na figura 1.3:

a *Negação*. A *negação* de uma proposição é construída, introduzindo-se a palavra *não* de forma apropriada ou prefixando-se a proposição por "não é fato que" (ou expressão equivalente).

Se p denota uma proposição, então a negação de p é denotada por:

$$\neg p \quad \text{ou} \quad \sim p$$

a qual é lida:

"não p"

b *Conjunção*. A conjunção de duas proposições p e q, denotada por:

$$p \wedge q$$

a qual é lida:

"**p** e **q**"

reflete uma noção de simultaneidade para ser verdadeira;

c *Disjunção*. A disjunção de duas proposições **p** e **q**, denotada por:

$$p \vee q$$

a qual é lida:

"**p** ou **q**"

reflete a noção de que pelo menos uma (eventualmente duas) das proposições componentes deve ocorrer para que a resultante seja verdadeira;

d *Condição*. A condição envolvendo duas proposições **p** e **q**, denotada por:

$$p \rightarrow q$$

a qual é lida:

"se **p** então **q**"

reflete a noção de que, a partir de uma *premissa* verdadeira (ou seja, **p** é verdadeira), obrigatoriamente deve-se chegar a uma *conclusão* verdadeira (ou seja, **q** é verdadeira), para que a proposição composta $p \rightarrow q$ seja verdadeira. Entretanto, partindo de uma premissa falsa, qualquer conclusão pode ser considerada;

e *Bicondição*. A bicondição envolvendo duas proposições **p** e **q**, denotada por:

$$p \leftrightarrow q$$

a qual é lida:

"**p** se e somente se **q**"

reflete a noção de condição "nos dois sentidos", ou seja, considera simultaneamente:

ida: **p** é premissa e **q** é conclusão;
volta: **q** é premissa e **p** é conclusão. ❑

p	q	$\neg p$	$p \wedge q$	$p \vee q$	$p \rightarrow q$	$p \leftrightarrow q$
V	V	F	V	V	V	V
V	F	F	F	V	F	F
F	V	V	F	V	V	F
F	F	V	F	F	V	V

figura 1.3 Tabela-verdade: **negação**, **e**, **ou**, **se-então**, **se-somente-se**.

Note-se que todos os operadores definidos acima são binários e denotados de forma infixada (operador no meio dos operandos), excetuando-se o de Negação, que é unário e denotado de forma prefixada.

Na realidade, o conetivo bicondicional pode ser visto como uma composição dos conetivos condicional e conjunção. De fato, sugere-se como exercício verificar que:

$$\mathbf{p} \leftrightarrow \mathbf{q} \text{ possui a mesma tabela-verdade que } (\mathbf{p} \rightarrow \mathbf{q}) \wedge (\mathbf{q} \rightarrow \mathbf{p})$$

ou seja, que para as mesmas atribuições de verdade a **p** e a **q**, as duas proposições têm o mesmo valor-verdade.

Os conetivos de condição e bicondição induzem as relações de *implicação* e de *equivalência* entre fórmulas, respectivamente. A importância dessas relações pode ser resumida como segue:

a *Relação de implicação*: está diretamente relacionada com o conceito de *teorema*, como será visto adiante;

b *Relação de equivalência*: permite definir a noção de "mesmo significado" entre duas fórmulas (sintaticamente) diferentes.

definição 1.21 – Relação de implicação, equivalência

Sejam **p** e **q** duas proposições. Então:

a *Relação de implicação*. Afirma-se que **p** *implica* **q**, fato que é denotado por:

$$\mathbf{p} \Rightarrow \mathbf{q} \quad \text{ou} \quad \mathbf{p} \models \mathbf{q}$$

se e somente se:

$$\mathbf{p} \rightarrow \mathbf{q} \text{ é uma tautologia}$$

b *Relação de equivalência*. Afirma-se que **p** *é equivalente a* **q**, fato que é denotado por:

$$\mathbf{p} \Leftrightarrow \mathbf{q}$$

se e somente se:

$$\mathbf{p} \leftrightarrow \mathbf{q} \text{ é uma tautologia}$$ ❑

É fácil verificar que $\Rightarrow$ e $\Leftrightarrow$ são relações de ordem e de equivalência, respectivamente.

exemplo 1.15 – Relação de implicação, equivalência

a *Adição* (tabela-verdade ilustrada na figura 1.4)

$$\mathbf{p} \Rightarrow \mathbf{p} \vee \mathbf{q}$$

b *Simplificação* (tabela-verdade ilustrada na figura 1.4)

$$\mathbf{p} \wedge \mathbf{q} \Rightarrow \mathbf{p}$$

c *Contraposição* (tabela-verdade ilustrada na figura 1.5)

$$\mathbf{p} \rightarrow \mathbf{q} \Leftrightarrow \neg\mathbf{q} \rightarrow \neg\mathbf{p}$$

d *Redução ao absurdo* (tabela-verdade ilustrada na figura 1.6)

$$\mathbf{p} \rightarrow \mathbf{q} \Leftrightarrow (\mathbf{p} \wedge \neg\mathbf{q}) \rightarrow \mathbf{F}$$ ❑

p	q	$p \vee q$	$p \rightarrow (p \vee q)$	$p \wedge q$	$(p \wedge q) \rightarrow p$
V	V	V	V	V	V
V	F	V	V	F	V
F	V	V	V	F	V
F	F	F	V	F	V

figura 1.4 Tabela-verdade: adição e simplificação.

p	q	$\neg p$	$\neg q$	$p \rightarrow q$	$\neg q \rightarrow \neg p$	$(p \rightarrow q) \leftrightarrow (\neg q \rightarrow \neg p)$
V	V	F	F	V	V	V
V	F	F	V	F	F	V
F	V	V	F	V	V	F
F	F	V	V	V	V	V

figura 1.5 Tabela-verdade: contraposição.

p	q	$\neg q$	$p \rightarrow q$	$p \wedge \neg q$	$((p \wedge \neg q) \rightarrow F)$	$(p \rightarrow q) \leftrightarrow ((p \wedge \neg q) \rightarrow F)$
V	V	F	V	F	V	V
V	F	V	F	V	F	V
F	V	F	V	F	V	V
F	F	V	V	F	V	V

figura 1.6 Tabela-verdade: redução ao absurdo.

1.4 → técnicas de demonstração

Um *teorema* é uma proposição do tipo:

$$p \rightarrow q$$

a qual, prova-se, é verdadeira sempre (tautologia), ou seja, que:

$$p \Rightarrow q$$

As proposições **p** e **q** são denominadas *hipótese* e *tese*, respectivamente. É usual denominar *corolário* um teorema que é uma consequência quase direta de um outro já demonstrado (ou seja, cuja prova é trivial ou imediata). Adicionalmente, um teorema auxiliar que possui um resultado importante para a prova de um outro é usualmente denominado *lema*.

Teoremas são fundamentais em computação e informática e, em particular, no estudo de linguagens formais e autômatos. Por exemplo, um teorema permite verificar se uma determinada implementação é correta. Sob este ponto de vista, um teorema pode ser visto como um algoritmo que, prova-se, sempre funciona. De fato, muitos dos teoremas apresentados ao longo do livro refletem essa ideia.

Antes de iniciar uma demonstração, é fundamental identificar claramente a hipótese e a tese. Por exemplo, considere o seguinte teorema:

> *a intersecção distribui-se sobre a união, ou seja,*
> $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$.

Uma reescrita identificando claramente a hipótese e a tese, é como segue:

> *se* A, B *e* C *são conjuntos quaisquer,*
> *então* $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$.

É usual um teorema ser apresentado na forma $p \leftrightarrow q$ como, por exemplo (suponha que A é um conjunto qualquer):

> A *é contável se e somente se existe uma função*
> *bijetora entre* A *e o conjunto dos números pares.*

Já foi sugerido como exercício verificar que:

$$p \leftrightarrow q \Leftrightarrow (p \rightarrow q) \wedge (q \rightarrow p)$$

Assim, neste caso, deve ser demonstrada a "ida" e a "volta", ou seja, que:

> *se um conjunto* A *é contável, então existe uma função*
> *bijetora entre* A *e o conjunto dos números pares*

e

> *se existe uma função bijetora entre* A *e o conjunto*
> *dos números pares, então* A *é contável.*

Para um determinado teorema $p \rightarrow q$, existem diversas técnicas para provar (demonstrar) que, de fato, $p \Rightarrow q$. As seguintes *técnicas de demonstração* destacam-se:

a *Prova direta*;
b *Prova por contraposição*;
c *Prova por redução ao absurdo* ou simplesmente *prova por absurdo*;
d *Prova por indução*.

A prova por indução é uma aplicação particular do *princípio da indução matemática*, a qual será estudada na seção 1.5 – Indução. Os demais tipos de prova são descritos a seguir. Destaque-se que, ao longo de todo o livro, cada demonstração é um exemplo de uso de tais

técnicas. Seguindo este raciocínio, os exercícios de demonstração também estão distribuídos ao longo de todo o livro.

1.4.1 prova direta

definição 1.22 – Prova direta

Uma prova é dita *prova direta* ou *demonstração direta* quando pressupõe verdadeira a hipótese e, a partir desta, prova ser verdadeira a tese. ❑

exemplo 1.16 – Prova direta

Considere o teorema:

> *a intersecção distribui-se sobre a união, ou seja,*
> $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$.

exemplificado acima. Inicialmente, lembre-se que:

a Por definição, $X = Y$ se e somente se $X \subseteq Y$ e $Y \subseteq X$

b Por definição, $X \subseteq Y$ se e somente se todos os elementos de X também são elementos de Y

Adicionalmente, é fácil verificar, usando tabela-verdade, que o operador lógico $\wedge$ distribui-se sobre o operador lógico $\vee$, ou seja, para quaisquer proposições p, q e r:

$$p \wedge (q \vee r) \Leftrightarrow (p \wedge q) \vee (p \wedge r)$$

Para provar que $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$, deve-se provar que:

$A \cap (B \cup C) \subseteq (A \cap B) \cup (A \cap C)$ *caso 1*
$(A \cap B) \cup (A \cap C) \subseteq A \cap (B \cup C)$ *caso 2*

Suponha que A, B e C são conjuntos quaisquer.

Caso 1. Suponha $x \in A \cap (B \cup C)$. Então:

$x \in A \cap (B \cup C) \Rightarrow$	definição de intersecção
$x \in A \wedge x \in (B \cup C) \Rightarrow$	definição de união
$x \in A \wedge (x \in B \vee x \in C) \Rightarrow$	distributividade do $\wedge$ sobre o $\vee$
$(x \in A \wedge x \in B) \vee (x \in A \wedge x \in C) \Rightarrow$	definição de intersecção
$x \in (A \cap B) \vee x \in (A \cap C) \Rightarrow$	definição de união
$x \in (A \cap B) \cup (A \cap C)$	
Portanto, $A \cap (B \cup C) \subseteq (A \cap B) \cup (A \cap C)$	

Caso 2. Suponha $x \in (A \cap B) \cup (A \cap C)$. Então:

$x \in (A \cap B) \cup (A \cap C) \Rightarrow$	definição de união
$x \in (A \cap B) \vee x \in (A \cap C) \Rightarrow$	definição de intersecção
$(x \in A \wedge x \in B) \vee (x \in A \wedge x \in C) \Rightarrow$	distributividade do $\wedge$ sobre o $\vee$
$x \in A \wedge (x \in B \vee x \in C) \Rightarrow$	definição de união

$x \in A \wedge x \in (B \cup C) \Rightarrow$ definição de intersecção
$x \in A \cap (B \cup C)$
Portanto, $(A \cap B) \cup (A \cap C) \subseteq A \cap (B \cup C)$

Logo, $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$ ❑

1.4.2 prova por contraposição

A prova por contraposição baseia-se no seguinte resultado (denominado de *contraposição*), o qual foi verificado no exemplo 1.15 – Relação de implicação, equivalência:

$$p \rightarrow q \Leftrightarrow \neg q \rightarrow \neg p$$

definição 1.23 – Prova por contraposição

Uma prova é dita *prova por contraposição* ou *demonstração por contraposição* quando, para provar $p \rightarrow q$, prova-se $\neg q \rightarrow \neg p$, pois são formas equivalentes. Para provar $\neg q \rightarrow \neg p$ basta, a partir de $\neg q$, obter $\neg p$ (prova direta). ❑

exemplo 1.17 – Prova por contraposição

Para demonstrar o seguinte teorema (supondo $n \in \mathbf{N}$):

$$n! > (n + 1) \rightarrow n > 2$$

pode-se, equivalentemente, demonstrar por contraposição que:

$$n \leq 2 \rightarrow n! \leq n + 1$$

Observe que é muito simples provar que $n \leq 2 \rightarrow n! \leq n + 1$, pois basta testar a proposição para os casos $n = 0$, $n = 1$ e $n = 2$, o que se sugere como exercício. ❑

1.4.3 prova por redução ao absurdo

A prova por redução ao absurdo baseia-se no seguinte resultado (denominado de *redução ao absurdo*), o qual foi verificado no exemplo 1.15 – Relação de implicação, equivalência:

$$p \rightarrow q \Leftrightarrow (p \wedge \neg q) \rightarrow F$$

definição 1.24 – Prova por redução ao absurdo

Uma prova é dita *prova (demonstração) por redução ao absurdo* ou simplesmente *prova (demonstração) por absurdo* quando a prova de $p \rightarrow q$ consiste em supor a hipótese p, supor a negação da tese $\neg q$ e concluir uma contradição (em geral, $q \wedge \neg q$). ❑

Observe que a técnica de demonstração conhecida como *prova por contraexemplo*, é uma demonstração por absurdo. De fato, em uma demonstração por absurdo, a construção da contradição $q \wedge \neg q$ é, em geral, a apresentação de um contraexemplo.

exemplo 1.18 – Prova por redução ao absurdo

Considere o seguinte teorema:

> *0 é o único elemento neutro da adição em **N***

ou seja, reescrevendo na forma de **p** → **q**:

> *se 0 é elemento neutro da adição em **N**,*
> *então 0 é o único elemento neutro da adição em **N**.*

Uma prova por redução ao absurdo é como segue:

a Suponha que (hipótese) **0** é elemento neutro da adição em **N** e que (negação da tese) **0** *não* é o único elemento neutro da adição em **N**. Seja **e** um elemento neutro da adição em **N** tal que $e \neq 0$ (se **0** não é o único, então existe um outro, diferente de **0**);

b Então;

- como **0** é elemento neutro, para qualquer $n \in N$, vale $n = 0 + n = n + 0$. Em particular, para $n = e$, vale $e = 0 + e = e + 0$
- como **e** é elemento neutro, para qualquer $n \in N$, vale $n = n + e = e + n$. Em particular, para $n = 0$, vale $0 = 0 + e = e + 0$
- portanto, como $e = 0 + e = e + 0$ e $0 = 0 + e = e + 0$, pela transitividade da igualdade, vale $e = 0$, o que é uma contradição, pois foi suposto que $e \neq 0$

Logo, é absurdo supor que o elemento neutro da adição em **N** não é único. Portanto, **0** é o único elemento neutro da adição em **N**. ❑

1.5 → indução

1.5.1 princípio da indução matemática

O *princípio da indução matemática* é uma técnica para lidar com tipos de dados que têm uma *relação de boa-ordem*, isto é, uma relação onde todo subconjunto não vazio do tipo de dado tem um elemento mínimo segundo essa relação de ordem. Um exemplo típico é o conjunto dos números naturais. Dada uma boa-ordem, pode-se aplicar indução para provar propriedades que valem para todo elemento do tipo de dado. Por simplicidade, o tipo de dados que tem uma relação de boa-ordem considerado é o conjunto dos números naturais **N** (ou qualquer outro conjunto isomorfo a **N**).

definição 1.25 – Princípio da indução matemática

Seja **p(n)** uma proposição sobre **M = {n ∈ N | n ≥ m** e **m ∈ N }**. O *princípio da indução matemática* é como segue:

a **p(m)** é verdadeira;
b Para qualquer **k ∈ M**, vale **p(k) ⇒ p(k + 1)**
c Então, para qualquer **n ∈ M**, **p(n)** é verdadeira.

Nesse caso, utiliza-se a seguinte denominação:

- *base de indução*: a proposição **p(m)**
- *hipótese de indução*: a proposição **p(k)**
- *passo de indução*: a implicação **p(k) ⇒ p(k + 1)** ❑

Na definição acima, o princípio da indução matemática foi apresentado na sua forma mais tradicional, usualmente denominada de *primeiro princípio da indução matemática*. Outras formulações alternativas do mesmo princípio são usuais. Observe que, embora o princípio da indução matemática seja definido sobre o conjunto dos números naturais, qualquer conjunto isomorfo a **N** pode ser considerado.

Duas aplicações do princípio da indução matemática destacam-se:

a *Prova indutiva* ou *prova por indução*, uma técnica de demonstração muito comum no contexto da computação e informática, a qual não é do domínio da lógica pura.
b *Definição indutiva* ou *definição recursiva*, igualmente comum no contexto da computação e informática, já usada anteriormente de maneira informal.

1.5.2 prova indutiva

A *prova indutiva* ou *prova por indução* limita-se a confirmar se uma determinada conjectura **p(n)** sobre **n ∈ M = {n ∈ N | n ≥ m** e **m ∈ N }** é correta.

Assim, em uma demonstração por indução, deve-se demonstrar a base de indução **p(m)** e, fixado um **k**, supor verdadeira a hipótese de indução **p(k)** e demonstrar o passo de indução, ou seja, que **p(k) → p(k + 1)** é, de fato, uma implicação.

exemplo 1.19 – Prova por indução: $1 + 2 + \ldots + n = (n^2 + n)/2$

Considere o seguinte teorema:

para qualquer $n \in \mathbf{N}$, *vale que* $1 + 2 + \ldots + n = (n^2 + n)/2$.

Uma prova por indução de **p(n): 1 + 2 + … + n = (n + n)/2** é como segue:

a *Base de indução*. Seja **k = 0**. Então:

$$(0^2 + 0)/2 = (0 + 0)/2 = 0/2 = 0$$

Portanto, **p(0)** é verdadeira. Note que:

$$1 + 2 + \ldots + k = 0 + 1 + 2 + \ldots + k$$

pois **0** é o elemento neutro da adição, e portanto, o somatório até $k = 0$ é perfeitamente definido;

b *Hipótese de indução*. Suponha que, para algum $k \in \mathbf{N}$:

$$p(k): 1 + 2 + \ldots + k = (k^2 + k)/2 \text{ é verdadeira}$$

c *Passo de indução*. Prova para:

$$p(k + 1): 1 + 2 + \ldots + k + (k + 1) = ((k + 1)^2 + k + 1)/2$$

é como segue:

$1 + 2 + \ldots + k + (k + 1) =$
$(1 + 2 + \ldots + k) + (k + 1) =$ pela hipótese de indução
$(k^2 + k)/2 + (k + 1) =$
$(k^2 + k)/2 + (2k + 2)/2 =$
$(k^2 + k + 2k + 2)/2 =$
$((k^2 + 2k + 1) + (k + 1))/2 =$
$((k + 1)^2 + (k + 1))/2$

Portanto, $p(k + 1): 1 + 2 + \ldots + k + (k + 1) = ((k + 1)^2 + (k + 1))/2$ é verdadeira.

Logo, para qualquer $n \in \mathbf{N}$, vale que $1 + 2 + \ldots + n = (n^2 + n)/2$ ❑

1.5.3 segundo princípio da indução matemática

Em diferentes momentos, pode ser conveniente trabalhar com outras formulações do princípio da indução matemática. Uma formulação especialmente importante para computação e informática é o *segundo princípio da indução matemática*.

definição 1.26 – Segundo princípio da indução matemática

Seja **p(n)** uma proposição sobre $M = \{n \in \mathbf{N} \mid n \geq m \text{ e } m \in \mathbf{N}\}$. O *segundo princípio da indução matemática* pode, equivalentemente, ser definido como segue:

a *Primeira versão*.

a.1) **p(m)** é verdadeira;

a.2) Para qualquer $k \in M$, vale:

$$p(m) \wedge p(m + 1) \wedge \ldots \wedge p(k) \Rightarrow p(k + 1)$$

a.3) Então, para qualquer $n \in \mathbf{N}$, **p(n)** é verdadeira.

b *Segunda versão*. Suponha $t \in \mathbf{N}$:

b.1) $p(m), p(m + 1), \ldots, p(m + t)$, são verdadeiras;

b.2) Para qualquer $k \in M$ tal que $k \geq m + t$, vale:

$$p(m) \wedge p(m + 1) \wedge \ldots \wedge p(k) \Rightarrow p(k + 1)$$

b.3) Então, para qualquer $n \in \mathbf{N}$, **p(n)** é verdadeira. ❑

Observe que a segunda versão do segundo princípio da indução matemática prova os **t** primeiros casos em separado para verificar a base de indução.

Uma aplicação usual desse princípio está na definição e na prova de propriedades de expressões, fórmulas, árvores, etc., razão pela qual esse princípio frequentemente é denominado de *indução em estrutura*, *indução estruturada*, ou ainda, *indução estrutural*.

exemplo 1.20 – Segundo princípio da indução matemática: proposição lógica

Considere o seguinte teorema:

> *Suponha que **p** é uma proposição lógica a qual contém exclusivamente os conetivos lógicos conjunção, disjunção e condição.*
> *Se o valor-verdade de todos os átomos de **p** é **V**, então o valor-verdade de **p** é **V**.*

Uma prova por indução (no número de átomos, usando a primeira versão do segundo princípio da indução) é como segue:

a *Base de indução*. Seja $k = 1$. Então **p** é um átomo. Portanto, por hipótese, o valor-verdade de **p** é **V**;

b *Hipótese de indução*. Suponha que, para algum $k \in \mathbf{N}$, e para qualquer $u \in \mathbf{N}$ tal que $u \leq k$, se o número de átomos de **p** é **u**, então o valor-verdade de **p** é **V**;

c *Passo de indução*. Seja **p** uma proposição com $k + 1$ átomos. Então **p** pode ser reescrita em um dos seguintes casos, sendo que **q** e **r** são proposições lógicas as quais, individualmente, possuem no máximo **k** átomos e, conjuntamente, possuem $k + 1$ átomos:

$$q \wedge r \qquad q \vee r \qquad q \rightarrow r$$

Como, por hipótese de indução o valor-verdade de **q** e **r** é **V**, vale que, em qualquer dos casos, o valor-verdade de **p** é **V**. ❑

1.5.4 definição indutiva

O princípio da indução matemática pode ser usado também em definições. Uma definição de uma construção usando esse princípio é denominada *definição indutiva* ou *definição recursiva*. Nesse caso, afirma-se que a construção é *indutivamente definida* ou *recursivamente definida*. Resumidamente, em uma definição indutiva:

- a base de indução explicita os casos elementares (mais simples);
- o passo de indução determina como os demais casos são definidos em termos dos anteriores.

Este tipo de definição já foi usado anteriormente quando da definição de fecho transitivo (veja Definição 1.7 – Fecho transitivo, fecho transitivo e reflexivo), a qual é reproduzida a seguir:

O fecho de **R** em relação ao conjunto de propriedades **{ transitiva }**, denominado *fecho transitivo* de **R** e denotado por $\mathbf{R}^+$, é definido como segue:

a Se $(a, b) \in R$, então $(a, b) \in R^+$
b Se $(a, b) \in R^+$ e $(b, c) \in R^+$, então $(a, c) \in R^+$
c Os únicos elementos de R^+ são os construídos como acima.

Nesse caso, vale que:

- item a) é a base de indução;
- item b) é o passo de indução (e a hipótese?);
- item c) garante que, de fato, esta é uma definição indutiva.

Quando se afirma que se trata de uma definição indutiva, o item c) acima é usualmente omitido (é subentendido).

1.6 → exercícios

exercício 1.1 Qual a relação entre linguagens formais e as análises léxica, sintática e semântica?

exercício 1.2 Para $A = \{ 1 \}$, $B = \{ 1, 2 \}$ e $C = \{ \{ 1 \}, 1 \}$, marque as afirmações corretas:

a $A \subset B$ [ ]
b $A \subseteq B$ [ ]
c $A \in B$ [ ]
d $A = B$ [ ]
e $A \subset C$ [ ]
f $A \subseteq C$ [ ]
g $A \in C$ [ ]
h $A = C$ [ ]
i $1 \in A$ [ ]
j $1 \in C$ [ ]
k $\{ 1 \} \in A$ [ ]
l $\{ 1 \} \in C$ [ ]
m $\varnothing \notin C$ [ ]
n $\varnothing \subseteq C$ [ ]

exercício 1.3 Para $A = \{ 1, 2 \}$ e $B = \{ \{ 1 \}, \{ 2 \}, 1, 2 \}$, determine o conjunto resultante em cada um dos seguintes itens:

a $A \cup B$
b $A \cap B$

c $\mathbf{B} \cup \varnothing$
d $\mathbf{B} \cap \varnothing$
e $\mathbf{B} \cup \mathbf{N}$
f $\mathbf{B} \cap \mathbf{N}$
g $\mathbf{B} \cup \mathbf{N} \cup \mathbf{R}$
h $(\mathbf{B} \cup \mathbf{N}) \cap \mathbf{R}$

exercício 1.4 Prove as seguintes propriedades (suponha que **A** e **B** são conjuntos):

Sugestão: soluções para este exercício podem depender de algum conhecimento de lógica. Caso você encontre alguma dificuldade ao tentar resolvê-lo, volte a esse exercício após o estudo da seção 1.3 – Noções de lógica e as correspondentes propriedades apresentadas no exercício 1.24.

a *Elemento neutro*.
a.1) $\mathbf{A} \cup \varnothing = \varnothing\ \mathbf{A} = \mathbf{A}$
a.2) $\mathbf{A} \cap \mathbf{U} = \mathbf{U} \cap \mathbf{A} = \mathbf{A}$

b *Idempotência*.
b.1) $\mathbf{A} \cup \mathbf{A} = \mathbf{A}$
b.2) $\mathbf{A} \cap \mathbf{A} = \mathbf{A}$

c *Comutativa*.
c.1) $\mathbf{A} \cup \mathbf{B} = \mathbf{B} \cup \mathbf{A}$
c.2) $\mathbf{A} \cap \mathbf{B} = \mathbf{B} \cap \mathbf{A}$

d *Associativa*.
d.1) $\mathbf{A} \cup (\mathbf{B} \cup \mathbf{C}) = (\mathbf{A} \cup \mathbf{B}) \cup \mathbf{C}$
d.2) $\mathbf{A} \cap (\mathbf{B} \cap \mathbf{C}) = (\mathbf{A} \cap \mathbf{B}) \cap \mathbf{C}$

e *Distributiva*.
e.1) $\mathbf{A} \cap (\mathbf{B} \cup \mathbf{C}) = (\mathbf{A} \cap \mathbf{B}) \cup (\mathbf{A} \cap \mathbf{C})$
e.2) $\mathbf{A} \cup (\mathbf{B} \cap \mathbf{C}) = (\mathbf{A} \cup \mathbf{B}) \cap (\mathbf{A} \cup \mathbf{C})$

f *Duplo complemento*.
$\sim(\sim\mathbf{A}) = \mathbf{A}$

g *DeMorgan*.
g.1) $\sim(\mathbf{A} \cup \mathbf{B}) = \sim\mathbf{A} \cap \sim\mathbf{B}$
g.2) $\sim(\mathbf{A} \cap \mathbf{B}) = \sim\mathbf{A} \cup \sim\mathbf{B}$

h *Universo e vazio*.
h.1) $\mathbf{A} \cup \sim\mathbf{A} = \mathbf{U}$
h.2) $\mathbf{A} \cap \sim\mathbf{A} = \varnothing$

exercício 1.5 Considere a propriedade de *DeMorgan*, relacionada com a operação de complemento e que envolve as operações de união e de intersecção. Prove que a intersecção (respectivamente, a união) pode ser calculada em termos das operações de complemento e de união (respectivamente, de intersecção), ou seja, que:

a $A \cap B = \sim(\sim A \cup \sim B)$
b $A \cup B = \sim(\sim A \cap \sim B)$

exercício 1.6 Referente às operações sobre conjuntos, discuta a validade das seguintes afirmações (suponha que **A** é um conjunto):

a A operação diferença é associativa e comutativa;
b A operação produto cartesiano é associativa e comutativa;
c O conjunto das partes do conjunto das partes de **A** é o conjunto das partes de **A**.

exercício 1.7 Determine em que condições as seguintes relações são conexas (suponha **A** um conjunto não vazio):

a $(2^A, \subseteq)$
b $(2^A, \subset)$

exercício 1.8 Exemplifique cada um dos casos abaixo:

a Relação que não é simétrica nem antissimétrica;
b Relação que é simultaneamente simétrica e antissimétrica.

exercício 1.9 Considere a seguinte relação em **{ 1, 2, 3, 4 }**:

$$R = \{ (1, 1), (1, 2), (2, 1) \}$$

Determine:

a O fecho transitivo R^+;
b O fecho transitivo e reflexivo R^*.

exercício 1.10 Sejam **R: A → A** uma endorrelação e **P** um conjunto de propriedades. Em que condições **R = FECHO-P(R)**?

exercício 1.11 Suponha que são conhecidos todos os trechos parciais que podem ser percorridos por um carteiro (exemplo: da casa **A** para a casa **B**). Usando a noção de grafo como uma relação e o conceito de fecho, como podem ser representados todos os caminhos que o carteiro pode fazer?

exercício 1.12 A definição de grafo como uma endorrelação permite a existência de arcos paralelos, ou seja, dois ou mais arcos com o mesmo nodo origem e destino? Justifique a sua resposta.

exercício 1.13 Toda relação de ordem é uma relação de equivalência e vice-versa? Justifique a sua resposta.

exercício 1.14 Um conjunto vazio pode ser (e, neste caso, em que condições):

a Uma relação de ordem?
b Uma relação de equivalência?

exercício 1.15 Determine uma partição do conjunto vazio.

exercício 1.16 Para cada item abaixo, justifique a sua resposta:

a Toda função é uma função parcial e vice-versa?
b Toda função parcial é uma relação e vice-versa?

exercício 1.17 Relativamente a $\varnothing \subseteq \mathbf{A} \times \mathbf{B}$ (justifique a sua resposta):

a Pode ser uma relação?
b Pode ser uma função parcial?
c Pode ser uma função?

exercício 1.18 Verifique a existência de funções entre os conjuntos de cada item abaixo. No caso de existir, quantas e quais funções distintas podem ser definidas?

a Domínio $\{ a \}$ e codomínio $\{ x, y, z \}$
b Domínio $\varnothing$ e codomínio $\{ x, y, z \}$
c Domínio $\{ a, b, c \}$ e codomínio $\{ x \}$
d Domínio $\{ a, b, c \}$ e codomínio $\varnothing$
e Domínio $\{ a, b \}$ e codomínio $\{ a, b \}$

exercício 1.19 Determine todas as composições de funções que podem ser definidas sobre as seguintes funções:

a Todas as funções com domínio e codomínio em $\{ a, b \}$
b Todas as funções com domínio em $\{ a, b, c \}$ e codomínio em $\{ x \}$

exercício 1.20 Reescreva as seguintes funções com domínio e codomínio em **R** em termos de composição de funções elementares:

a $f_1(x) = x + x^2$
b $f_2(x) = (x \text{ sen } x + \cos x)^2$

exercício 1.21 Seja **f: A → B** uma função. Uma função **g: B → A** é dita:

inversa à esquerda de **f** se $g \circ f = id_A$
inversa à direita de **f** se $f \circ g = id_B$

Prove que:

a Se **f** é injetora, então **f** tem inversa à esquerda;
b Se **f** é sobrejetora, então **f** tem inversa à direita.

exercício 1.22 Prove que **Q** é contável.

Sugestão: considere **Q** como sendo um conjunto de pares e, para construir uma função injetora de **Q** para **N**, lembre-se de que a decomposição de um número em seus fatores primos é única.

exercício 1.23 Prove que:

a A relação ⇒ é de ordem parcial. A relação é conexa?
b A relação ⇔ é de equivalência.

exercício 1.24 Prove as seguintes equivalências:

a *Idempotência*.

$p \wedge p \Leftrightarrow p$
$p \vee p \Leftrightarrow p$

b *Comutativa*.

$p \wedge q \Leftrightarrow q \wedge p$
$p \vee q \Leftrightarrow q \vee p$

c *Associativa*.

$p \wedge (q \wedge r) \Leftrightarrow (p \wedge q) \wedge r$
$p \vee (q \vee r) \Leftrightarrow (p \vee q) \vee r$

d *Distributiva*.

$p \wedge (q \vee r) \Leftrightarrow (p \wedge q) \vee (p \wedge r)$
$p \vee (q \wedge r) \Leftrightarrow (p \vee q) \wedge (p \vee r)$

e *Dupla negação*.

$\neg\neg p \Leftrightarrow p$

f *DeMorgan*.

$\neg(p \wedge q) \Leftrightarrow \neg p \vee \neg q$
$\neg(p \vee q) \Leftrightarrow \neg p \wedge \neg q$

g *Verdadeiro e falso*.

$p \vee \neg p \Leftrightarrow V$

$p \wedge \neg p \Leftrightarrow F$

exercício 1.25 Prove, usando tabela-verdade, que quaisquer dos conetivos estudados podem ser expressos usando somente os conetivos $\neg$ e $\wedge$.

exercício 1.26 Suponha que $A(n)$ denota $1 + 2 + \ldots + n = ((2n + 1)^2)/8$. Então:

a Prove que se $A(k)$ é verdadeiro para um $k \in N$, então $A(k + 1)$ também é verdadeiro;

b Considerando o item acima, discuta a afirmação:

> *portanto, por indução, $A(n)$ é verdadeiro para qualquer $n \in N$.*

c De fato, $A(n)$ é verdadeiro para qualquer $n \in N$? Prove a sua resposta.

exercício 1.27 Prove por indução que, para qualquer $n \in N$, vale que:

$$1 + 8 + 27 \ldots + n^3 = (1 + 2 + \ldots + n)^2$$

Dica: para verificar a base de indução ($n = 0$), lembre-se de que **zero** é o elemento neutro da adição.

exercício 1.28 Desenvolva uma prova por indução do seguinte teorema:

> *Qualquer valor de postagem igual ou maior que 12 reais pode ser formado usando exclusivamente selos de 4 e de 5 reais.*

Dica: faça uma prova por indução no número de selos, usando a segunda versão do segundo princípio da indução.

exercício 1.29 Por que a "prova por indução" que segue não é correta?

a *Proposição*. Dado um conjunto de n torcedores de futebol, se pelo menos um torcedor é gremista, então todos os demais torcedores também são gremistas;

b *"Prova"*. A proposição é trivialmente verdadeira para $n = 1$. O passo de indução pode ser facilmente entendido pelo seguinte exemplo:

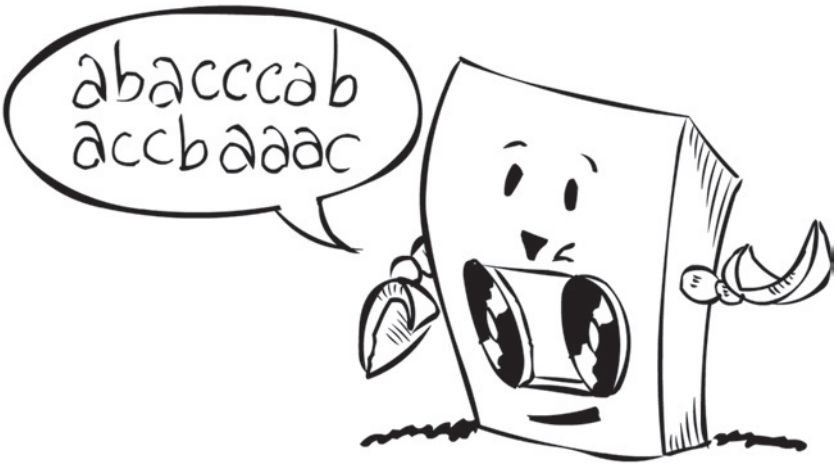
- suponha que a proposição é verdadeira para $n = 3$;
- sejam T_1, T_2, T_3 e T_4, quatro torcedores dos quais pelo menos um é gremista (suponha que é T_1);

- supondo o conjunto $\{ T_1, T_2, T_3 \}$ e a hipótese de que é verdadeiro para $n = 3$, então T_2 e T_3 são gremistas;
- analogamente para $\{ T_1, T_2, T_4 \}$, vale que T_2 e T_4 são gremistas;
- portanto, os quatro torcedores são gremistas!
- a generalização da construção acima para k e $k + 1$, é a prova desejada.

capítulo 2

linguagens e gramáticas

Este capítulo introduz algumas definições fundamentais como alfabeto, palavra e linguagem formal. Também apresenta a definição geral de gramática, essencial para todo o trabalho desenvolvido, e sobre a qual muitos conceitos e resultados de linguagens formais são construídos.



O Dicionário Aurélio define linguagem como:

> *o uso da palavra articulada ou escrita como meio de expressão e comunicação entre pessoas.*

Entretanto, esta definição não é suficientemente precisa para permitir o desenvolvimento matemático de uma teoria baseada em linguagens.

De fato, *linguagem* é um dos conceitos mais fundamentais em computação e informática. Entretanto, para definir linguagem é necessário antes introduzir os conceitos de *alfabeto* e de *palavra* ou *cadeia de caracteres*.

2.1 alfabeto

As definições que seguem são construídas usando como base a noção de *símbolo* ou *caractere*. Portanto, esta é uma entidade abstrata básica, não sendo definida formalmente. Letras e dígitos são exemplos de símbolos frequentemente usados.

definição 2.1 – Alfabeto

Um *alfabeto* é um conjunto finito de *símbolos* ou *caracteres*. ❑

Portanto:

- um conjunto infinito *não* é um alfabeto;
- o conjunto vazio é um alfabeto.

exemplo 2.1 – Alfabeto

a Os seguintes conjuntos *são* exemplos de alfabetos:

{ a, b, c }
∅ (conjunto vazio)

b Os seguintes conjuntos *não* são exemplos de alfabetos (por quê?):

N (conjunto dos números naturais)
{ a, b, aa, ab, ba, bb, aaa,... } ❑

exemplo 2.2 – Alfabeto: linguagem de programação

O alfabeto de uma linguagem de programação como Pascal é o conjunto de todos os símbolos usados na construção de programas, incluindo:

- letras;
- dígitos;
- caracteres especiais como ">", "/", etc.;
- espaço ou "branco". ❑

Ao longo de todo o texto, é usado, com frequência, um alfabeto binário (no caso, o alfabeto **{ a, b }**) pois, além de manter uma perfeita analogia com a representação interna dos computadores reais (o domínio de valores de um *bit* é binário), a manipulação de poucos símbolos simplifica as diversas abordagens desenvolvidas.

2.2 → palavra

definição 2.2 – Palavra, cadeia de caracteres, sentença

Uma *palavra*, *cadeia de caracteres* ou *sentença* sobre um alfabeto é uma sequência finita de símbolos (do alfabeto) justapostos. ❑

Portanto, uma cadeia sem símbolos é uma palavra válida, e o símbolo:

ε denota a *cadeia vazia* ou *palavra vazia*

definição 2.3 – Prefixo, sufixo, subpalavra

Um *prefixo* (respectivamente, *sufixo*) de uma palavra é qualquer sequência inicial (respectivamente, final) de símbolos da palavra.

Uma *subpalavra* de uma palavra é qualquer sequência de símbolos contíguos da palavra. ❑

exemplo 2.3 – Palavra, prefixo, sufixo, subpalavra

a **abcb** é uma palavra sobre o alfabeto **{ a, b, c }**

b Relativamente à palavra **abcb**, vale que:

ε, **a**, **ab**, **abc**, **abcb** são todos os prefixos;
ε, **b**, **cb**, **bcb**, **abcb** são todos os sufixos;

c Qualquer prefixo ou sufixo de uma palavra é uma subpalavra;

d Para a palavra **aa**, o conjunto de todos os prefixos, de todos os sufixos e de todas as subpalavras é o mesmo:

$$\{ \varepsilon, a, aa \}$$ ❑

exemplo 2.4 – Palavra: linguagem de programação

Em uma linguagem de programação como Pascal, uma palavra é um programa. ❑

definição 2.4 – Concatenação de palavras

A *concatenação de palavras* ou simplesmente *concatenação* é uma operação binária, definida sobre um conjunto de palavras, a qual associa a cada par de palavras uma palavra formada pela justaposição da primeira com a segunda.

Uma concatenação é denotada pela justaposição dos símbolos que representam as palavras componentes. A operação de concatenação satisfaz às seguintes propriedades (suponha **v**, **w**, **t** palavras quaisquer):

a *Associativa*.

$$v(w\,t) = (v\,w)t$$

b *Elemento neutro*.

$$\varepsilon\, w = w = w\, \varepsilon$$ ❑

Como a concatenação de palavras é uma operação associativa, é usual omitir os parênteses. Assim, **v(w t)** ou **(v w)t** pode ser denotado simplesmente por **v w t**.

exemplo 2.5 – Concatenação de palavras

Suponha o alfabeto $\Sigma = \{ a, b \}$. Então, para as palavras $v = baaaa$ e $w = bb$, vale que:

$$v\,w = baaaabb$$
$$v\,\varepsilon = v = baaaa$$ ❑

definição 2.5 – Concatenação sucessiva de uma palavra

A *concatenação sucessiva de uma palavra* (com ela mesma) ou simplesmente *concatenação sucessiva* representada na forma de um expoente (suponha **w** uma palavra):

w^n onde **n** é o número de concatenações sucessivas

é definida indutivamente a partir da operação de concatenação binária, como segue:

$$w^0 = \varepsilon$$
$$w^n = w\,w^{n-1}, \text{ para } n > 0$$ ❑

exemplo 2.6 – Concatenação sucessiva

Sejam **w** uma palavra e **a** um símbolo. Então:

$$w^3 = w\,w\,w$$
$$w^1 = w$$
$$a^5 = aaaaa$$
$$a^n = aaa\ldots a \quad \text{(o símbolo } a \text{ repetido } n \text{ vezes)}$$ ❑

Se Σ representa um alfabeto, então:

Σ^* denota o conjunto de todas as palavras possíveis sobre Σ
Σ^+ denota $\Sigma^* - \{ \varepsilon \}$

definição 2.6 – Conjunto de todas as palavras

Seja Σ um alfabeto. Então o conjunto de toda as palavras Σ^* é indutivamente definido como segue:

a *Base de indução*.
$\varepsilon \in \Sigma^*$
para qualquer $\mathbf{x} \in \Sigma$, vale $\mathbf{x} \in \Sigma^*$

b *Passo de indução*.
Se **u** e **v** são palavras de Σ^*,
então a concatenação **u v** é uma palavra de Σ^* ❑

Portanto, uma definição alternativa para *palavra*, *cadeia de caracteres* ou *sentença* sobre um alfabeto Σ é qualquer elemento **w** de Σ^*, ou seja:

$$\mathbf{w} \in \Sigma^*$$

exemplo 2.7 – Conjunto de todas as palavras

Se $\Sigma = \{ a, b \}$, então:

$\Sigma^+ = \{ a, b, aa, ab, ba, bb, aaa, \ldots \}$
$\Sigma^* = \{ \varepsilon, a, b, aa, ab, ba, bb, aaa, \ldots \}$ ❑

definição 2.7 – Comprimento, tamanho de uma palavra

O *comprimento* ou *tamanho* de uma palavra **w**, representado por $|\mathbf{w}|$, é o número de símbolos que compõem a palavra. Portanto, para um dado alfabeto Σ, comprimento é uma função com domínio em Σ^* e codomínio em **N**. ❑

exemplo 2.8 – Comprimento de uma palavra

a $|abcb| = 4$

b $|\varepsilon| = 0$ ❑

2.3 → linguagem formal

definição 2.8 – Linguagem formal

Uma *linguagem formal* ou simplesmente *linguagem* **L** sobre um alfabeto Σ, é um conjunto de palavras sobre Σ, ou seja:

$$\mathbf{L} \subseteq \Sigma^*$$ ❑

exemplo 2.9 – Linguagem formal

a O conjunto vazio $\varnothing$ e o conjunto formado pela palavra vazia $\{ \varepsilon \}$ são linguagens sobre qualquer alfabeto. Obviamente, vale que:

$$\varnothing \neq \{ \varepsilon \}$$

b Os conjuntos Σ^* e Σ^+ são linguagens sobre um alfabeto Σ qualquer. Obviamente, vale que:

$$\Sigma^* \neq \Sigma^+$$

c Suponha o alfabeto $\Sigma = \{ a, b \}$. Então, o conjunto de *palíndromos* (palavras que têm a mesma leitura da esquerda para a direita e vice-versa) sobre Σ é um exemplo de linguagem infinita. Assim, são palavras desta linguagem:

$$\varepsilon, a, b, aa, bb, aaa, aba, bab, bbb, aaaa, \ldots$$ ❑

exemplo 2.10 – Conjunto de todas as linguagens sobre um alfabeto

O conjunto de todas as linguagens sobre um alfabeto Σ é o conjunto das partes de Σ^*, ou seja:

$$2^{\Sigma^*}$$ ❑

exemplo 2.11 – Linguagem formal: linguagem de programação

Uma linguagem de programação como Pascal é formalmente definida pelo conjunto de todos os programas (palavras) da linguagem. ❑

2.4 gramática

Já foi dito que uma linguagem de programação é formalmente definida pelo conjunto de todos os programas (palavras) da linguagem. Como, em geral, o conjunto de todos os programas de uma linguagem de propósitos gerais como Pascal é infinito, não é uma definição adequada para ser implementada em computador. Uma maneira de especificar de forma finita linguagens (eventualmente) infinitas é usando o formalismo *gramática*.

Uma gramática é, basicamente, um conjunto *finito* de regras as quais, quando aplicadas sucessivamente, geram palavras. O conjunto de todas as palavras geradas por uma gramática define a linguagem. As gramáticas usadas para as linguagens naturais como português são as mesmas que as usadas para linguagens artificiais como Pascal. Eventualmente, gramáticas também são usadas para definir *semântica* de linguagens. Entretanto, para tratar semântica, em geral, são usados outros formalismos.

definição 2.9 – Gramática

Uma *gramática de Chomsky*, *gramática irrestrita* ou simplesmente *gramática* é uma quádrupla ordenada:

$$G = (V, T, P, S)$$

na qual:

a **V**, um conjunto finito de símbolos *variáveis* ou *não terminais*;

b **T**, um conjunto finito de símbolos *terminais* disjunto de **V**;

c **P:** $(V \cup T)^+ \rightarrow (V \cup T)^*$ é uma relação finita (ou seja, **P** é um conjunto finito de pares), denominada de *relação de produções* ou simplesmente *produções*. Cada par da relação é denominado de *regra de produção* ou simplesmente de *produção*;

d **S**, um elemento distinguido de **V** denominado *símbolo inicial* ou *variável inicial*. ❑

Uma regra de produção (α, β) é representada como segue:

$$\alpha \rightarrow \beta$$

Por simplicidade, um grupo de regras de produção da forma:

$$\alpha \rightarrow \beta_1,\ \alpha \rightarrow \beta_2,\ \ldots,\ \alpha \rightarrow \beta_n$$

(mesma componente no lado esquerdo) é usualmente abreviada como:

$$\alpha \rightarrow \beta_1 \mid \beta_2 \mid \ldots \mid \beta_n$$

As regras de produção definem as condições de geração das palavras da linguagem. A aplicação de uma regra de produção é denominada *derivação* de uma palavra e é formalmente definida como um par de uma relação. A aplicação sucessiva de regras de produção (fecho transitivo da relação de derivação) permite derivar as palavras da linguagem representada pela gramática.

definição 2.10 – Relação de derivação

Seja **G = (V, T, P, S)** uma gramática. Uma *derivação* é um par da *relação de derivação* denotada por $\Rightarrow$ com domínio em $(V \cup T)^+$ e codomínio em $(V \cup T)^*$. Um par $\langle \alpha, \beta \rangle$ da relação de derivação é representado de forma infixada como segue:

$$\alpha \Rightarrow \beta$$

A relação de derivação $\Rightarrow$ é indutivamente definida como segue:

a Para toda produção da forma $S \rightarrow \beta$ (**S** é o símbolo inicial de **G**), o seguinte par pertence à relação de derivação:

$$S \Rightarrow \beta$$

b Para todo par $\eta \Rightarrow \rho\ \alpha\ \sigma$ da relação de derivação, se $\alpha \rightarrow \beta$ é regra de **P**, então o seguinte par também pertence à relação de derivação:

$$\eta \Rightarrow \rho\ \beta\ \sigma$$ ❑

Portanto, uma derivação é a substituição de uma subpalavra de acordo com uma regra de produção.

Sucessivos passos de derivação são definidos como segue:

$\Rightarrow^*$ fecho transitivo e reflexivo da relação $\Rightarrow$, ou seja, zero ou mais passos de derivações sucessivos;

$\Rightarrow^+$ fecho transitivo da relação $\Rightarrow$, ou seja, um ou mais passos de derivações sucessivos;
$\Rightarrow^i$ exatos **i** passos de derivações sucessivos, sendo **i** um número natural.

Gramáticas são consideradas formalismos de geração, pois permitem derivar ("gerar") todas as palavras da linguagem que representam.

definição 2.11 – Linguagem gerada

Seja **G = (V, T, P, S)** uma gramática. A *linguagem gerada* pela gramática **G**, denotada por **L(G)** ou **GERA(G)**, é composta por todas as palavras de símbolos terminais deriváveis a partir do símbolo inicial **S**, ou seja:

$$L(G) = \{ w \in T^* \mid S \Rightarrow^+ w \}$$ ❑

exemplo 2.12 – Gramática, derivação, linguagem gerada: números naturais

Suponha que se deseja definir uma gramática capaz de gerar qualquer número natural válido em uma linguagem de programação. Assim, a gramática **G = (V, T, P, N)** na qual:

$$V = \{ N, D \}$$
$$T = \{ 0, 1, 2, \ldots, 9 \}$$
$$P = \{ N \rightarrow D, N \rightarrow DN, D \rightarrow 0 \mid 1 \mid \ldots \mid 9 \}$$

gera, sintaticamente, o conjunto dos números naturais. Note-se que se distinguem os zeros à esquerda. Por exemplo, distingue-se **123** de **0123** (sugere-se como exercício o desenvolvimento de uma gramática a qual não distingue zeros à esquerda). Como ilustração, uma derivação do número **243** é como segue (na coluna à direita, é apresentada a regra usada em cada passo de derivação):

$N \Rightarrow$	$N \rightarrow DN$
$DN \Rightarrow$	$D \rightarrow 2$
$2N \Rightarrow$	$N \rightarrow DN$
$2DN \Rightarrow$	$D \rightarrow 4$
$24N \Rightarrow$	$N \rightarrow D$
$24D \Rightarrow$	$D \rightarrow 3$
243	

Existe mais alguma derivação do número **243**? Portanto, pode-se indicar que:

$$S \Rightarrow^* 243$$
$$S \Rightarrow^+ 243$$
$$S \Rightarrow^6 243$$ ❑

Observe que, no exemplo acima, a seguinte interpretação indutiva pode ser dada à gramática em questão:

- *Base de indução*: todo dígito é um número natural;
- *Passo de indução*: se **n** é um número natural, então a concatenação de **n** com qualquer dígito também é um número natural.

O exemplo que segue pode parecer relativamente complexo para um segundo exemplo de gramática. O objetivo é apenas apresentar algumas das potencialidades que serão exploradas ao longo de todo o livro. De fato, nos capítulos subsequentes, são mostrados, em níveis crescentes de dificuldade, muitos outros exemplos de gramáticas. Neste ponto, alguma dificuldade de entendimento, ou de resolução dos exercícios propostos, é normal.

exemplo 2.13 – Gramática, derivação, linguagem gerada: palavra duplicada

A seguinte gramática:

$$G = (\{ S, X, Y, A, B, F \}, \{ a, b \}, P, S)$$

na qual:

$$\begin{aligned} P = \{ & S \rightarrow XY, \\ & X \rightarrow XaA \mid XbB \mid F \\ & Aa \rightarrow aA,\ Ab \rightarrow bA,\ AY \rightarrow Ya, \\ & Ba \rightarrow aB,\ Bb \rightarrow bB,\ BY \rightarrow Yb, \\ & Fa \rightarrow aF,\ Fb \rightarrow bF,\ FY \rightarrow \varepsilon \ \} \end{aligned}$$

gera a linguagem cujas palavras são tais que a primeira metade é igual à segunda metade:

$$\{ ww \mid w \text{ é palavra de } \{ a, b \}^* \}$$

Como ilustração, uma derivação da palavra **baba** é como segue (na coluna à direita, é apresentada a regra usada em cada passo de derivação):

$S \Rightarrow$	$S \rightarrow XY$
$XY \Rightarrow$	$X \rightarrow XaA$
$XaAY \Rightarrow$	$AY \rightarrow Ya$
$XaYa \Rightarrow$	$X \rightarrow XbB$
$XbBaYa \Rightarrow$	$Ba \rightarrow aB$
$XbaBYa \Rightarrow$	$BY \rightarrow Yb$
$XbaYba \Rightarrow$	$X \rightarrow F$
$FbaYba \Rightarrow$	$Fb \rightarrow bF$
$bFaYba \Rightarrow$	$Fa \rightarrow aF$
$baFYba \Rightarrow$	$FY \rightarrow \varepsilon$
baba	

Existe mais alguma derivação da palavra **baba**? A gramática apresentada gera o primeiro **w** após **X** e o segundo **w** após **Y**, como segue:

- a cada símbolo terminal gerado após **X**, é gerada uma variável correspondente;
- esta variável "caminha" na palavra até passar por **Y**, quando deriva o correspondente terminal;
- para encerrar, **X** deriva a variável **F**, a qual "caminha" até encontrar **Y**, quando **FY** deriva a palavra vazia. Lembre-se: ε é o elemento neutro da concatenação e, portanto, **baεba = baba**. ❑

definição 2.12 – Gramáticas equivalentes

Duas gramáticas G_1 e G_2 são ditas *gramáticas equivalentes* se e somente se:

$$GERA(G_1) = GERA(G_2)$$ ❑

No texto que segue, frequentemente são usadas as seguintes convenções:

- **A**, **B**, **C**,..., **S**, **T** para símbolos variáveis;
- **a**, **b**, **c**,..., **s**, **t** para símbolos terminais;
- **u**, **v**, **w**, **x**, **y**, **z** para palavras de símbolos terminais;
- α, β,... para palavras de símbolos variáveis ou terminais.

2.5 exercícios

exercício 2.1 Marque os conjuntos que são alfabetos:

a Conjunto dos números inteiros [ ]
b Conjunto dos números primos [ ]
c Conjunto das letras do alfabeto brasileiro [ ]
d Conjunto dos algarismos arábicos [ ]
e Conjunto dos algarismos romanos [ ]
f Conjunto **{ a, b, c, d }** [ ]
g Conjunto das partes de **{ a, b, c }** [ ]
h Conjunto das vogais [ ]
i Conjunto das letras gregas [ ]

exercício 2.2 Apresente os possíveis prefixos e sufixos de cada uma das seguintes palavras:

a teoria
b universidade
c **aaa**
d **abccba**
e **abcabc**

exercício 2.3 Exemplifique, comprovando ou negando as seguintes propriedades algébricas da operação de concatenação de palavras:

a Total (suponha a operação definida sobre uma determinada linguagem);
b Comutativa;
c Elemento neutro;
d Associativa;

e Elemento absorvente, genericamente definida como segue: para uma dada operação binária $\oplus$ sobre um conjunto **A**, afirma-se que a operação possui *elemento absorvente* se existe $a \in A$ tal que, para qualquer $x \in A$ vale que:

$$a \oplus x = x \oplus a = a$$

f Elemento inverso, genericamente definido como segue: para uma dada operação binária $\oplus$ sobre um conjunto **A**, afirma-se que a operação $\oplus$ possui *elemento inverso* se possui elemento neutro **e** e, para qualquer $a \in A$, existe $\underline{a} \in A$ tal que:

$$a \oplus \underline{a} = \underline{a} \oplus a = e$$

exercício 2.4 Sejam as linguagens L_1, L_2 e L_3. Verifique se é verdadeira a igualdade:

$$L_1 (L_2 \cap L_3) = L_1 L_2 \cap L_1 L_3$$

exercício 2.5 Relativamente ao exemplo 2.12 – Gramática, derivação, linguagem gerada: números naturais:

a Existe mais alguma derivação do número **243**? Caso exista, quantas?

b Modifique a gramática de tal forma a não distinguir zeros à esquerda.

exercício 2.6 Considere o exemplo 2.13 – Gramática, derivação, linguagem gerada: palavra duplicada. Existe mais alguma derivação da palavra **baba**?

exercício 2.7 Desenvolva uma gramática que gere a linguagem correspondente aos identificadores da linguagem Pascal (palavras formadas por uma ou mais letras ou dígitos, as quais sempre iniciam por uma letra). Analogamente para os identificadores em Pascal com tamanho máximo de seis caracteres.

exercício 2.8 Desenvolva uma gramática que gere expressões aritméticas com parênteses balanceados, dois operadores (representados por $*$ e $+$) e um operando (representado por **x**). Por exemplo, as seguintes palavras são expressões aritméticas válidas:

$$x \qquad x*(x+x) \qquad (((((x)))))$$

exercício 2.9 Desenvolva uma gramática que gere a seguinte linguagem:

$$\{ a^n b^n c^n \mid n \geq 0 \}$$

capítulo 3

linguagens regulares

Este capítulo é especialmente importante, não apenas pelo assunto tratado, mas também por desenvolver a base do raciocínio típico de linguagens formais. De certa forma, o correto entendimento deste capítulo é central em todo o estudo que segue. Estuda os formalismos regulares do tipo autômato, expressões e gramáticas e verifica a equivalência destes modelos.

O estudo das *linguagens regulares* ou *tipo 3*, é abordado usando os seguintes formalismos:

a *Autômato finito*. Trata-se de um formalismo operacional ou reconhecedor, sendo, basicamente, um *sistema de estados finitos*;

b *Expressão regular*. Trata-se de um formalismo denotacional, também considerado gerador (pois se pode inferir como construir todas as palavras da correspondente linguagem), o qual é definido a partir de conjuntos (linguagens) básicos e das operações de concatenação e de união;

c *Gramática regular*. Trata-se de um formalismo axiomático ou gerador o qual, como o nome indica, é uma gramática, mas com restrições da forma das regras de produção.

De acordo com a hierarquia de Chomsky, as linguagens regulares constituem a classe de linguagens mais simples, sendo possível desenvolver algoritmos de reconhecimento, de geração ou de conversão entre formalismos de pouca complexidade, de grande eficiência e de fácil implementação.

Entretanto, as linguagens regulares possuem fortes limitações de expressividade. Por exemplo, uma linguagem que possua duplo balanceamento não é uma linguagem regular. Como exemplo, considere um texto com parênteses balanceados, ou seja, parênteses em um número qualquer em um texto, eventualmente encadeados (aninhados), de tal forma a garantir que, para cada parêntese aberto, existe um correspondente parêntese fechado (e vice-versa).

Portanto, o tratamento de expressões aritméticas comuns não pode ser realizado usando um formalismo regular. Assim, a maioria das linguagens de programação de propósitos gerais como Pascal, C, Java, etc., são não regulares.

Por outro lado, relativamente à *complexidade de algoritmos*, autômatos finitos pertencem à classe de algoritmos mais eficientes em termos de tempo de processamento (supondo determinada condição detalhada adiante). De fato, qualquer autômato finito que solucione um problema é igualmente eficiente, ou seja, *qualquer solução é ótima*, a menos de eventual redundância de estados, a qual não influi no tempo de processamento. Tal redundância de estados pode ser facilmente eliminada, determinando-se um *autômato finito mínimo*.

Outra característica da classe das linguagens regulares é possuir importantes propriedades as quais podem ser usadas para:

- construir novas linguagens regulares a partir de linguagens regulares conhecidas (definindo uma álgebra);
- provar propriedades;
- construir algoritmos.

Considerando a eficiência e a simplicidade dos algoritmos, bem como das propriedades da classe de linguagens regulares, deve ficar bem claro que, se um problema tiver uma solução regular, esta deve ser considerada preferencialmente a qualquer outra solução não regular.

Um exemplo típico e simples de aplicação das linguagens regulares é a *análise léxica*. Observe que o universo de aplicações das linguagens regulares é muito grande e é constantemente ampliado por novas utilizações. Alguns exemplos mais recentes são *sistemas de animação*, *hipertextos* e *hipermídias*.

Minimização de autômatos finitos, propriedades da classe das linguagens regulares e algumas importantes aplicações são estudadas em capítulos subsequentes.

Para facilitar o estudo das linguagens regulares, a seguir são apresentadas algumas noções de *sistema de estados finitos* e de *composição de sistemas*.

3.1 → sistema de estados finitos

Um *sistema de estados finitos* é um modelo matemático de sistema com entradas e saídas discretas (em oposição ao contínuo). Pode assumir um número *finito* e *predefinido* de estados. Cada estado resume somente as informações do passado necessárias para determinar as ações para a próxima entrada. O fato de possuir um número finito e predefinido de estados significa que todos os estados possíveis do sistema podem ser mecanicamente explicitados antes de iniciar o processamento, ou seja, podem ser definidos de partida, por extensão.

Um forte motivacional para o estudo de sistemas de estados finitos é o fato de poderem ser associados a diversos tipos de sistemas naturais e construídos.

Um exemplo clássico e de simples entendimento é um elevador. Trata-se de um sistema que não memoriza as requisições anteriores. Cada "estado" sumaria as informações "andar corrente" e "direção de movimento". As entradas para o sistema são requisições pendentes.

Analisadores léxicos e processadores de texto (ou algumas ferramentas de processadores de texto) também são exemplos de sistemas de estados finitos, dos quais cada estado, basicamente, memoriza a estrutura do prefixo da palavra em análise.

Entretanto, nem todos os sistemas de estados finitos são adequados para serem estudados por esta abordagem. Um contraexemplo é o cérebro humano. Existem evidências de que um neurônio pode ser representado por um número finito de bits. O cérebro é composto por cerca de 2^{35} células. Portanto, a princípio, é possível representá-lo por um número finito de estados. Entretanto, o elevado número de combinações de células (e, consequentemente, de estados) determina uma abordagem pouco eficiente em termos práticos. Tal fenômeno é denominado *explosão de estados*.

Outro contraexemplo é o computador. Os estados determinados pelos processadores e memórias podem ser representados como um sistema de estados finitos. Entretanto, em um computador, podem ser agregadas memórias adicionais durante o processamento (discos, fitas, memórias auxiliares, etc.). Assim, o número de estados não necessariamente é predefinido antes de se iniciar uma computação, quebrando um dos princípios dos autômatos finitos. De fato, o estudo adequado da noção de *computabilidade* exige uma memória *sem limite predefinido*. Adiante, é apresentado um outro formalismo de autômato, a *máquina de Turing*, com tais características e portanto, adequado ao estudo da computabilidade. Note-se que o estudo da *computabilidade* e *solucionabilidade de problemas* é apenas introduzido nesta publicação (usualmente tais questões são tratadas na *teoria da computação*).

3.2 composição sequencial, concorrente e não determinista

A construção de um sistema é, em geral, composicional, no sentido em que sistemas (possivelmente complexos) são construídos a partir de sistemas conhecidos, e assim sucessivamente, até chegar ao nível mais elementar (como uma *ação atômica*). Três formas de composição se destacam:

a *Sequencial*. A execução da próxima componente depende da terminação da componente anterior;

b *Concorrente*. Resulta em componentes *independentes*, no sentido em que a ordem em que são executadas não é importante, e, portanto, podem ser processadas ao mesmo tempo;

c *Não Determinista*. A próxima componente a ser executada é uma escolha entre diversas componentes alternativas (em oposição à determinista, na qual, para as mesmas condições, a próxima componente a ser executada é sempre a mesma). O não determinismo pode ser:

- *interno*: o sistema escolhe aleatoriamente a próxima componente a ser executada;
- *externo*: a escolha da próxima componente a ser executada é externa ao sistema.

As três formas de composição são comuns nos sistemas reais. Por exemplo, considere-se um universo de pessoas em um banco, no qual as seguintes composições podem ocorrer, entre diversas outras possibilidades:

- sequencial: em uma fila, o atendimento do próximo cliente depende do atendimento do cliente anterior; o pagamento de uma conta depende do fornecimento de um valor;
- concorrente: os diversos caixas atendem independentemente diversos clientes;
- não determinista: quando dois ou mais caixas ficam disponíveis ao mesmo tempo, o próximo cliente pode escolher em qual caixa será atendido; o simples caminhar de um indivíduo pode aleatoriamente iniciar com a perna esquerda ou com a perna direita.

Portanto, na modelagem de sistemas em geral, as três forma de composição devem ser consideradas.

No caso específico das *linguagens formais*, as composições sequencial e não determinista são especialmente importantes, e é nestas que o estudo é centrado. Como curiosidade, destaca-se que a composição concorrente pode ser simulada, usando-se as composições sequencial e não determinista, e sendo, neste caso, denominada *concorrência falsa*.

Em particular, a semântica do *não determinismo* adotada em todo o texto que segue é a usual para *linguagens formais*, para *teoria da computação*, bem como para os demais estudos correlatos, e pode ser resumida como segue:

- considera o não determinismo interno;
- objetiva determinar a capacidade de reconhecer linguagens e de solucionar problemas. Assim, se pelo menos um caminho alternativo é capaz de reconhecer ou solucionar (mesmo que outros não o sejam), a máquina como um todo é considerada capaz de reconhecer ou solucionar.

Esta semântica, detalhada ao longo do texto, difere, por exemplo, da usualmente adotada no estudo dos *modelos para concorrência* (e consequentes aplicações como, por exemplo, no estudo dos *sistemas operacionais*), e pode causar alguma confusão com a semântica da concorrência.

3.3 → autômato finito

Autômato finito é um *sistema de estados finitos* (portanto possui um número finito e *predefinido* de estados) o qual constitui um modelo computacional do tipo sequencial muito comum em diversos estudos teórico-formais da computação e informática, com destaque para *linguagens formais*, *compiladores*, *semântica formal* e *modelos para concorrência*.

Trata-se de um formalismo operacional ou reconhecedor, o qual pode ser:

- *determinístico*: para o estado corrente e o símbolo lido da entrada, o sistema assume um *único* estado bem determinado;
- *não determinístico*: para o estado corrente e o símbolo lido da entrada, o sistema assume um estado pertencente a um *conjunto* de estados alternativos;

- *com movimentos vazios*: para o estado corrente e, independentemente de ler um símbolo ou não da entrada, o sistema assume um estado pertencente a um *conjunto* de estados alternativos (portanto é não determinístico). O movimento é dito *movimento vazio* se o sistema muda de estado sem uma correspondente leitura de símbolo. Movimentos vazios podem ser vistos como *transições encapsuladas* nas quais, excetuando-se por uma eventual mudança de estado, nada mais pode ser observado, de forma análoga à noção de *encapsulação* das *linguagens orientadas a objetos*.

Prova-se que os três tipos de autômatos acima são equivalentes em termos de poder computacional.

Um *autômato finito determinístico* ou simplesmente *autômato finito* pode ser visto como uma máquina constituída, basicamente, de três partes:

a *Fita*. Dispositivo de entrada que contém a informação a ser processada;

b *Unidade de controle*. Reflete o estado corrente da máquina. Possui uma unidade de leitura (*cabeça da fita*) a qual acessa uma célula da fita de cada vez e movimenta-se exclusivamente para a direita;

c *Programa*, *função programa* ou *função de transição*. Função que comanda as leituras e define o estado da máquina.

A fita é finita (à esquerda e à direita), sendo dividida em células, cada uma das quais armazena um símbolo. Os símbolos pertencem a um alfabeto de entrada. Não é possível gravar sobre a fita (e não existe memória auxiliar). Inicialmente, a palavra a ser processada (ou seja, a informação de entrada para a máquina) ocupa toda a fita.

A unidade de controle possui um número finito e predefinido de estados, originando o termo *controle finito*. A unidade de controle lê um símbolo da fita de cada vez. Após a leitura, a cabeça da fita move-se uma célula para a direita. Inicialmente, a cabeça está posicionada na célula mais à esquerda da fita, como ilustrado na figura 3.1.

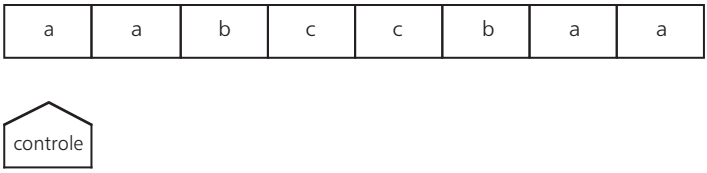


figura 3.1 Autômato finito como uma máquina com controle finito.

O programa é uma função parcial tal que:

> *dependendo do estado corrente e do símbolo lido,*
> *determina o novo estado do autômato.*

definição 3.1 – Autômato finito determinístico

Um *autômato finito determinístico* (abreviado por **AFD**) ou simplesmente *autômato finito* **M** é uma **5**-upla ordenada:

$$M = (\Sigma, Q, \delta, q_0, F)$$

na qual:

a Σ é um *alfabeto de símbolos de entrada*, ou simplesmente *alfabeto de entrada*;

b **Q** é um *conjunto de estados* possíveis do autômato o qual é finito;

c δ é uma *função programa* ou simplesmente *programa*, ou ainda *função de transição*:

$$\delta: Q \times \Sigma \rightarrow Q$$

a qual é uma função parcial. Supondo que a função programa é definida para um estado **p** e um símbolo **a**, resultando no estado **q**, então:

$$\delta(p, a) = q$$

é uma *transição* do autômato;

d q_0 é um elemento distinguido de **Q**, denominado *estado inicial*;

e **F** é um subconjunto de **Q**, denominado *conjunto de estados finais*. ❑

Um autômato finito pode ser representado na forma de um diagrama no qual:

- estados são nodos, representados por círculos;
- transições são arestas, ligando os correspondentes nodos. Por exemplo, uma transição do tipo $\delta(q, a) = p$ é como ilustrada na figura 3.2;
- estados iniciais e finais são representados de forma distinta dos demais, como ilustrado na figura 3.3;
- *transições paralelas* (mesmos nodos origem e destino) podem alternativamente ser representadas como ilustrado na figura 3.4 (supondo que $\delta(q, a) = p$ e $\delta(q, b) = p$).

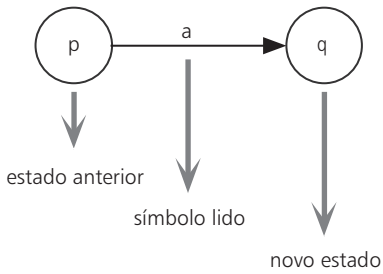


figura 3.2 Diagrama (**AFD**): transição.

figura 3.3 Diagrama (**AFD**): estado inicial (esquerda) e final (direita).

figura 3.4 Diagrama (**AFD**): representações alternativas para transições paralelas.

Uma forma alternativa e muito comum de representar uma função programa é como uma tabela de dupla entrada. Por exemplo, para uma transição do tipo $\delta(\mathbf{p}, \mathbf{a}) = \mathbf{q}$, a correspondente tabela é ilustrada na figura 3.5.

δ	**a**	...
p	**q**	...
q	...	...

figura 3.5 Função programa (**AFD**) na forma de tabela.

A *computação* de um *autômato finito*, para uma palavra de entrada **w**, consiste na sucessiva aplicação da função programa para cada símbolo de **w** (da esquerda para a direita) até ocorrer uma condição de parada.

Para os exemplos que seguem, lembre-se de que o autômato finito não possui memória de trabalho. Portanto, para armazenar as informações passadas necessárias ao processamento, deve-se usar o conceito de estado. Em cada exemplo, procure identificar claramente a informação "memorizada" por cada estado.

exemplo 3.1 – Autômato finito: ***aa*** ou ***bb*** como subpalavra

Considere a seguinte linguagem sobre o alfabeto **{ a, b }**:

$$\mathbf{L_1 = \{\ w \mid w \text{ possui } aa \text{ ou } bb \text{ como subpalavra}\ \}}$$

O autômato finito:

$$\mathbf{M_1 = (\{\ a, b\ \}, \{\ q_0, q_1, q_2, q_f\ \}, \delta_1, q_0, \{\ q_f\ \})}$$

onde δ_1 é dada pela tabela na figura 3.6, reconhece a linguagem $\mathbf{L_1}$. O autômato $\mathbf{M_1}$ é representado pelo diagrama na figura 3.7. O algoritmo apresentado usa os estados $\mathbf{q_1}$ e $\mathbf{q_2}$ para "memorizar" o símbolo anterior lido. Assim:

- q_1 representa "símbolo anterior é **a**";
- q_2 representa "símbolo anterior é **b**";
- qual a informação memorizada por q_0 e q_f?

Após identificar dois **a** ou dois **b** consecutivos, o autômato assume o estado q_f (final) e varre o sufixo da palavra de entrada, somente para terminar o processamento. A figura 3.8 ilustra a computação do autômato finito M_1 para a entrada **w = abba**, a qual é aceita. ❑

δ_1	a	b
q_0	q_1	q_2
q_1	q_f	q_2
q_2	q_1	q_f
q_f	q_f	q_f

figura 3.6 Função programa (**AFD**): sequência de dois símbolos iguais.

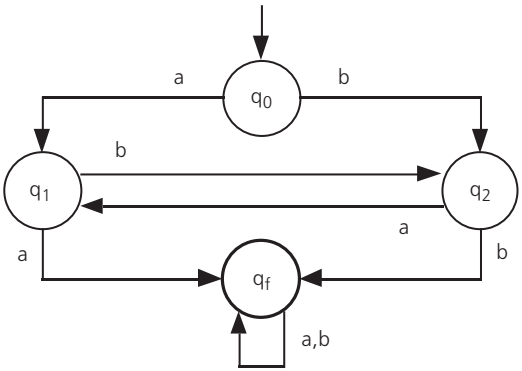


figura 3.7 Diagrama (**AFD**): sequência de dois símbolos iguais.

observação 3.2 – Autômato finito sempre para

Um autômato finito *sempre para* ao processar qualquer entrada pois, como qualquer palavra é finita, e como um novo símbolo da entrada é lido a cada aplicação da função programa, não existe a possibilidade de ciclo (*loop*) infinito. ❑

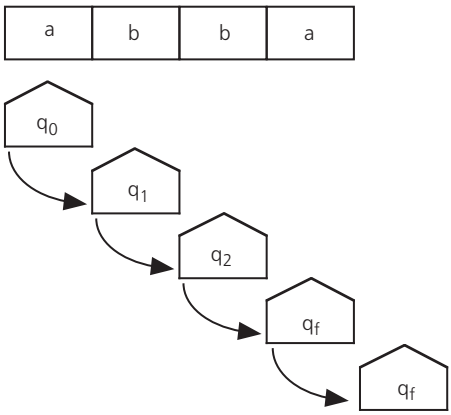


figura 3.8 Computação (**AFD**): sequência de dois símbolos iguais.

A parada do processamento de um autômato finito para uma entrada **w** pode ser de duas maneiras:

a *Aceita* a entrada **w**. Após processar o último símbolo da fita, o autômato finito assume um estado final;

b *Rejeita* a entrada **w**. São duas possibilidades:
- após processar o último símbolo da fita, o autômato finito assume um estado não final;
- em algum momento, ao longo do processamento de **w**, a função programa é indefinida para o argumento (estado corrente e símbolo lido da fita).

observação 3.3 – Autômato finito × grafo finito direto

O leitor com algum conhecimento de *teoria dos grafos* provavelmente está questionando qual a diferença entre um autômato finito e um *grafo finito direto* (aquele em que nodos e arcos são finitos e arcos são direcionados). Qualquer autômato finito pode ser visto como um grafo finito direto, no qual:

- podem existir arcos paralelos (arcos com mesmos nodos origem e destino);
- dois ou mais arcos podem ser identificados com a mesma etiqueta (no caso, símbolo do alfabeto);
- existe um nodo distinguido denominado estado inicial;
- existe um conjunto de nodos distinguidos, cujos elementos são denominados de estados finais.

De fato, muitas definições alternativas de autômato finito usam a definição de grafo como base. Ou seja, é usual considerar um autômato finito como um grafo finito direto especial. A vantagem desta abordagem é poder herdar uma série de resultados da teoria dos grafos. Alguns aspectos correlacionando a *teoria dos autômatos* com a teoria dos grafos são desenvolvidos ao longo do texto. ❑

Para definir formalmente o comportamento de um autômato finito (ou seja, para dar semântica à sintaxe de um autômato finito), é necessário estender a função programa, usando como argumento um estado e uma *palavra*, permitindo determinar as computações possíveis.

definição 3.4 – Função programa estendida, computação

Seja $\mathbf{M = (\Sigma, Q, \delta, q_0, F)}$ um autômato finito determinístico. A *função programa estendida* ou *computação* de **M**, denotada por:

$$\delta^*: Q \times \Sigma^* \rightarrow Q$$

é a função programa $\delta: Q \times \Sigma \rightarrow Q$ estendida para palavras e é indutivamente definida como segue:

$$\delta^*(q, \varepsilon) = q$$
$$\delta^*(q, aw) = \delta^*(\delta(q, a), w)$$ ❑

Portanto, como já comentado, a função programa estendida consiste na sucessiva aplicação da função programa para cada símbolo da palavra, a partir de um dado estado. Observe que, se a entrada for vazia, o autômato fica parado no estado corrente.

Assim, para verificar se um autômato finito aceita ou rejeita uma entrada **w**, deve-se aplicar a função programa estendida para a entrada **w**, a partir do estado inicial do autômato, conforme ilustrado no seguinte exemplo.

exemplo 3.2 – Função programa estendida

Considere o autômato finito $\mathbf{M_1 = (\{ a, b \}, \{ q_0, q_1, q_2, q_f \}, \delta_1, q_0, \{ q_f \})}$ definido no exemplo 3.1 – Autômato finito: **aa** ou **bb** como subpalavra. Então, a computação da palavra **abaa** a partir do estado inicial $\mathbf{q_0}$ é como segue:

$\delta^*(q_0, abaa) =$	função estendida sobre **abaa**
$\delta^*(\delta(q_0, a), baa) =$	processa ***a*baa**
$\delta^*(q_1, baa) =$	função estendida sobre **baa**
$\delta^*(\delta(q_1, b), aa) =$	processa ***b*aa**
$\delta^*(q_2, aa) =$	função estendida sobre **aa**
$\delta^*(\delta(q_2, a), a) =$	processa ***a*a**
$\delta^*(q_1, a) =$	função estendida sobre **a**
$\delta^*(\delta(q_1, a), \varepsilon) =$	processa ***a***
$\delta^*(q_f, \varepsilon) = q_f$	função estendida sobre ε: fim da indução

e, portanto, a palavra é aceita. ❑

A função programa estendida permite definir a *linguagem aceita*, bem como a *linguagem rejeitada* por um autômato finito.

definição 3.5 – Linguagem aceita, linguagem rejeitada

Seja $\mathbf{M = (\Sigma, Q, \delta, q_0, F)}$ um autômato finito determinístico. A *linguagem aceita* ou *linguagem reconhecida* por **M**, denotada por:

ACEITA(M) ou **L(M)**

é o conjunto de todas as palavras pertencentes a Σ^* aceitas por **M**, a partir do estado inicial q_0, ou seja:

$$L(M) = ACEITA(M) = \{ w \mid \delta^*(q_0, w) \in F \}$$

Analogamente, a *linguagem rejeitada* por **M**, denotada por:

$$REJEITA(M)$$

é o conjunto de todas as palavras pertencentes a Σ^* rejeitadas por **M**, a partir do estado inicial q_0, ou seja:

$$REJEITA(M) = \{ w \mid \delta^*(q_0, w) \notin F \text{ ou } \delta^*(q_0, w) \text{ é indefinida } \}$$ ❑

Supondo que Σ^* é o conjunto universo, as seguintes afirmações são verdadeiras (por quê?):

$ACEITA(M) \cap REJEITA(M) = \varnothing$
$ACEITA(M) \cup REJEITA(M) = \Sigma^*$
$\sim ACEITA(M) = REJEITA(M)$
$\sim REJEITA(M) = ACEITA(M)$

Portanto, cada autômato finito **M** definido sobre o alfabeto Σ induz uma partição do conjunto de todas as palavras Σ^* em duas classes de equivalência: **ACEITA(M)** e **REJEITA(M)**, conforme ilustrado na figura 3.9. Sendo preciso, no caso de um dos dois conjuntos ser vazio, então a partição induzida contém apenas um conjunto, o qual coincide com Σ^* (lembre-se de que uma classe de equivalência não pode ser vazia).

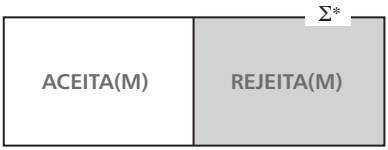


figura 3.9 Partição de Σ^*, induzida por um autômato finito **M**.

Diferentes autômatos finitos podem aceitar uma mesma linguagem. Tal fato induz a seguinte definição:

definição 3.6 – Autômatos finitos equivalentes

Dois autômatos finitos M_1 e M_2 são ditos *autômatos finitos equivalentes* se e somente se:

$$ACEITA(M_1) = ACEITA(M_2)$$ ❑

definição 3.7 – Linguagem regular, linguagem tipo 3

Uma linguagem **L** é dita uma *linguagem regular* ou *linguagem tipo 3* se existe pelo menos um autômato finito determinístico que aceita **L**. ❑

Portanto, a linguagem L_1 (exemplo 3.1 – Autômato finito: **aa** ou **bb** como subpalavra) é regular.

exemplo 3.3 – Autômato finito: vazia, todas as palavras

Considere as seguintes linguagens sobre o alfabeto **{ a, b }**:

$$L_2 = \varnothing \quad \text{e} \quad L_3 = \Sigma^*$$

Os autômatos finitos:

$$M_2 = (\{ a, b \}, \{ q_0 \}, \delta_2, q_0, \{ \ \}) \quad \text{e} \quad M_3 = (\{ a, b \}, \{ q_0 \}, \delta_3, q_0, \{ q_0 \})$$

ilustrados na figura 3.11, para os quais δ_2 e δ_3 são dadas pelas tabelas na figura 3.10, são tais que **ACEITA(M_2) = L_2** e **ACEITA(M_3) = L_3**. Portanto, as linguagens L_2 e L_3 são regulares.

Relativamente aos autômatos M_2 e M_3, sugere-se, como exercício, o esclarecimento das seguintes questões:

a Existe alguma diferença entre as funções δ_2 e δ_3?

b O que, exatamente, diferencia M_2 de M_3? ❑

δ_2	a	b
q_0	q_0	q_0

δ_3	a	b
q_0	q_0	q_0

figura 3.10 Função programa (**AFD**): vazia (esquerda) e todas as palavras (direita).

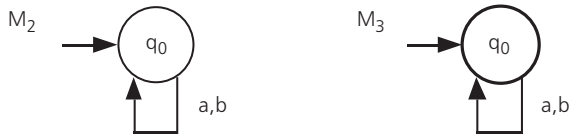


figura 3.11 Diagrama (**AFD**): vazia (esquerda) e todas as palavras (direita).

exemplo 3.4 – Autômato finito: número par de cada símbolo

Considere a seguinte linguagem sobre o alfabeto **{ a, b }**:

$$L_4 = \{ w \mid w \text{ possui um número par de } a \text{ e um número par de } b \}$$

O autômato finito:

$$M_4 = (\{ a, b \}, \{ q_0, q_1, q_2, q_3 \}, \delta_4, q_0, \{ q_0 \})$$

ilustrado na figura 3.12, é tal que **ACEITA(M_4) = L_4**. Portanto, L_4 é uma linguagem regular. Como seria um autômato para aceitar um número ímpar de cada símbolo? ❑

observação 3.8 – Função programa × função programa estendida

Objetivando simplificar a notação, uma função programa δ e a sua correspondente extensão δ^* podem ser ambas denotadas por δ. Esta simplificação de notação também é adotada para todas as funções estendidas definidas adiante. ❑

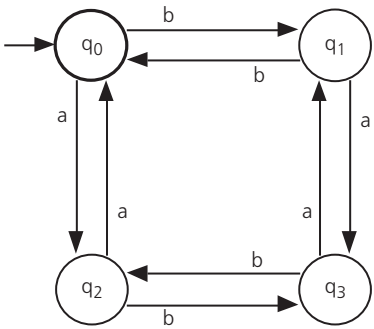


figura 3.12 Diagrama (**AFD**): número par de cada símbolo.

observação 3.9 – Computações × caminhos de um grafo

Relacionando teoria dos autômatos com teoria dos grafos, existe uma forte relação entre as computações de um autômato finito e os *caminhos* do correspondente grafo finito direto. De fato, dado um autômato, o enriquecimento do correspondente grafo com todos os caminhos (incluindo os de tamanho **zero**) é tal que:

- o conjunto de todos arcos (caminhos) corresponde às computações possíveis do autômato finito;
- o subconjunto de todos arcos com origem no estado inicial e destino em algum estado final corresponde à linguagem aceita;
- o subconjunto de todos arcos com origem no estado inicial e destino em algum estado não final corresponde à linguagem rejeitada.

Como ilustração, considere a figura 3.13, na qual:

a O autômato finito **M**, definido sobre o alfabeto **{ a, b, c, d }** é ilustrado à esquerda;

b O autômato **M**, enriquecido com seus caminhos (ou seja, suas computações), é ilustrado no centro. Observe que:

- em cada estado, a inclusão de um *endotransição* (transição com mesmo nodo origem e destino), etiquetado por ε, corresponde a ficar parado no mesmo estado, quando a entrada for vazia (um caminho de tamanho **zero**). Tais endotransições são interpretados como *faz nada* ou, em inglês, *no operation* (usualmente abreviado por *nop*, uma instrução existente nos sistemas computadores reais);
- cada caminho incluído é etiquetado, concatenando-se as etiquetas dos arcos componentes (lembre-se de que a palavra vazia ε é elemento neutro da concatenação). Observe que, se o autômato original possuir algum ciclo, existirão infinitos caminhos (por quê?), mas todos etiquetados por palavras (ou seja, cadeias de símbolos de tamanho finito);

c À direita, são representados:

- o conjunto de todas as computações possíveis (todos os arcos);
- a linguagem aceita (subconjunto de arcos com origem no estado inicial e destino em algum estado final). ❑

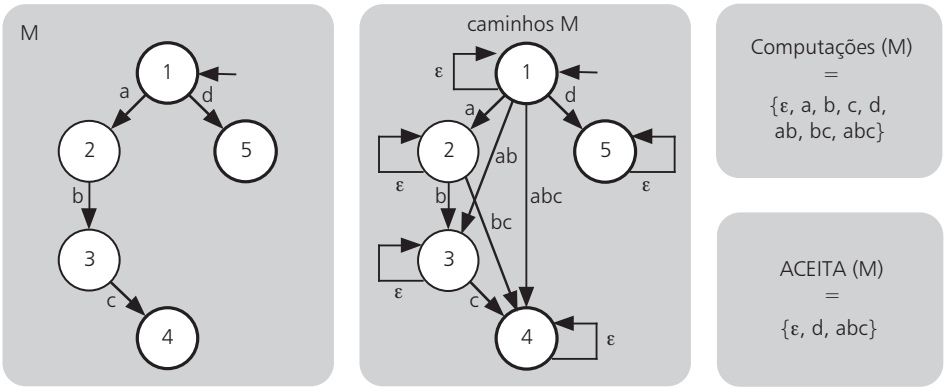
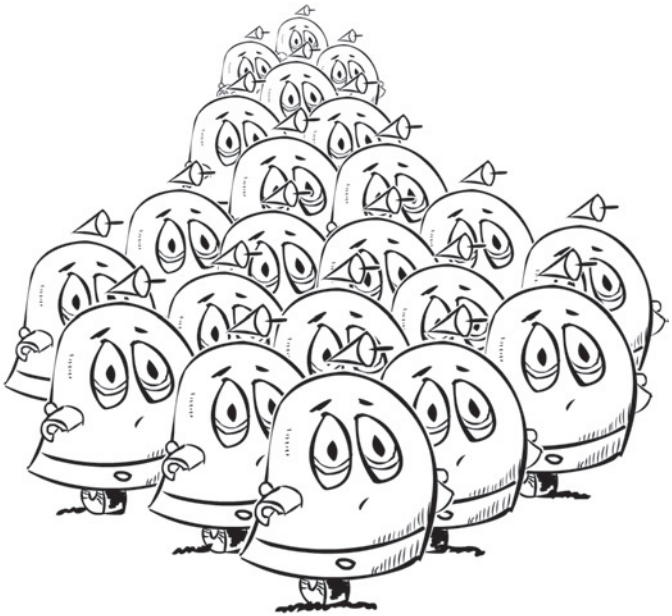


figura 3.13 Função programa estendida × caminhos de um grafo.



3.4 → autômato finito não determinístico

O *não determinismo* é uma importante generalização dos modelos de máquinas, sendo de fundamental importância no estudo dos *modelos para concorrência*, da *teoria da computação* e das *linguagens formais*, entre outros.

Já foi destacado que a semântica de não determinismo adotada é a usual no estudo das linguagens formais no sentido em que objetiva determinar a capacidade de reconhecer linguagens e de solucionar problemas, podendo causar alguma confusão com a semântica da concorrência adotada no estudo dos modelos para concorrência.

No contexto das linguagens formais, nem sempre a facilidade de não determinismo aumenta o poder de reconhecimento de linguagens de uma classe de autômatos. Por exemplo, conforme é mostrado adiante, qualquer autômato finito não determinístico pode ser simulado por um autômato finito determinístico.

A facilidade de *não determinismo* para autômatos finitos é expressa no programa, que é uma função tal que:

> *para o estado corrente e o símbolo lido da entrada, determina aleatoriamente um estado de um conjunto de estados alternativos.*

Portanto, a cada transição não determinista, novos caminhos alternativos são possíveis, definindo uma árvore de opções. Uma entrada é aceita se pelo menos um dos caminhos alternativos aceita a entrada (mesmo que os demais não aceitem). Assim, para um *autômato finito não determinístico* visto como uma máquina composta por fita, unidade de controle e programa, define-se uma semântica que assume um conjunto de estados alternativos, como se houvesse uma multiplicação da unidade de controle, uma para cada alternativa, processando independentemente, sem compartilhar recursos com as demais. Desta forma, o processamento de um caminho não influi no estado, símbolo lido e posição da cabeça dos demais caminhos alternativos e todos os caminhos alternativos são investigados simultaneamente.

definição 3.10 – Autômato finito não determinístico

Um *autômato finito não determinístico* (abreviado por **AFN**) **M** é uma **5**-upla ordenada:

$$M = (\Sigma, Q, \delta, q_0, F)$$

onde:

a Σ é um *alfabeto de símbolos de entrada*, ou simplesmente *alfabeto de entrada*;

b **Q** é um *conjunto de estados* possíveis do autômato o qual é finito;

c δ é uma *função programa* ou simplesmente *programa*, ou ainda *função de transição*:

$$\delta: Q \times \Sigma \rightarrow 2^Q$$

a qual é uma função total. Assim, para um estado **p** e um símbolo **a**:

$$\delta(p, a) = \{ q_1, q_2, \ldots, q_n \}$$

é uma *transição* do autômato;

d q_0 é um elemento distinguido de **Q**, denominado *estado inicial*;

e **F** é um subconjunto de **Q**, denominado *conjunto de estados finais*. ❑

Portanto, excetuando-se pela função programa δ, as componentes Σ, **Q**, $\mathbf{q_0}$ e **F** são como na definição do autômato finito determinístico.

Se para um estado **p** e um símbolo **a** ocorre que $\delta(\mathbf{p}, \mathbf{a}) = \varnothing$, então afirma-se que a transição é indefinida para o par **(p, a)** e portanto, o autômato para rejeitando a entrada. A representação da função programa como um diagrama é análoga a do autômato finito determinístico. Assim, a representação diagramática de uma transição do tipo:

$$\delta(\mathbf{p}, \mathbf{a}) = \{ \mathbf{q_1}, \mathbf{q_2}, \ldots, \mathbf{q_n} \}$$

resulta em diversas arestas etiquetadas por **a**, com origem em **p**, e com destino em cada estado $\mathbf{q_1}$, $\mathbf{q_2}$,..., $\mathbf{q_n}$, mostrando os diversos caminhos alternativos, como ilustrado na figura 3.14.

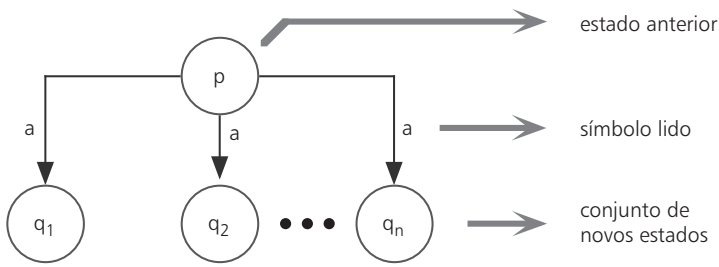


figura 3.14 Diagrama (**AFN**): transição.

Analogamente aos autômatos finitos determinísticos, a *computação* de um *autômato finito não determinístico*, para uma palavra de entrada **w**, consiste na sucessiva aplicação da função programa para cada símbolo de **w** (da esquerda para a direita) até ocorrer uma condição de parada. Como cada transição do autômato não determinístico resulta em um conjunto de estados, é necessário estender a definição da função programa, usando como argumento um conjunto finito de estados e uma palavra.

definição 3.11 – Função programa estendida, computação

Seja $\mathbf{M} = (\Sigma, \mathbf{Q}, \delta, \mathbf{q_0}, \mathbf{F})$ um autômato finito não determinístico. A *função programa estendida* ou *computação* de **M**, denotada por:

$$\delta^*: 2^{\mathbf{Q}} \times \Sigma^* \rightarrow 2^{\mathbf{Q}}$$

é a função programa $\delta: \mathbf{Q} \times \Sigma \rightarrow 2^{\mathbf{Q}}$ estendida para palavras e conjuntos de estados, indutivamente definida como segue:

$$\delta^*(\mathbf{P}, \varepsilon) = \mathbf{P}$$
$$\delta^*(\mathbf{P}, \mathbf{aw}) = \delta^*(\cup_{\mathbf{q} \in \mathbf{P}} \delta(\mathbf{q}, \mathbf{a}), \mathbf{w})$$ ❑

Portanto, como já comentado, a função programa estendida consiste na sucessiva aplicação da função programa a cada símbolo da palavra, a partir de um conjunto de estados. Observe que, se a entrada for vazia, o autômato fica parado nos estados correntes.

A *transição estendida* a um conjunto de estados é a união dos resultados da função programa aplicada a cada estado alternativo. Assim, para um conjunto de estados $\{ q_1, q_2, \ldots, q_n \}$ e para um símbolo **a**, vale que:

$$\delta^*(\{ q_1, q_2, \ldots, q_n \}, a) = \delta(q_1, a) \cup \delta(q_2, a) \cup \ldots \cup \delta(q_n, a)$$

A parada do processamento de um autômato finito não determinístico para uma entrada **w** pode ser de duas maneiras:

a *Aceita* a entrada **w**. Após processar o último símbolo da fita, *existe* pelo menos um *estado final* pertencente ao conjunto de estados alternativos atingidos;

b *Rejeita* a entrada **w**. São duas possibilidades:

- após processar o último símbolo da fita, *todos* os estados alternativos atingidos são *não finais*;
- em algum momento, ao longo do processamento de **w**, o conjunto de estados alternativos atingido é vazio. Neste caso, o autômato para por indefinição.

definição 3.12 – Linguagem aceita, linguagem rejeitada

Seja $M = (\Sigma, Q, \delta, q_0, F)$ um autômato finito não determinístico. A *linguagem aceita* ou *linguagem reconhecida* por **M**, denotada por:

$$\text{ACEITA}(M) \quad \text{ou} \quad L(M)$$

é o conjunto de todas as palavras pertencentes a Σ^* tais que existe pelo menos um caminho alternativo que aceita a palavra, a partir de $\{ q_0 \}$, ou seja:

$$\text{ACEITA}(M) = \{ w \mid \delta^*(\{ q_0 \}, w) \cap F \neq \varnothing \}$$

Analogamente, a *linguagem rejeitada* por **M**, denotada por:

$$\text{REJEITA}(M)$$

é o conjunto de todas as palavras pertencentes a Σ^* rejeitadas por todos os caminhos alternativos de **M**, a partir de $\{ q_0 \}$, ou seja:

$$\text{REJEITA}(M) = \{ w \mid \delta^*(\{ q_0 \}, w) \cap F = \varnothing \}$$ ❑

exemplo 3.5 – Autômato finito não determinístico: ***aa*** ou ***bb*** como subpalavra

Considere a seguinte linguagem sobre o alfabeto $\{ a, b \}$:

$$L_5 = \{ w \mid w \text{ possui } aa \text{ ou } bb \text{ como subpalavra} \}$$

O autômato finito não determinístico:

$$M_5 = (\{ a, b \}, \{ q_0, q_1, q_2, q_f \}, \delta_5, q_0, \{ q_f \})$$

onde δ_5 é dada pela tabela na figura 3.15, é tal que **ACEITA(M_5) = L_5** (compare com o exemplo 3.1 – Autômato finito: **aa** ou **bb** como subpalavra).

O autômato M_5 pode ser representado pelo diagrama ilustrado na figura 3.16. O algoritmo apresentado realiza uma varredura sobre a palavra de entrada. A cada ocorrência de **a** (respectivamente, de **b**) uma alternativa é iniciada, para verificar se o símbolo seguinte também é **a** (respectivamente, **b**). Assim, existem três alternativas de processamento:

- o ciclo em q_0 realiza uma varredura em toda a entrada;
- o caminho $q_0/q_1/q_f$ garante a ocorrência de **aa**;
- o caminho $q_0/q_2/q_f$ garante a ocorrência de **bb**.

A computação da palavra **abaa** a partir do conjunto de estados $\{ q_0 \}$ é como segue:

$\delta^*(\{ q_0 \}, abaa) =$	função estendida sobre **abaa**
$\delta^*(\delta(q_0, a), baa) =$	processa ***a**baa*
$\delta^*(\{ q_0, q_1 \}, baa) =$	função estendida sobre **baa**
$\delta^*(\delta(q_0, b) \cup \delta(q_1, b), aa) =$	processa ***b**aa*
$\delta^*(\{ q_0, q_2 \} \cup \varnothing, aa) =$	
$\delta^*(\{ q_0, q_2 \}, aa) =$	função estendida sobre **aa**
$\delta^*(\delta(q_0, a) \cup \delta(q_2, a), a) =$	processa ***aa***
$\delta^*(\{ q_0, q_1 \} \cup \varnothing, a) =$	
$\delta^*(\{ q_0, q_1 \}, a) =$	função estendida sobre **a**
$\delta^*(\delta(q_0, a) \cup \delta(q_1, a), \varepsilon) =$	processa ***a***
$\delta^*(\{ q_0, q_1 \} \cup \{ q_f \}, \varepsilon) =$	
$\delta^*(\{ q_0, q_1, q_f \}, \varepsilon) = \{ q_0, q_1, q_f \}$	função estendida sobre ε: fim da indução

e, portanto, a palavra é aceita pois $\{ q_0, q_1, q_f \} \cap F = \{ q_f \} \neq \varnothing$. ❑

δ_5	a	b
q_0	$\{q_0, q_1\}$	$\{q_0, q_2\}$
q_1	$\{q_f\}$	$\varnothing$
q_2	$\varnothing$	$\{q_f\}$
q_f	$\{q_f\}$	$\{q_f\}$

figura 3.15 Função programa (**AFN**): sequência de dois símbolos iguais.

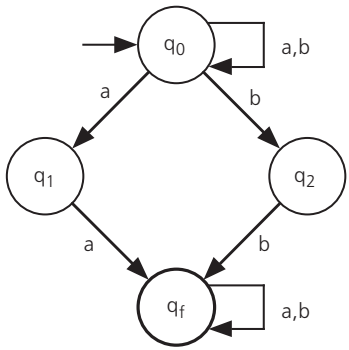


figura 3.16 Diagrama (**AFN**): sequência de dois símbolos iguais.

exemplo 3.6 – Autômato finito não determinístico: ***aaa*** como sufixo

Considere a seguinte linguagem sobre o alfabeto **{ a, b }**:

$$L_6 = \{ w \mid w \text{ possui } aaa \text{ como sufixo} \}$$

O autômato finito não determinístico:

$$M_6 = (\{ a, b \}, \{ q_0, q_1, q_2, q_f \}, \delta_6, q_0, \{ q_f \})$$

ilustrado na figura 3.17, é tal que $ACEITA(M_6) = L_6$.

A computação da palavra **baa** a partir do conjunto de estados $\{ q_0 \}$ é como segue:

$\delta^*(\{ q_0 \}, baa) =$	função estendida sobre **baa**
$\delta^*(\delta(q_0, b), aa) =$	processa ***b*aa**
$\delta^*(\{ q_0 \}, aa) =$	função estendida sobre **aa**
$\delta^*(\delta(q_0, a), a) =$	processa ***aa***
$\delta^*(\{ q_0, q_1 \}, a) =$	função estendida sobre **a**
$\delta^*(\delta(q_0, a) \cup \delta(q_1, a), \varepsilon) =$	processa ***a***
$\delta^*(\{ q_0, q_1 \} \cup \{ q_2 \}, \varepsilon) =$	
$\delta^*(\{ q_0, q_1, q_2 \}, \varepsilon) = \{ q_0, q_1, q_2 \}$	função estendida sobre ε: fim da indução

e, portanto, a palavra é rejeitada pois $\{ q_0, q_1, q_2 \} \cap F = \varnothing$. ❑

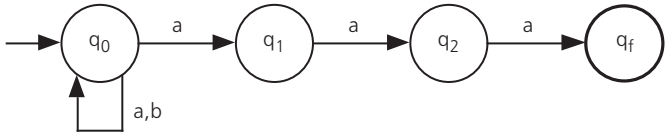


figura 3.17 Diagrama (**AFN**): **aaa** como sufixo.

Embora a facilidade de não determinismo seja, aparentemente, um significativo acréscimo ao autômato finito, na realidade não aumenta seu poder computacional. Assim, para cada **AFN**,

é possível construir um **AFD** equivalente que realiza as mesmas computações. O contrário também é verdadeiro.

teorema 3.13 – Equivalência entre **AFD** e **AFN**

A classe dos autômatos finitos determinísticos é equivalente à classe dos autômatos finitos não determinísticos.

➥ **Prova: (*por indução*)**

A prova consiste em mostrar que, a partir de um **AFN M** qualquer, é possível construir um **AFD** M_D que realiza as mesmas computações (ou seja, M_D simula **M**). A ideia central do algoritmo é a construção de estados de M_D que simulem as diversas combinações de estados alternativos de **M**. O contrário (construir um não determinístico a partir de um determinístico) não necessita ser mostrado, pois decorre trivialmente das definições (por quê?).

Seja $M = (\Sigma, Q, \delta, q_0, F)$ um **AFN** qualquer. Seja:

$$M_D = (\Sigma, Q_D, \delta_D, \langle q_0 \rangle, F_D)$$

um **AFD** construído a partir de **M** como segue:

a Q_D é o conjunto construído a partir de todas as combinações, sem repetições, de estados de **Q**. Cada estado de Q_D é denotado por:

$$\langle q_1 q_2 \ldots q_n \rangle$$

onde q_i pertence a **Q**, para **i** em $\{ 1, 2, \ldots, n \}$. Portanto, um estado de M_D representa uma imagem de todos os estados alternativos de **M**.

Importante: observe que a ordem dos elementos não distingue mais combinações. Por exemplo, $\langle q_u q_v \rangle = \langle q_v q_u \rangle$;

b δ_D: $Q_D \times \Sigma \rightarrow Q_D$ é tal que:

$$\delta_D(\langle q_1 \ldots q_n \rangle, a) = \langle p_1 \ldots p_m \rangle \quad \text{se e somente se} \quad \delta^*(\{ q_1, \ldots, q_n \}, a) = \{ p_1, \ldots, p_m \}$$

em particular:

$$\delta_D(\langle q_1 \ldots q_n \rangle, a) \text{ é indefinida} \quad \text{se e somente se} \quad \delta^*(\{ q_1, \ldots, q_n \}, a) = \varnothing$$

c $\langle q_0 \rangle$ é o estado inicial;

d F_D é o conjunto de todos os estados $\langle q_1 q_2 \ldots q_n \rangle$ pertencentes a Q_D tal que alguma componente q_i pertence a **F**, para **i** em $\{ 1, 2, \ldots, n \}$.

A demonstração de que o **AFD** M_D de fato simula exatamente as computações do **AFN M** é por indução no tamanho da palavra. Deve-se mostrar que (suponha **w** uma palavra qualquer de Σ^*):

$$\delta_D^*(\langle q_0 \rangle, w) = \langle q_1 \ldots q_u \rangle \quad \text{se e somente se} \quad \delta^*(\{ q_0 \}, w) = \{ q_1, \ldots, q_u \}$$

a *Base de indução*. Seja **w** tal que $|\, w \,| = 0$. Portanto $w = \varepsilon$:

$$\delta_D^*(\langle q_0 \rangle, \varepsilon) = \langle q_0 \rangle \quad \text{se e somente se} \quad \delta^*(\{ q_0 \}, \varepsilon) = \{ q_0 \}$$

o que é verdadeiro, por definição de função programa estendida;

b *Hipótese de indução*. Seja **w** tal que $|\ \mathbf{w}\ | = \mathbf{n}$ e $\mathbf{n} \geq \mathbf{1}$. Suponha verdadeiro que:

$$\delta_D^*(\langle q_0\rangle, w) = \langle q_1 \ldots q_u\rangle \quad \text{se e somente se} \quad \delta^*(\{\ q_0\ \}, w) = \{\ q_1, \ldots, q_u\ \}$$

c *Passo de indução*. Seja **w** tal que $|\ \mathbf{wa}\ | = \mathbf{n + 1}$ e $\mathbf{n} \geq \mathbf{1}$. Então:

$$\delta_D^*(\langle q_0\rangle, wa) = \langle p_1 \ldots p_v\rangle \quad \text{se e somente se} \quad \delta^*(\{\ q_0\ \}, wa) = \{\ p_1, \ldots, p_v\ \}$$

o que equivale, por hipótese de indução:

$$\delta_D(\langle q_1 \ldots q_u\rangle, a) = \langle p_1 \ldots p_v\rangle \quad \text{se e somente se} \quad \delta^*(\{\ q_1, \ldots, q_u\ \}, a) = \{\ p_1, \ldots, p_v\ \}$$

o que é verdadeiro, por definição de δ_D.

Logo, $\mathbf{M_D}$ simula **M** para qualquer entrada **w** pertencente a Σ^*. ❑

Portanto, uma linguagem aceita por um autômato finito não determinístico é uma *linguagem regular* ou *linguagem tipo 3*.

observação 3.14 – Determinismo × não determinismo

Muitas vezes é mais fácil desenvolver um autômato finito não determinístico do que um determinístico. Um exemplo interessante é desenvolver um autômato que aceita a seguinte linguagem sobre $\Sigma = \{\ a, b\ \}$, o qual é sugerido como exercício:

$$\{\ w\ |\ \text{o quinto símbolo da direita para a esquerda de } w \text{ é } a\ \}$$

A solução determinista não é trivial e resulta em um número relativamente grande de estados. Entretanto, uma solução não determinista é bem simples e necessita de poucos estados. Assim, em muitos casos, para construir um autômato finito determinístico, é preferível desenvolver inicialmente uma solução não determinista e, sobre esta solução, aplicar o algoritmo apresentado na prova do teorema 3.13 – Equivalência entre **AFD** e **AFN**. ❑

exemplo 3.7 – Construção de um ***AFD*** a partir de um ***AFN***

Considere o **AFN** (apresentado no exemplo 3.6 – Autômato finito não determinístico: **aaa** como sufixo):

$$M_6 = (\{\ a, b\ \}, \{\ q_0, q_1, q_2, q_f\ \}, \delta_6, q_0, \{\ q_f\ \})$$

O correspondente **AFD**:

$$M_{6D} = (\{\ a, b\ \}, Q_D, \delta_{6D}, \langle q_0\rangle, F_D)$$

construído conforme a prova do teorema 3.13 – Equivalência entre **AFD** e **AFN**, é tal que:

$Q_D = \{\ \langle q_0\rangle, \langle q_1\rangle, \langle q_2\rangle, \langle q_f\rangle, \langle q_0q_1\rangle, \langle q_0q_2\rangle, \ldots, \langle q_0q_1q_2q_f\rangle\ \}$
$F_D = \{\ \langle q_f\rangle, \langle q_0q_f\rangle, \langle q_1q_f\rangle, \ldots, \langle q_0q_1q_2q_f\rangle\ \}$
δ_{6D} é dada pela tabela na figura 3.18, na qual são explicitados apenas os estados para os quais a função programa é definida

O diagrama que representa $\mathbf{M_{6D}}$ é ilustrado na figura 3.19 em que, por simplicidade, os estados $\langle q_0\rangle$, $\langle q_0q_1\rangle$, $\langle q_0q_1q_2\rangle$ e $\langle q_0q_1q_2q_f\rangle$ foram renomeados para $\mathbf{p_0}$, $\mathbf{p_1}$, $\mathbf{p_2}$ e $\mathbf{p_f}$, respectivamente. ❑

δ_{δ_D}	a	b
$\langle q_0 \rangle$	$\langle q_0 q_1 \rangle$	$\langle q_0 \rangle$
$\langle q_0 q_1 \rangle$	$\langle q_0 q_1 q_2 \rangle$	$\langle q_0 \rangle$
$\langle q_0 q_1 q_2 \rangle$	$\langle q_0 q_1 q_2 q_f \rangle$	$\langle q_0 \rangle$
$\langle q_0 q_1 q_2 q_f \rangle$	$\langle q_0 q_1 q_2 q_f \rangle$	$\langle q_0 \rangle$

figura 3.18 Função programa: **AFD** a partir de um **AFN**.

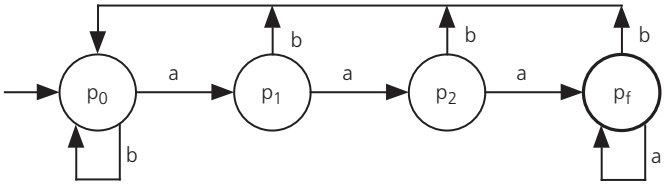


figura 3.19 Diagrama: **AFD** construído a partir do **AFN**.

3.5 → autômato finito com movimentos vazios

Movimentos vazios generalizam os movimentos não determinísticos. Um *movimento vazio* é uma transição sem a leitura de símbolo da fita. Pode ser interpretado como um *não determinismo interno* ao autômato, constituindo uma *transição encapsulada*, ou seja, excetuando-se por uma eventual mudança de estados, nada mais pode ser observado sobre um movimento vazio. Uma das vantagens dos autômatos finitos com movimentos vazios no estudo das linguagens formais é o fato de facilitarem algumas construções e demonstrações relacionadas com os autômatos.

No caso dos autômatos finitos, a facilidade de movimentos vazios não aumenta o poder de reconhecimento de linguagens. Conforme é mostrado adiante, qualquer autômato finito com movimentos vazios pode ser simulado por um autômato finito não determinístico.

definição 3.15 – Autômato finito com movimentos vazios

Um *autômato finito não determinístico e com movimentos vazios* (abreviado por **AFN**ε) ou simplesmente *autômato finito com movimentos vazios* **M** é uma **5**-upla ordenada:

$$M = (\Sigma, Q, \delta, q_0, F)$$

onde:

a Σ é um *alfabeto de símbolos de entrada*, ou simplesmente *alfabeto de entrada*;

b **Q** é um *conjunto de estados* possíveis do autômato o qual é finito;

c δ é uma *função programa* ou simplesmente *programa*, ou ainda *função de transição*:

$$\delta: Q \times (\Sigma \cup \{ \varepsilon \}) \rightarrow 2^Q$$

a qual é uma função total. Assim, para um estado **p** e o símbolo especial ε:

$$\delta(p, \varepsilon) = \{ q_1, q_2, \ldots, q_n \}$$

é um *movimento vazio* ou *transição vazia* do autômato;

d q_0 é um elemento distinguido de **Q**, denominado *estado inicial*;

e **F** é um subconjunto de **Q**, denominado *conjunto de estados finais*. ❑

Portanto, excetuando-se pela função programa δ, as componentes Σ, **Q**, **F** e q_0 são como na definição de **AFN**.

A representação da função programa como um diagrama é análoga à do autômato finito não determinístico. Como exemplo, supondo que:

$$\delta(q, \varepsilon) = \{ p_0 \} \quad \delta(q, a_1) = \{ p_1 \} \quad \ldots \quad \delta(q, a_n) = \{ p_n \}$$

o correspondente diagrama é ilustrado na figura 3.20.

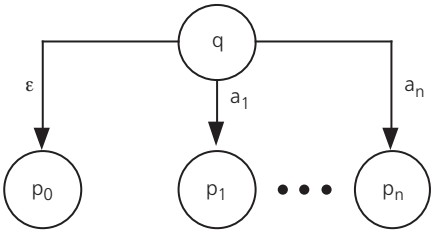


figura 3.20 Diagrama (**AFNε**): função programa com movimentos vazios.

A *computação* de um *autômato finito com movimentos vazios* é análoga à de um autômato finito não determinístico. Adicionalmente, o processamento de uma transição para uma entrada vazia também é não determinístico. Assim, um **AFNε**, ao processar uma entrada vazia, assume simultaneamente os estados destino e origem. Portanto, a origem de um movimento vazio sempre é um caminho alternativo.

exemplo 3.8 – Autômato finito com movimentos vazios: ***a***'s antecedem ***b***'s

Considere a seguinte linguagem sobre o alfabeto **{ a, b }**:

$$L_7 = \{ w \mid \text{qualquer símbolo } a \text{ antecede qualquer símbolo } b \}$$

O autômato finito com movimentos vazios:

$$M_7 = (\{ a, b \}, \{ q_0, q_f \}, \delta_7, q_0, \{ q_f \})$$

ilustrado na figura 3.22, onde δ_7 é dada pela tabela na figura 3.21, é tal que **ACEITA**(M_7) = L_7 ❑

δ_7	a	b	ε
q_0	$\{q_0\}$	∅	$\{q_f\}$
q_f	∅	$\{q_f\}$	∅

figura 3.21 Função programa (**AFNε**): qualquer símbolo **a** antecede qualquer símbolo **b**.

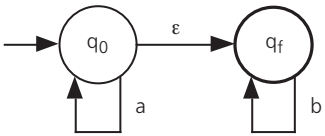


figura 3.22 Diagrama (**AFNε**): qualquer símbolo **a** antecede qualquer símbolo **b**.

A definição formal das computações de um autômato finito com movimentos vazios, é facilitada se primeiro for definida a computação exclusivamente de transições vazias (a partir de um estado ou de um conjunto finito de estados).

definição 3.16 – Computação vazia

Seja $M = (\Sigma, Q, \delta, q_0, F)$ um autômato finito com movimentos vazios.

a A *computação vazia* ou *função fecho vazio*, a partir de um estado, denotada por:

$$\delta\varepsilon: Q \rightarrow 2^Q$$

é definida como segue:

$$\delta\varepsilon(q) = \{ q \} \cup \delta(q, \varepsilon) \cup (\cup_{p \in \delta(q,\varepsilon)} \delta\varepsilon(p))$$

(e portanto, $\delta\varepsilon(q) = \{ q \}$, se $\delta(q, \varepsilon) = \varnothing$, ou seja, é indefinida);

b A *computação vazia* ou *função fecho vazio*, a partir de um conjunto de estados finito, denotada por:

$$\delta\varepsilon^*: 2^Q \rightarrow 2^Q$$

é tal que:

$$\delta\varepsilon^*(P) = \cup_{q \in P} \delta\varepsilon(q)$$ ❑

Como sempre, por simplicidade, as funções $\delta\varepsilon$ e $\delta\varepsilon^*$ são ambas denotadas por $\delta\varepsilon$.

exemplo 3.9 – Computação vazia

Considere o autômato finito com movimentos vazios M_7 (exemplo 3.8 – Autômato finito com movimentos vazios: **a**'s antecedem **b**'s). Então:

$\delta\varepsilon(q_0) = \{ q_0, q_f \}$
$\delta\varepsilon(q_f) = \{ q_f \}$
$\delta\varepsilon(\{ q_0, q_f \}) = \{ q_0, q_f \}$ ❑

A *computação* de um *autômato finito com movimentos vazios*, para uma palavra de entrada **w**, consiste na sucessiva aplicação da função programa para cada símbolo de **w** (da esquerda

para a direita), cada passo de aplicação intercalado com computações vazias, até ocorrer uma condição de parada. Assim, para cada conjunto de estados alternativos assumido pelo autômato, antes de processar a próxima transição, é necessário determinar todos os demais estados atingíveis exclusivamente por movimentos vazios.

definição 3.17 – Função programa estendida, computação

Seja $\mathbf{M = (\Sigma, Q, \delta, q_0, F)}$ um autômato finito com movimentos vazios. A *função programa estendida* ou *computação* de **M**, denotada por:

$$\delta^*: 2^Q \times \Sigma^* \rightarrow 2^Q$$

é a função programa $\delta: Q \times (\Sigma \cup \{ \varepsilon \}) \rightarrow 2^Q$ estendida para um conjunto finito de estados e para uma palavra e é indutivamente definida como segue:

$\delta^*(P, \varepsilon) = \delta\varepsilon(P)$
$\delta^*(P, wa) = \delta\varepsilon(R)$ onde $R = \{ r \mid r \in \delta(s, a) \text{ e } s \in \delta^*(P, w) \}$ ❑

A parada do processamento de um autômato finito com movimentos vazios é análoga à do autômato finito não determinístico, assim como as definições de linguagem aceita e linguagem rejeitada.

definição 3.18 – Linguagem aceita, linguagem rejeitada

Seja $\mathbf{M = (\Sigma, Q, \delta, q_0, F)}$ um autômato finito com movimentos vazios. A *linguagem aceita* ou *linguagem reconhecida* por **M**, denotada por:

$$\mathbf{ACEITA(M)} \quad \text{ou} \quad \mathbf{L(M)}$$

é o conjunto de todas as palavras pertencentes a Σ^* tais que existe pelo menos um caminho alternativo que aceita a palavra, a partir de $\{ q_0 \}$, ou seja:

$$\mathbf{ACEITA(M)} = \{ w \mid \delta^*(\{ q_0 \}, w) \cap F \neq \varnothing \}$$

Analogamente, a *linguagem rejeitada* por **M**, denotada por:

$$\mathbf{REJEITA(M)}$$

é o conjunto de todas as palavras pertencentes a Σ^* rejeitadas por todos os caminhos alternativos de **M**, a partir de $\{ q_0 \}$, ou seja:

$$\mathbf{REJEITA(M)} = \{ w \mid \delta^*(\{ q_0 \}, w) \cap F = \varnothing \}$$ ❑

exemplo 3.10 – Computação vazia, computação

Considere a seguinte linguagem sobre o alfabeto $\{ a, b, c \}$:

$$L_8 = \{ w \mid w \text{ possui como sufixo } \mathbf{a} \text{ ou } \mathbf{bb} \text{ ou } \mathbf{ccc} \}$$

O autômato finito com movimentos vazios:

$$M_8 = (\{ a, b, c \}, \{ q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_f \}, \delta_8, q_0, \{ q_f \})$$

ilustrado na figura 3.23, é tal que:

$$\mathbf{ACEITA}(M_8) = L_8$$

Relativamente à computação vazia, vale que, por exemplo:

$$\delta\varepsilon(\{ q_0 \}) = \{ q_0, q_1, q_2, q_4 \}$$

A computação da entrada **aab** a partir do estado inicial q_0, é como segue:

$$\delta^*(\{ q_0 \}, abb) = \delta\varepsilon(\{ r \mid r \in \delta(s, b) \text{ e } s \in \delta^*(\{ q_0 \}, ab) \}) \quad (1)$$

sendo que:

$$\delta^*(\{ q_0 \}, ab) = \delta\varepsilon(\{ r \mid r \in \delta(s, b) \text{ e } s \in \delta^*(\{ q_0 \}, a) \}) \quad (2)$$

sendo que:

$$\delta^*(\{ q_0 \}, a) = \delta\varepsilon(\{ r \mid r \in \delta(s, a) \text{ e } s \in \delta^*(\{ q_0 \}, \varepsilon) \}) \quad (3)$$

Como:

$$\delta^*(\{ q_0 \}, \varepsilon) \} = \delta\varepsilon(\{ q_0 \}) = \{ q_0, q_1, q_2, q_4 \}$$

o qual, quando considerado em (3):

$$\delta^*(\{ q_0 \}, a) = \{ q_0, q_1, q_2, q_4, q_f \}$$

o qual, quando considerado em (2):

$$\delta^*(\{ q_0 \}, ab) = \{ q_0, q_1, q_2, q_3, q_4 \}$$

o qual, quando considerado em (1), resulta na computação desejada:

$$\delta^*(\{ q_0 \}, abb) = \{ q_0, q_1, q_2, q_3, q_4, q_f \}$$ ❑

Assim como o não determinismo, a facilidade de movimentos vazios não aumenta o poder computacional dos autômatos finitos. De fato, para cada **AFNε**, é possível construir um **AFN** equivalente que realiza o mesmo processamento. O contrário é trivialmente verdadeiro.

teorema 3.19 – Equivalência entre **AFN** e **AFNε**

A classe dos autômatos finitos com movimentos vazios é equivalente à classe dos autômatos finitos não determinísticos.

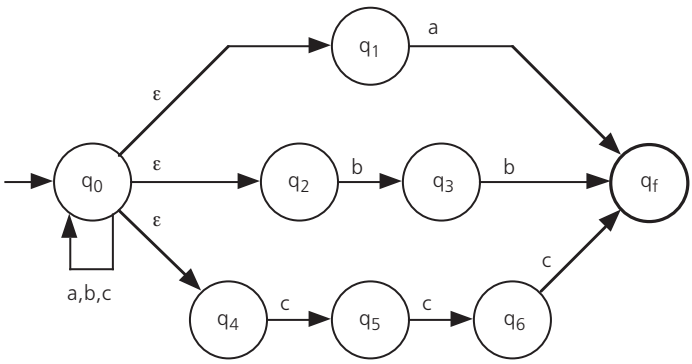


figura 3.23 Diagrama (**AFNε**): possui como sufixo **a** ou **bb** ou **ccc**.

➥ **Prova: (*por indução*)**
A prova consiste em mostrar que, a partir de um **AFNε M** qualquer, é possível construir um **AFN** M_N que realiza as mesmas computações (ou seja, M_N simula **M**). A ideia central do algoritmo é a construção de uma função programa sem movimentos vazios na qual o conjunto de estados destino de cada transição não vazia é ampliado com todos os demais estados possíveis de serem atingidos exclusivamente por transições vazias. O contrário (construir um **AFNε** a partir de um **AFN**) não necessita ser mostrado, pois decorre trivialmente das definições (por quê?).

Seja $M = (\Sigma, Q, \delta, q_0, F)$ um **AFNε** qualquer. Seja:

$$M_N = (\Sigma, Q, \delta_N, q_0, F_N)$$

um **AFN** construído a partir de **M** como segue:

a $\delta_N: Q \times \Sigma \rightarrow 2^Q$ é tal que:

$$\delta_N(q, a) = \delta^*(\{ q \}, a)$$

b F_N é o conjunto de todos os estados **q** pertencentes a **Q** tal que:

$$\delta\varepsilon(q) \cap F \neq \varnothing$$

ou seja, todos os estados que atingem estados finais via computações vazias.

A demonstração de que, de fato, o **AFN** M_N simula o **AFNε M** é feita por indução no tamanho da palavra e é sugerida como exercício. ❑

Portanto, uma linguagem aceita por um autômato finito com movimentos vazios é uma *linguagem regular* ou *linguagem tipo 3*.

exemplo 3.11 – Construção de um ***AFN*** a partir de um ***AFN*ε**
Considere o autômato finito com movimentos vazios:

$$M_9 = (\{ a, b \}, \{ q_0, q_1, q_2 \}, \delta_9, q_0, \{ q_2 \})$$

ilustrado na figura 3.25, sendo que δ_9 é dado pela tabela na figura 3.24 (qual a linguagem aceita por M_9?). O correspondente **AFN**:

$$M_{9N} = (\{ a, b \}, \{ q_0, q_1, q_2 \}, \delta_{9N}\ q_0, F_N)$$

construído conforme o algoritmo apresentado na prova do teorema 3.19 – Equivalência entre **AFN** e **AFNε**, é como segue e é ilustrado na figura 3.26:

$F_N = \{ q_0, q_1, q_2 \}$, pois:
$\delta\varepsilon(q_0) = \{ q_0, q_1, q_2 \}$
$\delta\varepsilon(q_1) = \{ q_1, q_2 \}$
$\delta\varepsilon(q_2) = \{ q_2 \}$
Na construção de δ_{9N}, note-se que:
$\delta_9^*(\{ q_0 \}, \varepsilon) = \{ q_0, q_1, q_2 \}$
$\delta_9^*(\{ q_1 \}, \varepsilon) = \{ q_1, q_2 \}$
$\delta_9^*(\{ q_2 \}, \varepsilon) = \{ q_2 \}$

Assim, δ_{9N} é tal que:

$\delta_{9N}(q_0, a) = \delta_9^*(\{ q_0 \}, a) = \delta\varepsilon(\{ r \mid r \in \delta(s, a) \text{ e } s \in \delta^*(\{ q_0 \}, \varepsilon) \}) = \{ q_0, q_1, q_2 \}$
$\delta_{9N}(q_0, b) = \delta_9^*(\{ q_0 \}, b) = \delta\varepsilon(\{ r \mid r \in \delta(s, b) \text{ e } s \in \delta^*(\{ q_0 \}, \varepsilon) \}) = \{ q_1, q_2 \}$
$\delta_{9N}(q_1, a) = \delta_9^*(\{ q_1 \}, a) = \delta\varepsilon(\{ r \mid r \in \delta(s, a) \text{ e } s \in \delta^*(\{ q_1 \}, \varepsilon) \}) = \{ q_2 \}$
$\delta_{9N}(q_1, b) = \delta_9^*(\{ q_1 \}, b) = \delta\varepsilon(\{ r \mid r \in \delta(s, b) \text{ e } s \in \delta^*(\{ q_1 \}, \varepsilon) \}) = \{ q_1, q_2 \}$
$\delta_{9N}(q_2, a) = \delta_9^*(\{ q_2 \}, a) = \delta\varepsilon(\{ r \mid r \in \delta(s, a) \text{ e } s \in \delta^*(\{ q_2 \}, \varepsilon) \}) = \{ q_2 \}$
$\delta_{9N}(q_2, b) = \delta_9^*(\{ q_2 \}, b) = \delta\varepsilon(\{ r \mid r \in \delta(s, b) \text{ e } s \in \delta^*(\{ q_2 \}, \varepsilon) \}) = \varnothing$ ❑

δ_9	a	b	ε
q_0	$\{q_0\}$	$\varnothing$	$\{q_1\}$
q_1	$\varnothing$	$\{q_1\}$	$\{q_2\}$
q_2	$\{q_2\}$	$\varnothing$	$\varnothing$

figura 3.24 Função programa (**AFNε**).

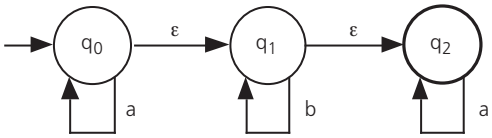


figura 3.25 Diagrama: autômato finito com movimentos vazios.

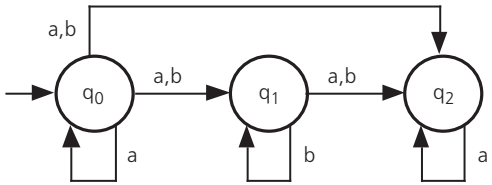


figura 3.26 Diagrama: autômato finito não determinístico equivalente.

3.6 → expressão regular

Toda linguagem regular pode ser descrita por uma expressão denominada *expressão regular*. Trata-se de um formalismo denotacional, também considerado gerador, pois se pode inferir como construir ("gerar") as palavras de uma linguagem. Uma expressão regular é definida a partir de conjuntos (linguagens) básicos e operações de concatenação e de união. As expressões regulares são consideradas adequadas para a comunicação humano × humano e, principalmente, para a comunicação humano × máquina.

definição 3.20 – Expressão regular, linguagem gerada

Uma *expressão regular* (abreviada por **ER**) sobre um alfabeto Σ é indutivamente definida como segue:

a *Base de indução*.

a.1) A expressão:

$$\varnothing$$

é expressão regular e denota a linguagem vazia:

$$\varnothing$$

a.2) A expressão:

$$\varepsilon$$

é uma expressão regular e denota a linguagem que contém exclusivamente a palavra vazia:

$$\{ \varepsilon \}$$

a.3) Para qualquer símbolo $\mathbf{x} \in \Sigma$, a expressão:

$$\mathbf{x}$$

é uma expressão regular e denota a linguagem que contém exclusivamente a palavra constituída pelo símbolo **x**:

$$\{ \mathbf{x} \}$$

b *Passo de indução*. Se **r** e **s** são expressões regulares e denotam as linguagens **R** e **S**, respectivamente, então:

b.1) *União*. A expressão:

$$(\mathbf{r} + \mathbf{s})$$

é expressão regular e denota a linguagem:

$$\mathbf{R} \cup \mathbf{S}$$

b.2) *Concatenação*. A expressão:

$$(\mathbf{rs})$$

é expressão regular e denota a linguagem:

$$\mathbf{R}\,\mathbf{S} = \{ \mathbf{uv} \mid \mathbf{u} \in \mathbf{R} \text{ e } \mathbf{v} \in \mathbf{S} \}$$

b.3) *Concatenação sucessiva*. A expressão:

$$(\mathbf{r}^*)$$

é expressão regular e denota a linguagem:

$$\mathbf{R}^*$$

Se **r** é uma expressão regular, a correspondente linguagem denotada é dita a *linguagem gerada* por **r**, sendo representada por:

L(r) ou **GERA(r)** ❑

A omissão de parênteses em uma **ER** é usual, respeitando as seguintes convenções:

- a concatenação sucessiva tem precedência sobre a concatenação e a união;
- a concatenação tem precedência sobre a união.

exemplo 3.12 – Expressão regular

Na tabela da figura 3.27, são apresentadas expressões regulares e as correspondentes linguagens geradas.

Detalhando a linguagem gerada pela expressão **(a + b)*(aa + bb)**, vale que:

a e **b** denotam **{ a }** e **{ b }**, respectivamente
a + b denota **{ a }** ∪ **{ b }** = **{ a, b }**
(a + b)* denota **{ a, b }***
aa e **bb** denotam **{ a } { a }** = **{ aa }** e **{ b } { b }** = **{ bb }**, respectivamente
(aa + bb) denota **{ aa }** ∪ **{ bb }** = **{ aa, bb }**
(a + b)*(aa + bb) denota **{ a, b }* { aa, bb }**

Portanto, **GERA((a + b)*(aa + bb))** corresponde à seguinte linguagem:

{ aa, bb, aaa, abb, baa, bbb, aaaa, aabb, abaa, abbb, baaa, babb, bbaa, bbbb,… } ❑

expressão regular	linguagem gerada
aa	somente a palavra **aa**
ba*	todas as palavras que iniciam por **b**, seguido por **zero** ou mais **a**
(a+b)*	todas as palavras sobre **{ a, b }**
(a+b)*aa(a+b)*	todas as palavras contendo **aa** como subpalavra
a*ba*ba*	todas as palavras contendo exatamente dois **b**
(a+b)*(aa+bb)	todas as palavras que terminam com **aa** ou **bb**
(a+ε)(b+ba)*	todas as palavras que não possuem dois **a** consecutivos

figura 3.27 Expressões regulares e correspondentes linguagens geradas.

Os dois teoremas a seguir mostram que a classe das expressões regulares denota exatamente a classe das linguagens regulares.

teorema 3.21 – Expressão regular → Linguagem regular

Se **r** é uma expressão regular, então **GERA(r)** é uma linguagem regular.

➥ **Prova: (*por indução*)**
Por definição, uma linguagem é regular se, e somente se, é possível construir um autômato finito (determinístico, não determinístico ou com movimentos vazios) que reconheça essa lin-

guagem. Assim, é necessário mostrar que, dada uma expressão regular **r** qualquer, é possível construir um autômato finito **M** tal que:

$$\mathbf{ACEITA(M) = GERA(r)}$$

Na construção do correspondente autômato finito com movimentos vazios **M** apresentada a seguir, a demonstração de que **ACEITA(M) = GERA(r)** é por indução (primeira versão do segundo princípio de indução) no número de operadores.

a *Base de indução.* Seja **r** uma expressão regular com **zero** operadores. Então **r** só pode ser da forma:

$r = \varnothing$
$r = \varepsilon$
$r = x$ (**x** pertencente a Σ)

Os autômatos finitos:

$M_1 = (\varnothing, \{ q_0 \}, \delta_1, q_0, \varnothing)$
$M_2 = (\varnothing, \{ q_f \}, \delta_2, q_f, \{ q_f \})$
$M_3 = (\{ x \}, \{ q_0, q_f \}, \delta_3, q_0, \{ q_f \})$

ilustrados na figura 3.28, aceitam as linguagens acima, respectivamente;

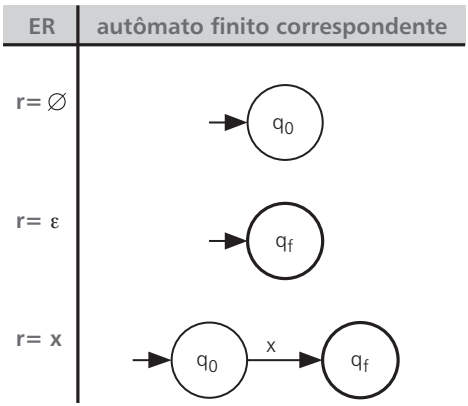


figura 3.28 Autômatos finitos correspondentes às expressões regulares com **zero** operadores.

b *Hipótese de indução*. Suponha que, para algum $n \in \mathbf{N}$, e para qualquer $u \in \mathbf{N}$ tal que $u \leq n$, se o número de operadores de **r** é **u**, é possível definir um autômato finito que aceita a linguagem gerada por **r**;

c *Passo de indução*. Seja **r** uma expressão regular com **n + 1** operadores. Então **r** pode ser representada por um dos seguintes casos, nos quais r_1 e r_2 individualmente, possuem no máximo **n** operadorers e, conjuntamente, possuem **n + 1** operadores:

$r = r_1 + r_2$
$r = r_1 r_2$
$r = r_1^*$

Portanto, por hipótese de indução, é possível construir os autômatos:

$$M_1 = (\Sigma_1, Q_1, \delta_1, q_{01}, \{ q_{f1} \}) \quad \text{e} \quad M_2 = (\Sigma_2, Q_2, \delta_2, q_{02}, \{ q_{f2} \})$$

tais que:

$$ACEITA(M_1) = GERA(r_1) \quad \text{e} \quad ACEITA(M_2) = GERA(r_2).$$

Note-se que, sem perda de generalidade, é possível assumir que M_1 e M_2 possuem exatamente um estado final (ver exercícios). Adicionalmente, suponha que o conjunto de estados dos dois autômatos são disjuntos (se não forem disjuntos, é suficiente realizar uma simples renomeação). Os autômatos finitos com movimentos vazios que aceitam a linguagem **GERA(r)** para cada caso são como segue:

c.1) $r = r_1 + r_2$. O autômato (suponha $q_0 \notin Q_1 \cup Q_2$ e $q_f \notin Q_1 \cup Q_2$):

$$M = (\Sigma_1 \cup \Sigma_2, Q_1 \cup Q_2 \cup \{ q_0, q_f \}, \delta, q_0, \{ q_f \})$$

ilustrado na figura 3.29 é tal que, a partir do estado inicial q_0, realiza transições vazias para os estados q_{01} e q_{02}. Assim, M_1 e M_2 processam de forma não determinística e, portanto, é suficiente um dos módulos aceitar a entrada para o autômato **M** aceitar;

c.2) $r = r_1 r_2$. O autômato:

$$M = (\Sigma_1 \cup \Sigma_2, Q_1 \cup Q_2, \delta, q_{01}, \{ q_{f2} \})$$

ilustrado na figura 3.29, ao processar os módulos M_1 e M_2 em sequência, aceita uma entrada se, e somente se, o prefixo pertencer a $ACEITA(M_1)$ e o sufixo pertencer a $ACEITA(M_2)$;

c.3) $r = r_1^*$. O autômato (suponha $q_0 \notin Q_1$ e $q_f \notin Q_1$):

$$M = (\Sigma_1, Q_1 \cup \{ q_0, q_f \}, \delta, q_0, \{ q_f \})$$

ilustrado na figura 3.29, é tal que a transição vazia de q_0 para q_f garante a aceitação da palavra vazia e o ciclo de q_{f1} para q_{01} permite o sucessivo processamento de M_1 para assegurar o reconhecimento de duas ou mais concatenações sucessivas. ❑

É interessante observar que, na prova do teorema 3.21 – Expressão regular → linguagem regular, algumas possíveis alterações na construção do autômato resultante podem gerar resultados indesejados, como por exemplo (procure justificar cada um dos seguintes casos):

- no caso $r = r_1 + r_2$, não introduzir os estados q_0 e q_f e identificar ("unificar") os estados iniciais e finais de M_1 com os correspondentes de M_2;
- no caso $r = r_1^*$, não introduzir o estado q_f e manter q_{f1} como o estado final do autômato resultante. Neste caso, a transição vazia que parte de q_0 teria q_{f1} como imagem.

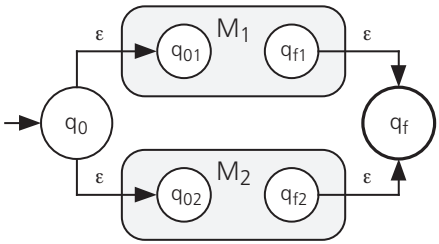
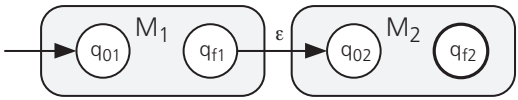
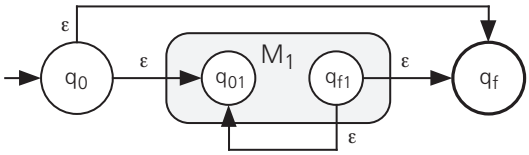
ER	autômato finito correspondente
$r = r_1 + r_2$	
$r = r_1 r_2$	
$r = r_1^*$	

figura 3.29 Autômatos finitos correspondentes às expressões regulares com **n+1** operadores.

exemplo 3.13 – Construção de um ***AFN*ε** a partir de uma expressão regular

A construção dos módulos de um **AFNε** que aceita a linguagem gerada pela expressão regular **a*(aa + bb)** é como ilustrada na figura 3.30 e o autômato resultante é ilustrado na figura 3.31 (por simplicidade, a identificação dos estados é omitida). ❑

O seguinte teorema não será provado.

teorema 3.22 – Linguagem regular → Expressão regular

Se **L** é uma linguagem regular, então existe uma expressão regular **r** tal que:

$$\text{GERA}(r) = L$$ ❑

ER	autômato finito correspondente
a	
b	
a*	
aa	
bb	
(aa + bb)	

figura 3.30 Expressões regulares e os correspondentes autômatos finitos.

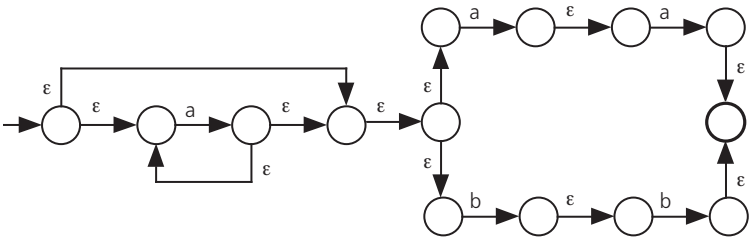


figura 3.31 Diagrama: **AFN**ε construído a partir da expressão regular.

3.7 gramática regular

Usando o conceito de gramáticas como apresentado no capítulo 2 – Linguagens e gramáticas, é possível definir tanto linguagens regulares como linguagens não regulares. Entretanto, é possível estabelecer restrições nas regras de produção, de tal forma a definir exatamente a classe das linguagens regulares.

Existe mais de uma forma de restringir as regras de produção de forma a definir uma *gramática regular*. A seguir, são apresentadas quatro dessas formas, denominadas *gramáticas lineares*.

definição 3.23 – Gramáticas lineares

Seja **G = (V, T, P, S)** uma gramática. Sejam **A** e **B** elementos de **V** e **w** uma palavra de **T***. Então **G** é uma *gramática linear* se *todas* as suas produções encontram-se em *uma* e em *somente uma* das seguintes formas:

a *Gramática linear à direita* (abreviada por **GLD**). Todas as regras de produção são da forma:

$$A \rightarrow wB \quad \text{ou} \quad A \rightarrow w$$

b *Gramática linear à esquerda* (abreviada por **GLE**). Todas as regras de produção são da forma:

$$A \rightarrow Bw \quad \text{ou} \quad A \rightarrow w$$

c *Gramática linear unitária à direita* (abreviada por **GLUD**). Todas as regras de produção são como na gramática linear à direita e, adicionalmente:

$$|\, w \,| \leq 1$$

d *Gramática linear unitária à esquerda* (abreviada por **GLUE**). Todas as regras de produção são como na gramática linear à esquerda e, adicionalmente:

$$|\, w \,| \leq 1$$ ❑

Note-se que as gramáticas lineares possuem forte restrição no formato de suas produções:

- o lado esquerdo possui exatamente uma variável;
- o lado direito de uma produção é constituído por, no máximo, uma variável. Adicionalmente, esta variável, se existir, sempre antecede (linear à esquerda) ou sucede (linear à direita) qualquer subpalavra (eventualmente vazia) de terminais.

Sugere-se como exercício verificar se é possível definir uma gramática que satisfaça simultaneamente as quatro formas lineares.

teorema 3.24 – Equivalência das gramáticas lineares

Seja **L** uma linguagem. Então:

L é gerada por uma **GLD** se, e somente se,
L é gerada por uma **GLE** se, e somente se,

L é gerada por uma **GLUD** se, e somente se,
L é gerada por uma **GLUE**. ❑

Ou seja, as diversas formas das gramáticas lineares são formalismos equivalentes. A demonstração do teorema é sugerida como exercício.

definição 3.25 – Gramática regular

Uma gramática **G** é dita uma *gramática regular*, eventualmente abreviada por **GR**, se **G** é uma gramática linear. ❑

definição 3.26 – Linguagem gerada

Seja **G = (V, T, P, S)** uma gramática. A *linguagem gerada* pela gramática **G**, denotada por:

$$L(G) \quad \text{ou} \quad GERA(G)$$

é tal que:

$$L(G) = \{ w \in T^* \mid S \Rightarrow^+ w \}$$ ❑

exemplo 3.14 – Gramática regular: ***a(ba)****

A linguagem **a(ba)*** é gerada pelas seguintes gramáticas regulares:

a *Linear à direita*. **G = ({ S, A }, { a, b }, P, S)**, e **P** possui as seguintes regras de produção:
$S \rightarrow aA$
$A \rightarrow baA \mid \varepsilon$

b *Linear à esquerda*. **G = ({ S }, { a, b }, P, S)**, e **P** possui as seguintes regras de produção:
$S \rightarrow Sba \mid a$

c *Linear unitária à direita*. **G = ({ S, A, B }, { a, b }, P, S)**, e **P** possui as seguintes regras de produção:
$S \rightarrow aA$
$A \rightarrow bB \mid \varepsilon$
$B \rightarrow aA$

d *Linear unitária à esquerda*. **G = ({ S, A }, { a, b }, P, S)**, e **P** possui as seguintes regras de produção:
$S \rightarrow Aa \mid a$
$A \rightarrow Sb$ ❑

exemplo 3.15 – Gramática regular: ***(a + b)****(aa + bb)***

A linguagem **(a + b)*(aa + bb)** é gerada pelas seguintes gramáticas regulares:

a *Linear à direita*. **G = ({ S, A }, { a, b }, P, S)**, e **P** possui as seguintes regras de produção:
$S \rightarrow aS \mid bS \mid A$
$A \rightarrow aa \mid bb$

b *Linear à esquerda*. **G = ({ S, A }, { a, b }, P, S)**, e **P** possui as seguintes regras de produção:
$S \rightarrow Aaa \mid Abb$
$A \rightarrow Aa \mid Ab \mid \varepsilon$ ❑

observação 3.27 – Gramática linear à esquerda e linear à direita

Suponha $|\ w\ | \geq 1$. Se uma gramática tiver simultaneamente produções do tipo $A \rightarrow wB$ (linear à direita) e do tipo $A \rightarrow Bw$ (linear à esquerda), então a correspondente linguagem gerada poderá não ser regular, ou seja, esta *não* é uma gramática regular. Por exemplo, já foi discutido que uma linguagem que possua duplo balanceamento não é regular. Em particular, a seguinte linguagem não é regular (adiante, é estudado como provar que tal linguagem não é regular):

$$\{\ a^n b^n \mid n \in \mathbf{N}\ \}$$

Entretanto, é possível desenvolver uma gramática, com produções lineares à direita e à esquerda, que gera esta linguagem, o que se sugere como exercício. ❑

Os dois teoremas a seguir mostram que a classe das gramáticas regulares denota exatamente a classe das linguagens regulares.

teorema 3.28 – Gramática regular → linguagem regular

Se **L** é uma linguagem gerada por uma gramática regular, então **L** é uma linguagem regular.

➥ **Prova: (*por indução*)**

Para mostrar que uma linguagem é regular, é suficiente construir um autômato finito que a reconheça. Suponha $G = (V, T, P, S)$ uma gramática linear unitária à direita. Então o **AFNε**:

$$M = (\Sigma, Q, \delta, q_0, F)$$

tal que (suponha $q_f \notin V$):

$\Sigma = T$
$Q = V \cup \{\ q_f\ \}$
$F = \{\ q_f\ \}$
$q_0 = S$
δ é construída como ilustrado na figura 3.32 (suponha **A** e **B** variáveis e **a** terminal) simula as derivações de **G**, ou seja:

$$ACEITA(M) = GERA(G)$$

tipo da produção	transição gerada
$A \rightarrow \varepsilon$	$\delta(A, \varepsilon) = \{q_f\}$
$A \rightarrow a$	$\delta(A, a) = \{q_f\}$
$A \rightarrow B_1 \mid \ldots \mid B_n$	$\delta(A, \varepsilon) = \{\ B_1, \ldots, B_n\ \}$
$A \rightarrow aB_1 \mid \ldots \mid aB_n$	$\delta(A, a) = \{\ B_1, \ldots, B_n\ \}$

figura 3.32 Transições construídas a partir das produções.

A demonstração que, de fato, **ACEITA(M) = GERA(G)** é por indução no número de derivações, como segue (suponha α elemento de $(T \cup V)^*$ e **w** elemento de T^*):

a *Base de indução*. Suponha que $S \Rightarrow^1 \alpha$. Então, se:

a.1) $\alpha = \varepsilon$, existe uma regra $S \rightarrow \varepsilon$. Por construção de **M**, $\delta(S, \varepsilon) = \{q_f\}$

a.2) $\alpha = a$, existe uma regra $S \rightarrow a$. Por construção de **M**, $\delta(S, a) = \{q_f\}$

a.3) $\alpha = A$, existe uma regra $S \rightarrow A$. Por construção de **M**, $\delta(S, \varepsilon) = \{A\}$

a.4) $\alpha = aA$, existe uma regra $S \rightarrow aA$. Por construção de **M**, $\delta(S, a) = \{A\}$

b *Hipótese de indução*. Suponha que $S \Rightarrow^n \alpha$, $n > 1$ tal que, se:

b.1) $\alpha = w$, então $\delta^*(S, w) = \{q_f\}$

b.2) $\alpha = wA$, então $\delta^*(S, w) = \{A\}$

c *Passo de indução*. Suponha que $S \Rightarrow^{n+1} \alpha$. Obrigatoriamente, ocorre somente a hipótese b.2) acima e, portanto:

$$S \Rightarrow^n wA \Rightarrow^1 \alpha$$

Então, se:

c.1) $\alpha = w\varepsilon = w$, existe uma regra $A \rightarrow \varepsilon$. Logo:

$$\delta^*(S, w\varepsilon) = \delta(\delta^*(S, w), \varepsilon) = \delta(A, \varepsilon) = \{q_f\}$$

c.2) $\alpha = wb$, existe uma regra $A \rightarrow b$. Logo:

$$\delta^*(S, wb) = \delta(\delta^*(S, w), b) = \delta(A, b) = \{q_f\}$$

c.3) $\alpha = wB$, existe uma regra $A \rightarrow B$. Logo:

$$\delta^*(S, w\varepsilon) = \delta(\delta^*(S, w), \varepsilon) = \delta(A, \varepsilon) = \{B\}$$

c.4) $\alpha = wbB$, existe uma regra $A \rightarrow bB$. Logo:

$$\delta^*(S, wb) = \delta(\delta^*(S, w), b) = \delta(A, b) = \{B\}$$ ❑

exemplo 3.16 – Construção de um ***AFN***ε a partir de uma gramática regular

Considere a seguinte gramática linear unitária à direita, introduzida no exemplo 3.14 – Gramática regular:

$$G = (\{ S, A, B \}, \{ a, b \}, P, S)$$

onde **P** é tal que:

$S \rightarrow aA$
$A \rightarrow bB \mid \varepsilon$
$B \rightarrow aA$

O **AFN**ε **M** que reconhece a linguagem gerada pela gramática **G** é:

$$M = (\{ a, b \}, \{ S, A, B, q_f \}, \delta, S, \{ q_f \})$$

o qual é ilustrado na figura 3.34, onde δ é dada pela tabela na figura 3.33. ❑

produção	transição
$S \rightarrow aA$	$\delta(S, a) = \{A\}$
$A \rightarrow bB$	$\delta(A, b) = \{B\}$
$A \rightarrow \varepsilon$	$\delta(A, \varepsilon) = \{q_f\}$
$A \rightarrow aA$	$\delta(B, a) = \{A\}$

figura 3.33 Transições construídas a partir das produções.

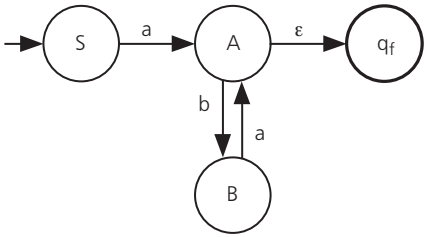


figura 3.34 Autômato finito construído a partir de uma gramática regular.

teorema 3.29 – Linguagem regular → Gramática regular

Se **L** é linguagem regular, então existe **G**, gramática regular que gera **L**.

➥ **Prova: (*por indução*)**

Se **L** é uma linguagem regular, então existe um **AFD** $M = (\Sigma, Q, \delta, q_0, F)$ tal que **ACEITA(M) = L**. A ideia central da demonstração é construir uma gramática linear à direita **G** a partir de **M** tal que:

$$GERA(G) = ACEITA(M)$$

ou seja, cuja derivação simula a função programa estendida. Seja a gramática regular:

$$G = (V, T, P, S)$$

tal que (suponha $S \notin Q$):

$V = Q \cup \{ S \}$
$T = \Sigma$
P é construído como ilustrado na figura 3.35 (suponha q_i, q_k elementos de **Q**, q_f elemento de **F** e **a** elemento de Σ)

transição	produção
-	$S \rightarrow q_0$
-	$q_f \rightarrow \varepsilon$
$\delta(q_i, a) = q_k$	$q_i \rightarrow aq_k$

figura 3.35 Produções construídas a partir de transições.

A demonstração de que, de fato, **GERA(G) = ACEITA(M)** é por indução no tamanho da palavra, como segue (suponha **w** elemento de Σ^*):

a *Base de indução*. Seja **w** tal que $|\ w\ | = 0$. Por definição, vale que $S \rightarrow q_0$ é produção. Se $\varepsilon \in$ **ACEITA(M)**, então q_0 é estado final e, por definição, vale que $q_0 \rightarrow \varepsilon$ é produção. Logo:

$$S \Rightarrow q_0 \Rightarrow \varepsilon$$

b *Hipótese de indução*. Seja **w** tal que $|\ w\ | = n\ (n \geq 1)$ e $\delta^*(q_0, w) = q$. Assim, se:

b.1) **q** não é estado final, então suponha que $S \Rightarrow^n wq$

b.2) **q** é estado final, então suponha que $S \Rightarrow^n wq \Rightarrow w$ (este caso não é importante para o passo de indução);

c *Passo de indução*. Seja **w** tal que $|\ wa\ | = n + 1$ e $\delta^*(q_0, wa) = p$. Então:

$$\delta(\delta^*(q_0, w), a) = \delta(q, a) = p$$

Portanto, obrigatoriamente, ocorre somente a hipótese b.1) acima e, se:

c.1) **p** não é estado final, então $S \Rightarrow^n wq \Rightarrow^1 wap$

c.2) **p** é estado final, então $S \Rightarrow^n wq \Rightarrow^1 wap \Rightarrow^1 wa$ ❑

exemplo 3.17 – Construção de uma gramática regular a partir de um *AFD*

Considere o autômato finito determinístico:

$$M = (\{ a, b, c \}, \{ q_0, q_1, q_2 \}, \delta, q_0, \{ q_0, q_1, q_2 \})$$

ilustrado na figura 3.37. A correspondente gramática regular construída é:

$$G = (\{ q_0, q_1, q_2, S \}, \{ a, b, c \}, P, S)$$

onde **P** é dado pela tabela na figura 3.36. ❑

transição	produção
-	$S \rightarrow q_0$
-	$q_0 \rightarrow \varepsilon$
-	$q_1 \rightarrow \varepsilon$
-	$q_2 \rightarrow \varepsilon$
$\delta(q_0, a) = q_0$	$q_0 \rightarrow aq_0$
$\delta(q_0, a) = q_1$	$q_0 \rightarrow bq_1$
$\delta(q_1, b) = q_1$	$q_1 \rightarrow bq_1$
$\delta(q_1, c) = q_2$	$q_1 \rightarrow cq_2$
$\delta(q_2, c) = q_2$	$q_2 \rightarrow cq_2$

figura 3.36 Produções construídas a partir de transições.

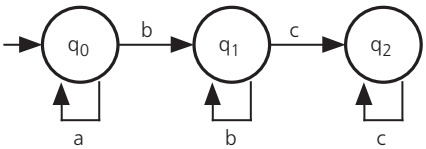


figura 3.37 Diagrama do autômato finito determinístico.

3.8 exercícios

exercício 3.1 Sobre as linguagens regulares:

a Qual a importância do seu estudo?
b Exemplifique suas aplicações (para os diversos formalismos);
c Você imagina algum tipo de linguagem cujo algoritmo de reconhecimento seja mais eficiente que o das regulares? E menos eficiente? Explique a sua resposta.

exercício 3.2 Desenvolva autômatos finitos determinísticos que reconheçam as seguintes linguagens sobre $\Sigma = \{ a, b \}$:

a { **w** | o sufixo de **w** é **aa** }
b { **w** | **w** possui **aaa** como subpalavra }
c { **w** | **w** possui número ímpar de **a** e número ímpar de **b** }
d { **w** | **w** possui número par de **a** e ímpar de **b** ou **w** possui número par de **b** e ímpar de **a** }
e { **w** | o quinto símbolo da direita para a esquerda de **w** é **a** }
Dica: o autômato resultante possui um número relativamente grande de estados.

exercício 3.3 Qual a condição para que a palavra vazia pertença a linguagem gerada por um autômato finito determinístico?

exercício 3.4 Desenvolva autômatos finitos não determinísticos que reconheçam as seguintes linguagens sobre o alfabeto $\Sigma = \{ a, b \}$

a Qualquer ocorrência de **a** é imediatamente sucedida por **b**;
b Qualquer ocorrência de **a** é imediatamente antecedida e imediatamente sucedida por **b**.

exercício 3.5 Quais das seguintes palavras são aceitas pelos autômatos finitos não determinísticos sobre o alfabeto $\Sigma = \{ a, b \}$:

a Ilustrado na figura 3.38:
ε?
aa?
bb?
aba?
bbaaba?
abababababa?

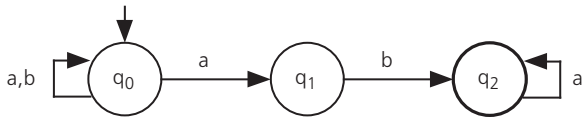


figura 3.38 Diagrama (**AFN**).

b Ilustrado na figura 3.39.

ε?
bb?
bab?
bbbaaa?
bbbbbbabababa?

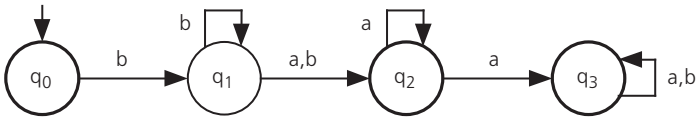


figura 3.39 Diagrama (**AFN**).

exercício 3.6 Desenvolva autômatos finitos não determinísticos, com ou sem movimentos vazios, que reconheçam as seguintes linguagens sobre o alfabeto $\Sigma = \{ a, b \}$:

a $\{ w_1 w_2 w_1 \mid w_2$ é qualquer e $\mid w_1 \mid = 3 \}$

b $\{ w \mid$ o décimo símbolo da direita para a esquerda de **w** é **a** $\}$

c $\{ w \mid$ **w** possui igual número de símbolos **a** e **b** e (qualquer prefixo de **w** possui, no máximo, dois **a** a mais que **b** ou qualquer prefixo de **w** possui, no máximo, dois **b** a mais que **a**) $\}$

exercício 3.7 Desenvolva, sobre o alfabeto $\Sigma = \{ a, b, c \}$:

a Autômato finito não determinístico que reconheça a seguinte linguagem:

$\{ w \mid$ **a** ou **bb** ou **ccc** é sufixo de **w** $\}$

b Autômato finito não determinístico com movimentos vazios que reconheça a seguinte linguagem:

$\{ w \mid$ **aa** ou **bb** é subpalavra e **cccc** é sufixo de **w** $\}$

exercício 3.8 Considere o autômato sobre o alfabeto $\Sigma = \{ \varepsilon, \eta, \mu \}$ ilustrado na figura 3.40. Justifique ou refute as seguintes afirmações:

a O autômato possui movimentos vazios;

b Aceita a linguagem $(\eta + \mu)^*$

Dica: observe bem a definição do alfabeto.

figura 3.40 Diagrama: autômato finito.

exercício 3.9 Por que se pode afirmar que um autômato finito com movimentos vazios sempre para (ao processar qualquer entrada)? Em particular, analise para as seguintes transições que caracterizam um ciclo (suponha que **q** e **p** são estados do autômato), considerando a definição de função programa estendida:

$\delta(q, \varepsilon) = \{p\}$
$\delta(p, \varepsilon) = \{q\}$

exercício 3.10 Complete a prova referente ao teorema 3.19 – Equivalência entre **AFN** e **AFN**ε.

exercício 3.11 Seja **L** uma linguagem regular. Prove que L^R também é regular sendo que:

$$L^R = \{ w \mid \text{o reverso de } w \text{ está em } L \}$$

Dica: se **L** é regular, então existe autômato finito que a reconhece. Como fazer o autômato "reverso"?

exercício 3.12 *Adição binária é regular.* A operação de adição de dois número binários pode ser expressa em termos de sequências de triplas. Por exemplo, considere a seguinte adição binária:

$$\begin{array}{cccc} 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ \hline 1 & 1 & 1 & 0 \end{array}$$

a qual pode ser expressa pela seguinte sequência de triplas sendo que cada tripla representa uma coluna:

$$(0, 1, 1), (1, 0, 1), (0, 0, 1), (1, 1, 0)$$

e que pode ser melhor visualizada usando a seguinte notação para sequência de triplas:

$$\begin{pmatrix} 0 \\ 1 \\ 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}$$

Seja $B = \{ 0, 1 \}$ e $\Sigma = B \times B \times B$. Construa uma autômato finito sobre o alfabeto Σ que reconhece todas as palavras que representam a operação de adição binária.

exercício 3.13 Descreva em palavras as linguagens geradas pelas seguintes expressões regulares:

a $(a + b + c)^*(a + b + c)$
b $(aa + b)^*(a + bb)$
c $(b + ab)^*(\varepsilon + a)$
d $c^*(a + b)(a + b + c)^*$
e $((b + c)^* + a(b + c)^*a)^*$
f $(aa + bb + (aa + bb)(ab + ba)(aa + bb))^*$

exercício 3.14 Desenvolva expressões regulares que gerem as seguintes linguagens sobre $\Sigma = \{ 0, 1 ,\ldots,9 ,a ,b ,\ldots,z ,+ ,- \}$:

a Identificadores em uma linguagem tipo Pascal, sendo que o primeiro caractere sempre é uma letra e os demais podem ser zero ou mais letras ou dígitos;

b Números inteiros em uma linguagem tipo Pascal, compostos por qualquer sequência não vazia de dígitos, precedidos ou não por um sinal.

exercício 3.15 Desenvolva expressões regulares que gerem as seguintes linguagens sobre $\Sigma = \{ a, b \}$:

a { **w** | **w** não possui **aba** como subpalavra }

b { **w** | qualquer par de **a** antecede qualquer par de **b** }

c { **w** | **w** tem no máximo um par de **a** como subpalavra e no máximo um par de **b** como subpalavra }

exercício 3.16 Aponte, justificando, quais das seguintes expressões regulares são equivalentes:

$(a + b)^*a^*$
$(a + b)^*$
$((a + b)a)^*$

exercício 3.17 Aplique o algoritmo de tradução de formalismo de expressão regular para autômato finito:

a $a(aa + bb)^* + \varepsilon$

b $(\varnothing^* + \varepsilon^*)^*$

c $(ab + ba)^*(aa + bb)^*$

d $ab(abb^* + baa^*)^*ba$

exercício 3.18 Desenvolva gramáticas regulares que gerem as seguintes linguagens sobre $\Sigma = \{ 0, 1 ,\ldots,9 ,a ,b ,\ldots,z ,+ ,- \}$:

a Identificadores em uma linguagem tipo Pascal, sendo que o primeiro caractere sempre é uma letra e os demais podem ser zero ou mais letras ou dígitos;

b Números inteiros em uma linguagem tipo Pascal, compostos por qualquer sequência não vazia de dígitos, precedidos ou não por um sinal.

exercício 3.19 Desenvolva gramáticas regulares que gerem as seguintes linguagens sobre $\Sigma = \{ a, b \}$:

a { **w** | **w** não possui **aba** como subpalavra }

b { **w** | qualquer par de **a** antecede qualquer par de **b** }

c { **w** | **w** tem no máximo um par de **a** como subpalavra e no máximo um par de **b** como subpalavra }

exercício 3.20 É possível definir uma gramática regular que satisfaça *simultaneamente* as quatro formas lineares? Justifique a sua resposta.

exercício 3.21 Demonstre a equivalência dos quatro tipos de gramáticas lineares.

exercício 3.22 Relativamente às gramáticas regulares, justifique ou refute as seguintes afirmações:

a Considere uma gramática com as seguintes produções (suponha que **A** é o símbolo inicial):

$A \rightarrow aB$
$B \rightarrow aC \mid bC$
$C \rightarrow aD \mid bD$
$D \rightarrow aS \mid bS \mid \varepsilon$
$S \rightarrow D$

Então:

a.1) A gramática é regular;
a.2) A linguagem gerada é composta por todas as palavras cujo terceiro símbolo da direita para a esquerda é **a**;

b Considere uma gramática com as seguintes produções (suponha que **S** é o símbolo inicial):

$S \rightarrow aS \mid Xb \mid \varepsilon$
$X \rightarrow Sb$

Então:

b.1) A gramática é não regular;
b.2) A linguagem gerada é regular.

exercício 3.23 Aplique os algoritmos de tradução de formalismos apresentados e, a partir da seguinte expressão regular:

$$(b + \varepsilon)(a + bb)^*$$

realize as diversas etapas até gerar a gramática regular correspondente, ou seja:

$$ER \rightarrow AFN\varepsilon \rightarrow AFN \rightarrow AFD \rightarrow GR$$

exercício 3.24 Foi afirmado que a seguinte linguagem é não regular:

$$\{ a^n b^n \mid n \in \mathbf{N} \}$$

Desenvolva uma gramática com produções lineares à direita *e* à esquerda que gera esta linguagem.

exercício 3.25 Foi declarado que uma importante aplicação das linguagens regulares é a especificação e o desenvolvimento de analisadores léxicos. Para reforçar tal ideia, apresente a seguinte linguagem baseada em unidades léxicas da linguagem de programação Pascal (ou alguma outra de seu domínio), usando um formalismo regular de sua escolha:

{ w | **w** é número inteiro ou **w** é número real ou **w** é identificador **}**

exercício 3.26 Foi afirmado que linguagens de propósitos gerais como Pascal, C, Java, etc., são não regulares, pois possuem estruturas de duplo balanceamento como parênteses em expressões aritméticas. Entretanto, é possível definir linguagens com duplo balanceamento

as quais são regulares: basta limitar o número máximo de encadeamentos (e portanto, limitar o número máximo de alternativas, ou seja, de estados possíveis, permitindo uma representação finita). De fato, muitas linguagens de programação, até a década de **1970**, possuíam limites desse tipo, com o objetivo de otimizar a compilação. Para exemplificar tal afirmação, desenvolva um formalismo regular (de qualquer tipo) sobre o alfabeto Σ **= { x, (,) }** o qual reconhece/gera qualquer palavra com parênteses balanceados, desde que limitados a três níveis de encadeamento. Por exemplo:

ε e **x(x(xx())x)** são palavras da linguagem;
((((x)))) não é palavra da linguagem.

exercício 3.27 Considere o alfabeto Σ **= {a, b}**. A linguagem L_n, para algum **n** natural fixo, é o conjunto de todas as palavras sobre Σ cujo comprimento máximo é **n**. Então, para qualquer **n** fixo, pode-se afirmar que sempre existe um formalismo regular que aceita/gera a linguagem L_n.

exercício 3.28 Desenvolva um programa em computador que simule o processamento de qualquer autômato finito determinístico, como segue:

- entrada: função de transição, estado inicial, conjunto de estados finais e as palavras a serem processadas;
- saída: condição de parada **ACEITA/REJEITA** e identificação do estado de parada.

Dica: a parte central do algoritmo de simulação é realmente pequena, e pode ser expressa em poucas unidades de linhas de código e consiste, basicamente, em um algoritmo que controla a mudança de estado do autômato a cada símbolo lido da entrada. Tal resultado comprova a simplicidade que é desenvolver um simulador genérico de autômato finito.

exercício 3.29 Desenvolva um programa em computador que implemente os seguintes algoritmos:

a Tradução de **AFN** para **AFD**;
b Tradução de **AFN**ε para **AFN**;
c Tradução de **ER** para **AFN**ε;
d Tradução de **GR** para **AFN**ε equivalente.

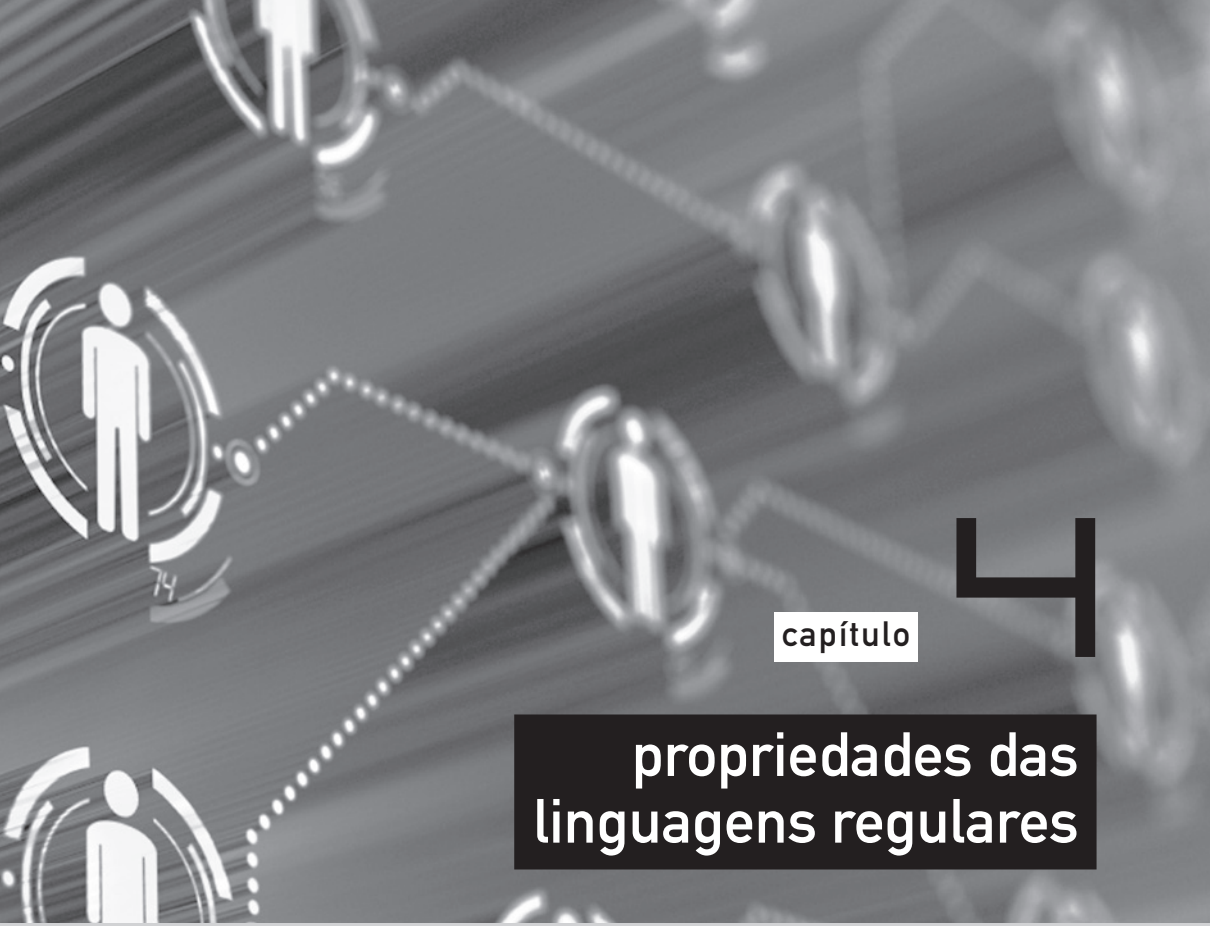
Dica: basta implementar o algoritmo apresentado em cada prova do correspondente teorema.

Observe que, usando os tradutores em combinação com o simulador do exercício acima, pode-se desenvolver um simulador de linguagens regulares tendo como entrada qualquer dos formalismos estudados.

exercício 3.30 Desenvolva um algoritmo que gere, em ordem lexicográfica, todas as palavras representadas por:

a Uma expressão regular qualquer;
b Uma gramática regular qualquer.

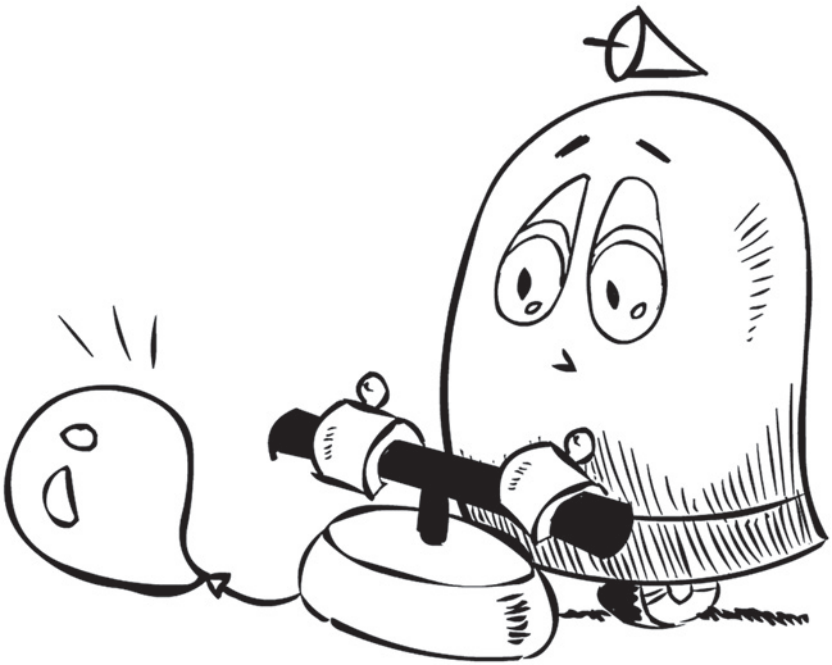
Dica: considere os algoritmos desenvolvidos nos dois exercícios acima.



capítulo 4

propriedades das linguagens regulares

Linguagens regulares são representadas por formalismos muito simples, sendo possível desenvolver algoritmos de pouca complexidade, de grande eficiência e de fácil implementação. Entretanto, possuem fortes limitações de expressividade. É por isso que o estudo de algumas das principais propriedades das linguagens regulares, assunto tratado nesse capítulo, é muito importante.



Uma das principais características das linguagens regulares é o fato de serem representadas por formalismos de pouca complexidade, grande eficiência e fácil implementação. Entretanto, por ser uma classe relativamente simples, é restrita e limitada, sendo fácil definir linguagens não regulares. Assim, algumas questões sobre linguagens regulares necessitam ser analisadas:

a Como determinar se uma linguagem é regular?

b A Classe das Linguagens Regulares é fechada para operações de união, concatenação e intersecção (ou seja, a operação de duas linguagens regulares resulta em uma linguagem regular)?

c Como verificar se uma linguagem regular é infinita ou finita (ou até mesmo vazia)?

d É possível analisar duas linguagens regulares quaisquer e concluir se são iguais ou diferentes?

No estudo que segue, a análise de cada propriedade é desenvolvida para um dos formalismos estudados. Para os demais formalismos, é suficiente traduzi-los, usando os algoritmos apresentados nos correspondentes teoremas, como ilustrado na figura 4.1 (note-se que a construção de uma expressão regular a partir de um autômato finito determinístico não foi provada).

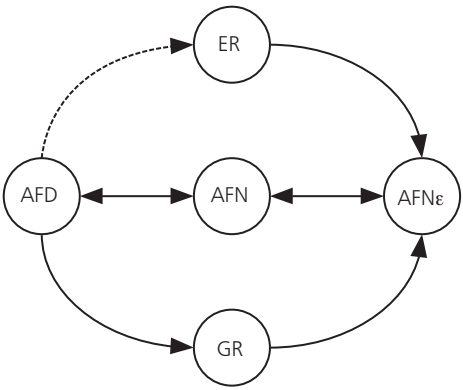


figura 4.1 Tradução dos formalismos das linguagens regulares.

observação 4.1 – Autômato finito × complexidade de algoritmos

Já foi comentado que, relativamente à *complexidade de algoritmos*, autômatos finitos pertencem à classe de algoritmos mais eficientes em termos de tempo de processamento, supondo que toda a entrada necessita ser lida. De fato, qualquer autômato finito que solucione um problema é igualmente eficiente, a menos de eventual redundância de estados, a qual não influi no tempo de processamento.

É importante observar que a eficiência que está em discussão supõe a leitura de toda a fita de entrada. Se tal suposição for relaxada, podem-se imaginar formalismos ainda mais eficientes para alguns tipos de problemas. Um exemplo seria reconhecer a linguagem Σ^*, para um dado alfabeto Σ (pois, como qualquer palavra de entrada deve ser aceita, nem sequer é necessário ler a fita de entrada). ❑

Portanto, usando autômatos finitos, *qualquer solução é ótima*, a menos de eventual redundância de estados. Tal redundância de estados pode ser facilmente eliminada, determinando-se um *autômato finito determinístico mínimo* ou simplesmente *autômato finito mínimo*.

4.1 → bombeamento para as linguagens regulares

O resultado a seguir, conhecido por *lema do bombeamento*, é útil no estudo das propriedades das linguagens regulares. Resumidamente, a ideia básica é a seguinte (veja a figura 4.2, supondo $\mathbf{q_0}$ o estado inicial e $\mathbf{q_f}$ um estado final):

- se uma linguagem é regular, então é aceita por um autômato finito determinístico o qual possui um número finito e predefinido de **n** estados;
- se o autômato reconhece uma entrada **w** de comprimento maior ou igual ao número de estados **n**, obrigatoriamente o autômato assume algum estado **q** mais de uma vez e, portanto, existe um ciclo na função programa que passa por **q**;

- logo, **w** pode ser dividida em três subpalavras **w = u v z** tal que $|\ u\ v\ | \leq n$, $|\ v\ | \geq 1$ e onde **v** é a parte de **w** reconhecida pelo ciclo;
- claramente, tal ciclo pode ser executado ("bombeado") **zero** ou mais vezes. Portanto, para qualquer $i \geq 0$, $u\ v^i\ z$, é aceita pelo autômato (ou seja, é palavra da linguagem).

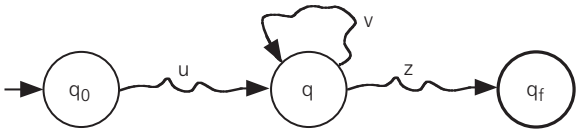


figura 4.2 Bombeamento para as linguagens regulares.

lema 4.2 – Bombeamento para as linguagens regulares

Se **L** é uma linguagem regular, então:

existe uma constante **n** tal que,
para qualquer palavra **w** de **L** onde $|\ w\ | \geq n$,
w pode ser definida como **w = u v z** onde:
$|\ u\ v\ | \leq n$,
$|\ v\ | \geq 1$
sendo que, para todo $i \geq 0$, $u\ v^i\ z$ é palavra de **L**.

➥ **Prova: (*direta*)**

Se **L** é uma linguagem regular, então existe um **AFD** $M = (\Sigma, Q, \delta, q_0, F)$ tal que:

$$ACEITA(M) = L$$

Suponha que:

n é o cardinal de **Q**
existe $w = a_1a_2\ldots a_m$ uma palavra de **L** de comprimento **m** tal que $m \geq n$
$\delta(q_0, a_1) = q_1$
$\delta(q_1, a_2) = q_2$
…
$\delta(q_{m-1}, a_m) = q_m$

Como $m \geq n$, então existem **r** e **s** com $0 \leq r < s \leq n$ tais que:

$q_r = q_s$
$\delta(q_0, a_1\ldots a_r) = q_r$
$\delta(q_r, a_{r+1}\ldots a_s) = q_s$
$\delta(q_s, a_{s+1}\ldots a_m) = q_m$

ou seja, o autômato passa mais de uma vez no estado $q_r = q_s$.

Sejam:

$u = a_1\ldots a_r$
$v = a_{r+1}\ldots a_s$
$z = a_{s+1}\ldots a_m$

Como $r < s \leq n$, então:

$$|v| \geq 1 \quad \text{e} \quad |uv| \leq n$$

Como $q_r = q_s$, então **v** é reconhecida em um ciclo.

Portanto, para todo $i \geq 0$, $u\,v^i\,z$ é palavra de **L**. ❑

exemplo 4.1 – Bombeamento para as linguagens regulares

Considere o autômato ilustrado na figura 4.3 (qual a linguagem aceita?). Exemplificando o lema 4.2 – Bombeamento para as linguagens regulares, vale que:

$n = 4$
no caso particular de $w = abbba$, vale que:
$q_r = q_s = q_1$
$u = a$, $v = bb$, $z = ba$ ❑

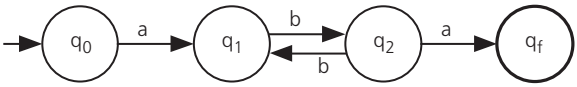


figura 4.3 Diagrama (**AFD**).

É importante observar que o lema do bombeamento garante que os formalismos regulares são capazes de expressar diversos bombeamentos (por quê?). Por exemplo, a seguintes linguagens são regulares:

- duplo bombeamento: $\{ a^n b^m \mid n \geq 0 \text{ e } m \geq 0 \}$
- triplo bombeamento: $\{ a^n b^m a^r \mid n \geq 0, m \geq 0 \text{ e } r \geq 0 \}$

4.2 → investigação se é linguagem regular

Para mostrar que uma determinada linguagem é regular, é suficiente representá-la usando um dos formalismos apresentados anteriormente (autômato finito, expressão regular ou gramática regular). Entretanto, a prova de que é não regular necessita ser desenvolvida para cada caso. Uma ferramenta útil é o lema 4.2 – Bombeamento para as linguagens regulares, como exemplificado a seguir.

exemplo 4.2 – Linguagem não regular

A seguinte linguagem sobre **{ a, b }** é não regular:

$$L = \{ w \mid w \text{ possui o mesmo número de símbolos } a \text{ e } b \}$$

A prova que segue é por absurdo.

Suponha que **L** é regular. Então existe um **AFD M** com **n** estados que aceita **L**.

Seja $w = a^n b^n$ palavra de **L** sendo $|w| = 2n \geq n$. Logo, pelo lema 4.2 – Bombeamento para as linguagens regulares, **w** pode ser definida como $w = u\,v\,z$ tal que:

$|\ u\ v\ | \leq n$
$|\ v\ | \geq 1$

sendo que, para todo $i \geq 0$, $u\ v^i\ z$ é palavra de **L**, o que é um absurdo, pois, como $|\ u\ v\ | \leq n$, **u v** obrigatoriamente é composta exclusivamente por símbolos **a**. Neste caso, por exemplo, $u\ v^2\ z$ não pertence a **L**, pois o número de símbolos **a** será maior que o número de símbolos **b**. ❑

4.3 → operações fechadas sobre as linguagens regulares

Operações sobre linguagens podem ser usadas para:

- construir novas linguagens regulares a partir de linguagens regulares conhecidas (definindo uma álgebra);
- provar propriedades;
- construir algoritmos.

Uma importante característica da classe de linguagens regulares é ser fechada para diversas operações, com destaque para:

- união;
- concatenação;
- complemento;
- intersecção.

No texto que segue, para uma determinada linguagem **L** sobre o alfabeto Σ, $\sim$**L** denota o seu complemento (em relação ao conjunto universo Σ^*).

teorema 4.3 – Operações fechadas sobre as linguagens regulares

A classe das linguagens regulares é fechada para as seguintes operações:

- união;
- concatenação;
- complemento;
- intersecção.

➥ **Prova:**
As provas referentes aos casos de união e concatenação decorrem trivialmente da definição de expressão regular (por quê?). Nos demais casos, valem que:

Complemento: (*direta*)
Suponha que **L** é uma linguagem regular sobre Σ^*. Então existe um **AFD**:

$$M = (\Sigma, Q, \delta, q_0, F)$$

tal que:

$$ACEITA(M) = L$$

A ideia do que segue consiste em inverter as condições de **ACEITA/REJEITA** de **M** para reconhecer **~L**. Entretanto, como **M** pode rejeitar por indefinição, é necessário modificar o autômato, garantindo que somente irá parar ao terminar de ler toda a entrada. Para tal, é suficiente introduzir um novo estado não final **d**, o qual será destino de todas as transições originalmente indefinidas. Um ciclo em **d** para todos os símbolos do alfabeto garante que o autômato terminará de ler toda a entrada. Por fim, a inversão das condições **ACEITA/REJEITA** pode ser obtida transformando os estados finais em não finais e vice-versa.

A construção do **AFD**:

$$M_C = (\Sigma, Q_C, \delta_C, q_0, F_C)$$

tal que:

$$ACEITA(M_C) = \sim L$$

é como segue (suponha $d \notin Q$):

$Q_C = Q \cup \{ d \}$
$F_C = Q_C - F$

δ_C é como δ, com as seguintes transições adicionais (para todo $a \in \Sigma$ e $q \in Q$):

$\delta_C(q, a) = d$ se $\delta(q, a)$ não é definida
$\delta_C(d, a) = d$

Claramente, o autômato finito M_C construído acima é tal que:

$$ACEITA(M_C) = \sim L \quad \text{ou seja} \quad ACEITA(M_C) = REJEITA(M)$$

Intersecção: (*direta*)
Suponha que L_1 e L_2 são linguagens regulares. Como consequência da propriedade de De-Morgan para conjuntos, a operação de intersecção pode ser expressa em termos das operações de união e de complemento. Em particular:

$$L_1 \cap L_2 = \sim(\sim L_1 \cup \sim L_2)$$

Como já foi verificado que a classe das linguagens regulares é fechada para as operações de complemento e de união, então também é fechada para a intersecção.

exemplo 4.3 – Complemento

Considere o autômato finito determinístico (com transições indefinidas):

$$M = (\{ a, b \}, \{ q_0, q_1, q_2, q_f \}, \delta, q_0, \{ q_f \})$$

ilustrado na figura 4.4, tal que:

$$ACEITA(M) = L$$

(qual a linguagem aceita?). Pela prova do teorema 4.3 – Operações fechadas sobre as linguagens regulares, o autômato finito determinístico:

$$M_C = (\{ a, b \}, \{ q_0, q_1, q_2, q_f, d \}, \delta_C, q_0, \{ q_0, q_1, q_2, d \})$$

ilustrado na figura 4.5 é tal que:

$$\text{ACEITA}(M_C) = \sim L \quad \text{ou seja} \quad \text{ACEITA}(M_C) = \text{REJEITA}(M) \qquad ❑$$

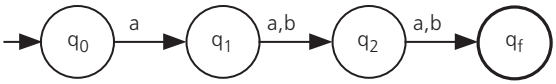


figura 4.4 Diagrama (**AFD**): aceita **L**.

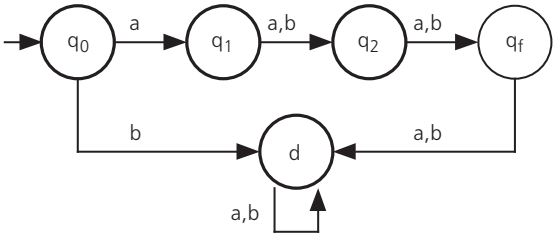


figura 4.5 Diagrama (**AFD**): aceita ~**L**.

4.4 investigação se uma linguagem regular é vazia, finita ou infinita

O teorema a seguir mostra que existe um algoritmo para verificar se uma linguagem regular representada por um autômato finito é vazia, finita ou infinita.

teorema 4.4 – Linguagem regular vazia, finita ou infinita

Se **L** é uma linguagem regular aceita por um autômato finito $M = (\Sigma, Q, \delta, q_0, F)$ com **n** estados (o cardinal de **Q** é **n**), então **L** é:

a *Vazia* se e somente se **M** não aceita qualquer palavra **w** tal que $|w| < n$

b *Finita* se e somente se **M** não aceita qualquer palavra **w** tal que $n \leq |w| < 2n$

c *Infinita* se e somente se **M** aceita pelo menos uma palavra **w** tal que $n \leq |w| < 2n$

➥ **Prova:**

Infinita (⇐ *direta*).

Para verificar se **M** aceita alguma palavra **w** tal que $n \leq |w| < 2n$ basta processar o autômato para todas as entradas neste intervalo de comprimento (o qual é finito). Se existe $w \in L$ tal que $n \leq |w| < 2n$, então, pelo lema 4.2 – Bombeamento para as linguagens regulares, **w** pode ser definida como $w = u\,v\,z$ tal que:

$$|u\,v| \leq n$$
$$|v| \geq 1$$

sendo que, para todo $i \geq 0$, $u\,v^i\,z$ é palavra de **L**

Logo, **L** é infinita.

Infinita (⇒).
Se **L** é infinita, então obviamente existe **w** tal que $|\mathbf{w}| \geq \mathbf{n}$. Assim:

Caso 1 (*direta*): $|\mathbf{w}| < \mathbf{2n}$
Se $|\mathbf{w}| < \mathbf{2n}$, então a prova está completa;

Caso 2 (*por absurdo*): $|\mathbf{w}| \geq \mathbf{2n}$
Suponha que não existe palavra de comprimento menor que **2n**. Então seja $|\mathbf{w}| \geq \mathbf{2n}$ mas de comprimento menor ou igual a qualquer outra palavra **t** tal que $|\mathbf{t}| \geq \mathbf{2n}$. Pelo lema 4.2 – Bombeamento para as linguagens regulares, **w** pode ser definida como $\mathbf{w = u\,v\,z}$ tal que:

$|\mathbf{u\,v}| \leq \mathbf{n}$
$|\mathbf{v}| \geq \mathbf{1}$

sendo que, para todo $\mathbf{i \geq 0}$, $\mathbf{u\,v^i\,z}$ é palavra de **L**. Em, particular, $\mathbf{1} \leq |\mathbf{v}| \leq \mathbf{n}$ (por quê?). Logo, **u z** é palavra de **L**, o que é um absurdo, pois:

a se $|\mathbf{u\,z}| \geq \mathbf{2n}$, então isso contradiz a suposição de que **w** é palavra de menor comprimento tal que $|\mathbf{w}| \geq \mathbf{2n}$

b se $|\mathbf{u\,z}| < \mathbf{2n}$, então $\mathbf{n} \leq |\mathbf{u\,z}| < \mathbf{2n}$ (pois $|\mathbf{u\,v\,z}| \geq \mathbf{2n}$, $\mathbf{1} \leq |\mathbf{v}| \leq \mathbf{n}$) e, portanto, isso contradiz a suposição de que não existe **w** tal que $\mathbf{n} \leq |\mathbf{w}| < \mathbf{2n}$

Vazia.

Processa **M** para todas as palavras de comprimento menor que **n**. Se rejeita todas as palavras, a linguagem é vazia. O detalhamento da prova é simples usando o lema 4.2 – Bombeamento para as linguagens regulares e é sugerida como exercício.

Finita (*por contraposição*).

Lembre-se que, por contraposição:

$$(\mathbf{p} \leftrightarrow \mathbf{q}) \Leftrightarrow (\neg\mathbf{p} \leftrightarrow \neg\mathbf{q})$$

Portanto, este é equivalente ao caso em que **L** é infinita, provado acima. Ou seja, processa **M** para toda palavra **w** de comprimento $\mathbf{n} \leq |\mathbf{w}| < \mathbf{2n}$. Se o autômato rejeita todas as palavras, então **L** é finita. ❑

exemplo 4.4 – Linguagem regular infinita

Considere o autômato ilustrado na figura 4.6 (qual a linguagem aceita?). Pelo teorema 4.4 – Linguagem regular vazia, finita ou infinita, a linguagem é infinita se, e somente se, o autômato aceita uma palavra **w** tal que $\mathbf{n} \leq |\mathbf{w}| < \mathbf{2n}$. Observe que **aabaa** (comprimento **5**) é a menor palavra aceita cujo comprimento é maior ou igual a **3**. Assim, como $\mathbf{3} \leq |\mathbf{aabaa}| < \mathbf{6}$, a linguagem é infinita. ❑

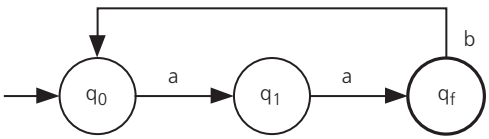


figura 4.6 Autômato finito determinístico.

4.5 igualdade de linguagens regulares

O teorema a seguir mostra que existe um algoritmo para verificar se dois autômatos finitos são equivalentes, ou seja, se reconhecem a mesma linguagem. Uma importante consequência deste resultado é que existe um algoritmo que permite verificar se duas implementações são equivalentes.

teorema 4.5 – Igualdade de linguagens regulares

Se M_1 e M_2 são autômatos finitos, então existe um algoritmo para determinar se:

$$ACEITA(M_1) = ACEITA(M_2)$$

➥ **Prova: (*direta*)**
Suponha M_1 e M_2 autômatos finitos tais que $ACEITA(M_1) = L_1$ e $ACEITA(M_2) = L_2$. Portanto, é possível construir um autômato finito M_3 tal que $ACEITA(M_3) = L_3$ onde:

$$L_3 = (L_1 \cap \sim L_2) \cup (\sim L_1 \cap L_2)$$

É fácil verificar que $L_1 = L_2$ se, e somente se, L_3 é vazia, o que sugere-se como exercício. Como visto no teorema 4.4 – Linguagem regular vazia, finita ou infinita, existe um algoritmo para verificar se uma linguagem regular é vazia. ❑

4.6 minimização de um autômato finito

A implementação de um simulador de autômato finito determinístico (sugerida como exercício) consiste, basicamente, em um algoritmo que controla a mudança de estado do autômato a cada símbolo lido da entrada. Este exercício ajuda a deixar claro que o tempo de processamento de um autômato finito determinístico necessário para aceitar ou rejeitar é diretamente proporcional ao tamanho da entrada. Relativamente à *complexidade de algoritmos*, autômatos finitos pertencem à classe de algoritmos mais eficientes em termos de tempo de processamento (supondo que toda a fita de entrada necessita ser lida).

Deve-se destacar que o tempo de processamento *não* depende do autômato de reconhecimento considerado, ou seja, *qualquer* autômato finito determinístico que reconheça a linguagem terá a *mesma eficiência* de tempo de processamento. Entretanto, uma otimização possível é a minimização do número de estados, ou seja, do espaço necessário para armazenar o autômato.

Dado um autômato finito determinístico qualquer, o objetivo da minimização é gerar o autômato finito equivalente com o menor número de estados possível, denominado de *autômato finito determinístico mínimo* ou simplesmente *autômato finito mínimo*. Entretanto, em algumas aplicações especiais, a minimização do número de estados não implica necessariamente o menor custo de implementação. Um exemplo típico seria o desenho de *circuitos lógicos* ou *redes lógicas*, quando pode ser desejável introduzir estados intermediários em determinadas posições (do circuito eletrônico), de forma a melhorar a eficiência, ou simplesmente facilitar as ligações físicas. Portanto, o algoritmo de minimização, nesses casos, deve ser modificado, prevendo as variáveis específicas da aplicação.

O autômato finito mínimo é único (a menos de isomorfismo). Assim, dois autômatos distintos que aceitam a mesma linguagem, ao serem minimizados, geram o mesmo autômato finito mínimo, diferenciando-se, eventualmente, na identificação dos estados.

Basicamente, o algoritmo de minimização unifica os estados equivalentes, definidos a seguir.

definição 4.6 – Estados equivalentes

Seja $\mathbf{M = (\Sigma, Q, \delta, q_0, F)}$ um autômato finito determinístico qualquer. Dois estados **q** e **p** de **Q** são ditos *Estados Equivalentes* se, e somente se, para qualquer palavra **w** pertencente a Σ^*, simultaneamente $\delta(\mathbf{q, w}) \in \mathbf{ACEITA(M)}$ e $\delta(\mathbf{p, w}) \in \mathbf{ACEITA(M)}$ ou simultaneamente $\delta(\mathbf{q, w}) \in \mathbf{REJEITA(M)}$ e $\delta(\mathbf{p, w}) \in \mathbf{REJEITA(M)}$. ❑

Portanto, o processamento de uma entrada qualquer a partir de estados equivalentes resulta na mesma condição de **ACEITA/REJEITA**.

Na definição que segue, lembre-se de que, para um dado conjunto **A**, **#A** denota o cardinal de **A**.

definição 4.7 – Autômato finito mínimo

Para uma dada linguagem regular **L**, o correspondente *autômato finito determinístico mínimo* ou simplesmente *autômato finito mínimo* é um autômato finito determinístico:

$$\mathbf{M_m = (\Sigma, Q_m, \delta_m, q_{0_m}, F_m)}$$

tal que $\mathbf{ACEITA(M_m) = L}$ e que, para qualquer outro autômato finito determinístico $\mathbf{M = (\Sigma, Q, \delta, q_0, F)}$ tal que $\mathbf{ACEITA(M) = L}$, ocorre:

$$\#Q \geq \#Q_m$$ ❑

observação 4.8 – Pré-requisitos do algoritmo de minimização

Um autômato finito a ser minimizado deve satisfazer os seguintes pré-requisitos:

a Deve ser determinístico;

b Todos os estados do autômato devem ser *estados alcançáveis* a partir do estado inicial. Ou seja, não pode ter *estados inacessíveis*;

c A função programa deve ser total (a partir de qualquer estado, são previstas transições para todos os símbolos do alfabeto).

Caso o autômato não satisfaça algum dos pré-requisitos, é necessário gerar um autômato equivalente, como segue, respectivamente:

a Gerar um autômato determinístico equivalente, usando os algoritmos de tradução apresentados nos correspondentes teoremas como ilustrado na figura 4.1;

b Eliminar os estados inacessíveis e suas correspondentes transições. Trata-se de um algoritmo relativamente simples que é sugerido como exercício;

c Para transformar a função programa em total, é suficiente introduzir um novo estado não final **d** e incluir as transições não previstas, tendo **d** como estado destino. Por fim, incluir um ciclo em **d** para todos os símbolos do alfabeto (uma construção análoga, mas com diferente objetivo, foi realizada na prova do teorema 4.3 – Operações fechadas sobre as linguagens regulares). ❑

O algoritmo apresentado a seguir identifica os estados equivalentes por exclusão. A partir de uma tabela de estados, são marcados os estados não equivalentes. Ao final do algoritmo, as referências não marcadas representam os estados equivalentes.

definição 4.9 – Algoritmo de minimização

Suponha um autômato finito determinístico $\mathbf{M = (\Sigma, Q, \delta, q_0, F)}$ que satisfaz à observação 4.8 – Pré-requisitos do algoritmo de minimização. Os passos do *algoritmo de minimização* são os seguintes:

Passo 1: Construção da tabela. Construir uma tabela, relacionando os estados distintos, sendo que cada par (não ordenado) de estados ocorre somente uma vez, como ilustrado na figura 4.7;

Passo 2: Marcação dos estados trivialmente não equivalentes. Marcar todos os pares do tipo **{ estado final, estado não final }**, pois, obviamente, estados finais não são equivalentes a não finais;

Passo 3: Marcação dos estados não equivalentes. Para cada par $\{ q_u, q_v \}$ não marcado e para cada símbolo $a \in \Sigma$, suponha que:

$$\delta(q_u, a) = p_u \quad \text{e} \quad \delta(q_v, a) = p_v$$

Assim:

a Se $p_u = p_v$, então q_u é equivalente a q_v para o símbolo **a** e não deve ser marcado;

b Se $p_u \neq p_v$ e o par $\{ p_u, p_v \}$ não está marcado, então $\{ q_u, q_v \}$ é incluído em uma lista a partir de $\{ p_u, p_v \}$ para posterior análise;

c Se $p_u \neq p_v$ e o par $\{ p_u, p_v \}$ está marcado, então:
- $\{ q_u, q_v \}$ não é equivalente e deve ser marcado;
- se $\{ q_u, q_v \}$ encabeça uma lista de pares, então marcar todos os pares da lista (e, recursivamente, se algum par da lista encabeça outra lista);

Passo 4: Unificação dos estados equivalentes. Os estados dos pares não marcados são equivalentes e podem ser unificados como segue:
- a equivalência de estados é transitiva;
- pares de estados não finais equivalentes podem ser unificados como um único estado não final;
- pares de estados finais equivalentes podem ser unificados como um único estado final;
- se algum dos estados equivalentes é inicial, então o correspondente estado unificado é inicial;
- todas as transições com origem (respectivamente, com destino) em um estado equivalente, são preservadas, mas passam a ter origem (respectivamente, destino) no correspondente estado unificado;

Passo 5: Exclusão dos estados inúteis. Por fim, os estados chamados inúteis devem ser excluídos. Um estado **q** é dito um *estado inútil* se é não final e a partir de **q** não é possível atingir um estado final. Deve-se observar que o estado **d** (se incluído) sempre é inútil (o algoritmo para excluir os estados inúteis é simples e é sugerido como exercício). Todas as transições com origem ou destino em um estado inútil são excluídas. ❑

q_1					
q_2					
...					
q_n					
d					
	q_0	q_1	...	q_{n-1}	q_n

figura 4.7 Minimização: tabela de estados.

exemplo 4.5 – Minimização

Considere o autômato finito determinístico ilustrado na figura 4.8 (qual a linguagem aceita?). O autômato satisfaz os pré-requisitos de minimização (e, consequentemente, não é necessário incluir o estado **d**). Os passos do algoritmo são como segue:

Passo 1. Construção da tabela, como ilustrado na figura 4.9;
Passo 2. Marcação dos pares do tipo **{ estado final, estado não final }**, como ilustrado na figura 4.9;
Passo 3. Análise dos pares de estado não marcados, sendo que a tabela resultante é ilustrada na figura 4.10, e sendo o símbolo ⊗ usado para representar os pares marcados nesta etapa:

a Análise do par $\{ q_0, q_4 \}$:

$$\delta(q_0, a) = q_2 \qquad \delta(q_0, b) = q_1$$
$$\delta(q_4, a) = q_3 \qquad \delta(q_4, b) = q_2$$

Como $\{ q_1, q_2 \}$ e $\{ q_2, q_3 \}$ são não marcados, então $\{ q_0, q_4 \}$ é incluído nas listas encabeçadas por $\{ q_1, q_2 \}$ e $\{ q_2, q_3 \}$;

b Análise do par $\{ q_0, q_5 \}$:

$$\delta(q_0, a) = q_2 \qquad \delta(q_0, b) = q_1$$
$$\delta(q_5, a) = q_2 \qquad \delta(q_5, b) = q_3$$

Como $\{ q_1, q_3 \}$ é não marcado (e como $\{ q_2, q_2 \}$ é trivialmente equivalente), então $\{ q_0, q_5 \}$ é incluído na lista encabeçada por $\{ q_1, q_3 \}$;

c Análise do par $\{ q_1, q_2 \}$:

$$\delta(q_1, a) = q_1 \qquad \delta(q_1, b) = q_0$$
$$\delta(q_2, a) = q_4 \qquad \delta(q_2, b) = q_5$$

Como { q_1, q_4 } é marcado, então { q_1, q_2 } também é marcado.
Como { q_1, q_2 } encabeça uma lista, o par { q_0, q_4 } também é marcado;

d Análise do par { q_1, q_3 }:

$$\delta(q_1, a) = q_1 \qquad \delta(q_1, b) = q_0$$
$$\delta(q_3, a) = q_5 \qquad \delta(q_3, b) = q_4$$

Como { q_1, q_5 } bem como { q_0, q_4 } são marcados, então { q_1, q_3 } também é marcado.
Como { q_1, q_3 } encabeça uma lista, o par { q_0, q_5 } também é marcado;

e Análise do par { q_2, q_3 }:

$$\delta(q_2, a) = q_4 \qquad \delta(q_2, b) = q_5$$
$$\delta(q_3, a) = q_5 \qquad \delta(q_3, b) = q_4$$

Como { q_4, q_5 } é não marcado, então { q_2, q_3 } é incluído na lista encabeçada por { q_4, q_5 };

f Análise do par { q_4, q_5 }:

$$\delta(q_4, a) = q_3 \qquad \delta(q_4, b) = q_2$$
$$\delta(q_5, a) = q_2 \qquad \delta(q_5, b) = q_3$$

Como { q_2, q_3 } é não marcado, então { q_4, q_5 } é incluído na lista encabeçada por { q_2, q_3 };

Passo 4. Como os pares { q_2, q_3 } e { q_4, q_5 } são não marcados, as seguintes unificações podem ser feitas:

- q_{23} representa a unificação dos estados não finais q_2 e q_3;
- q_{45} representa a unificação dos estados finais q_4 e q_5.

O autômato mínimo resultante possui quatro estados e é ilustrado na figura 4.11. Observe a preservação das transições cujos estados origem ou destino foram unificados. ❑

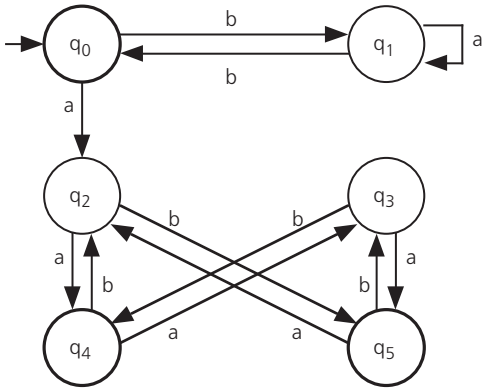


figura 4.8 Diagrama (**AFD**): autômato finito a ser minimizado.

q_1	×				
q_2	×				
q_3	×				
q_4		×	×	×	
q_5		×	×	×	
	q_0	q_1	q_2	q_3	q_4

figura 4.9 Minimização: tabela de estados com estados trivialmente não equivalentes marcados.

Os dois teoremas a seguir garantem que o autômato finito mínimo de uma linguagem é aquele construído pelo algoritmo de minimização apresentado e é único. Os teoremas não serão demonstrados

teorema 4.10 – Autômato finito mínimo

O autômato construído usando o algoritmo de minimização apresentado é um autômato finito determinístico com menor número de estados que aceita a linguagem. ❑

q_1	×				
q_2	×	⊗			
q_3	×	⊗			
q_4	⊗	×	×	×	
q_5	⊗	×	×	×	
	q_0	q_1	q_2	q_3	q_4

$\{q_0, q_5\}$

$\{q_0, q_4\}$

$\{q_0, q_4\} \rightarrow \{q_4, q_5\}$

$\{q_2, q_3\}$

figura 4.10 Minimização: tabela de estados resultante.

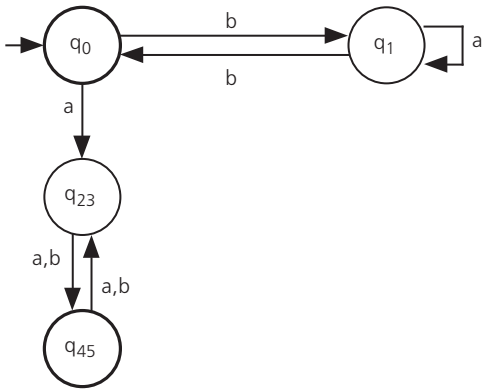


figura 4.11 Diagrama (**AFD**): autômato finito mínimo.

teorema 4.11 – Unicidade do autômato finito mínimo

O autômato finito determinístico mínimo de uma linguagem é único, a menos de isomorfismo. ❑

Assim, dois autômatos distintos que aceitam a mesma linguagem, ao serem minimizados, geram o mesmo autômato finito mínimo, diferenciando-se, eventualmente, na identificação dos estados. A definição formal de *isomorfismo de autômatos finitos* não será apresentada.

Como um autômato finito mínimo é único, a menos de isomorfismo, é usual ser referido como *o* (e não como *um*) autômato finito mínimo.

4.7 exercícios

exercício 4.1 Considere o autômato finito ilustrado na figura 4.12. Então, determine e justifique se as seguintes palavras na forma **w = u v z** satisfazem ao enunciado do lema 4.2 – Bombeamento para as linguagens regulares:

a **u = ε, v = aab, z = aab**

b **u = aa, v = baa, z = b**

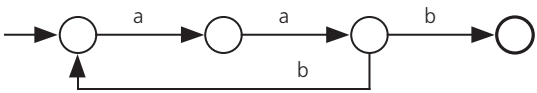


figura 4.12 Diagrama (**AFD**).

exercício 4.2 Para cada linguagem abaixo, desenvolva um correspondente autômato finito e exemplifique o lema 4.2 – Bombeamento para as linguagens regulares de forma a ilustrar a existência de mais de um bombeamento:

a $\{ a^n b^m \mid n \geq 0 \text{ e } m \geq 0 \}$

b $\{ a^n b^m a^r \mid n \geq 0, m \geq 0 \text{ e } r \geq 0 \}$

exercício 4.3 Prove que as seguintes linguagens *não* são regulares (suponha que $n \in \mathbf{N}$ e $m \in \mathbf{N}$):

a $\{ ww \mid w \text{ é palavra de } \{ a, b \}^* \}$

b $\{ a^n b^n \}$

c $\{ w \mid (w = a^n b^m \text{ ou } w = a^m b^n), n \geq m \}$

exercício 4.4 Justifique a seguinte afirmação que seria um corolário do lema 4.2 – Bombeamento para as linguagens regulares: o lema de bombeamento garante que, se uma linguagem regular é infinita, então existe um conjunto finito de palavras que podem gerar todas as demais usando "bombeamentos"

exercício 4.5 No teorema 4.3 – Operações fechadas sobre as linguagens regulares, por que a prova referente aos casos de união e de concatenação decorrem trivialmente da definição de expressão regular?

exercício 4.6 Algumas importantes propriedades podem ser demonstradas usando expressões regulares e as correspondentes linguagens (conjuntos) denotadas. Assim, para as seguintes propriedades das linguagens regulares, desenvolva provas baseadas em expressões regulares (suponha que **r**, **s** e **t** são expressões regulares):

a *Comutatividade da união*.

$r + s = s + r$

b *Associatividade da união*.

$(r + s) + t = r + (s + t)$

c *Não comutatividade da concatenação*.

$rs = sr$ pode não ser satisfeita

d *Associatividade da concatenação*.

$(rs)t = r(st)$

e *Distributividade (à esquerda e à direita) da concatenação sobre a união*.

$t(r + s) = tr + ts$

$(r + s)t = rt + st$

f *Dupla concatenação sucessiva*.

$(r^*)^* = r^*$

g *Outras propriedades da concatenação sucessiva*.

$(r + \varepsilon)^* = r^*$

$(r^*s^*)^* = (r + s)^*$

exercício 4.7 Prove que as linguagens regulares são fechadas para o produto cartesiano, ou seja, se L_1 e L_2 são linguagens regulares, então:

$$L_1 \times L_2 = \{ (w_1, w_2) \mid w_1 \in L_1 \text{ e } w_2 \in L_2 \}$$

é linguagem regular.

Dica: detalhe o alfabeto da linguagem resultante.

exercício 4.8 Refute a seguinte afirmação:

se uma linguagem é regular, então qualquer sublinguagem desta também é regular.

exercício 4.9 Relativamente ao teorema 4.5 – Igualdade de linguagens regulares, prove que

$L_1 = L_2$ *se, e somente se,* $(L_1 \cap \sim L_2) \cup (\sim L_1 \cap L_2)$ *é vazia.*

exercício 4.10 Relativamente à observação 4.8 – Pré-requisitos do algoritmo de minimização, desenvolva um algoritmo que elimine os *estados inacessíveis* de um autômato finito determinístico.

exercício 4.11 Minimize os autômatos finitos ilustrados na figura 4.13.

Dica: não esqueça de verificar os pré-requisitos do algoritmo de minimização.

exercício 4.12 Minimize o autômato finito ilustrados na figura 4.14.

Dica: especial atenção aos pré-requisitos do algoritmo de minimização.

exercício 4.13 Dado um alfabeto Σ, usando o algoritmo de minimização, determine o autômato finito mínimo que aceita as seguintes linguagens:

a Linguagem vazia;

b Linguagem cuja única palavra é a palavra vazia.

exercício 4.14 Por que os itens da observação 4.8 – Pré-requisitos do algoritmo de minimização devem ser verificados antes de realizar uma minimização? A resposta deve ter como base uma análise sobre a definição 4.9 – Algoritmo de minimização.

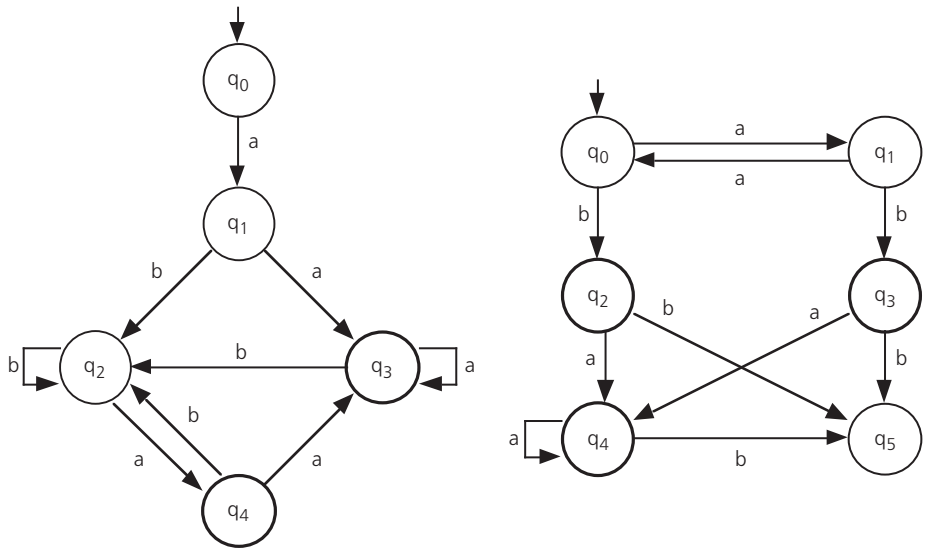


figura 4.13 Diagramas (**AFD**): autômatos finitos a serem minimizados.

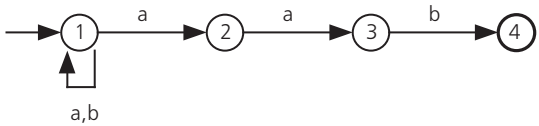


figura 4.14 Diagrama: autômato finito a ser minimizado.

exercício 4.15 Desenvolva algoritmos mais otimizados que os apresentados (e compare a eficiência) para determinar se uma linguagem regular é vazia, finita ou infinita, usando as seguintes sugestões:

a Vazia, eliminando os estados inacessíveis;

b Finita, combinando os algoritmos de minimização e aquele apresentado no lema 4.2 – Bombeamento para as linguagens regulares;

c Infinita, analisando a função programa do autômato minimizado.

exercício 4.16 Como pode ser verificada a equivalência de dois autômatos finitos, usando o algoritmo de minimização?

exercício 4.17 Como pode ser verificado se duas linguagens regulares são complementares?

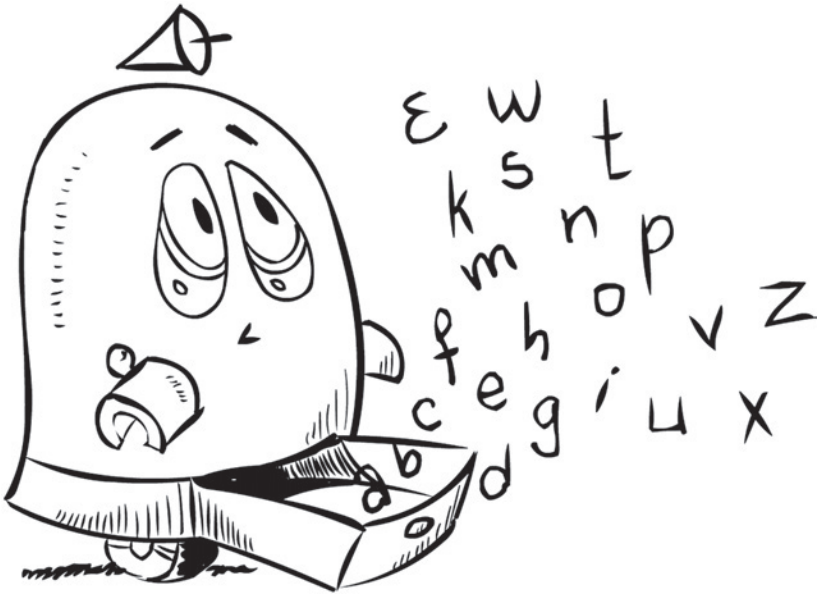
exercício 4.18 Desenvolva um programa em computador que implemente em computador a minimização de um autômato finito determinístico.

Dica: não esqueça dos pré-requisitos do algoritmo de minimização.

capítulo 5

autômato finito com saída

Autômato finito possui aplicações práticas restritas, pois a saída é limitada à informação aceita/rejeita. Assim, a definição de autômato finito é estendida prevendo saídas associadas aos estados (máquina de Moore) ou às transições (máquina de Mealy). São apresentadas aplicações com destaque para hipertexto, hipermídia e animações.



O conceito básico de autômato finito possui aplicações práticas restritas, pois a informação de saída é limitada à lógica binária aceita/rejeita. Sem alterar a classe de linguagens reconhecidas, é possível estender a definição de autômato finito, incluindo-se a geração de uma palavra de saída. As saídas podem ser associadas:

- às transições (*máquina de Mealy*);
- aos estados (*máquina de Moore*).

Em ambas as máquinas, a saída não pode ser lida, ou seja, não pode ser usada como memória auxiliar, e é como segue:

- é definida sobre um alfabeto especial, denominado *alfabeto de símbolos de saída*, o qual pode ser igual ao alfabeto de entrada;
- a saída é armazenada em uma *fita de saída* independente da de entrada;
- a *cabeça da fita de saída* move uma célula para a direita a cada símbolo gravado;
- o resultado do processamento do autômato finito é o seu estado final (condição de aceita/rejeita) e a informação contida na fita de saída.

As máquinas de Mealy e Moore são modificações sobre o autômato finito determinístico. A extensão da definição, prevendo as facilidades de não determinismo e movimentos vazios, é sugerida como exercício.

Independentemente de aplicações tradicionais dos autômatos finitos com saída, duas aplicações, com ênfase na *www* (*world wide web*), são destacadas:

- *hipertexto* e *hipermídia*;
- *animação quadro a quadro*.

5.1 → máquina de Mealy

A máquina de Mealy é um autômato finito modificado de forma a gerar uma palavra de saída (a qual pode ser vazia) para cada transição da máquina.

definição 5.1 – Máquina de Mealy

Uma máquina de Mealy **M** é um autômato finito determinístico com saídas associadas às transições. É representada por uma **6**-upla:

$$M = (\Sigma, Q, \delta, q_0, F, \Delta)$$

na qual:

a Σ é um *alfabeto de símbolos de entrada*, ou simplesmente *alfabeto de entrada*;

b **Q** é um *conjunto de estados* possíveis do autômato o qual é finito;

c δ é uma *função programa* ou *função de transição*:

$$\delta: Q \times \Sigma \rightarrow Q \times \Delta^*$$

a qual é uma função parcial;

d q_0 é um elemento distinguido de **Q**, denominado *estado inicial*;

e **F** é um subconjunto de **Q**, denominado *conjunto de estados finais*;

f Δ é um *alfabeto de símbolos de saída*, ou simplesmente *alfabeto de saída*. ❑

Portanto, as componentes Σ, **Q**, q_0 e **F** são como no autômato finito determinístico. A função programa pode ser representada como um diagrama como nos autômatos finitos, adicionando, na etiqueta de cada transição, a saída associada, quando diferente da palavra vazia.

A *computação* de uma máquina de Mealy, para uma palavra de entrada **w**, consiste na sucessiva aplicação da função programa para cada símbolo de **w** (da esquerda para a direita) até ocorrer uma condição de parada. A palavra vazia como saída da função programa indica que nenhuma gravação é realizada e, obviamente, não move a cabeça da fita de saída. Se todas as transições geram saída vazia, então a máquina de Mealy processa como se fosse um autômato finito. A definição formal da função programa estendida de uma máquina de Mealy é sugerida como exercício.

exemplo 5.1 – Máquina de Mealy: diálogo

Uma aplicação comum e frequentemente recomendada para os autômatos com saída é o projeto de diálogo entre um programa (de computador) e o seu usuário, gerando, eventualmente, ações internas ao sistema. Basicamente, um diálogo pode ser de dois tipos:

- comandado pelo programa;
- comandado pelo usuário.

Em qualquer caso, uma das principais dificuldades do projetista é a visualização do conjunto de eventos e ações que definam um diálogo adequado para as diversas situações possíveis.

O exemplo que segue é uma máquina de Mealy que trata algumas situações típicas de um diálogo que cria e atualiza arquivos. A seguinte simbologia é adotada no diagrama da função de transição:

⟨…⟩ entrada fornecida pelo usuário (em um teclado, por exemplo);
"…" saída gerada pelo programa (em um vídeo, por exemplo);
[…] ação interna ao programa, sem comunicação com o usuário;
(…) resultado de uma ação interna ao programa; é usado como entrada no diagrama.

A máquina de Mealy:

$$M = (\Sigma, \{ q_0, q_1, \ldots, q_8, q_f \}, \delta, q_0, \{ q_f \}, \Delta)$$

é como ilustrado na figura 5.1, na qual Σ e Δ representam o conjunto de símbolos (palavras do português) de entrada e de saída válidos no diálogo. ❑

5.2 máquina de Moore

A máquina de Moore possui uma segunda função, que gera uma palavra de saída (a qual pode ser vazia) para cada estado da máquina.

definição 5.2 – Máquina de Moore

Uma máquina de Moore **M** é um autômato finito determinístico com saídas associadas aos estados. É representada por uma **7**-upla:

$$M = (\Sigma, Q, \delta, q_0, F, \Delta, \delta_S)$$

na qual:

a Σ é um *alfabeto de símbolos de entrada*, ou simplesmente *alfabeto de entrada*;
b **Q** é um *conjunto de estados* possíveis do autômato o qual é finito;
c δ é uma *função programa* ou *função de transição*:

$$\delta: Q \times \Sigma \rightarrow Q$$

a qual é uma função parcial;
d q_0 é um elemento distinguido de **Q**, denominado *estado inicial*;
e **F** é um subconjunto de **Q**, denominado *conjunto de estados finais*;
f Δ é um *alfabeto de símbolos de saída*, ou simplesmente *alfabeto de saída*;
g δ_S é uma *função de saída*:

$$\delta_S: Q \rightarrow \Delta^*$$

a qual é uma função total. ❑

Portanto, as componentes Σ, **Q**, δ, q_0 e **F** são como no autômato finito determinístico, e Δ é como na máquina de Mealy. A função programa pode ser representada como um diagrama como nos autômatos finitos, adicionando, na etiqueta de cada estado, a saída associada, quando diferente da palavra vazia.

A *computação* de uma máquina de Moore, para uma palavra de entrada **w**, consiste na sucessiva aplicação da função programa para cada símbolo de **w** (da esquerda para a direita) até ocorrer uma condição de parada, juntamente com a sucessiva aplicação da função de saída a cada estado atingido. A palavra vazia como resultado da função de saída indica que nenhuma gravação é realizada e, obviamente, não move a cabeça da fita de saída. Se todos os estados geram saída vazia, então a máquina de Moore processa como se fosse um autômato finito. A definição formal da função programa estendida de uma máquina de Moore é sugerida como exercício.

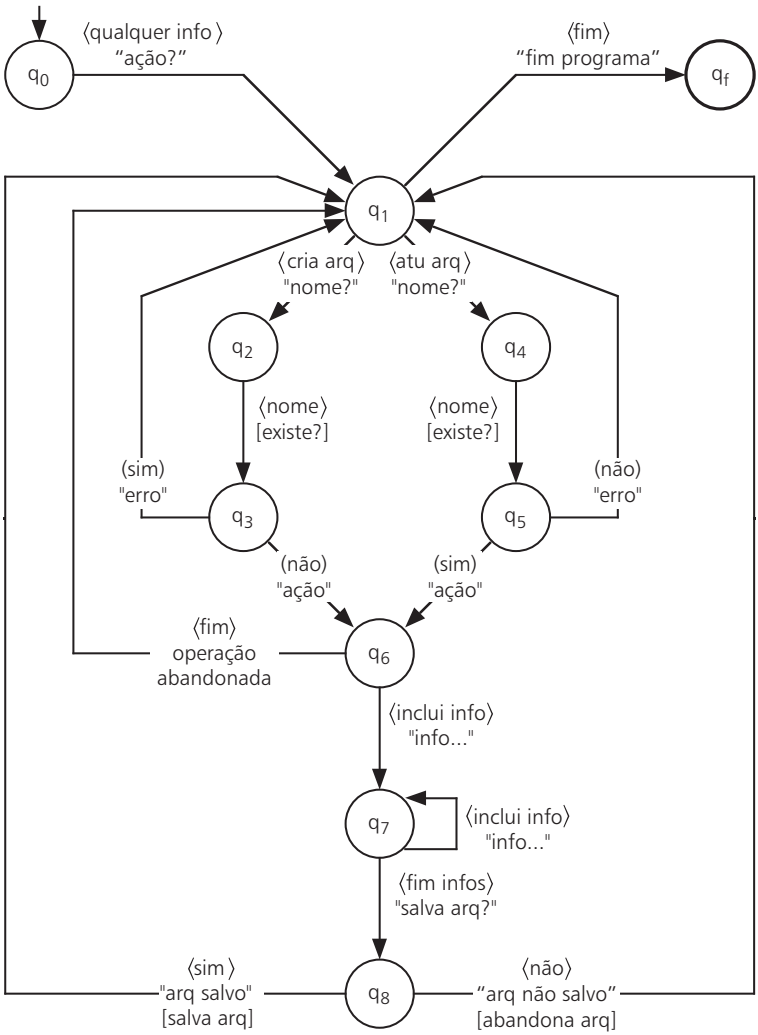


figura 5.1 Diagrama (máquina de Mealy): diálogo.

exemplo 5.2 – Máquina de Moore: análise léxica

Um exemplo comum de aplicação do conceito de máquina de Moore é o desenvolvimento de analisadores léxicos de compiladores ou tradutores de linguagens em geral. Basicamente, um analisador léxico é um autômato finito (em geral, determinístico) que identifica os componentes básicos da linguagem como, por exemplo, números, identificadores, separadores, etc. Uma máquina de Moore como um analisador léxico é como segue:

- um estado final é associado a cada unidade léxica;
- cada estado final possui uma saída (definida pela Função de Saída) que descreve ou codifica a unidade léxica identificada;
- para os demais estados (não finais), em geral, a saída gerada é a palavra vazia. Eventualmente pode ser não vazia, se alguma informação adicional à codificação da unidade léxica é necessária. ❑

5.3 → equivalência das máquinas de Moore e de Mealy

A equivalência dos dois modelos de autômato finito com saída não é válida para a entrada vazia. Neste caso, enquanto a máquina de Moore gera a palavra correspondente ao estado inicial, a máquina de Mealy não gera qualquer saída, pois não executa transição alguma. Entretanto, para os demais casos, a equivalência pode ser facilmente mostrada.

teorema 5.3 – Máquina de Moore → máquina de Mealy

Toda máquina de Moore pode ser simulada por uma máquina de Mealy, para entradas não vazias.

➥ **Prova: (*por indução*)**
Suponha $\mathbf{M} = (\Sigma, \mathbf{Q}, \delta, \mathbf{q_0}, \mathbf{F}, \Delta, \delta_S)$, uma máquina de Moore qualquer. Seja (suponha que $\mathbf{q_e}$ não é um estado de $\mathbf{Q}$):

$$\mathbf{ME} = (\Sigma, \mathbf{Q} \cup \{ \mathbf{q_e} \}, \delta_{ME}, \mathbf{q_e}, \mathbf{F}, \Delta)$$

uma máquina de Mealy na qual a função δ_{ME} é definida como segue (suponha $\mathbf{q}$ um estado de $\mathbf{Q}$ e $\mathbf{a}$ um símbolo de Σ):

a $\delta_{ME}(q_e, a) = (\delta(q_0, a), \delta_S(q_0)\delta_S(\delta(q_0, a)))$
b $\delta_{ME}(q, a) = (\delta(q, a), \delta_S(\delta(q, a)))$

Em b), é construída a função programa da máquina de Mealy a partir das funções de transição e de saída da máquina de Moore. O estado $\mathbf{q_e}$ introduzido em a) é referenciado somente na primeira transição a ser executada. Seu objetivo é garantir a geração da saída referente ao estado inicial $\mathbf{q_0}$ de Moore (lembre-se de que Mealy necessita executar a transição para gerar a saída), como ilustrado na figura 5.2.

Uma indução em $\mathbf{n > 0}$ prova que, ao reconhecer a entrada $\mathbf{a_1 \ldots a_n}$, se $\mathbf{M}$ passa pelos estados $\mathbf{q_0}, \mathbf{q_1}, \ldots, \mathbf{q_n}$ e gera as saídas $\mathbf{u_0}, \mathbf{u_1}, \ldots, \mathbf{u_n}$, então $\mathbf{ME}$ passa pelos estados $\mathbf{q_e}, \mathbf{q_0}, \mathbf{q_1}, \ldots, \mathbf{q_n}$ e gera as saídas $\mathbf{u_0 u_1}, \ldots, \mathbf{u_n}$. ❑

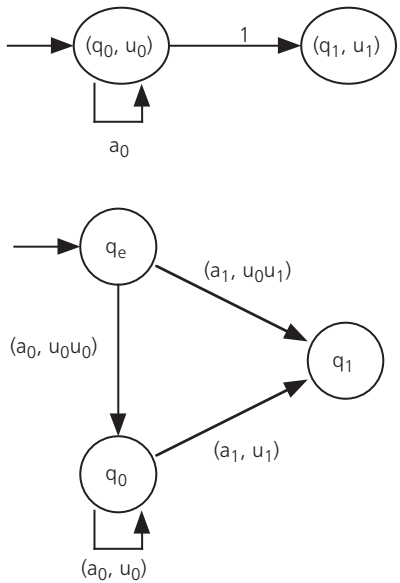


figura 5.2 Diagramas: Máquina de Moore (acima) e correspondente máquina de Mealy (abaixo).

Para facilitar a visualização e evitar confusões, no texto que segue, um par ordenado **(a, b)** pode ser equivalentemente representado por ⟨**a, b**⟩.

teorema 5.4 – Máquina de Mealy → máquina de Moore

Toda máquina de Mealy pode ser simulada por uma máquina de Moore.

➥ **Prova: (*por indução*)**
Suponha $\mathbf{M} = (\Sigma, Q, \delta, q_0, F, \Delta)$, uma máquina de Mealy qualquer. Seja $\mathbf{S}(\delta)$ a imagem da função programa δ restrita à componente da palavra de saída (ou seja, o conjunto de todas as saídas possíveis de **M**).

A construção da máquina de Moore:

$$\mathbf{MO} = (\Sigma, (Q \times S(\delta)) \cup \{ \langle q_0, \varepsilon \rangle \}, \delta_{MO}, \langle q_0, \varepsilon \rangle, F \times S(\delta), \Delta, \delta_S)$$

correspondente determina, em geral, um maior número de estados que a máquina **M** que está sendo simulada. Isto é necessário, pois, se diversas transições com saídas diferentes atingem um mesmo estado, este necessita ser simulado por diversos estados, um para cada palavra de saída. Ou seja, em Moore, os estados são construídos como um par ordenado, no qual a segunda componente representa a palavra de saída, como ilustrado na Figura 5.3.

Assim, as funções da máquina de Moore podem ser construídas a partir de δ, e a função programa δ_{MO} é como segue:

- para **a** em Σ, se $\delta(q_0, a) = (q, u)$, então:

$$\delta_{MO}(\langle q_0, \varepsilon \rangle, a) = \langle q, u \rangle$$

- para **b** em Σ e para **q** em **Q**, se $\delta(q, b) = (p, v)$, então, para a_i em Σ e para q_i em **Q** tais que $\delta(q_i, a_i) = (q, u_i)$, vale que:

$$\delta_{MO}(\langle q, u_i\rangle, b) = \langle p, v\rangle$$

e a função de saída δ_S é tal que, para o estado $\langle q, u\rangle$ de **MO**:

$$\delta_S(\langle q, u\rangle) = u$$

Uma indução em **n** prova que, ao reconhecer a entrada $a_1 \ldots a_n$, se **M** passa pelos estados $q_0, q_1, \ldots, q_n$ e gera as saídas $u_1, \ldots, u_n$ então **MO** passa pelos estados $\langle q_0, \varepsilon\rangle, \langle q_1, u_1\rangle, \ldots, \langle q_n, u_n\rangle$ e gera as saídas $\varepsilon, u_1, \ldots, u_n$. ❑

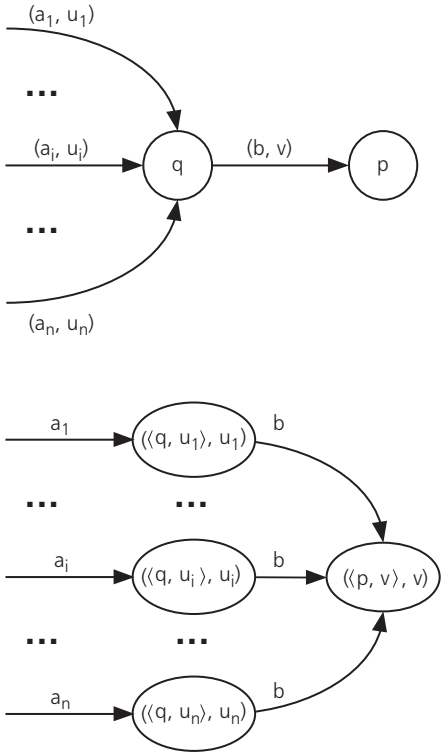


figura 5.3 Diagramas: máquina de Mealy (acima) e correspondente máquina de Moore (abaixo).

exemplo 5.3 – Máquina de Mealy → máquina de Moore

Considere a máquina de Mealy $M = (\{ a, \beta \}, \{ q, p \}, \delta, q, \{ q, p \}, \{ a, \beta \})$ que compacta os brancos de um texto, na qual **a** representa um símbolo qualquer do texto e β o símbolo branco, como ilustrado na figura 5.4 (na etiqueta de uma transição, a primeira componente representa o símbolo lido, e a segunda, a palavra gravada).

A máquina de Moore construída conforme a prova do teorema 5.4 – Máquina de Mealy → máquina de Moore é **MO = ({ a, β }, Q, δ_{MO}, ⟨q, ε⟩, F, { a, β }, δ_S)**, tal que **Q = F = { q, p } × { ε, a, β }**, ilustrada na figura 5.5, em que a segunda componente de cada estado representa a saída. ❑

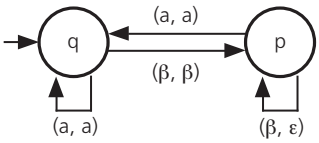


figura 5.4 Diagrama: máquina de Mealy.

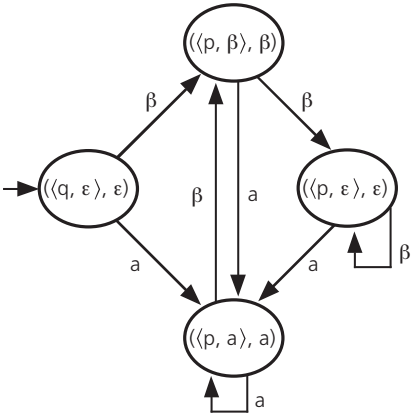


figura 5.5 Diagrama: correspondente máquina de Moore.

observação 5.5 – Máquina de Mealy × máquina de Moore

Na prova do teorema 5.4 – Máquina de Mealy → máquina de Moore, foi destacado que a máquina de Moore possui, em geral, mais estados que a correspondente máquina de Mealy. Por esta razão, em aplicações práticas, sempre que possível, deve-se usar a máquina de Mealy preferencialmente à máquina de Moore. Entretanto, em experimentos reais, observou-se uma significativa preferência das pessoas a associar as saídas aos estados (e não às transições). Assim, sugere-se especial atenção a este fato, principalmente nos primeiros exercícios. ❑

5.4 → hipertexto e hipermídia como autômato finito com saída

Um *hipertexto* é um texto cuja principal característica é a existência de ponteiros ou *links* entre suas diversas páginas. Assim, certas partes do texto possuem "âncoras" as quais apontam

para outras páginas do documento. *Hipermídia* é extensão desta noção para recursos *multimídia* como, por exemplo, imagens, animações e sons.

A noção de hipertexto foi proposta por Vannevar Bush em **1945** com o objetivo de armazenar uma grande quantidade de documentos interligados de acordo com uma semântica de associação, objetivando flexibilizar e otimizar os tempos de recuperação de informações.

A associação dos conceitos de hipertexto e de hipermídia à *www* (*world wide web*) resultou em documentos nos quais, em geral, os ponteiros são fisicamente codificados nas páginas. Entretanto, tal solução compromete a reusabilidade e a atualização dos recursos utilizados na composição do hiperdocumento. De fato, idealmente, um hipertexto (ou hipermídia) deve possuir estrutura navegacional independente da base de dados sobre a qual é construído.

Neste contexto, hipertextos e hipermídias podem ser vistos como autômatos finitos com saída, nos quais:

a O alfabeto de entrada corresponde ao conjunto de rótulos dos ponteiros;

b A função programa corresponde à estrutura navegacional;

c O alfabeto de saída corresponde ao conjunto de recursos hipertexto/hipermídia armazenados na base de dados;

d Cada palavra de saída gerada corresponde a uma página, composta por símbolos do alfabeto de saída (recursos hipertexto/hipermídia) "concatenados".

Assim, cada autômato define um hipertexto/hipermídia o qual consiste de um conjunto de recursos independentes disponibilizados na base de dados. Mesmos recursos podem ser usados em diferentes hipertextos/hipermídias. A função de transição determina a estruturação lógica, e as palavras de saída geradas constituem as páginas. O resultado é a estrutura de páginas e ponteiros de um hipertexto/hipermídia em um *sítio* (ou *site*) na www. Cada autômato com saída corresponde a uma diferente visão da mesma base de dados.

Uma hipermídia vista como um autômato finito com saída pode possuir restrições nos controles de tempos e/ou sincronizações entre diversas mídias, decorrentes das limitações de expressividade das linguagens regulares (e, consequentemente, nas limitações sobre o que os autômatos finitos podem computar).

exemplo 5.4 – Hiperdocumento como autômato finito com saída

Suponha um hipertexto com objetivo de disponibilizar um curso sobre autômatos com saída na www. Seguindo a abordagem de hipertextos vistos como autômatos finitos com saída, é ilustrado, na figura 5.6, um exemplo de máquina de Moore com tal objetivo, na qual:

a O alfabeto de entrada é o seguinte conjunto:

{ próxima, exercício, anterior, resumos, saída }

b O alfabeto de saída é o seguinte conjunto:

{ A, B, C, D, E, F, G, H, I, J, K, L, M }

sendo que cada letra corresponde ao seguinte fragmento de hipertexto na base de dados:

A Introdução aos autômatos finitos
B Definição de autômato finito determinístico
C Exemplo de autômato finito determinístico
D Exercício sobre autômato finito determinístico
E Introdução aos autômatos finitos com saída
F Definição da máquina de Mealy
G Exemplo de máquina de Mealy
H Exercício sobre máquina de Mealy
I Definição da máquina de Moore
J Exemplo de máquina de Moore
K Exercício sobre máquina de Moore
L Conclusões
M Fim

Observe que:

- os fragmentos de hipertextos são concatenados, compondo páginas;
- mesmos fragmentos são usados em mais de uma página, facilitando o reuso de fragmentos de hipertextos;
- se um fragmento for alterado na base de dados, todas as suas referências são automaticamente alteradas no autômato;
- símbolos do alfabeto de entrada são rótulos de ponteiros.

Sugere-se como exercício o desenvolvimento de uma solução baseada em máquina de Mealy com o mesmo objetivo. ❑

A estruturação de um hipertexto/hipermídia como um autômato finito com saída possui as seguintes vantagens:

- alto grau de modularização dos recursos que constituem a base de dados;
- facilidade de reuso desses recursos;
- independência da estrutura navegacional (função de transição) do conteúdo das páginas. Assim, modificações na estrutura navegacional não influem no conteúdo das páginas e vice-versa;
- facilidade de criação e de manutenção de hipertextos/hipermídias;
- facilidade de criação de hipertextos/hipermídias sobre um hipertexto/hipermídia já existente;
- interface gráfica simples e direta (decorrente da representação de um autômato finito como um diagrama);
- facilidade de implementação (lembre-se de que a implementação de um simulador de autômatos finitos é trivial).

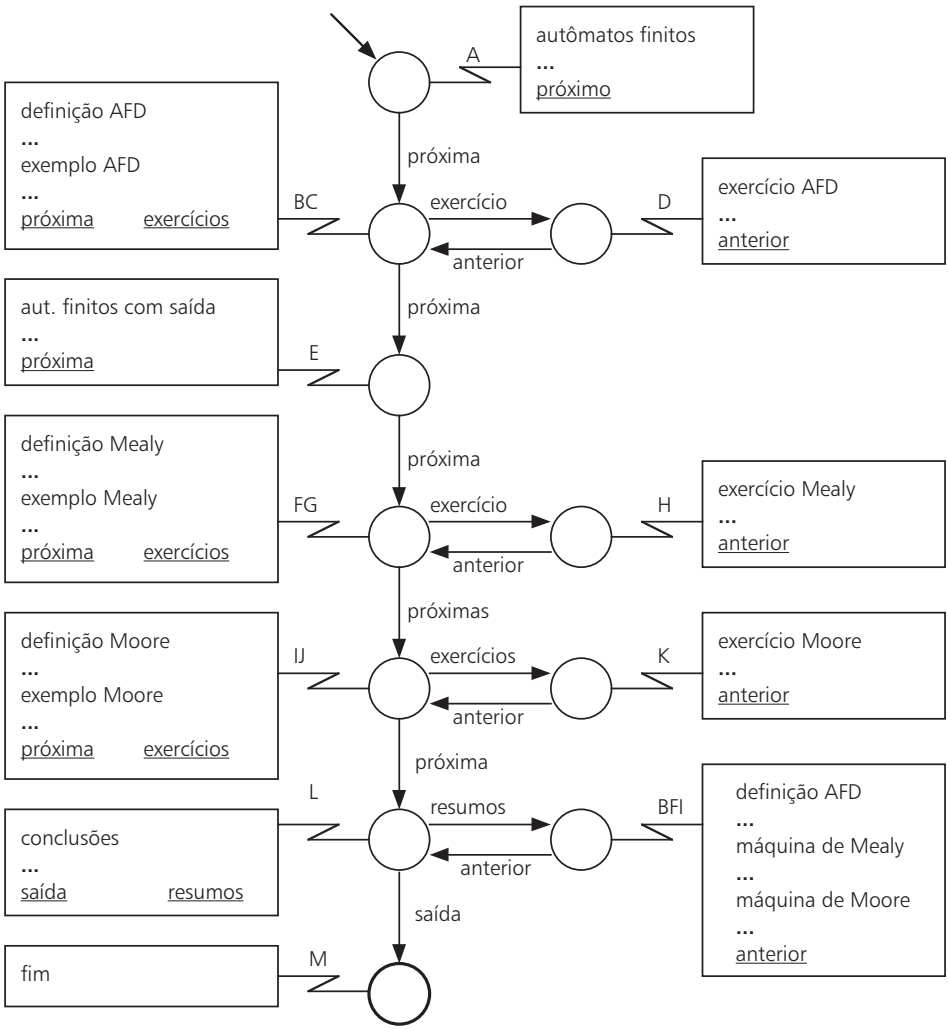


figura 5.6 Diagrama (**máquina de Moore**): hipertexto como autômato finito com saída.

Uma interessante generalização deste modelo é obtida, usando-se a facilidade de não determinismo, cuja interpretação no contexto de hipertextos/hipermídias na www é sugerida como exercício.

5.5 → animação como autômato finito com saída

Os *sistemas de animação* usados para criação e apresentação de animações podem ser classificados em dois tipos, a saber:

a *Tempo real*, no qual cada imagem a ser exibida é computada no momento de sua visualização;

b *Quadro-a-quadro*, no qual cada imagem a ser exibida é previamente computada e armazenada.

Correntemente, sistemas de animação são especialmente importantes no contexto da *world wide web*, pois grande parte de seu conteúdo contém animações. Portanto questões como taxa de transmissão, espaço de armazenamento e tempo de processamento são importantes.

Entre os sistemas de animação quadro-a-quadro encontrados na www, destacam-se:

- AVI – *audio video interleave*;
- MPEG – *moving picture expert group*;
- QuickTime;
- GIF – *graphics interchange format*.

Entre as características desejáveis de um sistema de animação, destacam-se:

- reutilização de uma sequência de imagens ou de partes específicas de imagens para compor animações a partir de animações existentes;
- busca de informações sobre a ocorrência de determinadas condições ao longo da animação. Trata-se de uma característica especialmente importante em animações mais complexas.

Analogamente aos hipertextos e hipermídias, animações quadro-a-quadro podem ser vistas como autômatos finitos com saída. Neste contexto, cada autômato corresponde a um ator, e a composição de atores em camadas constituem animações. Cada ator possui uma fita de entrada independente. Assim, para cada ator:

a O alfabeto de saída corresponde ao conjunto de imagens e sons elementares do ator;

b Cada palavra de saída gerada corresponde a uma imagem e/ou som do ator, a cada instante da animação;

c O alfabeto de entrada corresponde ao conjunto de ações possíveis;

d A função programa corresponde ao comportamento do ator.

É desejável estender o modelo de autômato com saída, prevendo facilidades específicas para animações, como controle de tempos e transformações aplicadas a imagem ou som. Uma solução é prever que cada célula de fita de entrada é composta por uma tripla ordenada, contendo:

- um símbolo do alfabeto de entrada;
- o tempo de processamento da transição (e consequentemente, da exibição da correspondente imagem);

- a transformação aplicada à imagem (posicionamento, rotação, etc.) e/ou ao som (volume, equalização, etc.) de saída.

exemplo 5.5 – Animação como autômato finito com saída

Considere os seguintes atores:

- uma cobra capaz de se movimentar, abocanhar e rir;
- uma maçã que pode estar ou não mordida;

Suponha que é desejada uma animação composta pelos dois atores, na qual a cobra eventualmente abocanha a maçã. Na figura 5.7, é ilustrada uma solução baseada em máquina de Mealy com tal objetivo. Na figura 5.8, é ilustrada uma possível animação. Observe que:

- as imagens dos atores são organizadas em camadas, compondo quadros das animações;
- a sincronização dos atores é dada por controle de tempos;
- se algum símbolo do alfabeto de saída (imagem elementar) de um dos atores for alterado, todas as suas referências são automaticamente alteradas;
- cada um dos atores pode ser reusado na composição de uma outra animação;
- um mesmo ator pode ser usado diversas vezes em uma mesma animação. Por exemplo, suponha uma animação com diversas cobras agindo independentemente.

Sugere-se como exercício o desenvolvimento de uma solução baseada em máquina de Moore com o mesmo objetivo ❑

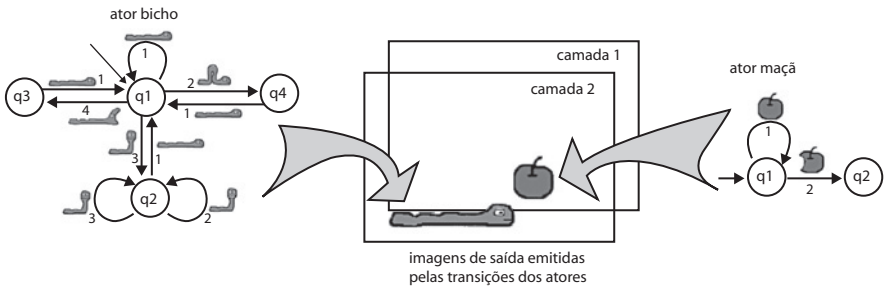


figura 5.7 Diagrama (**máquinas de Mealy**): animações como autômato finito com saída.

A estruturação de uma animação como um autômato finito com saída possui as seguintes vantagens, as quais são análogas às do hipertexto/hipermídia:

- o encapsulamento das propriedades estéticas e comportamentais em uma unidade básica (ator) favorece que estes sejam reusados em diferentes animações ou instanciando mais de uma mesma animação. Em qualquer caso, existe apenas um autômato (e diversas fitas de entrada, uma para cada instância);

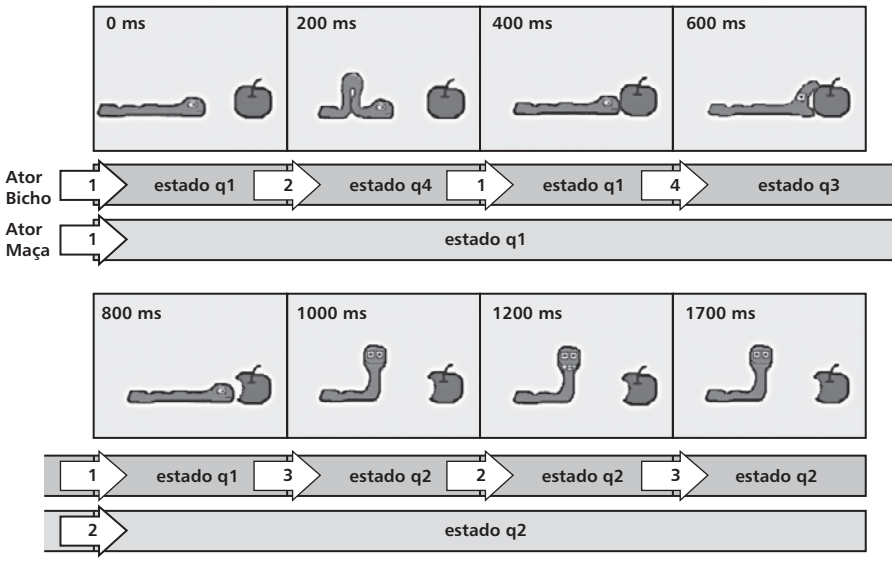


figura 5.8 Animação composta pelos atores cobra e maçã.

- independência da estrutura comportamental (função de transição) do conteúdo das imagens/sons. Assim, modificações na estrutura comportamental não influem no conteúdo das imagens/sons e vice-versa;
- facilidade de criação e de manutenção de atores e animações;
- facilidade de criação de atores/animações sobre atores/animações já existentes;
- interface gráfica simples e direta (decorrente da representação de um autômato finito como um diagrama);
- facilidade de implementação (lembre-se de que a implementação de um simulador de autômatos finitos é trivial).

Adicionalmente, existe uma importante vantagem: usando a estrutura de estados e algumas informações adicionais, pode-se facilmente implementar buscas de informações sobre a ocorrência de determinadas condições ao longo de uma animação (especialmente vantajoso no desenvolvimento de animações complexas).

Por fim, comparativamente com os demais modelos usuais quadro-a-quadro, animações vistas como autômatos com saída possuem uma importante vantagem em termos de tamanho de arquivo ou de taxa de transferência (no caso de aplicações na www). De fato, a animação baseada em autômatos pode montar cada quadro no momento em que é exibido, usando o alfabeto de saída. Neste caso, a mesma imagem pode ser exibida como saída

pelo ator em diferentes momentos da animação, sem a necessidade de codificá-la (e/ou transmiti-la) novamente. O mesmo ocorre para diferentes instâncias do mesmo ator, bem como para diferentes atores que usam o mesmo alfabeto de saída. Em estudos de casos reais, é fácil atingir porcentagens inferiores a **20%** do espaço usualmente requerido por um GIF equivalente, por exemplo.

Uma interessante generalização deste modelo é obtida, usando-se a facilidade de não determinismo, cuja interpretação no contexto de animações é sugerida como exercício.

5.6 exercícios

exercício 5.1 Defina formalmente:

a A função programa estendida para a máquina de Mealy;

b As seguintes funções estendidas para a máquina de Moore:

b.1) Função programa;

b.2) Função de saída.

exercício 5.2 Considere o exemplo 5.1 – Máquina de Mealy: diálogo. Defina, por extensão, os alfabetos de entrada e de saída.

exercício 5.3 Considere o exemplo 5.1 – Máquina de Mealy: diálogo. Então:

a Desenvolva uma máquina de Moore que realize o mesmo processamento;

b Escolha uma das máquinas e amplie o diálogo prevendo:

- operações de modificação e exclusão de informações;
- facilidade de *help* (opção que fornece ao usuário informações de ajuda sobre como proceder no diálogo, no ponto em que se encontra).

exercício 5.4 Desenvolva uma:

a Máquina de Mealy;

b Máquina de Moore;

que realize a conversão da representação de valores monetários de dólares para reais. Por exemplo, dado o valor **US$25.010,59** na fita de entrada, deve ser gravado o valor **R$25.010,59** na fita de saída (atenção para o uso da vírgula e do ponto nos dois valores). Adicionalmente, o autômato deve verificar se a entrada é um valor monetário válido:

exercício 5.5 Desenvolva uma:

a Máquina de Mealy;

b Máquina de Moore;

sobre o alfabeto de entrada **{ x, β, • }**. O objetivo é tratar brancos (β) corretamente em um texto. Assim, a máquina deve analisar um texto (palavra sobre o alfabeto), garantindo que:

- não existam brancos contíguos;
- o texto deve iniciar por **x** e terminar por •
- sejam eliminados eventuais β antes de um •
- antes de um ponto exista **x**

Note-se que o autômato somente pode alterar os brancos no texto. Caso o resto do texto não esteja de acordo, deve ser rejeitado (neste caso, a saída pode ser qualquer). Por exemplo:

- a entrada ββ**xx**ββ**xx**ββ**xx**ββ•βββ deve ser aceita e gera a saída **xx**β**xx**β**xx**•
- a entrada •**x** deve ser rejeitada.

exercício 5.6 Considere as máquinas de Moore e de Mealy ilustradas na figura 5.9 (esquerda e direita, respectivamente). As máquinas são equivalentes? Justifique a sua resposta.

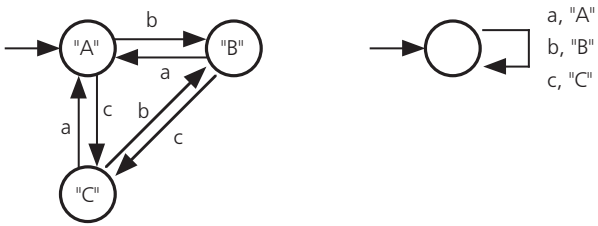


figura 5.9 Diagramas: máquina de Moore (esquerda) e máquina de Mealy (direita).

exercício 5.7 Desenvolva uma máquina de Mealy tal que:

- reconhece como entrada um dígito **d** (com valor variando de **1** a **3**) seguido de uma palavra não vazia **w** sobre o alfabeto **{a, b}**
- para cada símbolo **s** de **w**, grava na saída s^d (o símbolo **s** repetido **d** vezes)

Por exemplo, para a entrada **3abba**, grava **aaabbbbbbaaa**

exercício 5.8 Justifique a seguinte afirmação:

> *a linguagem gerada (ou seja, o conjunto de palavras que podem ser gravadas na fita) por uma máquina de Mealy ou de Moore é regular.*

exercício 5.9 Modifique a definição de:

a Máquina de Mealy;
b Máquina de Moore;

para cada um dos seguintes casos:

- a saída é restrita a um símbolo ou à palavra vazia;
- não possui estados finais.

Alguma dessas modificações altera o poder computacional das máquinas?

exercício 5.10 Modifique a definição de:

a Máquina de Mealy;
b Máquina de Moore;

prevendo cada uma das seguintes facilidades:

- não determinismo;
- movimentos vazios.

Para cada facilidade acima, em relação à equivalência dos dois tipos de máquina, como fica o caso referente à palavra vazia?

exercício 5.11 Modifique o algoritmo de minimização de autômato finito, para cada um dos seguintes casos:

a Minimização da máquina de Mealy;
b Minimização da máquina de Moore.

exercício 5.12 Considere o exemplo 5.4 – Hiperdocumento como autômato finito com saída. Desenvolva uma solução baseada em máquina de Mealy com o mesmo objetivo.

Dica: a solução deve prever que a primeira página é exibida apenas no momento da primeira transição.

exercício 5.13 Considerando hipertextos e hipermídias vistos como autômatos finitos com saída, que interpretação pode ser dada para a facilidade de não determinismo no contexto da www?

Dica: existem pelo menos duas importantes interpretações.

exercício 5.14 Considerando hipertextos e hipermídias vistos como autômatos finitos com saída, qual a consequência nos seguintes casos:

a Substituição de símbolos no alfabeto de entrada?
b Substituição de símbolos no alfabeto de saída?

exercício 5.15 Considere o exemplo 5.5 – Animação como autômato finito com saída. Desenvolva uma solução baseada em máquina de Moore com o mesmo objetivo.

exercício 5.16 Considerando animações vistas como autômatos finitos com saída, que interpretação pode ser dada para a facilidade de não determinismo?

Dica: existem pelo menos duas importantes interpretações.

exercício 5.17 Considerando animações vistas como autômatos finitos com saída, qual a consequência nos seguintes casos:

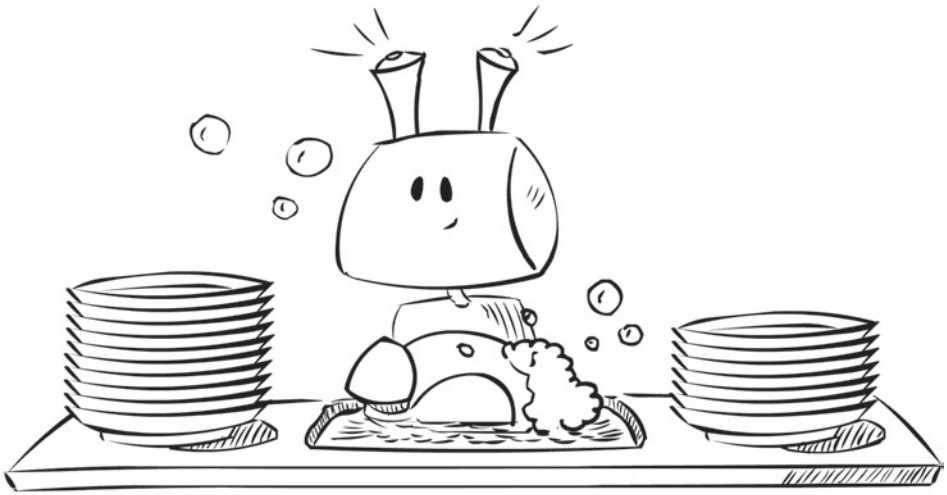
a Substituição de símbolos no alfabeto de entrada?

b Substituição de símbolos no alfabeto de saída?

capítulo 6

linguagens livres do contexto

O estudo das linguagens livres do contexto é de fundamental importância, pois compreende a sintaxe da maioria das linguagens de programações de propósitos gerais como Pascal, C e Java. Dois formalismos livres do contexto são apresentados: gramática livre do contexto e autômato com pilha. Alguns estudos e resultados são desenvolvidos objetivando reconhecimento de linguagens e prova de propriedades como árvores de derivação, simplificação de gramáticas e formas normais de gramáticas.



O estudo da *classe das linguagens livres do contexto* ou *tipo 2* é de fundamental importância na computação e informática pois:

- compreende um universo mais amplo de linguagens (comparativamente com o das regulares), tratando, adequadamente, questões como parênteses balanceados, construções bloco-estruturadas, entre outras, típicas de linguagens de programação como Pascal, C, Java, etc.;
- os algoritmos reconhecedores e geradores que implementam as linguagens livres do contexto são relativamente simples e possuem uma eficiência razoável;
- exemplos típicos de aplicações dos conceitos e resultados referentes às linguagens livres do contexto são centrados em *linguagens artificiais* e, em especial, nas linguagens de programação. Em particular, destacam-se *analisadores sintáticos*, *tradutores de linguagens* e *processadores de texto* em geral.

De acordo com a hierarquia de Chomsky, a classe das linguagens livres do contexto contém propriamente a classe das linguagens regulares. Entretanto, constitui uma classe de linguagens relativamente restrita, sendo fácil definir linguagens que não pertencem a esta classe.

O estudo das *linguagens livres do contexto* é abordado, usando-se os seguintes formalismos:

a *Gramática livre do contexto*. Trata-se de um formalismo axiomático ou gerador o qual, como o nome indica, é uma gramática, mas com restrições na forma das regras de produção. Tais restrições são definidas de maneira mais livre que na gramática regular;

b *Autômato com pilha*. Trata-se de um formalismo operacional ou reconhecedor cuja estrutura básica é análoga à do autômato finito não determinístico, adicionada de uma memória auxiliar tipo *pilha* (a qual pode ser lida ou gravada).

Relativamente às gramáticas livres do contexto, os seguintes tópicos são desenvolvidos e são importantes tanto para desenvolvimento e otimização de algoritmos reconhecedores, como na prova de teoremas:

a *Árvore de derivação*. Representa a derivação de uma palavra na forma de *árvore*, partindo do símbolo inicial como a *raiz*, e terminando em símbolos terminais como *folhas*;

b *Gramática ambígua*. Se existe pelo menos uma palavra que possua duas ou mais árvores de derivação nesta gramática;

c *Simplificação de gramática*. Simplifica as produções sem reduzir o poder de geração da gramática;

d *Forma normal*. Estabelece restrições rígidas na forma das produções, sem reduzir o poder de geração da gramática.

A construção de um autômato com pilha a partir de uma gramática livre do contexto qualquer, permite estabelecer as seguintes conclusões:

- a construção de um reconhecedor para uma linguagem livre do contexto a partir de sua gramática é simples e imediata;
- qualquer linguagem livre do contexto pode ser reconhecida por um autômato com pilha com somente um estado de controle lógico, o que implica que a estrutura de pilha é suficiente como única memória, não sendo necessário usar os estados para "memorizar" informações passadas.

6.1 → gramática livre do contexto

As *linguagens livres do contexto* ou *tipo 2* são definidas a partir das *gramáticas livres do contexto*.

definição 6.1 – Gramática livre do contexto

Uma *gramática livre do contexto* **G** é uma gramática:

$$G = (V, T, P, S)$$

com a restrição de que qualquer regra de produção de **P** é da forma:

$$A \rightarrow \alpha$$

onde **A** é uma variável de **V**, e α, uma palavra de $(V \cup T)^*$. ❑

Portanto, uma gramática livre do contexto é uma gramática na qual o lado esquerdo das produções contém exatamente uma variável.

definição 6.2 – Linguagem livre do contexto, linguagem tipo 2

Seja **G** uma gramática livre do contexto. A *linguagem gerada* pela gramática **G**:

$$GERA(G) = \{ w \in T^* \mid S \Rightarrow^+ w \}$$

é dita uma *linguagem livre do contexto* ou *linguagem tipo 2*. ❑

Portanto, toda linguagem regular é livre do contexto. A relação entre as classes de linguagens estudadas até o momento é ilustrada na figura 6.1 (inclusões próprias).

figura 6.1 Relação entre as classes de linguagens.

O nome "livre do contexto" deve-se ao fato de representar a mais geral classe de linguagens cuja produção é da forma $\mathbf{A} \rightarrow \alpha$. Ou seja, em uma derivação, a variável **A** deriva α sem depender (sendo "livre") de qualquer análise dos símbolos que antecedem ou sucedem **A** (o "contexto") na palavra que está sendo derivada.

exemplo 6.1 – Gramática livre do contexto: duplo balanceamento

Considere a linguagem:

$$L_1 = \{ a^n b^n \mid n \geq 0 \}$$

A seguinte gramática livre do contexto:

$G_1 = (\{ S \}, \{ a, b \}, P_1, S)$, em que:
$P_1 = \{ S \rightarrow aSb \mid S \rightarrow \varepsilon \}$

é tal que $\mathbf{GERA}(G_1) = L_1$. Por exemplo, a palavra **aabb** pode ser gerada pela seguinte sequência de derivação:

$$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aa\varepsilon bb = aabb$$ ❑

O exemplo acima é um caso clássico e de fundamental importância no estudo da computação e informática, pois permite estabelecer analogia com estruturas de duplo balanceamento em linguagens de programação como Pascal, C, Java, etc. Por exemplo:

a Linguagens bloco-estruturadas como `begin`^{n} `end`^{n} e similares;
b Linguagens com parênteses balanceados em expressões na forma `(`^{n} `)`^{n}

O exemplo a seguir ilustra parênteses balanceados em expressões aritméticas.

exemplo 6.2 – Gramática livre do contexto: expressões aritméticas

A linguagem $\mathbf{L_2}$ gerada pela gramática livre do contexto abaixo é composta de expressões aritméticas contendo colchetes balanceados, dois operadores e um operando:

$\mathbf{G_2 = (\{ E \}, \{ +, *, [,], x \}, P_2, E)}$, em que:
$\mathbf{P_2 = \{ E \rightarrow E{+}E \mid E{*}E \mid [E] \mid x \}}$

Por exemplo, a expressão **[x+x]∗x** pode ser gerada pela seguinte sequência de derivação (identifique, em cada derivação, qual a regra de produção usada):

$$\mathbf{E \Rightarrow E{*}E \Rightarrow [E]{*}E \Rightarrow [E{+}E]{*}E \Rightarrow [x{+}E]{*}E \Rightarrow [x{+}x]{*}E \Rightarrow [x{+}x]{*}x}$$

É possível gerar a mesma expressão com outra sequência de derivação? Quantas sequências distintas são possíveis?

Identifique quais produções controlam o duplo balanceamento de parênteses. ❑

observação 6.3 – BNF: *Backus Naur Form*

Em computação e informática, uma maneira usual de representar uma gramática livre do contexto é o uso da *forma de Backus Naur* ou simplesmente *BNF* (do inglês, *Backus Naur form*). Em uma BNF, vale que:

- as variáveis são palavras delimitadas pelos símbolos ⟨ e ⟩
- as palavras não delimitadas são terminais;
- uma regra de produção $\mathbf{A} \rightarrow \alpha$ é representada por:

$$\mathbf{A} ::= \alpha$$ ❑

exemplo 6.3 – BNF: identificador em Pascal

Suponha que se deseja definir uma BNF capaz de gerar qualquer identificador válido na linguagem de programação Pascal, ou seja, na qual:

- toda letra é um identificador;
- se **S** é um identificador, então a concatenação à direita de **S** com qualquer letra ou dígito também é um identificador.

Uma BNF é como segue (a variável ⟨**identificador**⟩ é o símbolo inicial):

⟨**identificador**⟩ ::= ⟨**letra**⟩ | ⟨**identificador**⟩⟨**letra**⟩ | ⟨**identificador**⟩⟨**dígito**⟩
⟨**letra**⟩ ::= **a** | **b** | … | **z**
⟨**dígito**⟩ ::= **0** | **1** | … | **9** ❑

6.2 ⇢ árvore de derivação

Em algumas aplicações como *compiladores* e *processadores de textos*, frequentemente é conveniente representar a derivação de palavras na forma de *árvore*, partindo do símbolo inicial como a *raiz*, e terminando em símbolos terminais como *folhas*.

definição 6.4 – Árvore de derivação

Para uma determinada gramática livre do contexto, a representação da derivação de palavras na forma de árvore, denominada *árvore de derivação*, é como segue (considere a figura 6.2):

a A *raiz* é o símbolo inicial da gramática;

b Os *vértices interiores* obrigatoriamente são variáveis. Se **A** é um vértice interior e X_1, X_2,…,X_n são os "filhos" de **A**, então:
- $A \rightarrow X_1X_2\ldots X_n$ é uma produção da gramática;
- os vértices X_1, X_2,…,X_n são ordenados da esquerda para a direita;

c Um *vértice folha*, ou simplesmente *folha*, é um símbolo terminal, ou o símbolo vazio. Neste caso, o vazio é o único filho de seu pai ($A \rightarrow \varepsilon$). ❑

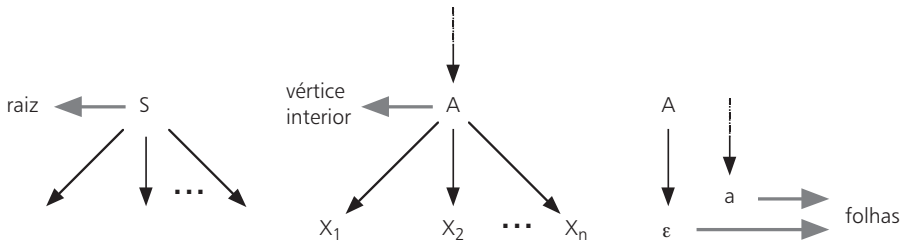


figura 6.2 Árvore de derivação: representação.

exemplo 6.4 – Árvore de derivação

A palavra **aabb** e a expressão **[x+x]∗x** dos seguintes exemplos, respectivamente:

- exemplo 6.1 – Gramática livre do contexto: duplo balanceamento;
- exemplo 6.2 – Gramática livre do contexto: expressões aritméticas;

são geradas pelas árvores de derivação ilustradas na figura 6.3, à esquerda e à direita, respectivamente. ❑

Uma única árvore de derivação pode representar derivações distintas de uma mesma palavra, como ilustrado no exemplo a seguir.

exemplo 6.5 – Árvore de derivação × derivações

Na árvore representada na figura 6.4, a palavra **x+x∗x** pode ser gerada por diversas derivações distintas, como segue:

a $E \Rightarrow E+E \Rightarrow x+E \Rightarrow x+E*E \Rightarrow x+x*E \Rightarrow x+x*x$

b $E \Rightarrow E+E \Rightarrow E+E*E \Rightarrow E+E*x \Rightarrow E+x*x \Rightarrow x+x*x$

c $E \Rightarrow E+E \Rightarrow E+E*E \Rightarrow x+E*E \Rightarrow x+x*E \Rightarrow x+x*x$

d etc…

Observe que, no item a), em cada passo de derivação, sempre é derivada a variável mais à esquerda. Analogamente para a variável mais a direita no item b). ❑

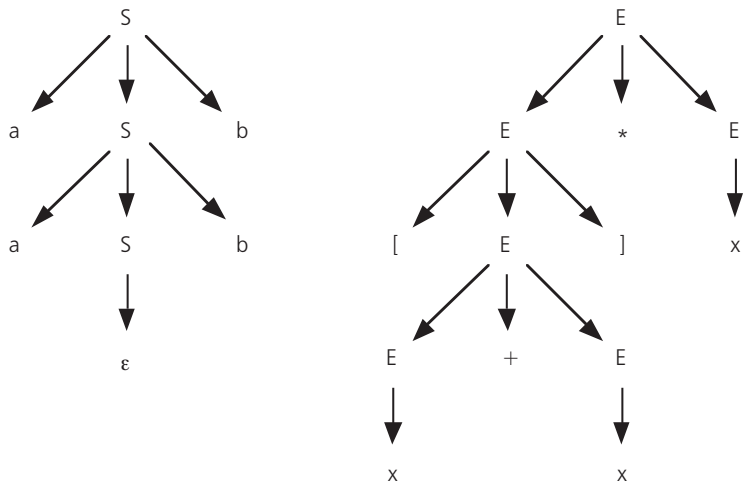


figura 6.3 Árvores de derivação: **aabb** e **[x+x]∗x**.

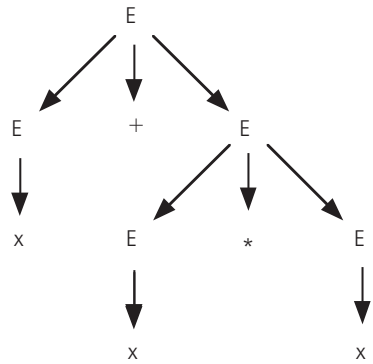


figura 6.4 Árvore de derivação: **x+x∗x**.

definição 6.5 – Derivação mais à esquerda (direita)

Dada uma árvore de derivação, uma *derivação mais à esquerda* (respectivamente, *derivação mais à direita*) de uma palavra é a sequência de produções aplicada sempre à variável mais à esquerda (respectivamente, mais à direita) da palavra, em cada passo de derivação. ❑

exemplo 6.6 – Derivação mais à esquerda (direita)

Observe novamente o exemplo 6.5 – Árvore de derivação × derivações. Então:

- o item a) representa uma derivação mais à esquerda;
- o item b) representa uma derivação mais à direita. ❑

6.3 gramática livre do contexto ambígua

Eventualmente, uma mesma palavra pode ser associada a duas ou mais árvores de derivação, caracterizando uma *gramática ambígua*. Em muitas aplicações como, por exemplo, no desenvolvimento e otimização de alguns tipos de algoritmos de reconhecimento, é conveniente que a gramática usada seja não ambígua. Entretanto, nem sempre é possível eliminar ambiguidades. Na realidade, é fácil definir linguagens para as quais qualquer gramática livre do contexto é ambígua.

definição 6.6 – Gramática ambígua

Uma gramática livre do contexto é dita uma *gramática livre do contexto ambígua* ou simplesmente *gramática ambígua*, se existe pelo menos uma palavra que possua duas ou mais árvores de derivação nesta gramática. ❑

exemplo 6.7 – Gramática ambígua

Considere o exemplo 6.2 – Gramática livre do contexto: expressões aritméticas. A palavra **x+x∗x** pode ser gerada por árvores distintas, como ilustrado na figura 6.5. Portanto, a gramática em questão é ambígua. ❑

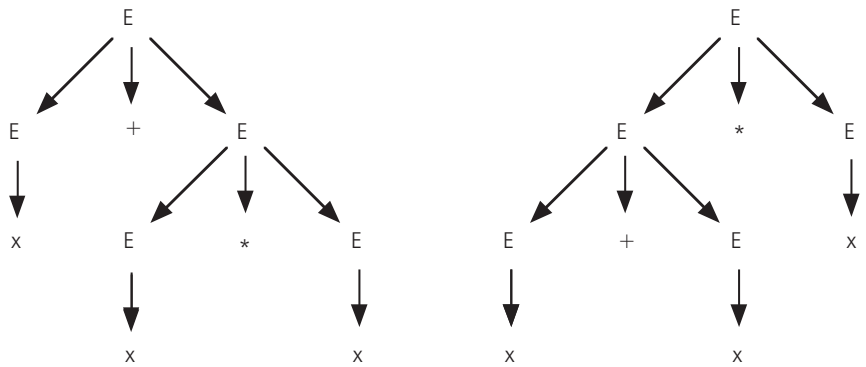


figura 6.5 Gramática ambígua: árvores diferentes para a palavra **x+x∗x**.

Considere a gramática ambígua apresentada no exemplo 6.2 – Gramática livre do contexto: expressões aritméticas. A palavra **x+x∗x** possui mais de uma derivação à esquerda (respectivamente, mas à direita), como segue:

a Derivação mais à esquerda:

$$E \Rightarrow E+E \Rightarrow x+E \Rightarrow x+E*E \Rightarrow x+x*E \Rightarrow x+x*x$$
$$E \Rightarrow E*E \Rightarrow E+E*E \Rightarrow x+E*E \Rightarrow x+x*E \Rightarrow x+x*x$$

b Derivação mais à direita:

$$E \Rightarrow E+E \Rightarrow E+E*E \Rightarrow E+E*x \Rightarrow E+x*x \Rightarrow x+x*x$$
$$E \Rightarrow E*E \Rightarrow E*x \Rightarrow E+E*x \Rightarrow E+x*x \Rightarrow x+x*x$$

Uma forma equivalente de definir *gramática ambígua* é verificando a existência de pelo menos uma palavra com duas ou mais derivações mais à esquerda (ou, alternativamente, mais à direita). O seguinte teorema não será demonstrado.

teorema 6.7 – Gramática ambígua

Uma gramática livre do contexto **G** é uma *gramática ambígua* se existe pelo menos uma palavra tal que possui:

- duas ou mais derivações mais à esquerda; ou
- duas ou mais derivações mais à direita. ❑

definição 6.8 – Linguagem inerentemente ambígua

Uma linguagem é dita uma *linguagem inerentemente ambígua* se qualquer gramática livre do contexto que a define é ambígua. ❑

exemplo 6.8 – Linguagem inerentemente ambígua

A seguinte linguagem é inerentemente ambígua:

$$\{ w \mid w = a^n b^n c^m d^m \text{ ou } w = a^n b^m c^m d^n, n \geq 1, m \geq 1 \}$$ ❑

exemplo 6.9 – Linguagem inerentemente ambígua: contraexemplo

Considere o exemplo 6.2 – Gramática livre do contexto: expressões aritméticas. A linguagem em questão *não* é inerentemente ambígua.

A construção de uma gramática não ambígua para esta linguagem é sugerida como exercício. O correto entendimento da solução deste exercício é especialmente importante, pois justifica a maneira aparentemente "estranha" de definir expressões na maioria das gramáticas das linguagens de programação. De fato, como já foi destacado, no desenvolvimento e na otimização de alguns tipos de algoritmos de reconhecimento, é conveniente que a gramática usada seja não ambígua. ❑

6.4 → simplificação de gramática livre do contexto

É possível simplificar alguns tipos de produções sem reduzir o poder de geração das gramáticas livres do contexto. Em geral, as simplificações de gramáticas são usadas na construção e na otimização de algoritmos e na demonstração de teoremas.

As seguintes simplificações são apresentadas:

a *Símbolos inúteis*. Exclusão de variáveis ou terminais não usados para gerar palavras;

b *Produções vazias*. Exclusão de produções da forma $A \rightarrow \varepsilon$ (se a palavra vazia pertence à linguagem, é incluída uma produção vazia específica para tal fim);

c *Produções que substituem variáveis*. Exclusão de produções da forma $A \rightarrow B$, ou seja, que simplesmente substituem uma variável por outra e, consequentemente, não adicionam qualquer informação de geração de palavras.

No texto que segue, são omitidas as provas de que os algoritmos de simplificação apresentados, de fato, atingem os objetivos propostos.

6.4.1 símbolos inúteis

A exclusão de *símbolos inúteis* (símbolos não usados na geração de palavras de terminais) é realizada, excluindo-se as produções que fazem referência a esses símbolos, bem como os próprios símbolos inúteis. Não é necessária qualquer modificação adicional nas produções da gramática. O algoritmo apresentado é dividido em duas etapas, como segue:

a *Etapa 1: qualquer variável gera terminais.* O algoritmo restringe o conjunto de variáveis, analisando as produções da gramática a partir de terminais gerados. Inicialmente, considera todas as variáveis que geram terminais diretamente (exemplo: **A** → **a**). A seguir, são adicionadas, sucessivamente, as variáveis que geram terminais indiretamente (exemplo: **B** → **Ab**);

b *Etapa 2: qualquer símbolo é atingível a partir do símbolo inicial*. Após a execução da etapa acima, o algoritmo analisa as produções da gramática a partir do símbolo inicial. Inicialmente, considera exclusivamente o símbolo inicial. Após, sucessivamente as produções da gramática são aplicadas, e os símbolos referenciados são adicionados aos novos conjuntos.

definição 6.9 – Algoritmo: exclusão dos símbolos inúteis

Seja **G = (V, T, P, S)** uma gramática livre do contexto. O *algoritmo para exclusão dos símbolos inúteis* é composto por duas etapas, como segue:

Etapa 1: qualquer variável gera terminais. A gramática resultante desta etapa é:

$$G_1 = (V_1, T, P_1, S)$$

na qual $V_1 \subseteq V$ é construído conforme o algoritmo apresentado na figura 6.6.
O conjunto P_1 possui os mesmos elementos que **P**, excetuando-se as produções cujas variáveis não pertencem a V_1;

Etapa 2: qualquer símbolo é atingível a partir do símbolo inicial. A gramática resultante desta etapa é:

$$G_2 = (V_2, T_2, P_2, S)$$

na qual $V_2 \subseteq V_1$ e $T_2 \subseteq T$ são construídos conforme o algoritmo apresentado na figura 6.7 (suponha que $\alpha, \beta \in (T \cup V_1)^*$):
O conjunto P_2 possui os mesmos elementos que P_1, excetuando-se as produções cujos símbolos não pertencem a V_2 ou T_2. ❑

```
V1 = ∅;
repita  V1 = V1 ∪ { A | A → α ∈ P e α ∈ (T ∪ V1)* }
até     que o cardinal de V1 não aumente;
```

figura 6.6 Algoritmo: exclusão dos símbolos inúteis – etapa 1.

```
T2 = ∅;
V2 = { S };
repita  V2 = V2 ∪ { A | X → α A β ∈ P1, X ∈ V2 };
        T2 = T2 ∪ { a | X → α a β ∈ P1, X ∈ V2 }
até que os cardinais de V2 e T2 não aumentem;
```

figura 6.7 Algoritmo: exclusão dos símbolos inúteis – etapa 2.

Deve-se reparar que se as etapas acima forem executadas em ordem inversa (*etapa 2* antes da *etapa 1*), o algoritmo pode não atingir o resultado esperado. Para demonstrar, é suficiente apresentar um contraexemplo, o que se sugere como exercício (lembre-se de que uma demonstração por contraexemplo é, de fato, uma demonstração por absurdo).

exemplo 6.10 – Exclusão dos símbolos inúteis

Considere a seguinte gramática livre do contexto:

G = ({ S, A, B, C }, { a, b, c }, P, S), na qual:
P = { S → aAa | bBb, A → a | S, C → c }

A exclusão dos símbolos inúteis é como segue:

Etapa 1: qualquer variável gera terminais. Considere a figura 6.8. A coluna "iteração" representa o número de ciclos do comando `repita-até`, e a coluna "variáveis", o conjunto de variáveis construído após a iteração.
Observe que o algoritmo para na terceira iteração, pois nenhuma variável foi adicionada ao conjunto.
A produção **S → bBb** é excluída, pois **B** não pertence ao novo conjunto de variáveis;
A gramática resultante desta etapa é a seguinte:

$$G_1 = (\{ A, C, S \}, \{ a \}, \{ S \to aAa, A \to a \mid S, C \to c \}, S)$$

Etapa 2: qualquer símbolo é atingível a partir do símbolo inicial. Considere a figura 6.9.
A produção **C → c** é excluída, pois **C** e **c** não pertencem aos novos conjuntos de variáveis e terminais, respectivamente.
A gramática resultante desta etapa é a seguinte:

$$G_2 = (\{ S, A \}, \{ a \}, \{ S \to aAa, A \to a \mid S \}, S)$$ ❑

iteração	variáveis
início	∅
1	{A, C}
2	{A, C, S}
3	{A, C, S}

figura 6.8 Exclusão dos símbolos inúteis – etapa 1.

iteração	variáveis	terminais
início	{S}	∅
1	{S, A}	{a}
2	{S, A}	{a}

figura 6.9 Exclusão dos símbolos inúteis – etapa 2.

6.4.2 produções vazias

A exclusão de *produções vazias* (produções da forma $A \rightarrow \varepsilon$) pode determinar modificações diversas nas produções da gramática. O algoritmo é dividido em três etapas, como segue:

a *Etapa 1: variáveis que constituem produções vazias*. Considera, inicialmente, todas as variáveis que geram diretamente a palavra vazia (exemplo: $A \rightarrow \varepsilon$). A seguir, são determinadas, sucessivamente, as variáveis que indiretamente geram a palavra vazia (exemplo: $B \rightarrow A$);

b *Etapa 2: exclusão de produções vazias*. Inicialmente, são consideradas todas as produções não vazias. A seguir, cada produção cujo lado direito possui uma variável que gera a palavra vazia, determina uma produção adicional, sem essa variável;

c *Etapa 3: geração da palavra vazia, se necessário*. Se a palavra vazia pertence à linguagem, então é incluída uma produção para gerar a palavra vazia.

definição 6.10 – Algoritmo: exclusão das produções vazias

Seja $G = (V, T, P, S)$ gramática livre do contexto. O *algoritmo para exclusão das produções vazias* é composto por três etapas, como segue:

Etapa 1: variáveis que constituem produções vazias. O algoritmo para construir o conjunto das variáveis que geram ε, denotado por V_ε, é apresentado na figura 6.10;

Etapa 2: exclusão de produções vazias. A gramática resultante desta etapa é:

$$G_1 = (V, T, P_1, S)$$

onde P_1 é construído conforme o algoritmo apresentado na figura 6.11;

Etapa 3: geração da palavra vazia, se necessário. Se a palavra vazia pertence à linguagem, então a seguinte produção é incluída:

$$S \rightarrow \varepsilon$$

resultando na seguinte gramática:

$G_2 = (V, T, P_2, S)$ onde:

$P_2 = P_1 \cup \{ S \rightarrow \varepsilon \}$ ❑

```
Vε = { A | A → ε ∈ P };
repita  Vε = Vε ∪ { X | X → X1...Xn ∈ P tal que X1,..., Xn ∈ Vε }
até     que o cardinal de Vε não aumente;
```

figura 6.10 Algoritmo: exclusão das produções vazias – etapa 1.

```
P1 = { A → α | A → α ∈ P e α ≠ ε };
repita  para   toda A → α ∈ P1, X ∈ Vε tal que α = α1 X α2, α1 α2 ≠ ε
        faça   P1 = P1 ∪ { A → α1 α2 }
até     que o cardinal de P1 não aumente;
```

figura 6.11 Algoritmo: exclusão das produções vazias – etapa 2.

exemplo 6.11 – Exclusão das produções vazias

Considere a seguinte gramática livre do contexto:

G = ({ S, X, Y }, { a, b }, P, S), na qual:
P = { S → aXa | bXb | ε, X → a | b | Y, Y → ε }

A exclusão das produções vazias é como segue:

Etapa 1: variáveis que constituem produções vazias. O conjunto V_ε é construído conforme ilustrado na figura 6.12;

Etapa 2: exclusão de produções vazias. O novo conjunto de produções é construído conforme ilustrado na figura 6.13. A gramática resultante desta etapa é a seguinte:

$$G_1 = (\{ S, X, Y \}, \{ a, b \}, \{ S \to aXa \mid bXb \mid aa \mid bb, X \to a \mid b \mid Y \}, S)$$

Etapa 3: geração da palavra vazia, se necessário. Como a palavra vazia pertence à linguagem, a produção $S \to \varepsilon$ é incluída no conjunto de produções.

A gramática resultante é a seguinte:

$$G_2 = (\{ S, X, Y \}, \{ a, b \}, \{ S \to aXa \mid bXb \mid aa \mid bb \mid \varepsilon, X \to a \mid b \mid Y \}, S)$$

Observe que **Y**, originalmente um símbolo útil, resultou em um símbolo inútil. Ou seja, a exclusão de produções vazias gerou um símbolo inútil. De fato, não é qualquer combinação de simplificações de gramática que atinge o resultado desejado. Veja adiante a seção 6.4.4 – Simplificações combinadas. ❑

iteração	V_ε
início	{S, Y}
1	{S, Y, X}
2	{S, Y, X}

figura 6.12 Exclusão das produções vazias – etapa 1.

iteração	produções
início	{S → aXa | bXb, X → a | b | Y}
1	{S → aXa | bXb | aa | bb, X → a | b | Y}
2	{S → aXa | bXb | aa | bb, X → a | b | Y}

figura 6.13 Exclusão das produções vazias – etapa 2.

6.4.3 produções que substituem variáveis

Uma produção que substitui diretamente uma variável por outra, ou seja, do tipo **A** → **B** não adiciona informação alguma em termos de geração de palavras, a não ser o fato de que, neste caso, a variável **A** pode ser substituída por **B**. Assim, se **B** → α, então a produção **A** → **B** pode ser substituída por **A** → α. A generalização desta ideia é o algoritmo proposto, dividido em duas etapas, como segue:

a *Etapa 1: fecho transitivo de cada variável*. Entende-se por fecho transitivo de uma variável o conjunto de variáveis que podem substituí-la transitivamente. Por exemplo, se **A** → **B** e **B** → **C**, então **B** e **C** pertencem ao fecho de **A**;

b *Etapa 2: exclusão das produções que substituem variáveis*. Substitui as produções da forma **A** → **B** por produções da forma **A** → α, na qual α é atingível a partir de **A** através de seu fecho.

definição 6.11 – Algoritmo: exclusão das produções que substituem variáveis

Seja **G = (V, T, P, S)** gramática livre do contexto. O *algoritmo para exclusão das produções que substituem variáveis* é composto por duas etapas, como segue:

Etapa 1: fecho transitivo de cada variável. O algoritmo para construir o fecho transitivo é apresentado na figura 6.14;

Etapa 2: exclusão das produções que substituem variáveis. A gramática resultante desta etapa é:

$$G_1 = (V, T, P_1, S)$$

na qual P_1 é construído conforme o algoritmo apresentado na figura 6.15. ❑

exemplo 6.12 – Exclusão das produções que substituem variáveis

Considere a seguinte gramática livre do contexto:

G = ({ S, X }, { a, b }, P, S), onde:
P = { S → aXa | bXb, X → a | b | S | ε }

A exclusão da produção **X** → **S** é como segue:

Etapa 1: fecho transitivo da cada variável.
FECHO-S = ∅
FECHO-X = { S }

```
para   toda A ∈ V
faça   FECHO-A = { B | A ≠ B e A ⇒+ B usando exclusivamente
            produções de P da forma X → Y };
```

figura 6.14 Algoritmo: exclusão das produções que substituem variáveis – etapa 1.

```
P1 = { A → α | A → α ∈ P e α ∉ V };
   para      toda   A ∈ V e B ∈ FECHO-A
   faça      se     B → α ∈ P e α ∉ V
             então  P1 = P1 ∪ { A → α };
```

figura 6.15 Algoritmo: exclusão das produções que substituem variáveis – etapa 2.

iteração	produções
início	{S → aXa | bXb, X → a | b | ε}
S	{S → aXa | bXb, X → a | b | ε}
X	{S → aXa | bXb, X → a | b | ε | aXa | bXb}

figura 6.16 Exclusão das produções que substituem variáveis – etapa 2.

*Etapa 2: exclusão das produções da forma **A → B***. Construção do conjunto de produções (a coluna "iteração" representa a execução do algoritmo para a variável referenciada) é conforme ilustrado na figura 6.16.

A gramática resultante é a seguinte:

$$G_1 = (\{ S, X \}, \{ a, b \}, \{ S \to aXa \mid bXb, X \to a \mid b \mid \varepsilon \mid aXa \mid bXb \}, S) \quad ❑$$

6.4.4 simplificações combinadas

Deve-se observar que não é qualquer combinação de simplificações de gramática livre do contexto que atinge o resultado desejado. Por exemplo, em uma gramática *sem* símbolos inúteis, mas *com* produções que substituem variáveis, o algoritmo para excluir esse tipo de produção pode gerar símbolos inúteis (por quê?). Portanto, caso os algoritmos sejam combinados, a seguinte sequência de simplificação é recomendada:

a *Exclusão das produções vazias*;
b *Exclusão das produções que substituem variáveis*;
c *Exclusão dos símbolos inúteis*.

6.5 → formas normais

As formas normais estabelecem restrições rígidas na forma das produções, sem reduzir o poder de geração das gramáticas livres do contexto (excetuando-se a geração da palavra vazia). São usadas principalmente no desenvolvimento de algoritmos (com destaque para reconhecedores de linguagens) e na prova de teoremas.

As formas normais apresentadas são as seguintes (suponha que **A**, **B** e **C** são variáveis, **a** é terminal e α é uma palavra de variáveis):

- *Forma normal de Chomsky,* na qual as produções são da forma:

$$A \to BC \quad \text{ou} \quad A \to a$$

- *Forma normal de Greibach,* na qual as produções são da forma:

$$A \rightarrow a\alpha$$

Para cada caso, é apresentado um algoritmo de conversão de uma gramática livre do contexto qualquer para a correspondente forma normal. As provas de que os algoritmos atingem os objetivos propostos são omitidas.

6.5.1 forma normal de Chomsky

definição 6.12 – Forma normal de Chomsky

Uma gramática livre do contexto **G = (V, T, P, S)** é dita estar na *Forma normal de Chomsky* se todas as suas produções são da forma (suponha que **A**, **B** e **C** são variáveis de **V** e **a** é terminal de **T**):

$$A \rightarrow BC \quad \text{ou} \quad A \rightarrow a$$ ❑

Portanto, a palavra vazia não pertence à linguagem gerada por uma gramática na Forma normal de Chomsky.

O algoritmo a seguir transforma uma gramática livre do contexto qualquer, cuja linguagem gerada não possua a palavra vazia, em uma gramática na Forma normal de Chomsky. O algoritmo é dividido em três etapas, como segue:

a *Etapa 1: simplificação da gramática*. Simplifica a gramática, excluindo as produções vazias (como a linguagem não possui a palavra vazia, todas as produções da forma $A \rightarrow \varepsilon$ podem ser excluídas), produções da forma $A \rightarrow B$ (se o lado direito de alguma produção tiver somente um símbolo, este será terminal) e, opcionalmente, os símbolos inúteis;

b *Etapa 2: transformação do lado direito das produções de comprimento maior ou igual a dois*. Garante que o lado direito das produções de comprimento maior ou igual a dois é composto exclusivamente por variáveis. A exclusão de um terminal **a** pode ser realizada, substituindo-se este por uma variável intermediária C_a e incluindo a produção $C_a \rightarrow a$;

c *Etapa 3: transformação do lado direito das produções de comprimento maior ou igual a três, em produções com exatamente duas variáveis*. Garante que o lado direito das produções de comprimento maior do que um é composto exatamente por duas variáveis. Após a execução da etapa acima, o lado direito das produções da forma $A \rightarrow B_1B_2\ldots B_n$ ($n \geq 2$) é composto exclusivamente por variáveis. Portanto, para concluir a transformação, é suficiente garantir que o lado direito é composto por exatamente duas variáveis. Isto é possível, gerando-se $B_1B_2\ldots B_n$ em diversas etapas, e usando-se variáveis intermediárias.

definição 6.13 – Algoritmo: forma normal de Chomsky

Seja **G = (V, T, P, S)** uma gramática livre do contexto tal que $\varepsilon \notin$ **GERA(G)**. O *algoritmo para transformar na forma normal de Chomsky* é como segue:

Etapa 1: simplificação da gramática. As seguintes simplificações:

- produções vazias;
- produções que substituem variáveis;
- símbolos inúteis (opcional);

devem ser realizadas usando os algoritmos de simplificação descritos anteriormente, resultando na gramática:

$$G_1 = (V_1, T_1, P_1, S)$$

Etapa 2: transformação do lado direito das produções de comprimento maior ou igual a dois. A gramática resultante desta etapa é:

$$G_2 = (V_2, T_1, P_2, S)$$

na qual V_2 e P_2 são construídos conforme o algoritmo apresentado na figura 6.17 (para cada variável **a**, suponha $C_a \notin V_2$);

Etapa 3: transformação do lado direito das produções de comprimento maior ou igual a três em produções com exatamente duas variáveis. A gramática resultante desta etapa é:

$$G_3 = (V_3, T_1, P_3, S)$$

na qual V_3 e P_3 são construídos conforme o algoritmo apresentado na figura 6.18 (a cada ciclo, suponha $D_1 \notin V_3, \ldots, D_{n-2} \notin V_3$). ❑

```
V2 = V1;
P2 = P1;
para     toda A → X1X2Xn ∈ P2 tal que n ≥ 2
faça     se     para r ∈ { 1,...,n }, Xr é um símbolo terminal
         então  (suponha Xr = a)
                V2 = V2 ∪ { Ca };
                substitui a por Ca em A → X1X2...Xn ∈ P2;
                P2 = P2 ∪ { Ca → a };
```

figura 6.17 Algoritmo: forma normal de Chomsky – etapa 2.

```
V3 = V2;
P3 = P2;
para     toda A → B1B2...Bn ∈ P3 tal que n ≥ 3
faça     P3 = P3 – { A → B1B2...Bn };
         V3 = V3 ∪ { D1,...,Dn-2 };
         P3 = P3 ∪ { A → B1D1, D1 → B2D2,...,
                     Dn-3 → Bn-2Dn-2, Dn-2 → Bn-1Bn };
```

figura 6.18 Algoritmo: forma normal de Chomsky – etapa 3.

exemplo 6.13 – Algoritmo: forma normal de Chomsky

Considere a seguinte gramática apresentada no exemplo 6.2 – Gramática livre do contexto: expressões aritméticas:

$\mathbf{G = (\{ E \}, \{ +, *, [,], x \}, P, E)}$, na qual:
$\mathbf{P = \{ E \rightarrow E{+}E \mid E{*}E \mid [E] \mid x \}}$

Etapa 1: simplificação da gramática. A gramática já está simplificada;
Etapa 2: transformação do lado direito das produções de comprimento maior ou igual a dois. Excetuando-se a produção $\mathbf{E \rightarrow x}$, as demais devem ser substituídas como segue:

$\mathbf{E \rightarrow E\ C_+\ E \mid E\ C_*\ E \mid C_[\ E\ C_]}$
$\mathbf{C_+ \rightarrow +}$
$\mathbf{C_* \rightarrow *}$
$\mathbf{C_[\rightarrow [}$
$\mathbf{C_] \rightarrow]}$

Etapa 3: transformação do lado direito das produções de comprimento maior ou igual a três em produções com exatamente duas variáveis. As produções:

$\mathbf{E \rightarrow E\ C_+\ E \mid E\ C_*\ E \mid C_[\ E\ C_]}$

necessitam ser substituídas como segue:

$\mathbf{E \rightarrow E\ D_1 \mid E\ D_2 \mid C_[\ D_3}$
$\mathbf{D_1 \rightarrow C_+\ E}$
$\mathbf{D_2 \rightarrow C_*\ E}$
$\mathbf{D_3 \rightarrow E\ C_]}$

A gramática resultante, na forma normal de Chomsky, é a seguinte:

$\mathbf{G_{FNC} = (\{ E, C_+, C_*, C_[, C_], D_1, D_2, D_3 \}, \{ +, *, [,], x \}, P_{FNC}, E)}$, onde:
$\mathbf{P_{FNC} = \{ E \rightarrow E\ D_1 \mid E\ D_2 \mid C_[\ D_3 \mid x,}$
$\mathbf{D_1 \rightarrow C_+\ E,\ D_2 \rightarrow C_*\ E,\ D_3 \rightarrow E\ C_],}$
$\mathbf{C_+ \rightarrow +,\ C_* \rightarrow *,\ C_[\rightarrow [,\ C_] \rightarrow] \}}$ ❑

6.5.2 forma normal de Greibach

definição 6.14 – Forma normal de Greibach

Uma gramática livre do contexto é dita estar na *forma normal de Greibach* se todas as suas produções são da forma:

$$\mathbf{A \rightarrow a\alpha}$$

na qual **A** é uma variável de **V**, **a** é um terminal de **T**, e α é uma palavra de **V***. ❑

Portanto, a palavra vazia não pertence à linguagem gerada por uma gramática na forma normal de Greibach.

O algoritmo a seguir transforma uma gramática livre do contexto qualquer, cuja linguagem gerada não possua a palavra vazia, em uma gramática na forma normal de Greibach. O algoritmo é dividido nas seguintes etapas:

a *Etapa 1: simplificação da gramática*. Análoga à correspondente etapa do algoritmo referente à Forma normal de Chomsky;

b *Etapa 2: renomeação das variáveis em uma ordem crescente qualquer*. As variáveis da gramática são renomeadas em uma ordem crescente qualquer, como, por exemplo, A_1, A_2,..., A_n, em que **n** é o cardinal do conjunto de variáveis. Diferentes critérios de renomeação podem resultar em diferentes gramáticas na forma normal de Greibach. Entretanto, todas são equivalentes (geram a mesma linguagem);

c *Etapa 3: transformação de produções para a forma $A_r \rightarrow A_s\alpha$, em que $r \leq s$*. As produções são modificadas, garantindo que a primeira variável do lado direito é maior ou igual que a do lado esquerdo, considerando-se a ordenação da etapa anterior. As produções $A_r \rightarrow A_s\alpha$ tais que $r > s$ são modificadas, substituindo-se a variável A_s pelas suas correspondentes produções ($A_s \rightarrow \beta_1 \mid \ldots \mid \beta_m$), resultando em $A_r \rightarrow \beta_1\alpha \mid \ldots \mid \beta_m\alpha$ e assim sucessivamente. Entretanto, como o conjunto de variáveis é finito, existe um limite para as produções crescentes, que pode ser a geração de um terminal ($A_r \rightarrow a\alpha$) ou de uma recursão ($A_r \rightarrow A_r\alpha$);

d *Etapa 4: exclusão das recursões da forma $A_r \rightarrow A_r\alpha$*. As recursões (à esquerda) podem existir originalmente na gramática, ou serem geradas pela etapa anterior. A eliminação da recursão à esquerda pode ser realizada, introduzindo-se variáveis auxiliares e incluindo-se recursão à direita ($B_r \rightarrow \alpha B_r$);

e *Etapa 5: um terminal no início do lado direito de cada produção*. Após a execução da etapa anterior, todas as produções da forma $A_r \rightarrow A_s\alpha$ são tais que $r < s$. Consequentemente, as produções da maior variável A_n só podem iniciar por um terminal no lado direito. Assim, se, em $A_{n-1} \rightarrow A_n\alpha$, for substituído A_n pelas suas correspondentes produções (exemplo: $A_n \rightarrow a\beta$), o lado direito das produções de A_{n-1} também iniciará por um terminal (exemplo: $A_{n-1} \rightarrow a\beta\alpha$). A repetição do algoritmo para A_{n-2},...,A_1 resultará em produções exclusivamente da forma $A_r \rightarrow a\alpha$;

f *Etapa 6: produções na forma $A \rightarrow a\alpha$ em que α é composta por variáveis*. É análoga à correspondente etapa do algoritmo relativo à forma normal de Chomsky.

definição 6.15 – Algoritmo: forma normal de Greibach

Seja **G = (V, T, P, S)** uma gramática livre do contexto tal que $\varepsilon \notin$ **GERA(G)**. O *algoritmo para transformar na forma normal de Greibach* é como segue:

Etapa 1: simplificação da gramática. As seguintes simplificações:

- produções vazias;
- produções que substituem variáveis;
- símbolos inúteis (opcional);

devem ser realizadas, usando os algoritmos de simplificação apresentados anteriormente e resultando na gramática:

$$G_1 = (V_1, T_1, P_1, S)$$

Etapa 2: renomeação das variáveis em uma ordem crescente qualquer. A gramática resultante desta etapa é (suponha que A_i corresponde à renomeação de **S**):

$$G_2 = (V_2, T_1, P_2, A_i)$$

Etapas 3 e 4: transformação de produções para a forma $A_r \rightarrow A_s\alpha$, na qual $r \leq s$ e *exclusão das recursões da forma $A_r \rightarrow A_r\alpha$*. A gramática resultante destas duas etapas realizadas em conjunto é:

$$G_3 = (V_3, T_1, P_3, A_i)$$

onde V_3, P_3 são construídos conforme o algoritmo apresentado na figura 6.19, supondo que o cardinal de V_2 é **n** (a cada ciclo, suponha $B_r \notin V_3$)

Etapa 5: um terminal no início do lado direito de cada produção. A gramática resultante desta etapa é:

$$G_4 = (V_3, T_1, P_4, A_i)$$

onde P_4 é construído conforme o algoritmo apresentado na figura 6.20. Também é necessário garantir que as produções relativas às variáveis auxiliares B_r iniciam por um terminal do lado direito, conforme o algoritmo apresentado na figura 6.21;

Etapa 6: produções na forma $A \rightarrow a\alpha$ na qual α é composta por variáveis. É análoga à correspondente etapa do algoritmo relativo à forma normal de Chomsky. ❑

exemplo 6.14 – Algoritmo: forma normal de Greibach

Considere a seguinte gramática livre do contexto:

$G = (\{ S, A \}, \{ a, b \}, P, S)$, onde:
$P = \{ S \rightarrow AA \mid a, A \rightarrow SS \mid b \}$

A transformação na correspondente forma normal de Greibach é como segue:

Etapa 1: simplificação da gramática. A gramática já está simplificada;

Etapa 2: renomeação das variáveis em uma ordem crescente qualquer. As variáveis **S** e **A** são renomeadas para A_1 e A_2, respectivamente. As produções da gramática ficam como segue:

$A_1 \rightarrow A_2A_2 \mid a$
$A_2 \rightarrow A_1A_1 \mid b$

Etapas 3 e 4: transformação de produções para a forma $A_r \rightarrow A_s\alpha$, na qual $r \leq s$ e *exclusão das recursões da forma $A_r \rightarrow A_r\alpha$*. A produção $A_2 \rightarrow A_1A_1$ necessita ser modificada. As produções da gramática ficam como segue:

$A_1 \rightarrow A_2A_2 \mid a$
$A_2 \rightarrow A_2A_2A_1 \mid aA_1 \mid b$

```
P3 = P2
para    r variando de 1 até n
faça
        para   s variando de 1 até r-1                          Etapa 3
        faça   para   toda A_r → A_s α ∈ P_3
               faça   excluir A_r → A_s α de P_3;
                      para   toda A_s → β ∈ P_3
                      faça   P_3 = P_3 ∪ { A_r → β α }
        para   toda A_r → A_r α ∈ P_3                           Etapa 4
        faça   excluir A_r → A_r α de P_3;
               V_3 = V_3 ∪ { B_r };
               P_3 = P_3 ∪ { B_r → α } ∪ { B_r → α B_r };
        para   toda A_r → ϕ ∈ P_3 tal que ϕ não inicia por A_r e
               alguma A_r → A_r α foi excluída
        faça   P_3 = P_3 ∪ { A_r → ϕ B_r };
```

figura 6.19 Algoritmo: forma normal de Greibach – etapas 3 e 4.

```
P_4 = P_3;
para    r variando de n-1 até 1 e toda A_r → A_s α ∈ P_4
faça    excluir A_r → A_s α de P_4;
        para    toda A_s → β de P_4
        faça    P_4 = P_4 ∪ { A_r → β α };
```

figura 6.20 Algoritmo: forma normal de Greibach – etapas 5 (parte 1).

```
para    toda B_r → A_s β_r
faça    excluir B_r → A_s β_r de P_4;
        para    toda A_s → a α
        faça    P_4 = P_4 ∪ { B_r → a α β_r };
```

figura 6.21 Algoritmo: forma normal de Greibach – etapa 5 (parte 2).

A produção $A_2 \rightarrow A_2A_2A_1$ contém uma recursão. Portanto, é necessário incluir uma variável auxiliar **B**, como segue:

$A_1 \rightarrow A_2A_2 \mid a$
$A_2 \rightarrow aA_1 \mid b \mid aA_1B \mid bB$
$B \rightarrow A_2A_1 \mid A_2A_1B$

Etapa 5: um terminal no início do lado direito de cada produção. O lado direito das produções da maior variável A_2 inicia por um terminal. Substituindo-se A_2 no lado direito de $A_1 \rightarrow A_2A_2$ pelas suas correspondentes derivações, as produções de A_1 também iniciarão por um terminal:

$A_1 \rightarrow aA_1A_2 \mid bA_2 \mid aA_1BA_2 \mid bBA_2 \mid a$
$A_2 \rightarrow aA_1 \mid b \mid aA_1B \mid bB$
$B \rightarrow A_2A_1 \mid A_2A_1B$

As produções referentes à variável **B** também são modificadas:

$$B \rightarrow aA_1A_1 \mid bA_1 \mid aA_1BA_1 \mid bBA_1 \mid aA_1A_1B \mid bA_1B \mid aA_1BA_1B \mid bBA_1B$$

Etapa 6: produções na forma $A \rightarrow a\alpha$ na qual α é composta por variáveis. Nenhum procedimento é necessário, pois as produções já se encontram nessa forma.

A gramática resultante, na forma normal de Greibach, é a seguinte:

$G_{FNG} = (\{ A_1, A_2, B \}, \{ a, b \}, P_{FNG}, A_1)$, na qual:
$P_{FNG} = \{ A_1 \rightarrow aA_1A_2 \mid bA_2 \mid aA_1BA_2 \mid bBA_2 \mid a,$
$A_2 \rightarrow aA_1 \mid b\ aA_1B \mid bB,$
$B \rightarrow aA_1A_1 \mid bA_1 \mid aA_1BA_1 \mid bBA_1 \mid aA_1A_1B \mid bA_1B \mid aA_1BA_1B \mid bBA_1B\}$ ❑

6.6 recursão à esquerda

Em diversas situações, como no desenvolvimento de algoritmos reconhecedores, é desejável que a gramática que define a linguagem não seja recursiva à esquerda.

Entende-se por *recursão à esquerda* a ocorrência da seguinte situação:

$$A \Rightarrow^+ A\alpha$$

Ou seja, uma variável deriva ela mesma, de forma direta ou indireta, como o símbolo mais à esquerda de uma subpalavra gerada.

A transformação de uma gramática qualquer em uma equivalente sem recursão à esquerda pode ser realizada, executando-se as quatro primeiras etapas do algoritmo referente à forma normal de Greibach.

definição 6.16 – Algoritmo: gramática sem recursões à esquerda

Seja **G = (V, T, P, S)** uma gramática livre do contexto. O *algoritmo para transformar em gramática sem recursão à esquerda* é constituído pelas seguintes etapas do algoritmo tal como na definição 6.15 – Algoritmo: forma normal de Greibach:

a *Etapa 1: simplificação da gramática*;
b *Etapa 2: renomeação das variáveis em uma ordem crescente qualquer*;
c *Etapa 3: transformação de produções para a forma $A_r \rightarrow A_s\alpha$, na qual $r \leq s$*;
d *Etapa 4: exclusão das recursões da forma $A_r \rightarrow A_r\alpha$.* ❑

6.7 → autômato com pilha

Analogamente às linguagens regulares, a classe das linguagens livres do contexto pode ser associada a um formalismo do tipo autômato, denominado *autômato com pilha*.

O autômato com pilha é análogo ao autômato finito, incluindo uma *pilha* como memória auxiliar e a facilidade de não determinismo. A pilha é independente da fita de entrada e não possui limite máximo de tamanho, o que implica uma noção de "tão grande quanto se queira". Portanto, é baseada na noção de conjunto *infinitamente contável*. Estruturalmente, sua principal característica é que o último símbolo gravado é o primeiro a ser lido, como ilustrado na figura 6.22. A *base* de uma pilha é fixa e define o seu início. O *topo* é variável e define a posição do último símbolo gravado.

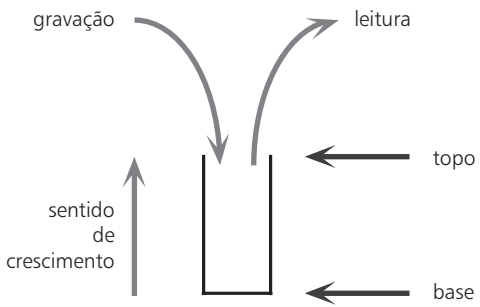


figura 6.22 Estrutura do tipo pilha.

A facilidade de não determinismo é importante e necessária, pois aumenta o poder computacional dos autômatos com pilha, permitindo reconhecer exatamente a classe das linguagens livres do contexto. Por exemplo, o reconhecimento da linguagem na qual a segunda metade de cada palavra é a reversa da primeira metade, ou seja (suponha que $\mathbf{w}^r$ é **w** escrita de forma reversa):

$$\{ \mathbf{ww}^r \mid \mathbf{w} \text{ é palavra sobre } \{ \mathbf{a, b} \} \}$$

só é possível por um *autômato com pilha não determinístico*.

Um resultado interessante mostrado adiante é que qualquer linguagem livre do contexto pode ser reconhecida por um autômato com pilha com somente um estado (ou três estados, dependendo da definição). Isso significa que a estrutura de pilha é suficiente como única memória, não sendo necessário usar os estados para "memorizar" informações passadas. Ou seja, a estrutura de estados no autômato com pilha poderia ser excluída sem se reduzir o poder computacional. Como consequência, e considerando-se que a pilha não possui tamanho máximo, o autômato com pilha pode assumir tantos estados quanto se queira.

6.7.1 definição do autômato com pilha

O modelo autômato com pilha possui duas definições universalmente aceitas que diferem no critério de parada do autômato, como segue:

- o valor inicial da pilha é vazio e o autômato para aceitando ao atingir um estado final;
- a pilha contém, inicialmente, um símbolo especial denominado símbolo inicial da pilha. Não existem estados finais, e o autômato para aceitando quando a pilha estiver vazia.

As duas definições são equivalentes (possuem o mesmo poder computacional), sendo fácil modificar um autômato com pilha para satisfazer a outra definição. Neste livro, é adotada a definição que usa estados finais.

Um *autômato com pilha não determinístico* ou *autômato com pilha* é composto, basicamente, por quatro partes, como segue:

a *Fita*. Análoga à do autômato finito;

b *Pilha*. Memória auxiliar do tipo pilha que pode ser usada para leitura e gravação;

c *Unidade de controle*. Reflete o estado corrente da máquina. Possui uma cabeça de fita e uma cabeça de pilha;

d *Programa*, *função programa* ou *função de transição*. Comanda a leitura da fita, leitura e gravação da pilha e define o estado da máquina.

A pilha é dividida em células, armazenando, cada uma, um símbolo do alfabeto auxiliar (pode ser igual ao alfabeto de entrada). A leitura ou gravação é sempre no topo. Não possui tamanho fixo, nem máximo, sendo seu tamanho corrente igual ao tamanho da palavra armazenada. Seu valor inicial é vazio (contém a palavra vazia).

A unidade de controle possui um número *finito* e *predefinido* de estados. Possui uma cabeça de fita e uma cabeça de pilha, como segue:

a *Cabeça da fita*. Unidade de leitura que acessa uma célula da fita de cada vez e se movimenta exclusivamente para a direita. É possível testar se a entrada foi completamente lida;

b *Cabeça da pilha*. Unidade de leitura e gravação a qual se move para a esquerda (ou "para cima") ao gravar, e para a direita (ou "para baixo") ao ler um símbolo. Acessa um símbolo de cada vez, estando sempre posicionada no topo. A leitura exclui o símbolo lido. É possível testar se a pilha está vazia. Em uma mesma gravação, é possível armazenar uma palavra composta por mais de um símbolo. Neste caso, o símbolo do topo é o mais à esquerda da palavra gravada.

Observe que, embora a unidade de controle possua um número finito e predefinido de estados, tal unidade não é dita de controle finito (em oposição aos autômatos finitos), pois o conteúdo da pilha também caracteriza o estado geral do sistema.

O programa é uma função parcial tal que:

> *dependendo do estado corrente, símbolo lido da fita e símbolo lido da pilha, determina o novo estado e a palavra a ser gravada (na pilha)*

possui a facilidade de movimento vazio (análoga à do autômato finito), permitindo mudar de estado sem ler da fita.

definição 6.17 – Autômato com pilha

Um *autômato com pilha não determinístico* ou simplesmente *autômato com pilha* (frequentemente abreviado por **AP**) **M** é uma **6**-upla:

$$\mathbf{M} = (\Sigma, \mathbf{Q}, \delta, \mathbf{q}_0, \mathbf{F}, \mathbf{V})$$

na qual:

a Σ é um *alfabeto de símbolos de entrada* ou simplesmente *alfabeto de entrada*;

b **Q** é um *conjunto de estados* possíveis do autômato o qual é finito;

c δ é uma *função programa* ou simplesmente *programa* ou ainda, *função de transição* (suponha que ε e **?** não são símbolos do alfabeto de entrada):

$$\delta\colon \mathbf{Q} \times (\Sigma \cup \{\varepsilon, ?\}) \times (\mathbf{V} \cup \{\varepsilon, ?\}) \rightarrow 2^{\mathbf{Q}\times\mathbf{V}^*}$$

a qual é uma função total. Assim, para $\mathbf{p} \in \mathbf{Q}$, $\mathbf{x} \in \Sigma \cup \{\varepsilon, ?\}$ e $\mathbf{y} \in \mathbf{V} \cup \{\varepsilon, ?\}$:

$$\delta(\mathbf{p}, \mathbf{x}, \mathbf{y}) = \{ (\mathbf{q}_1, \mathbf{v}_1),\ldots,(\mathbf{q}_n, \mathbf{v}_n) \}$$

é uma *transição* do autômato;

d $\mathbf{q}_0$ é um elemento distinguido de **Q**, denominado *estado inicial*;

e **F** é um subconjunto de **Q**, denominado *conjunto de estados finais*;

f **V** é um *alfabeto auxiliar* ou *alfabeto da pilha*. ❑

As seguintes características da função programa devem ser consideradas:

- se para $\mathbf{p} \in \mathbf{Q}$, $\mathbf{x} \in \Sigma \cup \{\varepsilon, ?\}$ e $\mathbf{y} \in \mathbf{V} \cup \{\varepsilon, ?\}$ ocorre que $\delta(\mathbf{p}, \mathbf{x}, \mathbf{y}) = \varnothing$, então afirma-se que a transição é indefinida para **(p, x, y)** e portanto, o autômato para rejeitando a entrada;
- trata-se de uma função parcial e, portanto, pode ser indefinida para alguns argumentos do conjunto de partida;
- a omissão do parâmetro de leitura, representada por "**?**", indica o teste de pilha vazia ou toda palavra de entrada lida;
- o símbolo ε na leitura indica a facilidade de *movimento vazio* da fita ou da pilha (o autômato não lê, nem move a cabeça). Note-se que, para o movimento ser considerado não determinístico, é suficiente que o movimento seja vazio na fita;
- o símbolo ε na gravação indica que nenhuma gravação é realizada na pilha (e não move a cabeça).

Por exemplo:

$$\delta(p, ?, \varepsilon) = \{ (q, \varepsilon) \}$$

indica que, no estado **p**, se a entrada foi completamente lida, não lê da pilha, assume o estado **q** e não grava na pilha.

Analogamente aos autômatos finitos, a função programa de um autômato com pilha pode ser representada na forma de um diagrama, como ilustrado na figura 6.23 (suponha a transição $\delta(p, x, y) = \{ (q, v) \}$).

A *computação* de um *autômato com pilha*, para uma palavra de entrada **w**, consiste na sucessiva aplicação da função programa para cada símbolo de **w** (da esquerda para a direita) até ocorrer uma condição de parada. Entretanto, é possível que um autômato com pilha nunca atinja uma condição de parada. Neste caso, fica processando indefinidamente (ciclo ou *loop* infinito). Um exemplo simples de ciclo infinito é um programa que empilha e desempilha um mesmo símbolo indefinidamente, sem ler da fita.

Um autômato com pilha pode parar, aceitando ou rejeitando a entrada, ou ficar em *loop* infinito, como segue:

a *Aceita* a entrada **w**. Pelo menos um dos caminhos alternativos atinge um estado final (não importa se leu ou não toda a entrada): o autômato para, e a palavra **w** é aceita;

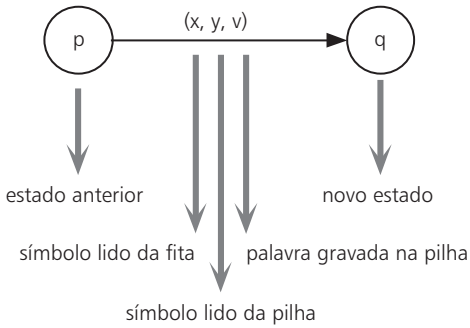


figura 6.23 Diagrama (**AP**): transição.

b *Rejeita* a entrada **w**. Todos os caminhos alternativos rejeitam a entrada (a função programa é indefinida para cada caso): o autômato para, e a palavra **w** é rejeitada;

c Fica em *loop* para a entrada **w**. Pelo menos um caminho alternativo está em *loop* infinito e os demais rejeitam ou também estão em *loop* infinito: o autômato está em *loop* infinito.

Para definir formalmente o comportamento de um autômato com pilha, é necessário estender a definição da função programa, usando-se como argumento um estado e uma palavra. Esta extensão é sugerida como exercício.

definição 6.18 – Linguagem aceita, linguagem rejeitada, linguagem *loop*

Seja **M = (Σ, Q, δ, q_0, F, V)** um autômato com pilha. Então:

a A *linguagem aceita* ou *linguagem reconhecida* por **M**, denotada por:

ACEITA(M) ou **L(M)**

é o conjunto de todas as palavras pertencentes a Σ* aceitas por **M**, a partir do estado inicial q_0;

b A *linguagem rejeitada* por **M**, denotada por:

REJEITA(M)

é o conjunto de todas as palavras pertencentes a Σ* rejeitadas por **M**, a partir do estado inicial q_0;

c A *linguagem loop* de **M**, denotada por:

LOOP(M)

é o conjunto de todas as palavras pertencentes a Σ* para as quais **M** fica processando indefinidamente a partir do estado inicial q_0. ❑

Cada autômato com pilha **M** definido sobre o alfabeto Σ induz uma partição do conjunto de todas as palavras Σ* em três classes de equivalência: **ACEITA(M)**, **REJEITA(M)** e **LOOP(M)**, conforme ilustrado na figura 6.24. Sendo preciso, se um ou dois dos conjuntos for vazio, então a partição induzida contém um ou dois conjuntos a menos (lembre-se de que uma classe de equivalência não pode ser vazia).

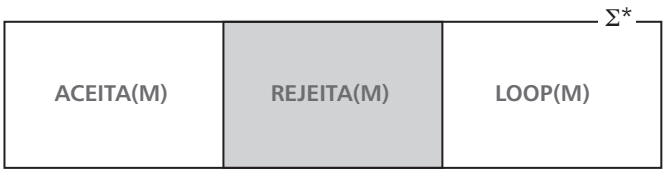


figura 6.24 Partição de Σ*, induzida por um autômato com pilha **M**.

exemplo 6.15 – Autômato com pilha: duplo balanceamento

Considere o exemplo 6.1 – Gramática livre do contexto: duplo balanceamento. O seguinte autômato com pilha:

$$M_1 = (\{ a, b \}, \{ q_0, q_1, q_f \}, \delta_1, q_0, \{ q_f \}, \{ B \})$$

no qual δ_1 é como abaixo, é tal que **ACEITA(M_1) = L_1** (o conjunto **LOOP(M_1)** é vazio?):

$\delta_1 (q_0, a, \varepsilon) = \{ (q_0, B) \}$
$\delta_1 (q_0, b, B) = \{ (q_1, \varepsilon) \}$
$\delta_1 (q_0, ?, ?) = \{ (q_f, \varepsilon) \}$
$\delta_1 (q_1, b, B) = \{ (q_1, \varepsilon) \}$
$\delta_1 (q_1, ?, ?) = \{ (q_f, \varepsilon) \}$

O diagrama do autômato é ilustrado na figura 6.25. Observe que o autômato é determinístico. No estado q_0, para cada símbolo **a** lido da fita, é armazenado um símbolo **B** na pilha. No estado q_1, é realizado um batimento, verificando se, para cada símbolo **b** da fita, existe um correspondente **B** na pilha. O algoritmo somente aceita se, ao terminar de ler toda a palavra de entrada, a pilha estiver vazia. ❑

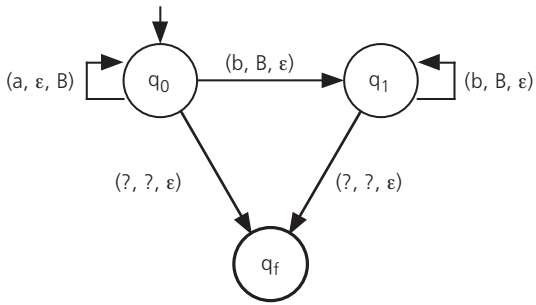


figura 6.25 Diagrama (**AP**): duplo balanceamento.

exemplo 6.16 – Autômato com pilha: palavra e sua reversa

Considere a seguinte linguagem sobre o alfabeto **{ a, b }**:

$$L_3 = \{ ww^r \mid w \text{ pertence a } \{ a, b \}^* \}$$

O autômato M_3 ilustrado na figura 6.26 é tal que **ACEITA(M_3) = L_3** (**LOOP(M_3)** é vazio?). O autômato é não determinístico devido ao movimento vazio de q_0 para q_1. O alfabeto auxiliar é igual ao de entrada. Em q_0 é empilhado o reverso do prefixo. A cada símbolo empilhado, ocorre um movimento não determinístico para q_1 o qual verifica se o sufixo da palavra é igual ao conteúdo da pilha. ❑

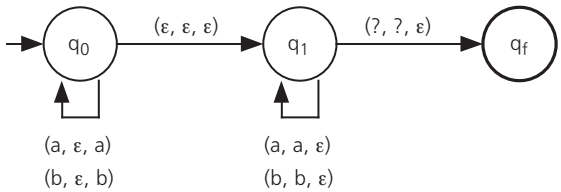


figura 6.26 Diagrama (**AP**): ww^r.

exemplo 6.17 – Autômato com pilha: $a^n b^m a^{n+m}$

Considere a seguinte linguagem sobre o alfabeto **{ a, b }**:

$$L_4 = \{ a^n b^m a^{n+m} \mid n \geq 0, m \geq 0 \}$$

O autômato não determinístico M_4 ilustrado na figura 6.27 é tal que **ACEITA(M_4) = L_4** (o conjunto **LOOP(M_4)** é vazio?). M_4 empilha um símbolo auxiliar **X** para cada **a** ou **b** em q_0 ou q_1, respectivamente. Após, em q_2, verifica se o número de **a** no sufixo é igual ao de **X** empilhado. ❑

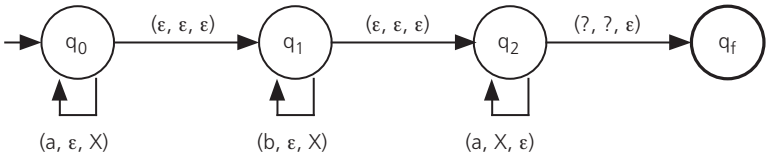


figura 6.27 Diagrama (**AP**): $a^n b^m a^{n+m}$.

6.7.2 autômato com pilha e linguagens livres do contexto

A classe das linguagens reconhecidas pelos autômatos com pilha é igual à classe das linguagens livres do contexto (ou seja, é igual à classe das linguagens geradas pelas gramáticas livres do contexto).

O teorema a seguir apresenta a construção de um autômato com pilha a partir de uma gramática livre do contexto qualquer, permitindo estabelecer as seguintes conclusões:

- a construção de um reconhecedor para uma linguagem livre do contexto a partir de sua gramática é simples e imediata;
- qualquer linguagem livre do contexto pode ser aceita por um autômato com pilha com somente um estado de controle lógico, o que significa que a facilidade de memorização de informações através de estados (como nos autômatos finitos) não aumenta o poder computacional dos autômatos com pilha.

teorema 6.19 – Gramática livre do contexto → autômato com pilha

Se **L** é uma linguagem livre do contexto, então existe **M**, autômato com pilha tal que **ACEITA(M) = L**.

➥ **Prova:**
Suponha que a palavra vazia não pertence a **L**. A demonstração consiste na construção de um autômato com pilha a partir da gramática transformada na forma normal de Greibach (produções da forma $A \rightarrow a\alpha$, α palavra de variáveis). O autômato com pilha gerado simula a derivação mais à esquerda, como segue (suponha a produção $A \rightarrow a\alpha$):

- lê o símbolo **a** da fita;
- lê o símbolo **A** da pilha;
- empilha a palavra α.

A simulação acima é realizada para cada produção, usando um único estado de controle. A construção do autômato com pilha **M** a partir da gramática **G = (V, T, P, S)** é como segue (veja a figura 6.28):

- seja $G_{FNG} = (V_{FNG}, T_{FNG}, P_{FNG}, S)$, a transformação de **G** na forma normal de Greibach;
- seja $M = (T_{FNG}, \{ q_0, q_1, q_f \}, \delta, q_0, \{ q_f \}, V_{FNG})$, em que:
  $\delta(q_0, \varepsilon, \varepsilon) = \{ (q_1, S) \}$
  $\delta(q_1, a, A) = \{ (q_1, \alpha) \mid A \rightarrow a\alpha \in P_{FNG} \}$
  $\delta(q_1, ?, ?) = \{ (q_f, \varepsilon) \}$

A demonstração de que $ACEITA(M) = GERA(G_{FNG})$ é realizada por indução no número de movimentos de **M** (ou de derivações de G_{FNG}) e é sugerida como exercício. Como o autômato pode ser modificado para tratar a palavra vazia? ❑

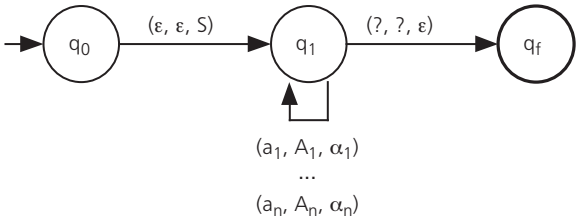


figura 6.28 Diagrama (**AP**): a partir de uma gramática na forma normal de Greibach.

exemplo 6.18 – Gramática livre do contexto → Autômato com pilha

Considere a seguinte linguagem (compare com a apresentada no exemplo 6.1 – Gramática livre do contexto: duplo balanceamento. Por que a diferença?):

$$L_5 = \{ a^n b^n \mid n \geq 1 \}$$

representada pela seguinte gramática na forma normal de Greibach:

G_5 = ({ S, B }, { a, b }, P_5, S), na qual:
P_5 = { S → aB | aSB, B → b }

é aceita pelo seguinte autômato com pilha (ver figura 6.29) construído a partir de **G_5**:

$$M_5 = (\{ a, b \}, \{ q_0, q, q_f \}, \delta_5, q_0, \{ q_f \}, \{ S, B \})$$ ❑

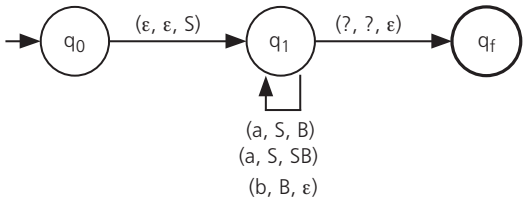


figura 6.29 Diagrama (**AP**): a partir de uma gramática na forma normal de Greibach.

As seguintes proposições são corolários do teorema – 6.19 – Gramática livre do contexto → autômato com pilha.

corolário 6.20 – Autômato com pilha × número de estados

Se **L** é uma linguagem livre do contexto, então:

a Existe **M**, autômato com pilha com controle de aceitação por estados finais, com somente três estados, tal que **ACEITA(M) = L**;

b Existe **M**, autômato com pilha com controle de aceitação por pilha vazia, com somente um estado tal que **ACEITA(M) = L**. ❑

O detalhamento da demonstração do item b) do corolário acima é simples e é sugerida como exercício.

corolário 6.21 – Existência de um autômato com pilha que sempre para

Se **L** é uma linguagem livre do contexto, então existe **M**, autômato com pilha, tal que:

ACEITA(M) = L
REJEITA(M) = Σ* - L
LOOP(M) = ∅ ❑

Ou seja, para qualquer linguagem livre do contexto, existe um autômato com pilha que sempre para para qualquer entrada (por quê?).

A demonstração do seguinte teorema é omitida.

teorema 6.22 – Autômato com pilha → gramática livre do contexto

Se **L** é aceita por um autômato com pilha, então **L** é linguagem livre do contexto. ❑

observação 6.23 – Estados × poder computacional dos autômatos com pilha

A combinação dos seguintes resultados:

- Corolário 6.20 – Autômato com pilha × número de estados;
- Corolário 6.21 – Existência de um autômato com pilha que sempre para;
- Teorema 6.22 – Autômato com pilha → gramática livre do contexto;

comprovam que o uso dos estados como "memória" não aumenta o poder de reconhecimento do autômato com pilha. ❑

6.7.3 número de pilhas e o poder computacional

O autômato com pilha é um modelo frequentemente referenciado em estudos aplicados e formais pois, além de a estrutura de pilha ser adequada para implementação em computadores, poucas modificações sobre a definição básica determinam significativas alterações no poder computacional. Especificamente, os principais estudos de linguagens e computabilidade podem ser desenvolvidos, usando-se exclusivamente o autômato com pilha, variando-se o número de pilhas com ou sem a facilidade de não determinismo. Os seguintes resultados podem ser enunciados:

a *Autômato com pilha, sem usar a estrutura de pilha*. Se a estrutura de pilha não for usada, a única forma de memorizar informações passadas é usar os estados. Assim, claramente,

o autômato com pilha sem usar a pilha é muito semelhante ao autômato finito. De fato, é fácil mostrar que os autômatos com pilha, sem usar a pilha, com ou sem a facilidade de não determinismo, reconhecem a classe das linguagens regulares (o que se sugere como exercício);

b *Autômato com pilha determinístico*. Trata-se de um autômato com pilha, mas sem a facilidade de ser não determinístico. Esta classe de autômatos aceita um importante subconjunto próprio da classe das linguagens livres do contexto, denominado *classe das linguagens livres do contexto determinísticas*. Uma das razões é que a implementação de um autômato com pilha determinístico em um computador é simples e eficiente, facilitando, assim, o desenvolvimento de analisadores sintáticos. Algumas propriedades da classe das linguagens livres do contexto determinísticas são as seguintes:

b.1) É possível definir um tipo de gramática que gera exatamente tal classe. Entretanto, não são restrições simples sobre a definição geral de gramática (o que se sugere como exercício de pesquisa);

b.2) É fechada para a operação de complemento;

b.3) Não é fechada para as operações de união, intersecção e concatenação;

c *Autômato com pilha não determinístico*. Já foi visto que os autômatos com pilha não determinísticos aceitam exatamente a classe das linguagens livres do contexto;

d *Autômato com duas pilhas*. O autômato com duas pilhas possui o mesmo poder computacional da *máquina de Turing* (estudada no capítulo 8 – Linguagens recursivamente enumeráveis e sensíveis ao contexto), considerada o dispositivo mais geral de computação. Assim, se existe um algoritmo para resolver um problema (por exemplo, reconhecer uma determinada linguagem), então esse algoritmo pode ser expresso como um autômato com duas pilhas. A facilidade de não determinismo não aumenta o poder computacional deste tipo de autômato;

e *Autômato com múltiplas pilhas*. O poder computacional de um autômato com mais de duas pilhas é equivalente ao do autômato com duas pilhas. Ou seja, se um problema é solucionado por um autômato com múltiplas pilhas, então o mesmo problema pode ser solucionado por um autômato com duas pilhas.

6.8 → exercícios

exercício 6.1 Sobre as linguagens livres do contexto:

a Qual a importância do seu estudo?

b Exemplifique suas aplicações;

c Faça um quadro comparativo com as linguagens regulares, destacando as principais características, semelhanças e diferenças.

exercício 6.2 Considere o exemplo 6.2 – Gramática livre do contexto: expressões aritméticas:

a É possível gerar a mesma expressão **[x+x]∗x** com uma sequência de derivação diferente da apresentada no exemplo? Neste caso, quantas sequências distintas são possíveis?

b Identifique quais produções controlam o duplo balanceamento de parênteses.

exercício 6.3 Desenvolva gramáticas livres do contexto que gerem as seguintes linguagens (suponha que w^r é **w** escrita de forma reversa):

a ∅

b { ε }

c { a, b }*

d { ww^r | **w** é palavra de { a, b }* }

e { **w** | **w** é palíndromo em { a, b }* } (palíndromo significa que $w = w^r$)
Dica: diferencie claramente esta linguagem da apresentada no item anterior;

f { $a^i b^j c^k$ | i = j ou j = k e i, j, k ≥ 0 }

g { **w** | **w** é palavra de { x, y, (,) }* com parênteses balanceados }

h { $a^n w b^n$ | **w** é palavra de { a, a }* }

i { **w** | **w** é expressão regular sobre o alfabeto { x } }

exercício 6.4 Para a linguagem de programação esboçada abaixo (inspirada em Pascal), construa uma BNF:

- os `comandos` podem ser como segue:
  `simples`,
  `composto`,
  `enquanto-faça`,
  `repita-até`;

- `comando simples`:
  qualquer palavra de { `a, b` }*;

- `comando composto`:
  `i` (início),
  seguido de um ou mais comandos separados por "`;`",
  seguidos de `t` (término);

- `comando enquanto-faça`:
  `e` (enquanto),
  seguido de uma `expressão`,
  seguida de `f` (faça),
  seguida de um `comando`;

- `comando repita-até`:
  `r` (repita),
  seguido de um `comando`,
  seguido de `a` (até),
  seguida de uma `expressão`;
- `expressão`: como definida no exemplo 6.2 – Gramática livre do contexto: expressões aritméticas, excetuando-se que possui **a** e **b** como operandos.

exercício 6.5 Considere a seguinte gramática:

G = ({ S }, { a, b }, P, S), na qual:
P = { S → SS | aSa bSb | ε }

a Qual a linguagem gerada?
b A gramática é ambígua?
c Para a palavra **aabbaaaa**:
- construa uma árvore de derivação;
- para a árvore construída, determine as derivações mais à esquerda e a mais à direita.

exercício 6.6 Considere:

- exemplo 6.2 – Gramática livre do contexto: expressões aritméticas;
- exemplo 6.7 – Gramática ambígua;

Construa uma gramática não ambígua equivalente.

Sugestão: faça uma pesquisa bibliográfica e verifique como são definidas, usando gramáticas, expressões em algumas linguagens de programação reais. A definição de uma "expressão simples" na linguagem Pascal é um bom exemplo.

exercício 6.7 Considere a seguinte gramática:

G = ({ S, A, B }, { a, b }, P, S), na qual:
P = { S → aS | bA | B | _, B → aA | S, A → aAb | SB }

Justifique ou refute:

a Está simplificada relativamente aos símbolos inúteis;
b Está simplificada relativamente às produções vazias;
c Está simplificada relativamente às produções que substituem variáveis.

exercício 6.8 Considere a seguinte gramática:

$G = (\{ S, X, Y, Z, A, B \}, \{ a, b, u, v \}, P, S)$, onde:
$P = \{ S \rightarrow XYZ,$
$X \rightarrow AXA \mid BXB \mid Z \mid \varepsilon,$
$Y \rightarrow AYB \mid BYA \mid Z \mid \varepsilon,$
$A \rightarrow a, B \rightarrow b$
$Z \rightarrow Zu \mid Zv \mid \varepsilon \}$

a Qual a linguagem gerada?

b Simplifique a gramática.

exercício 6.9 Considere o definição 6.9 – Algoritmo: exclusão dos símbolos inúteis. Por que, se a *etapa 2* for executada antes da *etapa 1*, o resultado pode não ser o esperado?

exercício 6.10 Sobre simplificação de gramáticas:

a Quais algoritmos de simplificação apenas *restringem* (excluem elementos de conjuntos) a gramática original (se necessário)?

b Quais algoritmos de simplificação *modificam* (modificam elementos de conjuntos) a gramática original (se necessário)?

exercício 6.11 Sobre simplificação de gramáticas:

a Considere uma gramática resultante da aplicação do algoritmo símbolos inúteis. Qual o efeito sobre a gramática se o algoritmo for aplicado novamente?

b Analogamente para o algoritmo produções vazias?

c Analogamente para o algoritmo produções que substituem variáveis?

exercício 6.12 Considere a seção 6.4.4 – Simplificações combinadas. Por que a ordem recomendada deve ser respeitada? Justifique a ordem de cada algoritmo.

exercício 6.13 Para as gramáticas abaixo, construa as gramáticas equivalentes na forma normal de Chomsky:

a Gramática construída para a seguinte linguagem no **Exercício 6.3**:

$$\{ w \mid w \text{ é expressão regular sobre o alfabeto } \{ x \} \}$$

b Gramática construída para a linguagem de programação do **Exercício 6.4**.

exercício 6.14 Para as gramáticas abaixo, construa as gramáticas equivalentes na forma normal de Greibach:

a Gramática construída para a seguinte linguagem no **Exercício 6.3**:

$$\{ w \mid w \text{ é expressão regular sobre o alfabeto } \{ x \} \}$$

b Gramática construída para a linguagem de programação do **Exercício 6.4**.

exercício 6.15 Por que os seguintes algoritmos de simplificação de gramáticas:

- Produções vazias;
- Produções que substituem variáveis;

devem ser executados antes dos algoritmos para transformar uma gramática livre do contexto na:

a Forma normal de Chomsky?
b Forma normal de Greibach?

exercício 6.16 Relativamente às formas normais:

a Considere uma gramática resultante da aplicação do algoritmo forma normal de Chomsky. Qual o efeito sobre a gramática se o algoritmo for aplicado novamente?
b Analogamente para a forma normal de Greibach?

exercício 6.17 Uma gramática pode estar simultaneamente na forma normal de Greibach e na forma normal de Chomsky?

exercício 6.18 Qual o efeito da aplicação do algoritmo forma normal de Greibach sobre uma gramática na forma normal de Chomsky? E vice-versa?

exercício 6.19 Relativamente às formas normais e gramáticas regulares, justifique ou refute:

Dica: revise a definição de gramática regular.

a O algoritmo forma normal de Chomsky, quando aplicado a uma gramática regular, sempre resulta em uma gramática regular;
b Analogamente para o algoritmo forma normal de Greibach.

exercício 6.20 Explique, em palavras, como recursões à esquerda de uma gramática livre do contexto podem ser substituídas por recursões à direita.

exercício 6.21 Desenvolva autômatos com pilha que aceitem as seguintes linguagens:

a $\varnothing$
b $\{ \varepsilon \}$
c $\{ a, b \}^*$
d $\{ w \mid w$ é palíndromo em $\{ a, b \}^* \}$
e $\{ a^n w b^n \mid w$ é palavra de $\{ a, a \}^* \}$
f $\{ w \mid w$ é expressão regular sobre o alfabeto $\{ x \} \}$
g $\{ u a^n v a^n w \mid n \in \{ 1, 2 \}, u, v, w$ são palavras de $\{ a, b \}^*$ e $|u| = |v| = 5 \}$

exercício 6.22 Construa um autômato com pilha que nunca para (fica em *loop* infinito) para qualquer entrada.

exercício 6.23 O autômato com pilha ilustrado na figura 6.30, para ou fica em *loop* infinito? Justifique a sua resposta.

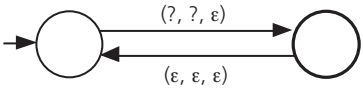


figura 6.30 Diagrama (**AP**).

exercício 6.24 A linguagem aceita pelo autômato com pilha ilustrado na figura 6.31 é regular? Justifique a sua resposta.

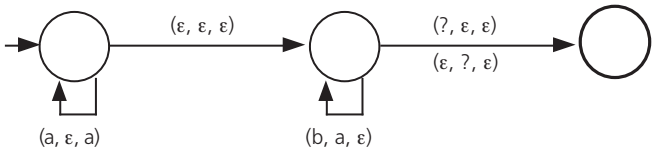


figura 6.31 Diagrama (**AP**).

exercício 6.25 Para a gramática na forma normal de Greibach desenvolvida no **exercício 6.21** relativamente à linguagem:

{ w | **w** é expressão regular sobre o alfabeto **{ x } }**

a Construa o correspondente autômato com pilha, conforme o teorema 6.19 – Gramática livre do contexto → Autômato com pilha;

b Usando o autômato construído, faça o reconhecimento da entrada **(x+x)***

exercício 6.26 Por que se pode afirmar que, para qualquer linguagem livre do contexto, é possível garantir que existe pelo menos um autômato com pilha que aceita a linguagem e que sempre para para qualquer entrada?

exercício 6.27 Considere o teorema 6.19 – Gramática livre do contexto → autômato com pilha. Refute a seguinte afirmação:

> *o tempo de processamento de um autômato com pilha gerado a partir de uma gramática livre do contexto (conforme o teorema em questão) é sempre diretamente proporcional ao tamanho da entrada pois, a cada símbolo lido, executa exatamente uma transição.*

exercício 6.28 Estenda a função programa do autômato com pilha, usando como argumento um estado e uma palavra, de forma similar à realizada para os autômatos finitos.

exercício 6.29 Demonstre que o autômato com pilha que não usa a estrutura de pilha para armazenar informações do processamento possui o mesmo poder computacional do autômato finito.

exercício 6.30 Explique intuitivamente por que as seguintes linguagens não são livres do contexto:

Dica: para cada caso, tente desenvolver o correspondente autômato com pilha.

a $\{ ww \mid w$ é palavra de $\{ a, b \}^* \}$

b $\{ a^n b^n a^m \mid n \geq 0, m \geq 0$ e $n \neq m \}$

exercício 6.31 Desenvolva um programa em computador que implemente os seguintes algoritmos:

a Simplificação (símbolos inúteis, produções vazias e produções que substituem variáveis e suas combinações);

b Forma normal de Chomsky;

c Forma normal de Greibach;

d Recursão à esquerda.

exercício 6.32 Desenvolva um programa em computador que simule o processamento de uma autômato com pilha. A composição deste programa com o da forma normal de Greibach (exercício anterior) resulta em um algoritmo de reconhecimento genérico para linguagens livres do contexto (como tratar a palavra vazia?).

Dica: especial atenção na simulação do não determinismo.

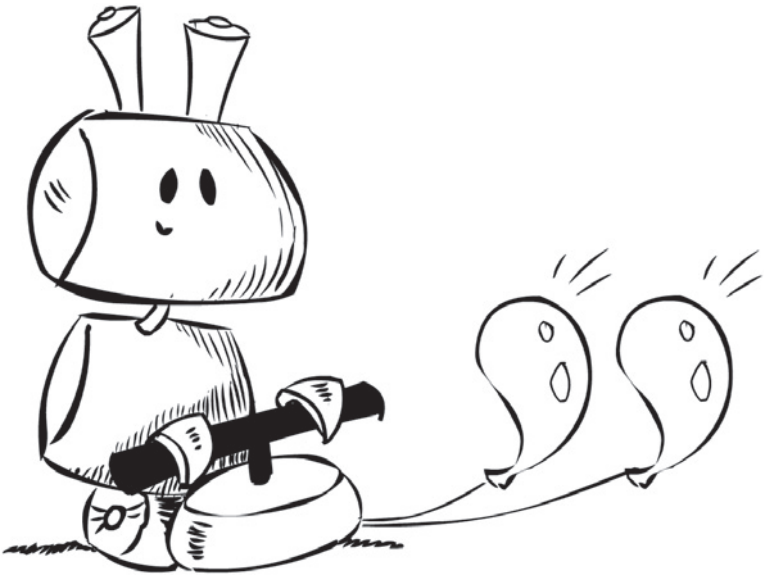
exercício 6.33 Intuitivamente, das linguagens do **exercício 6.21**, quais são *linguagens livres do contexto determinísticas*? Justifique cada caso.

exercício 6.34 Faça uma pesquisa bibliográfica sobre o tipo de gramática que gera exatamente a *classe das linguagens livres do contexto determinísticas.*

capítulo 7

propriedades e reconhecimento das linguagens livres do contexto

A classe das linguagens livres do contexto contém propriamente a classe das linguagens regulares e compreende a sintaxe da maioria das linguagens de programação de propósitos gerais. Portanto, é uma classe especialmente importante e o estudo de suas propriedades é central no estudos das linguagens formais, com destaque para as operações fechadas nesta classe e algoritmos de reconhecimento.



Embora a classe das linguagens livres do contexto contenha propriamente a classe das linguagens regulares, ainda é relativamente restrita. De fato, é fácil definir linguagens que não são livres do contexto, como, por exemplo:

a *Palavra duplicada*. Linguagem especialmente importante para computação e informática, pois permite estabelecer analogia com questões similares presentes em diversas linguagens de programação (em geral, verificações semânticas). Por exemplo, garantir a declaração de uma variável antes do seu uso. A linguagem é a seguinte:

$$\{ ww \mid w \text{ é palavra sobre } \{ a, b \} \}$$

b *Triplo balanceamento*. Caracteriza uma importante limitação das linguagens livres do contexto. De fato, a grande maioria das linguagens de programação não possui (propositalmente) estruturas desse tipo. A linguagem é a seguinte:

$$\{ a^n b^n c^n \mid n \geq 0 \}$$

Sugere-se como exercício verificar que comandos encadeados do tipo `se-então-senão` não constituem um triplo balanceamento.

Para ambas as linguagens, usando o formalismo autômato com pilha, é fácil intuir por que não são livres do contexto.

Assim, algumas questões sobre a classe das linguagens livres do contexto necessitam ser analisadas:

a Como determinar se uma linguagem é livre do contexto?

b A classe das linguagens livres do contexto é fechada para operações como união, intersecção, concatenação e complemento (ou seja, a operação de duas linguagens livres do contexto resulta em uma linguagem livre do contexto)?

c Como verificar se uma linguagem livre do contexto é infinita ou finita (ou até mesmo vazia)?

Entretanto, algumas questões relativas a esta classe de linguagens não possuem solução computacional. Um exemplo simples e importante é o seguinte:

d Não existe um algoritmo capaz de analisar duas linguagens livres do contexto quaisquer e concluir se são iguais ou diferentes. A solucionabilidade (ou não) de problemas é apenas introduzida neste livro (capítulo 8 – Linguagens recursivamente enumeráveis e sensíveis ao contexto), e seu detalhamento é usualmente desenvolvido no estudo da *teoria da computação*.

Verificar se uma determinada palavra pertence ou não a uma linguagem é uma das principais questões relacionadas com o estudo de linguagens formais. Relativamente aos algoritmos de reconhecimento, é importante determinar a "quantidade de recursos" (por exemplo: tempo e espaço) de que o algoritmo necessita ao longo de seu processamento. Deve-se destacar que o objetivo é gerar algoritmos de reconhecimento válidos para qualquer linguagem dentro de uma classe. Neste capítulo, os seguintes algoritmos apresentados são específicos para as linguagens livres do contexto:

- *Autômato com pilha descendente*;
- *Algoritmo de Cocke-Younger-Kasami*;
- *Algoritmo de Early*.

Esses algoritmos de reconhecimento são construídos a partir de uma gramática livre do contexto que define a linguagem. Os reconhecedores que usam autômato com pilha são muito simples, mas, em geral, ineficientes. Para uma entrada **w**, seu tempo de processamento é proporcional a $\mathbf{k}^{|\mathbf{w}|}$ (o valor de **k** depende do autômato), não sendo recomendáveis para entradas de tamanhos consideráveis. Existe uma série de algoritmos bem mais eficientes (não baseados em autômato com pilha), com tempo de processamento proporcional a $|\mathbf{w}|^3$ ou até um pouco menos. Não é provado se o tempo proporcional a $|\mathbf{w}|^3$ é efetivamente necessário para que um algoritmo genérico reconheça linguagens livres do contexto.

7.1 → propriedades das linguagens livres do contexto

7.1.1 investigação se é linguagem livre do contexto

Para mostrar que uma determinada linguagem é livre do contexto, é suficiente expressá-la usando os formalismos gramática livre do contexto ou autômato com pilha. Entretanto, a demonstração de que uma linguagem não é livre do contexto necessita ser realizada caso a caso.

Analogamente às linguagens regulares, as linguagens livres do contexto possuem um *lema do bombeamento*, o qual é útil no estudo das propriedades. Em particular, pode ser usado para verificar se determinada linguagem não é livre do contexto.

O lema é enunciado a seguir. Observe que estabelece a possibilidade de um duplo bombeamento balanceado.

lema 7.1 – Bombeamento para as linguagens livres do contexto

Se **L** é uma linguagem livre do contexto, então:

existe uma constante **n** tal que,
para qualquer palavra **w** de **L** onde $|w| \geq n$,
w pode ser definida como $w = u\,x\,v\,y\,z$ na qual:
$|x\,v\,y| \leq n$,
$|x\,y| \geq 1$
sendo que, para todo $i \geq 0$, $u\,x^i\,v\,y^i\,z$ é palavra de **L**. ❑

➥ **Prova:**
Uma maneira de demonstrar o lema do bombeamento é usar gramáticas na forma normal de Chomsky. Neste caso, se a gramática possui **s** variáveis, pode-se assumir que $n = 2^s$. A prova não será detalhada. ❑

No lema, para $w = u\,x\,v\,y\,z$, observe que **x** ou **y** pode ser a palavra vazia (mas não ambas), o que resulta em bombeamentos não balanceados, como nas linguagens regulares (de fato, toda linguagem regular é livre do contexto). Assim, comparativamente com as linguagens regulares, o duplo bombeamento balanceado é uma importante característica das linguagens livres do contexto. Tal fato pode ser facilmente verificado em algumas linguagens já apresentadas como, por exemplo:

- "duplo balanceamento": $\{ a^n b^n \mid n \geq 0 \}$
- "palavra e sua reversa": $\{ ww^r \mid w$ pertence a $\{ a, b \}^* \}$

As seguintes conclusões podem ser obtidas a partir dos lemas do bombeamento introduzidos:

- os formalismos regulares são capazes de expressar apenas bombeamentos sem qualquer balanceamento;
- os formalismos livres do contexto são capazes de expressar bombeamentos balanceados dois a dois.

Entretanto, os formalismos livres do contexto não são capazes de expressar um triplo bombeamento balanceado, como ilustrado no exemplo a seguir.

exemplo 7.1 – Linguagem não livre do contexto: triplo balanceamento

A seguinte linguagem sobre $\{ a, b, c \}$ não é livre do contexto:

$$L = \{ a^n\, b^n\, c^n \mid n \geq 0 \}$$

A prova que segue é por absurdo.

Suponha que **L** é livre do contexto. Então existe uma gramática na forma normal de Chomsky **G** com **s** variáveis que gera as palavras não vazias de **L**.

Sejam $r = 2^s$ e $w = a^r b^r c^r$

Pelo bombeamento, **w** pode ser definida como $w = u\,x\,v\,y\,z$ na qual:

$$|x\,v\,y| \leq r$$
$$|x\,y| \geq 1$$

sendo que, para todo $i \geq 0$, $u\,x^i\,v\,y^i\,z$ é palavra de **L**, o que é um absurdo. De fato, como $|x\,v\,y| \leq r$, não é possível supor que **x y** possui símbolos **a** e **c**, pois quaisquer ocorrências de **a** e **c** estão separadas por, pelo menos, **r** ocorrências de **b**. Consequentemente, **x y** jamais possuirá ocorrências de **a**, **b** e **c** simultaneamente. Assim, tem-se que:

- se **x y** possui somente símbolos **a**, é fácil verificar que a aplicação do bombeamento pode desbalancear as ocorrências de **a**, **b** e **c**;
- analogamente para os seguintes casos: somente símbolos **b**, somente símbolos **c**, somente símbolos **a** e **b** e somente símbolos **b** e **c**. ❑

7.1.2 operações fechadas sobre as linguagens livres do contexto

Como já foi visto, operações sobre linguagens podem ser usadas para:

- construir novas linguagens a partir de linguagens conhecidas (definindo-se uma álgebra);
- provar propriedades;
- construir algoritmos.

A classe das linguagens livres do contexto é fechada para as seguintes operações:

- união;
- concatenação;

Entretanto (e contrariamente à classe das linguagens regulares), não é fechada para as seguintes operações:

- intersecção;
- complemento.

No texto que segue, para uma determinada linguagem **L** sobre o alfabeto Σ^*, $\sim L$ denota o seu complemento (em relação ao conjunto universo Σ^*).

teorema 7.2 – Operações fechadas sobre linguagens livres do contexto

A classe das linguagens livres do contexto é fechada para as seguintes operações:

a união;
b concatenação.

➥ **Prova:**
União: (*direta*)
A demonstração que segue é baseada no formalismo autômato com pilha e usa a facilidade de não determinismo. Sugere-se como exercício a demonstração que usa o formalismo gramática livre do contexto.

Suponha L_1 e L_2, linguagens livres do contexto. Então, existem autômatos com pilha:

$$M_1 = (\Sigma_1, Q_1, \delta_1, q_{01}, F_1, V_1) \quad \text{e} \quad M_2 = (\Sigma_2, Q_2, \delta_2, q_{02}, F_2, V_2)$$

tais que $ACEITA(M_1) = L_1$ e $ACEITA(M_2) = L_2$. Seja M_3 construído como segue e ilustrado na figura 7.1 (suponha que $Q_1 \cap Q_2 \cap \{ q_0 \} = \varnothing$ e $V_1 \cap V_2 = \varnothing$):

$$M_3 = (\Sigma_1 \cup \Sigma_2, Q_1 \cup Q_2 \cup \{ q_0 \}, \delta_3, q_0, F_1 \cup F_2, V_1 \cup V_2)$$

Claramente, M_3 reconhece $L_1 \cup L_2$.

Concatenação: (*direta*)
A demonstração a seguir usa o formalismo gramática livre do contexto. Sugere-se como exercício a demonstração que usa o formalismo autômato com pilha.

Suponha L_1 e L_2, linguagens livres do contexto. Então, existem gramáticas livres do contexto:

$$G_1 = (V_1, T_1, P_1, S_1) \quad \text{e} \quad G_2 = (V_2, T_2, P_2, S_2)$$

tais que $GERA(G_1) = L_1$ e $GERA(G_2) = L_2$. Seja G_3 construída como segue (suponha que $V_1 \cap V_2 \cap \{ S \} = \varnothing$):

$$G_3 = (V_1 \cup V_2 \cup \{ S \}, T_1 \cup T_2, P_1 \cup P_2 \cup \{ S \rightarrow S_1 S_2 \}, S)$$

a qual é livre do contexto (por quê?). Como a única produção com a variável **S** no lado esquerdo é $S \rightarrow S_1 S_2$, claramente qualquer palavra gerada por G_3 terá, como prefixo, uma palavra de L_1 e, como sufixo, uma palavra de L_2. Logo, $L_1 L_2$ é linguagem livre do contexto. ❑

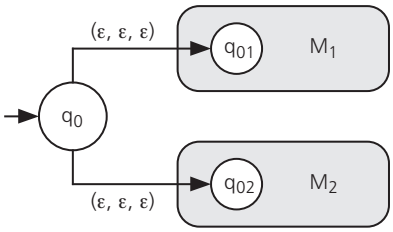


figura 7.1 Diagrama (**AP**): união.

O teorema a seguir mostra que a classe das linguagens livres do contexto não é fechada para as operações de intersecção e complemento. Observe que o fato de não ser fechada para a operação de complemento é, aparentemente, uma contradição, pois:

- foi verificado que, se **L** é linguagem livre do contexto, então existe **M**, autômato com pilha, tal que **ACEITA(M) = L** e **REJEITA(M) = ~L**. Ou seja, **M** é capaz de rejeitar qualquer palavra que não pertença a **L**;

- o teorema a seguir mostra que, se **L** é linguagem livre do contexto, não se pode afirmar que **~L** também é livre do contexto.

Assim, é perfeitamente possível um autômato com pilha *rejeitar* o complemento de uma linguagem livre do contexto, embora nem sempre seja possível *aceitar* o complemento da mesma. Uma explicação intuitiva, usando o formalismo autômato com pilha, é a seguinte, na qual, sem perda de generalidade, suponha que a função programa é total (por que sem perda de generalidade?):

- a condição para um autômato com pilha aceitar uma entrada é que pelo menos um dos caminhos alternativos reconheça a palavra (mesmo que os demais rejeitem). Essa condição pode ser representada pelo diagrama ilustrado na figura 7.2;
- se os estados de aceita e rejeita forem invertidos, na tentativa de aceitar o complemento, a situação resultante continua sendo de aceitação (e não de rejeição), pois permanecerá pelo menos um caminho alternativo reconhecendo a entrada. O diagrama ilustrado na figura 7.3 representa a "negação" da condição acima.

Portanto, considerando-se a facilidade de não determinismo, o fato de existir um autômato com pilha capaz de rejeitar o complemento de uma linguagem não implica que existe um autômato com pilha capaz de aceitar o mesmo complemento.

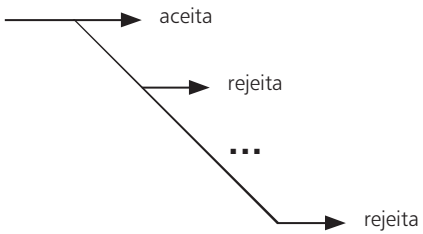


figura 7.2 Exemplo de situação não determinística para aceitação.

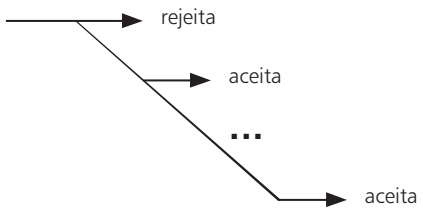


figura 7.3 Inversão das condições de **ACEITA/REJEITA**.

teorema 7.3 – Operações não fechadas sobre linguagens livres do contexto

A classe das linguagens livres do contexto não é fechada para as seguintes operações:

a intersecção;
b complemento.

➥ **Prova:**
Intersecção: (*direta*)
Para mostrar que a classe das linguagens livres do contexto não é fechada para a operação de intersecção, é suficiente apresentar um contraexemplo. Assim, sejam:

$$L_1 = \{ a^n b^n c^m \mid n \geq 0 \text{ e } m \geq 0 \} \quad \text{e} \quad L_2 = \{ a^m b^n c^n \mid n \geq 0 \text{ e } m \geq 0 \}$$

É fácil mostrar que L_1 e L_2 são linguagens livres do contexto. Entretanto, L_3 como abaixo, resultante da intersecção de L_1 com L_2, não é linguagem livre do contexto:

$$L_3 = \{ a^n b^n c^n \mid n \geq 0 \}$$

Complemento: (*direta*)
Como a operação de intersecção pode ser representada em termos da união e do complemento, e considerando que a classe das linguagens livres do contexto não é fechada para a operação de intersecção, então não se pode afirmar que o complemento de uma linguagem livre do contexto é livre do contexto. ❑

7.1.3 investigação se é vazia, finita ou infinita

O teorema a seguir mostra que é possível construir algoritmos para determinar se uma linguagem livre do contexto é vazia, finita ou infinita.

teorema 7.4 – Linguagem livre do contexto vazia, finita ou infinita

É possível determinar se uma linguagem livre do contexto qualquer é:

a vazia;
b finita;
c infinita.

➥ **Prova:** Suponha **L** uma linguagem livre do contexto.
Vazia: (*direta*)
Seja $G = (V, T, P, S)$, uma gramática livre do contexto tal que $GERA(G) = L$. Seja $G_{SI} = (V_{SI}, T_{SI}, P_{SI}, S)$ equivalente a **G**, excluindo os símbolos inúteis.

Se P_{SI} for vazio, então **L** é vazia.

Finita e infinita: (*direta*)
Seja $G = (V, T, P, S)$ uma gramática livre do contexto tal que $GERA(G) = L$. Seja $G_{FNC} = (V_{FNC}, T_{FNC}, P_{FNC}, S)$ equivalente a **G** na forma normal de Chomsky. Então:

a Se existe **A** variável de V_{FNC}, tal que:

- $A \rightarrow BC$, ou seja, **A** é referenciada no lado esquerdo de uma produção que não gera diretamente terminais;
- $X \rightarrow YA$ ou $X \rightarrow AY$, ou seja, se **A** é referenciada no lado direito de alguma produção;

então existe um ciclo em **A** do tipo:

$$A \Rightarrow^{+} \alpha\, A\, \beta$$

Portanto, **A** é capaz de gerar palavras de tamanho tão grande quanto se queira, e, consequentemente, **L** é infinita;

b Caso não exista tal **A**, então **L** é finita. ❑

7.2 → algoritmos de reconhecimento

Os algoritmos de reconhecimento podem ser classificados como segue:

a *Top-down* ou *preditivo*. Constrói uma árvore de derivação para a palavra de entrada (a ser reconhecida) a partir da raiz (símbolo inicial da gramática), gerando os ramos em direção às folhas (símbolos terminais que compõem a palavra);

b *Bottom-up*. É o oposto do *top-down*, partindo das folhas e construindo a árvore de derivação em direção à raiz.

7.2.1 autômato com pilha como reconhecedor

A construção de reconhecedores usando autômato com pilha é relativamente simples e imediata, havendo uma relação quase direta entre as produções da gramática e as transições do autômato. Os algoritmos apresentados são do tipo *top-down* e simulam a derivação mais à esquerda da palavra a ser reconhecida. A facilidade de não determinismo é usada para testar as diversas produções alternativas da gramática para gerar os símbolos terminais.

Já foi mostrado que qualquer linguagem livre do contexto pode ser especificada por um autômato com pilha. De fato, foi mostrado como construir um autômato com pilha a partir de uma gramática na forma normal de Greibach. Como cada produção (na forma normal de Greibach) gera exatamente um terminal, uma palavra **w** é gerada em $|w|$ etapas de derivação. Entretanto, como cada variável pode ter mais de uma produção associada, é necessário testar, sistematicamente, as diversas alternativas. Assim, o número de passos para reconhecer **w** é proporcional a $k^{|w|}$, sendo que **k** depende do autômato com pilha (uma aproximação para **k** é a metade do número médio de produções associadas às diversas variáveis). Portanto, o autômato construído possui um tempo de reconhecimento proporcional ao expoente em $|w|$, o que pode ser muito ineficiente para entradas mais longas.

Existe uma forma alternativa de construir um autômato com pilha a partir de uma gramática livre do contexto. Trata-se de um algoritmo igualmente simples e com o mesmo nível de eficiência. O autômato resultante, denominado de *autômato com pilha descendente*, é construído a partir de uma gramática sem recursão à esquerda e simula a derivação mais à esquerda, como segue:

- inicialmente, empilha o símbolo inicial;
- sempre que existir uma variável no topo da pilha, substitui esta (de forma não determinística) por todas as produções da variável;
- se o topo da pilha for um terminal, verifica se é igual ao próximo símbolo da entrada.

definição 7.5 – Algoritmo: autômato com pilha descendente

O *algoritmo para construção do autômato com pilha descendente* **M** a partir de uma gramática **G = (V, T, P, S)**, sem recursão à esquerda, é como segue e é ilustrado na figura 7.4:

$M = (T, \{ q_0, q_1, q_f \}, \delta, q_0, \{ q_f \}, V \cup T)$, na qual:
$\delta(q_0, \varepsilon, \varepsilon) = \{ (q_1, S) \}$
$\delta(q_1, \varepsilon, A) = \{ (q_1, \alpha) \mid A \rightarrow \alpha \in P \}$, para toda variável **A** de **V**
$\delta(q_1, a, a) = \{ (q_1, \varepsilon) \}$, para todo terminal **a** de **T**
$\delta(q_1, ?, ?) = \{ (q_f, \varepsilon) \}$ ❑

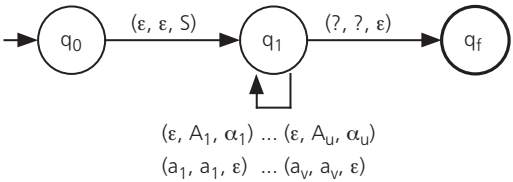


figura 7.4 Diagrama (**AP** descendente): a partir de uma gramática sem recursão à esquerda.

exemplo 7.2 – Autômato com pilha descendente: duplo balanceamento

A linguagem $L = \{ a^n b^n \mid n \geq 1 \}$, definida pela gramática (sem recursão à esquerda):

$G = (\{ S \}, \{ a, b \}, P, S)$, na qual:
$P = \{ S \rightarrow aSb \mid ab \}$

é reconhecida pelo seguinte autômato com pilha descendente, o qual é ilustrado na figura 7.5:

$M = (\{ a, b \}, \{ q_0, q_1, q_f \}, \delta, q_0, \{ q_f \}, \{ S, a, b \})$ ❑

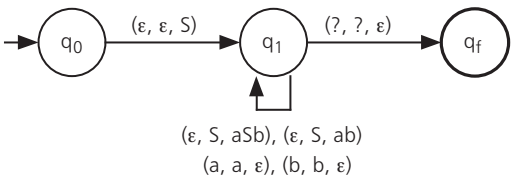


figura 7.5 Diagrama (**AP** descendente): duplo balanceamento.

7.2.2 algoritmo de Cocke-Younger-Kasami

O *algoritmo de Cocke-Younger-Kasami* foi desenvolvido independentemente por J. Cocke, D. H. Younger e T. Kasami, em 1965. A partir de uma gramática na forma normal de Chomsky, gera *bottom-up* todas as árvores de derivação da entrada em um tempo de processamento proporcional a $|w|^3$, para uma entrada **w**. A ideia básica do algoritmo é a construção de uma tabela triangular de derivação, sendo que cada célula contém o conjunto de raízes que pode gerar a correspondente subárvore.

definição 7.6 – Algoritmo de Cocke-Younger-Kasami

Suponha:

$G = (V, T, P, S)$ uma gramática livre do contexto na forma normal de Chomsky;
$w = a_1a_2...a_n$ uma entrada a ser verificada.

O *algoritmo de Cocke-Younger-Kasami* (ou simplesmente *algoritmo de CYK*) é composto pelas seguintes etapas, em que V_{rs} representa as células de uma tabela triangular de derivação genericamente ilustrada na figura 7.6:

*Etapa 1: variáveis que geram diretamente terminais (**A → a**)*. O algoritmo é apresentado na figura 7.7;

*Etapa 2: produção que gera duas variáveis (**A → BC**)*. O algoritmo é apresentado na figura 7.8. Observe que:

- limite de iteração para **r** é $(n - s + 1)$, pois a tabela é triangular;
- os vértices V_{rk} e $V_{(r+k)(s-k)}$ são as raízes das subárvores de V_{rs};
- se uma célula for vazia, isso significa que esta célula não gera qualquer subárvore;

Etapa 3: condição de aceitação da entrada. Se o símbolo inicial da gramática pertence ao vértice V_{1n} (raiz da árvore de derivação de toda palavra), então a entrada é aceita. ❑

exemplo 7.3 – Algoritmo de Cocke-Younger-Kasami

Considere a seguinte gramática:

$G = (\{ S, A \}, \{ a, b \}, P, S)$, na qual:
$P = \{ S \rightarrow AA \mid AS \mid b, A \rightarrow SA \mid AS \mid a \}$

A tabela triangular de derivação para a palavra de entrada **abaab** é ilustrada na figura 7.9. ❑

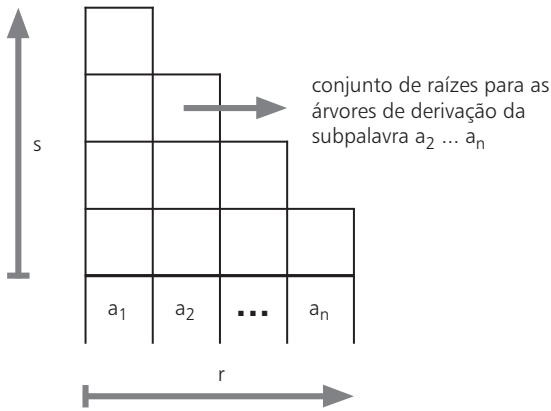


figura 7.6 Algoritmo de CYK: tabela triangular de derivação.

```
para   r variando de 1 até n
faça   V_r1 = { A | A → a_r ∈ P }
```

figura 7.7 Algoritmo: Cocke-Younger-Kasami – etapa 1.

```
para   s variando de 2 até n
faça   para   r variando de 1 até n - s + 1
       faça   V_rs = ∅
              para   k variando de 1 até s - 1
              faça   V_rs = V_rs ∪ { A | A → BC ∈ P, B ∈ V_rk e C ∈ V_(r+k)(s-k) }
```

figura 7.8 Algoritmo: Cocke-Younger-Kasami – etapa 2.

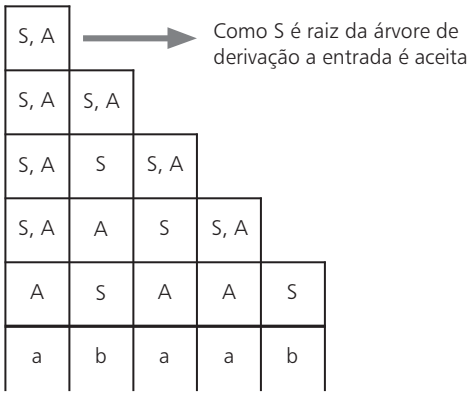


figura 7.9 Algoritmo de CYK: tabela triangular de derivação para **abaab**.

7.2.3 algoritmo de Early

O *algoritmo de Early* foi desenvolvido em 1968. É possivelmente o mais rápido algoritmo de reconhecimento conhecido para gramáticas livres do contexto. É construído a partir de uma gramática livre do contexto sem produções vazias e possui tempo de processamento proporcional a $|\mathbf{w}|^3$, para uma entrada **w**. Entretanto, para gramáticas não ambíguas, pode ser implementado em um tempo proporcional a $|\mathbf{w}|^2$. Adicionalmente, para muitas gramáticas de interesse prático, o tempo é proporcional ao tamanho da entrada, ou seja, $|\mathbf{w}|$.

Trata-se de um algoritmo do tipo *top-down*, que parte do símbolo inicial e executa sempre a derivação mais à esquerda. Cada ciclo gera um terminal, o qual é comparado com o símbolo da entrada. A comparação com sucesso determina a construção de um conjunto de produções que, potencialmente, pode gerar o próximo símbolo.

definição 7.7 – Algoritmo de Early

Suponha:

- **G = (V, T, P, S)** uma gramática livre do contexto sem produções vazias;
- $\mathbf{w} = \mathbf{a_1a_2...a_n}$ uma palavra a ser verificada.

No que segue, o símbolo "•" é usado como um marcador, antecedendo a posição, em cada produção, que será analisada na tentativa de gerar o próximo símbolo terminal. Adicionalmente, o sufixo (genericamente representado por) "**/u**" adicionado a cada produção indica o **u**-ésimo ciclo em que esta produção passou a ser considerada. As etapas do *algoritmo de Early* são as seguintes:

Etapa 1: construção de D_0, o primeiro conjunto de produções. São incluídas em $\mathbf{D_0}$ todas as produções que partem do símbolo inicial **S**, bem como todas as produções que podem ser aplicadas em sucessivas derivações mais à esquerda (a partir de **S**). O algoritmo é apresentado na figura 7.10. Observe que:

(1) representa as produções que partem de **S**;
(2) inclui todas as produções que podem derivar (mais à esquerda) o próximo símbolo;

Etapa 2: construção dos demais conjuntos de produção. São construídos **n** conjuntos de produção a partir de $\mathbf{D_0}$, sendo que $\mathbf{n} = |\mathbf{w}|$. Ao gerar o símbolo $\mathbf{a_r}$ de **w**, o algoritmo constrói o conjunto $\mathbf{D_r}$, contendo as produções que potencialmente podem gerar o símbolo $\mathbf{a_{r+1}}$. O algoritmo é apresentado na figura 7.11. Observe que:

(1) cada ciclo gera um conjunto de produções $\mathbf{D_r}$;
(2) gera o símbolo $\mathbf{a_r}$;
(3) inclui todas as produções que podem derivar (mais à esquerda) o próximo símbolo (análogo ao realizado na etapa a) acima);
(4) uma subpalavra de **w** foi reduzida à variável **A**: inclui em $\mathbf{D_r}$ todas as produções de $\mathbf{D_s}$ que referenciaram •**A**;

Etapa 3: condição de aceitação da entrada. Se uma produção da forma **S** → α•/0 pertence a D_n, então a palavra **w** de entrada foi aceita. Deve-se reparar que **S** → α•/0 é uma produção que:

- parte do símbolo inicial **S**;
- foi incluída em D_0 ("/0");
- todo o lado direito da produção foi analisado com sucesso (o marcador "•" está no final de α). ❑

Observe que, para otimizar o algoritmo de Early, nas etapas apresentadas na figura 7.10 e na figura 7.11, os ciclos `repita-até` podem ser restritos exclusivamente às produções recentemente incluídas em D_r ou em D_0 ainda não analisadas.

```
D0 = ∅
para   toda S → α ∈ P                                    (1)
faça   D0 = D0 ∪ { S → •α/0 }
repita para   toda A → •Bβ/0 ∈ D0                        (2)
       faça   para toda B → ϕ ∈ P
              faça   D0 = D0 ∪ { B → •ϕ/0 }
até    que o cardinal de D0 não aumente
```

figura 7.10 Algoritmo: Early – etapa 1.

```
para   r variando de 1 até n                             (1)
faça   Dr = ∅;
       para   toda A → α•arβ/s ∈ Dr-1                    (2)
       faça   Dr = Dr ∪ { A → αar•β/s };
       repita
              para   toda A → α•Bβ/s ∈ Dr                (3)
              faça   para toda B → ϕ ∈ P
                     faça Dr = Dr ∪ { B → •ϕ/r }
              para   toda A → α•/s de Dr                 (4)
              faça   para toda B → β•Aϕ/k ∈ Ds
                     faça   Dr = Dr ∪ { B → βA•ϕ/k }
       até    que o cardinal de Dr não aumente
```

figura 7.11 Algoritmo: Early – etapa 2.

exemplo 7.4 – Algoritmo de Early: expressão simples (tipo Pascal)

A seguinte gramática gera expressões com parênteses balanceados e duas operações (análoga à "expressão simples" da linguagem Pascal):

G = ({ E, T, F }, { +, ∗, [,], x }, P, E), na qual:
P = { E → T | E+T, T → F | T∗F, F → [E] | x }

O reconhecimento da palavra **x∗x** é como segue:

D_0:

E → •T/0	produções que partem
E → •E+T/0	do símbolo inicial
T → •F/0	produções que podem ser aplicadas
T → •T∗F/0	em derivação mais à esquerda
F → •[E]/0	a partir do símbolo inicial
F → •x/0	

D_1: reconhecimento de **x** em **x∗x**

F → x•/0	**x** foi reduzido a **F**
T → F•/0	inclui todas as produções de D_0 que
T → T•∗F/0	referenciaram **•F** direta ou indiretamente, (pois **F → x•/0**)
E → T•/0	movendo o marcador "•"
E → E•+T/0	um símbolo para a direita

D_2: reconhecimento de ∗ em **x∗x**

T → T∗•F/0	gerou ∗; o próximo será gerado por **F**
F → •[E]/2	inclui todas as produções de **P** que
F → •x/2	podem gerar o próximo terminal a partir de **F•**

D_3: reconhecimento de **x** em **x∗x**

F → x•/2	**x** foi reduzido à **F**
T → T∗F•/0	incluído de D_2 (pois **F → x/2**); a entrada foi reduzida à **T**
E → T•/0	incluído de D_0 (pois **T → T*F/0**); a entrada foi reduzida à **E**
T → T•∗F/0	incluído de D_0 (pois **T → T∗F•/0**)
E → E•+T/0	incluído de D_0 (pois **E → T•/0**)

Como **w = x∗x** foi reduzida ao símbolo inicial **E**, ou seja, **E → T•/0** pertence a D_3, a entrada foi aceita. ❑

7.3 → exercícios

exercício 7.1 Explique como o lema de bombeamento para as linguagens livres do contexto pode ser aplicado como o bombeamento para as linguagens regulares.

exercício 7.2 Prove que as seguintes linguagens não são livres do contexto:

a **{ ww | w** é palavra de **{ a, b }* }**

b **{ $a^n b^n a^m$ | n ≥ 0, m ≥ 0** e **n ≠ m }**

exercício 7.3 Demonstre que a classe das linguagens livres do contexto é fechada para as seguintes operações:

a União, usando o formalismo de gramática;

b Concatenação, usando o formalismo de autômato.

exercício 7.4 Demonstre que a classe das linguagens livres do contexto é fechada para concatenação sucessiva, ou seja, se **L** é uma linguagem livre do contexto, então **L*** também é livre do contexto.

exercício 7.5 Demonstre que a classe das linguagens livres do contexto é fechada para o produto cartesiano, ou seja, se L_1 e L_2 são linguagens livres do contexto, então $L_1 \times L_2$ definida abaixo também é livre do contexto:

$$L_1 \times L_2 = \{ (w_1, w_2) \mid w_1 \in L_1 \text{ e } w_2 \in L_1 \}$$

exercício 7.6 As linguagens geradas pelas gramáticas cujas produções estão representadas abaixo são vazias, finitas ou infinitas?

a

$S \rightarrow AB \mid CA$
$A \rightarrow a$
$B \rightarrow BC$
$C \rightarrow AB \mid \varepsilon$

b

$S \rightarrow aS \mid aSbS \mid X$
$X \rightarrow SS$

exercício 7.7 Por que os algoritmos de reconhecimento baseados em autômatos com pilha são, em geral, tão ineficientes em termos de tempo de processamento?

exercício 7.8 No algoritmo autômato com pilha descendente, qual a provável consequência se a gramática usada tiver recursão à esquerda?

exercício 7.9 Dada a seguinte gramática livre do contexto que denota o conjunto das expressões regulares sobre o alfabeto **{ x }** (observe que ε é um símbolo terminal):

G = ({ S }, { ∅, ε, x, (,), +, * }, P, S), na qual:
P = **{ S → ∅ | ε | (S+S) | (SS) | S* }**

faça o reconhecimento da entrada **(x+x)*** para cada um dos seguintes algoritmos de reconhecimento:

a Autômato com pilha descendente;
b Cocke-Younger-Kasami (CYK);
c Early.

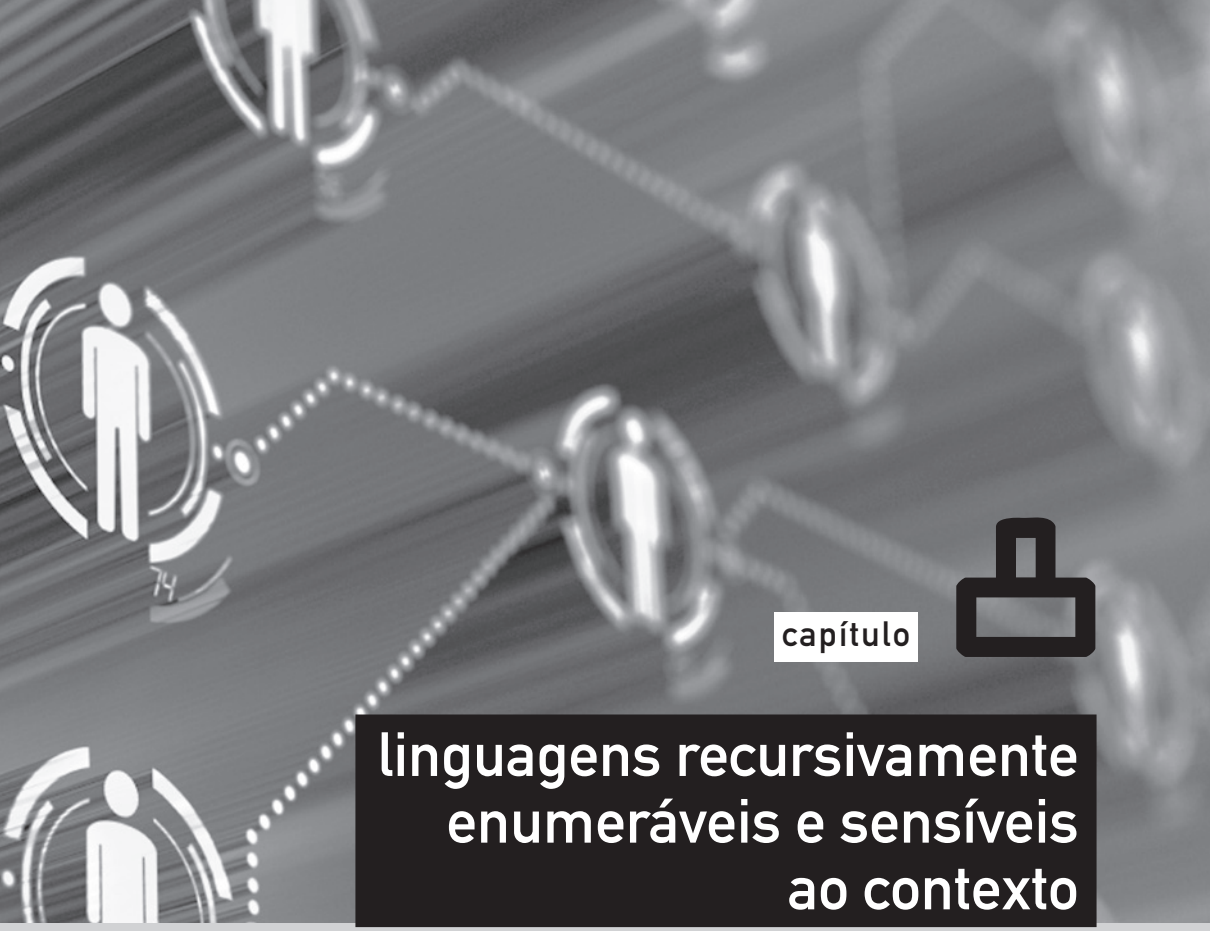
exercício 7.10 Afirma-se que o algoritmo CYK gera todas as árvores de derivação da entrada. Entretanto, tal fato não é evidente, pois o algoritmo é definido sobre uma tabela triangular. Assim:

a Detalhe exatamente como a tabela resultante representa todas as árvores de derivações;
b Justifique ou refute:

> *no caso de a entrada pertencer à linguagem, é perfeitamente possível recuperar a(s) árvore(s) de derivação da entrada, a partir das informações contidas na tabela.*

exercício 7.11 Relativamente ao algoritmo de Early:

a O que significa o fato de algum conjunto D_r ser vazio?
b Como o algoritmo procede a partir desse momento?
c Como ficam os demais conjuntos D_i, para $i > r$?



capítulo

linguagens recursivamente enumeráveis e sensíveis ao contexto

O estudo das linguagens recursivamente enumeráveis e sensíveis ao contexto explora os limites do que pode ser reconhecido computacionalmente. Em particular, mostra que existem infinitas, mas contáveis, linguagens computáveis, e infinitas e não contáveis linguagens não computáveis. Ou seja, existem mais linguagens não computáveis do que computáveis. O estudo é desenvolvido usando os formalismos máquina de Turing e gramática. São estabelecidas algumas classes de linguagens computáveis, bem como provadas algumas propriedades dessas classes.

Ciência da computação é o conhecimento sistematizado da *computação*. Sua origem é milenar, tendo se desenvolvido em diversas regiões e momentos ao longo da história da humanidade, com destaque para culturas como o Mesopotâmia, Egito, Grécia, Babilônia, Índia, China, Roma e Asteca.

No início do século XX, ocorreu um grande impulso nas pesquisas com o objetivo de definir um modelo computacional suficientemente genérico, capaz de implementar qualquer *função computável* base para viabilizar a exploração dos limites do que pode ser computado.

Em 1936, Alan Turing propôs um formalismo para a representação de procedimentos efetivos. O trabalho de Turing é particularmente significativo por ter sido o primeiro a identificar programas escritos para uma "máquina computacional", como noções intuitivas do efetivamente computável. A *máquina de Turing* é um formalismo muito simples, universalmente conhecido e provavelmente o mais usado como modelo teórico de computação. A intenção do modelo foi simular, tanto quanto possível, as atitudes humanas relacionadas à computação. O resultado foi uma fundamentação teórica para o desenvolvimento do computador como conhecemos hoje. Atualmente, é aceita como uma formalização de um *procedimento efetivo* (*algoritmo* ou *função computável*).

Ainda em 1936, Alonzo Church apresentou a *tese de Church*, a qual afirma que qualquer função computável pode ser processada por uma máquina de Turing, ou seja, existe um procedimento expresso na forma de uma máquina de Turing capaz de processar tal função. Contudo, como a noção intuitiva de procedimentos não é matematicamente precisa, é impossível demonstrar formalmente se a máquina de Turing é, de fato, o mais genérico dispositivo de computação. Entretanto, a evidência de que a tese é verdadeira, foi reforçada na medida em que todos os demais modelos propostos possuem, no máximo, a mesma capacidade computacional dessa máquina. Assim, a tese de Church foi assumida como uma hipótese para toda a teoria da computação, razão pela qual também é conhecida como *hipótese de Church*.

Resumidamente, uma máquina de Turing é um autômato cuja fita não possui tamanho máximo e pode ser usada simultaneamente como dispositivo de entrada, de saída e de memória de trabalho.

As *linguagens recursivamente enumeráveis* ou *linguagens tipo 0* são aquelas que podem ser aceitas por uma máquina de Turing. Considerando que, segundo a hipótese de Church, a máquina de Turing é o mais geral dispositivo de computação, então a classe das linguagens recursivamente enumeráveis representa o conjunto de todas as linguagens que podem ser reconhecidas mecanicamente e em um tempo finito.

Analogamente às demais classes de linguagens, é possível representar as linguagens recursivamente enumeráveis usando um formalismo axiomático ou gerador, na forma de gramática, denominado *gramática irrestrita*. Como o próprio nome indica, uma gramática irrestrita não possui qualquer restrição sobre a forma das produções. Portanto, o formalismo gramática possui o mesmo poder computacional que o formalismo máquina de Turing.

Uma consequência importante do estudo das linguagens recursivamente enumeráveis é que, computacionalmente falando, existem mais *problemas não computáveis* (para os quais não existem máquinas de Turing capazes de processá-los) do que *problemas computáveis* (caso contrário). De fato, existem infinitos, mas contáveis problemas computáveis, e infinitos e não contáveis problemas não computáveis.

A classe das linguagens recursivamente enumeráveis inclui algumas para as quais é impossível determinar mecanicamente se uma palavra não pertence à linguagem, embora seja possível determinar se pertence. Portanto, um problema computável pode ser um *problema parcialmente solucionável*. Se **L** é uma destas linguagens, então, para qualquer máquina de Turing **M** que aceita **L**, existe pelo menos uma palavra **w** não pertencente a **L** que, ao ser processada por **M**, a máquina entra em *loop* infinito. Assim, pode-se afirmar que:

- se **w** pertence a **L**, **M** para e aceita a entrada;
- se **w** não pertence a **L**, **M** pode parar, rejeitando a palavra ou permanecer processando indefinidamente.

Portanto, é conveniente definir um subclasse da classe das linguagens enumeráveis recursivamente, composta pelas *linguagens recursivas*, para as quais existe pelo menos uma máquina de Turing que para para qualquer entrada, aceitando ou rejeitando. Todo problema enquadrado nesta classe de linguagens é denominado *problema solucionável*.

As *linguagens sensíveis ao contexto* ou *tipo 1* são aquelas que podem ser aceitas por uma *máquina de Turing com fita limitada*, ou seja, uma máquina de Turing com limitação finita no tamanho da fita (igual ao tamanho da entrada mais duas células de controle). O correspondente formalismo axiomático é a *gramática sensível ao contexto*, em oposição ao termo "livre do contexto": o lado esquerdo das produções da gramática pode ser uma palavra de variáveis ou terminais, definindo um "contexto" de derivação.

A classe das linguagens sensíveis ao contexto está contida propriamente na classe das linguagens recursivas. Trata-se de uma classe especialmente importante, pois inclui a grande maioria das linguagens aplicadas. A relação entre as diversas classes de linguagens é ilustrada na figura 8.1.

figura 8.1 Relação entre as classes de linguagens.

No texto que segue, para uma determinada linguagem **L** sobre o alfabeto Σ, $\sim$**L** denota o seu complemento (em relação ao conjunto universo Σ^*).

8.1 máquina de Turing

Já foi afirmado que a noção de algoritmo não é matematicamente precisa. Entretanto, intuitivamente, um algoritmo deve possuir as seguintes características, entre outras:

a Ter uma descrição finita;

b Consistir de passos:

- discretos (em oposição ao contínuo);
- executáveis mecanicamente;
- executáveis em um tempo finito.

8.1.1 noção intuitiva

A *máquina de Turing*, proposta por Alan Turing em 1936, é um mecanismo simples que formaliza a ideia de uma pessoa que realiza cálculos. Lembra, em muito, os computadores atuais, embora tenha sido proposta anos antes do primeiro computador digital. Apesar de sua simplicidade, o modelo máquina de Turing possui, no mínimo, o mesmo poder computacional de qualquer computador de propósito geral.

O ponto de partida de Turing foi analisar a situação na qual uma pessoa, equipada com um instrumento de escrita e um apagador, realiza cálculos em uma folha de papel, organizada em quadrados.

Inicialmente, suponha que a folha de papel contém somente os dados iniciais do problema. O trabalho da pessoa pode ser resumido em sequências de operações simples como segue:

- ler um símbolo de um quadrado;
- alterar um símbolo em um quadrado;
- mover os olhos para outro quadrado.

Quando é encontrada alguma representação satisfatória para a resposta desejada, a pessoa termina seus cálculos. Para viabilizar esse procedimento, as seguintes hipóteses são aceitáveis:

- a natureza bidimensional do papel não é um requerimento essencial para os cálculos. Pode ser assumido que o papel consiste de uma fita infinita organizada em quadrados;
- o conjunto de símbolos deve ser finito. Para representar informações mais complexas, é possível utilizar sequências de símbolos, analogamente à noção de alfabeto e linguagem natural como o português;
- o conjunto de estados da mente da pessoa durante o processo de cálculo é finito. Mais ainda, entre esses estados, existem dois em particular: "estado inicial" e "estado final", correspondendo ao início e ao fim dos cálculos, respectivamente;
- o comportamento da pessoa, a cada momento, é determinado somente pelo seu estado presente e pelo símbolo para o qual sua atenção está voltada;
- a pessoa é capaz de observar e alterar o símbolo de apenas um quadrado de cada vez, bem como de transferir sua atenção para somente um dos quadrados adjacentes.

8.1.2 modelo

Essa noção de uma pessoa calculando pode ser vista como uma máquina, constituída de três partes, como segue:

a *Fita*. Usada simultaneamente como dispositivo de entrada, de saída e de memória de trabalho;

b *Unidade de controle*. Reflete o estado corrente da máquina. Possui uma unidade de leitura e gravação (cabeça da fita), a qual acessa uma célula da fita de cada vez e se movimenta para a esquerda ou para a direita;

c *Programa*, *função programa* ou *função de transição*. Função que define o estado da máquina e comanda as leituras, as gravações e o sentido de movimento da cabeça.

A fita é finita à esquerda e *infinita* (tão grande quanto necessário) à direita, sendo dividida em células, cada uma armazenando um símbolo. Os símbolos podem:

- pertencer ao alfabeto de entrada;
- pertencer ao alfabeto auxiliar;
- ser "branco";
- ser "marcador de início de fita".

Inicialmente, a palavra a ser processada (ou seja, a informação de entrada para a máquina) ocupa as células mais à esquerda após o marcador de início de fita, ficando as demais com "branco", como ilustrado na figura 8.2 (os símbolos β e **b** representam "branco" e "marcador de início de fita", respectivamente).

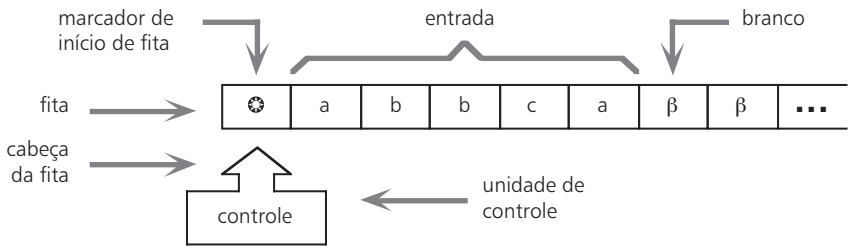


figura 8.2 Máquina de Turing: fita e unidade de controle.

A unidade de controle possui um número *finito* e *predefinido* de estados. A *cabeça da fita* lê o símbolo de uma célula de cada vez e grava um novo símbolo. Após a leitura/gravação (a gravação é realizada na mesma célula de leitura), a cabeça move uma célula para a direita ou para a esquerda. O símbolo gravado e o sentido do movimento são definidos pelo programa.

definição 8.1 – Máquina de Turing

Uma *máquina de Turing* (frequentemente abreviada por **MT**) **M** é uma **8**-upla:

$$\mathbf{M} = (\Sigma, \mathbf{Q}, \delta, \mathbf{q_0}, \mathbf{F}, \mathbf{V}, \beta, ✪)$$

na qual:

a Σ é um *alfabeto de símbolos de entrada*, ou simplesmente *alfabeto de entrada*;

b **Q** é um *conjunto de estados* possíveis da máquina, o qual é finito;

c δ é uma *função programa* ou simplesmente *programa*, ou ainda, *função de transição* (suponha que $\Sigma \cup \mathbf{V}$ e $\{ \beta, ✪ \}$ são conjuntos disjuntos):

$$\delta: \mathbf{Q} \times (\Sigma \cup \mathbf{V} \cup \{ \beta, ✪ \}) \rightarrow \mathbf{Q} \times (\Sigma \cup \mathbf{V} \cup \{ \beta, ✪ \}) \times \{ \mathbf{E}, \mathbf{D} \}$$

a qual é uma função parcial. Supondo que a função programa é definida para $\mathbf{p} \in \mathbf{Q}$, $\mathbf{x} \in \Sigma \cup \mathbf{V} \cup \{ \beta, ✪ \}$, resultando em $\mathbf{q} \in \mathbf{Q}$, $\mathbf{y} \in \Sigma \cup \mathbf{V} \cup \{ \beta, ✪ \}$ e $\mathbf{m} \in \{ \mathbf{E}, \mathbf{D} \}$, então:

$$\delta(\mathbf{p}, \mathbf{x}) = (\mathbf{q}, \mathbf{y}, \mathbf{m})$$

é uma *transição* da máquina;

d $\mathbf{q_0}$ é um elemento distinguido de **Q**, denominado *estado inicial*;

e **F** é um subconjunto de **Q**, denominado *conjunto de estados finais*;

f **V** é um *alfabeto auxiliar* (pode ser vazio);

g β é o símbolo especial *branco*;

h ✪ é o *símbolo de início* ou *marcador de início* da fita. ❑

O símbolo de início de fita sempre ocorre exatamente uma vez e na célula mais à esquerda da fita, auxiliando na identificação de que a cabeça da fita encontra-se na célula mais à esquerda da fita.

A função programa considera o estado corrente e o símbolo lido da fita para determinar o novo estado, o símbolo a ser gravado e o sentido de movimento da cabeça, sendo que esquerda e direita são representados por **E** e **D**, respectivamente. A função programa pode

ser interpretada como um diagrama, como ilustrado na figura 8.3 (suponha a transição **δ(p, x) = (q, y, m)**). Neste caso, os estados inicial e finais são representados como nos autômatos finitos.

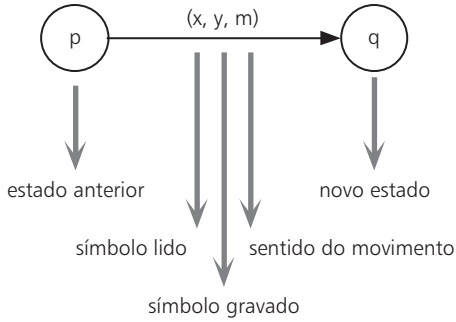


figura 8.3 Diagrama (**MT**): transição.

A *computação* de uma máquina de Turing **M**, para uma palavra de entrada **w**, consiste na sucessiva aplicação da função programa a partir do estado inicial e da cabeça posicionada na célula mais à esquerda da fita até ocorrer uma condição de parada. O processamento de **M** para a entrada **w** pode parar ou ficar processando indefinidamente (ciclo ou *loop* infinito).

A parada do processamento de uma máquina de Turing para uma entrada **w** pode ser de duas maneiras:

a *Aceita* a entrada **w**. Atinge um estado final: a máquina para, e a palavra **w** é aceita;

b *Rejeita* a entrada **w**. São duas possibilidades:

- a função programa é indefinida para o argumento (símbolo lido e estado corrente): a máquina para, e a palavra **w** é rejeitada;
- o argumento corrente da função programa define um movimento à esquerda, e a cabeça da fita já se encontra na célula mais à esquerda: a máquina para, e a palavra **w** é rejeitada.

Para definir formalmente o comportamento de uma máquina de Turing, é necessário estender a definição da função programa, usando como argumento um estado e uma palavra. Esta extensão é sugerida como exercício.

definição 8.1 – Linguagem aceita, linguagem rejeitada, linguagem *loop*

Seja **M = (Σ, Q, δ, q_0, F, V, β, ✪)** uma máquina de Turing. Então:

a A *linguagem aceita* ou *linguagem reconhecida* por **M**, denotada por:

ACEITA(M) ou **L(M)**

é o conjunto de todas as palavras pertencentes a Σ* aceitas por **M**, a partir do estado inicial q_0;

b A *linguagem rejeitada* por **M**, denotada por:

REJEITA(M)

é o conjunto de todas as palavras pertencentes a Σ^* rejeitadas por **M**, a partir do estado inicial q_0;

c A *linguagem loop* de **M**, denotada por:

LOOP(M)

é o conjunto de todas as palavras pertencentes a Σ^* para as quais **M** fica processando indefinidamente a partir do estado inicial q_0. ❑

Supondo que Σ^* é o conjunto universo, as seguintes afirmações são verdadeiras (por quê?):

$$\text{ACEITA(M)} \cap \text{REJEITA(M)} \cap \text{LOOP(M)} = \varnothing$$
$$\text{ACEITA(M)} \cup \text{REJEITA(M)} \cup \text{LOOP(M)} = \Sigma^*$$
$$\sim\text{ACEITA(M)} = \text{REJEITA(M)} \cup \text{LOOP(M)}$$
$$\sim\text{REJEITA(M)} = \text{ACEITA(M)} \cup \text{LOOP(M)}$$
$$\sim\text{LOOP(M)} = \text{ACEITA(M)} \cup \text{REJEITA(M)}$$

Portanto, cada máquina de Turing **M** definida sobre o alfabeto Σ induz uma partição do conjunto de todas as palavras Σ^* em três classes de equivalência: **ACEITA(M)**, **REJEITA(M)** e **LOOP(M)** conforme ilustrado na figura 8.4. Sendo preciso, se um ou dois dos conjuntos forem vazios, então a partição induzida contém um ou dois conjuntos a menos (lembre-se de que uma classe de equivalência não pode ser vazia).

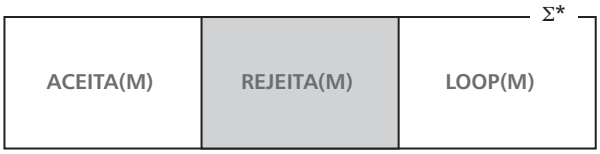


figura 8.4 Partição de Σ^*, induzida por uma máquina de Turing **M**.

exemplo 8.1 – Máquina de Turing: duplo balanceamento

Considere a linguagem:

$$L = \{ a^n b^n \mid n \geq 0 \}$$

A máquina de Turing:

$$M = (\{ a, b \}, \{ q_0, q_1, q_2, q_3, q_4 \}, \delta, q_0, \{ q_4 \}, \{ A, B \}, \beta, ✪)$$

ilustrada na figura 8.5 e na qual δ é como na tabela da figura 8.6, é tal que:

$$\text{ACEITA(M)} = L \quad \text{e} \quad \text{REJEITA(M)} = \sim L$$

e, portanto, **LOOP(M)** $= \varnothing$.

O algoritmo apresentado reconhece o primeiro símbolo **a**, o qual é marcado como **A**, e movimenta a cabeça da fita à direita, procurando o **b** correspondente, o qual é marcado como **B**. Este ciclo é repetido sucessivamente até identificar, para cada **a** o seu correspondente **b**. Adicionalmente, o algoritmo garante que qualquer outra palavra que não esteja na forma $a^n b^n$ é rejeitada.

A figura 8.7 ilustra a sequência do processamento da máquina de Turing **M** para a entrada **w = aabb**. ❑

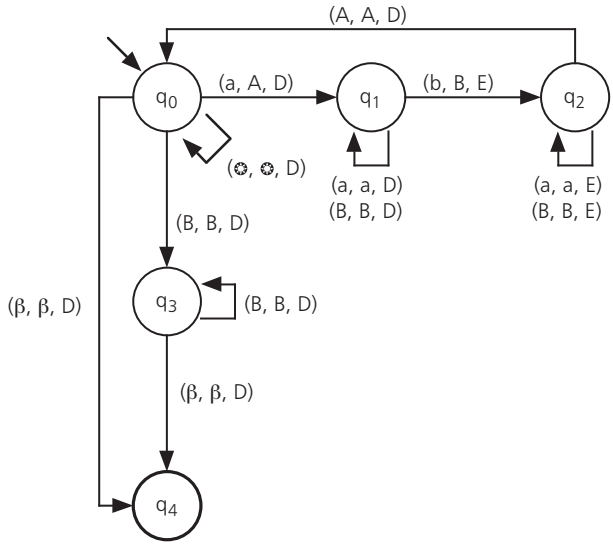


figura 8.5 Diagrama (**MT**): duplo balanceamento.

δ	⊗	a	b	A	B	β
q_0	$(q_0, ⊗, D)$	(q_1, A, D)			(q_3, B, D)	(q_4, β, D)
q_1		(q_1, a, D)	(q_2, B, E)		(q_1, B, D)	
q_2		(q_2, a, E)		(q_0, A, D)	(q_2, B, E)	
q_3					(q_3, B, D)	(q_4, β, D)
q_4						

figura 8.6 Função programa (**MT**): duplo balanceamento.

figura 8.7 Computação (**MT**): duplo balanceamento para a entrada **aabb**.

observação 8.2 – Máquina de Turing × algoritmo

Em diversos momentos, foi afirmado que a *máquina de Turing* é aceita como uma formalização do conceito de *algoritmo*. Entretanto, também é usual considerar que o conceito de algoritmo corresponde a uma máquina de Turing que sempre para para qualquer entrada. Nesse caso, uma máquina que eventualmente fica processando indefinidamente (em *loop* infinito) não seria considerada um algoritmo. ❑

8.2 modelos equivalentes à máquina de Turing

Uma das razões para considerar a máquina de Turing como o mais geral dispositivo de computação é o fato de que todos os demais modelos e máquinas propostos, bem como as diversas modificações da máquina de Turing, possuem, no máximo, o mesmo poder computacional da máquina de Turing:

a *Autômato com múltiplas pilhas*. Conforme citado no estudo das linguagens livres do contexto, o poder computacional do *autômato com duas pilhas* é equivalente ao da máquina de Turing. Adicionalmente, um maior número de pilhas também não aumenta a capacidade computacional.

A definição formal do autômato com duas pilhas e do autômato com múltiplas pilhas e a equivalência deles ao modelo da máquina de Turing são sugeridas como exercício.

É interessante observar que, como são necessárias duas pilhas para que o autômato possua o mesmo poder computacional que uma máquina de Turing, pode-se afirmar que a estrutura de *fita* é mais expressiva do que a de *pilha*;

b *Máquina de Turing não determinística*. A facilidade de não determinismo não aumenta o poder computacional da máquina de Turing;

c *Máquina de Turing com fita infinita à esquerda e à direita*. A modificação da definição básica da máquina de Turing, permitindo que a fita seja infinita dos dois lados, não aumenta o seu poder computacional. Na realidade, a fita infinita à esquerda e à direita pode ser facilmente simulada por uma fita tradicional, como ilustrado na figura 8.8, na qual as células pares representam a parte direita da fita, e as ímpares, a parte esquerda;

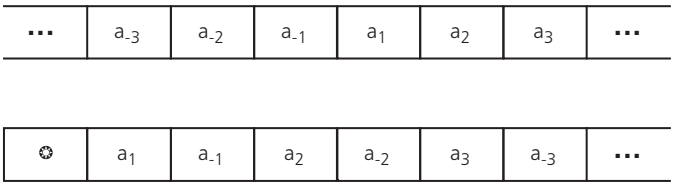


figura 8.8 Fita (**MT**): simulação de uma fita infinita à esquerda e à direita.

d *Máquina de Turing com múltiplas fitas*. A máquina de Turing com múltiplas fitas possui **k** fitas infinitas à esquerda e à direita e **k** cabeças de fita. A função programa é como segue:

- dependendo do estado corrente da máquina e do símbolo lido em cada uma das fitas;
- grava um novo símbolo em cada uma das fitas;
- move cada uma das cabeças independentemente;
- a máquina assume um (único) novo estado.

Inicialmente, a palavra de entrada é armazenada na primeira fita, ficando as demais com valor branco;

e *Máquina de Turing multidimensional*. Neste modelo, a fita tradicional é substituída por uma estrutura do tipo arranjo **k**-dimensional, infinita em todas as **2k** direções;

f *Máquina de Turing com múltiplas cabeças*. A máquina de Turing com esta modificação possui **k** cabeças de leitura e gravação sobre a mesma fita. Cada cabeça possui movimento independente. Assim, o processamento depende do estado corrente e do símbolo lido em cada uma das cabeças;

g *Combinações de modificações sobre a máquina de Turing*. A combinação de algumas ou todas as modificações apresentadas não aumenta o poder computacional da máquina de Turing. Por exemplo, uma máquina de Turing não determinística com múltiplas fitas e múltiplas cabeças pode ser simulada por uma máquina de Turing tradicional.

8.3 hipótese de Church

O modelo abstrato de computação proposto por A. Turing em 1936, conhecido como *máquina de Turing*, tinha como objetivo explorar os limites da capacidade de expressar soluções de problemas. Trata-se, portanto, de uma proposta de definição formal da noção intuitiva de algoritmo. Diversos outros trabalhos, como *Cálculo Lambda* (Church, 1936) e *funções recursivas* (Kleene, 1936), resultaram em conceitos equivalentes ao de Turing. O fato de todos esses trabalhos independentes gerarem o mesmo resultado em termos de capacidade de expressar computabilidade é um forte reforço no que é conhecido como *tese de Church* ou *tese de Turing-Church*:

> *"A capacidade de computação representada pela máquina de Turing é o limite máximo que pode ser atingido por qualquer dispositivo de computação".*

Em outras palavras, a hipótese de Church afirma que qualquer outra forma de expressar algoritmos terá, no máximo, a mesma capacidade computacional da máquina de Turing. Como a noção de algoritmo ou função computável é intuitiva, a hipótese de Church não é demonstrável. Entretanto, como é assumida como verdadeira (ou seja, é assumida como hipótese) para toda ciência da computação, também é conhecida como *hipótese de Church* ou *hipótese de Turing-Church*.

8.4 máquina de Turing como reconhecedor

As seguintes classes de linguagens são definidas a partir do formalismo máquina de Turing:

a Classe das linguagens recursivamente enumeráveis: composta pelas linguagens para as quais existe uma máquina de Turing capaz de determinar se uma determinada palavra **w** pertence à linguagem, ou seja, se **w** ∈ **L**. Entretanto, se **w** ∈ ~**L**, o algoritmo pode (ver figura 8.9, direita):

- *parar*, identificando que a palavra não pertence à linguagem;
- *ficar em loop* infinito (processando indefinidamente).

b Classe das linguagens recursivas: composta pelas linguagens para as quais existe pelo menos uma máquina de Turing que *sempre para*, capaz de determinar se uma determinada palavra **w** pertence ou não à linguagem, ou seja, para uma dada linguagem **L**, existe uma máquina de Turing que determina se **w** ∈ **L** ou **w** ∈ ~**L**, como ilustrado na figura 8.9, esquerda.

A existência dessas duas classes contradiz a intuição da maioria das pessoas, pois estabelece que:

> *reconhecer o complemento de uma linguagem pode ser impossível, mesmo que seja possível reconhecer a linguagem.*

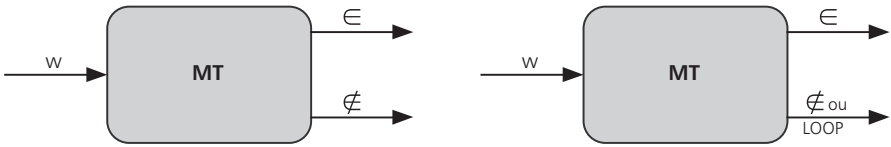


figura 8.9 Existência de **MT**: recursiva (esquerda) e recursivamente enumerável (direita).

8.4.1 linguagem recursivamente enumerável

definição 8.3 – Linguagem recursivamente enumerável, linguagem tipo 0

Uma linguagem aceita por uma máquina de Turing é dita *linguagem recursivamente enumerável* ou *linguagem tipo 0*. ❑

exemplo 8.2 – Linguagem recursivamente enumerável

As seguintes linguagens são exemplos de linguagens recursivamente enumeráveis, sendo que, para a primeira, a correspondente máquina de Turing foi construída no exemplo 8.1 – Máquina de Turing: duplo balanceamento (as demais são sugeridas como exercício):

a $\{ a^n b^n \mid n \geq 0 \}$
b $\{ w \mid w$ tem o mesmo número de símbolos **a** e **b** $\}$
c $\{ a^i b^j c^k \mid i = j$ ou $j = k \}$ ❑

Considerando que, segundo a Hipótese de Church, a máquina de Turing é o mais geral dispositivo de computação, pode-se afirmar que a classe das linguagens recursivamente enumeráveis representa todas as linguagens que podem ser reconhecidas ("compiladas") mecanicamente. Trata-se, portanto, de uma classe de linguagens muito rica. Existem, entretanto, conjuntos que não são recursivamente enumeráveis, ou seja, linguagens para as quais não é possível desenvolver uma máquina de Turing que as reconheça.

No teorema a seguir, mostra-se que existe pelo menos uma linguagem que não é recursivamente enumerável.

teorema 8.4 – Linguagem não recursivamente enumerável

Seja $\Sigma = \{ a, b \}$.

a Suponha que X_i representa o **i**-ésimo elemento na ordenação lexicográfica de Σ^*. Por exemplo (o valor de $i \in \mathbf{N}$ e o correspondente elemento de Σ^* na ordenação lexicográfica estão na primeira e segunda colunas, respectivamente):

0	ε
1	a
2	b
3	aa
…	…

b Conforme é proposto nos exercícios, é possível codificar todas as máquinas de Turing como uma palavra sobre Σ de tal forma que cada código represente uma única máquina

de Turing. Suponha o conjunto dos códigos ordenados lexicograficamente e suponha que T_i representa o **i**-ésimo código nesta ordenação.

Então o conjunto que segue não é linguagem recursivamente enumerável:

$$L = \{ X_i \mid X_i \text{ não é aceita por } T_i \}$$

➥ **Prova: (*por absurdo*)**
Suponha que **L** é uma linguagem recursivamente enumerável. Então, por definição, existe uma máquina de Turing que aceita **L**. Seja T_k a codificação desta máquina de Turing, ou seja, $ACEITA(T_k) = L$. Assim, as seguintes afirmações são válidas:

a Por definição de **L**, X_k pertence a **L** se, e somente se, X_k não é aceita por T_k;

b Entretanto, como T_k aceita a linguagem **L**, X_k pertence a **L** se, e somente se, X_k é aceita por T_k.

Claramente b) contradiz a). Portanto, é absurdo supor que **L** é recursivamente enumerável. Logo, **L** não é linguagem recursivamente enumerável. ❑

observação 8.5 – Cardinal dos problemas > cardinal dos algoritmos

Considere o teorema 8.4 – Linguagem não recursivamente enumerável. Na prova do teorema, afirma-se que existe uma codificação de todas as máquinas de Turing. Portanto, o conjunto destas codificações é isomorfo a um subconjunto infinito dos números naturais. Logo, o conjunto de todas as máquinas de Turing (ou de todas as linguagens recursivamente enumeráveis ou ainda, de todos os problemas computáveis) é enumerável (infinitamente contável). Em contrapartida, o cardinal do conjunto das linguagens que não são recursivamente enumeráveis (ou de todos os problemas não computáveis) é não contável.

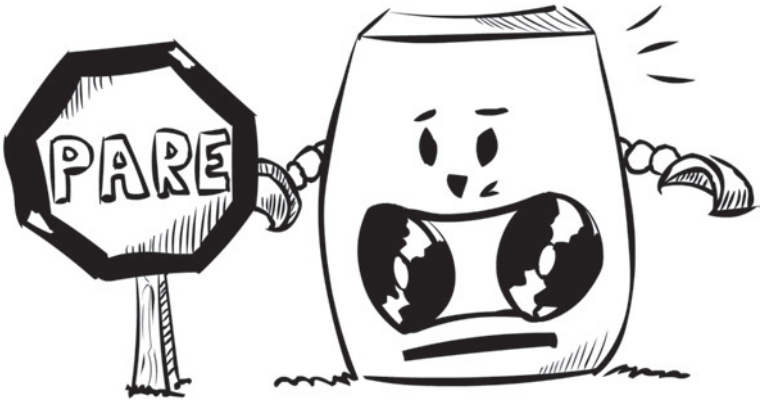
Como o cardinal de um conjunto enumerável é $\aleph_0$ (o menor cardinal dos conjuntos infinitos), em particular, é menor do que o cardinal de qualquer conjunto não contável. Portanto, computacionalmente falando:

existem mais problemas do que algoritmos para resolvê-los.

Como ilustração, considere o conjunto de todas as funções nos naturais:

$$\{ f: \mathbf{N} \rightarrow \mathbf{N} \mid f \text{ é função} \}$$

Claramente, trata-se de uma classe muito particular de problemas (ou de linguagens). Prova-se que este conjunto é isomorfo a **R** e portanto, seu cardinal é $2^{\aleph_0}$. Ou seja, é muito maior do que $\aleph_0$, o cardinal do conjunto de todas as máquinas de Turing. ❑



8.4.2 linguagem recursiva

definição 8.6 – Linguagem recursiva

Uma linguagem **L** é dita *linguagem recursiva* se existe pelo menos uma máquina de Turing **M** tal que:

a **ACEITA(M) = L**
b **REJEITA(M) = ~L** ❑

Ou seja, uma linguagem é recursiva se existe uma máquina de Turing que aceita a linguagem e sempre para para qualquer entrada.

exemplo 8.3 – Linguagem recursiva

É fácil verificar que as seguintes linguagens são recursivas:

a $\{ a^n b^n \mid n \geq 0 \}$
b $\{ a^n b^n c^n \mid n \geq 0 \}$
c $\{ w \mid w \in \{ a, b \}^*$ e tem o dobro de símbolos **a** que **b** $\}$ ❑

8.5 → gramática irrestrita e a linguagem recursivamente enumerável

Uma *gramática irrestrita* é simplesmente uma gramática sem qualquer restrição nas produções. Portanto, o termo "irrestrita" serve somente para destacar tal fato.

exemplo 8.4 – Gramática irrestrita: triplo balanceamento

A linguagem **{ $a^n b^n c^n$ | n ≥ 0 }** é gerada pela seguinte gramática irrestrita:

G = ({ S, C }, { a, b, c }, P, S), na qual:
P = { S → abc | ε,
ab → aabbC,
Cb → bC,
Cc → cc }

A palavra **aaabbbccc** pode ser derivada como segue (existe alguma outra sequência de derivação que gera a mesma palavra?):

S ⇒ abc ⇒ aabbCc ⇒ aaabbCbCc ⇒ aaabbCbcc ⇒ aaabbbCcc ⇒ aaabbbccc

Observe que a variável **C** "caminha" na palavra até a posição correta para gerar um terminal **c**. ❑

O seguinte teorema não será demonstrado.

teorema 8.7 – Linguagem recursivamente enumerável × gramática irrestrita

L é uma linguagem recursivamente enumerável se, e somente se, **L** é gerada por uma gramática irrestrita. ❑

8.6 ⇢ linguagem sensível ao contexto

8.6.1 gramática sensível ao contexto

definição 8.8 – Gramática sensível ao contexto

Uma *gramática sensível ao contexto* **G** é uma gramática:

$$G = (V, T, P, S)$$

com a restrição de que qualquer regra de produção de **P** é da forma $\alpha \rightarrow \beta$, onde:

a β é uma palavra de $(V \cup T)^*$

b α é uma palavra de $(V \cup T)^+$ tal que $|\alpha| \leq |\beta|$, excetuando-se, eventualmente, para **S** → ε. Neste caso, **S** não pode estar presente no lado direito de qualquer produção. ❑

Portanto, em uma gramática sensível ao contexto, a cada etapa de derivação, o tamanho da palavra derivada não pode diminuir, excetuando-se para gerar a palavra vazia, se esta pertencer à linguagem. Observe que nem toda gramática livre do contexto é uma gramática sensível ao contexto (por quê?).

definição 8.9 – Linguagem sensível ao contexto, linguagem tipo 1

Uma linguagem gerada por uma gramática sensível ao contexto é dita *linguagem sensível ao contexto* ou *linguagem tipo 1*. ❑

exemplo 8.5 – Linguagem sensível ao contexto: palavra duplicada

A linguagem

$$\{ ww \mid w \text{ é palavra de } \{ a, b \}^* \}$$

é gerada pela seguinte gramática sensível ao contexto:

$G = (\{ S, X, Y, A, B, \langle aa\rangle, \langle ab\rangle, \langle ba\rangle, \langle bb\rangle \}, \{ a, b \}, P, S)$, na qual:

$$\begin{aligned}
P = \{\ & S \to XY \mid aa \mid bb \mid \varepsilon, \\
& X \to XaA \mid XbB \mid aa\langle aa\rangle \mid ab\langle ab\rangle \mid ba\langle ba\rangle \mid bb\langle bb\rangle, \\
& Aa \to aA,\ Ab \to bA,\ AY \to Ya, \\
& Ba \to aB,\ Bb \to bB,\ BY \to Yb, \\
& \langle aa\rangle a \to a\langle aa\rangle,\ \langle aa\rangle b \to b\langle aa\rangle,\ \langle aa\rangle Y \to aa, \\
& \langle ab\rangle a \to a\langle ab\rangle,\ \langle ab\rangle b \to b\langle ab\rangle,\ \langle ab\rangle Y \to ab, \\
& \langle ba\rangle a \to a\langle ba\rangle,\ \langle ba\rangle b \to b\langle ba\rangle,\ \langle ba\rangle Y \to ba, \\
& \langle bb\rangle a \to a\langle bb\rangle,\ \langle bb\rangle b \to b\langle bb\rangle,\ \langle bb\rangle Y \to bb\ \}
\end{aligned}$$

Excetuando-se para $|w| \leq 1$, a gramática apresentada gera o primeiro **w** após **X**, e o segundo **w** após **Y**, como segue:

- a cada símbolo terminal gerado após **X**, é gerada uma variável correspondente;
- essa variável "caminha" na palavra até passar por **Y**, quando deriva o correspondente terminal;
- para encerrar, **X** deriva uma subpalavra de dois terminais e uma correspondente variável a qual "caminha" até encontrar **Y**, quando é derivada a mesma subpalavra de dois terminais. Note-se que, se a gramática fosse modificada para **X** derivar uma subpalavra de somente um terminal (e a correspondente variável) como por exemplo $X \to u\langle u\rangle$, ao encontrar **Y**, seria necessário derivar usando uma produção do tipo $\langle u\rangle Y \to u$ na qual o lado direito possui comprimento menor que o lado esquerdo e, consequentemente, a gramática não seria sensível ao contexto. ❑

8.6.2 máquina de Turing com fita limitada

Uma *máquina de Turing com fita limitada*, é, basicamente, uma máquina de Turing com a fita limitada ao tamanho da entrada mais duas células contendo marcadores de início e de fim de

fita. Como não é conhecido se a facilidade de não determinismo aumenta o poder computacional das máquinas de Turing com fita limitada, a sua definição prevê não determinismo.

definição 8.10 – Máquina de Turing com fita limitada

Uma *máquina de Turing com fita limitada* (frequentemente abreviada por **MTFL**) **M** é uma **8**-upla:

$$M = (\Sigma, Q, \delta, q_0, F, V, ✪, †)$$

na qual:

a Σ é um *alfabeto de símbolos de entrada* ou simplesmente *alfabeto de entrada*;

b **Q** é um *conjunto de estados* possíveis da máquina o qual é finito;

c δ é uma *função programa*, ou simplesmente *programa*, ou ainda, *função de transição*:

$$\delta: Q \times (\Sigma \cup V \cup \{ ✪, † \}) \rightarrow 2^{Q \times (\Sigma \cup V \cup \{ ✪, † \}) \times \{ E, D \}}$$

a qual é uma função total. Assim, para o par $(p, x) \in Q \times (\Sigma \cup V \cup \{ ✪, † \})$:

$$\delta(p, x) = \{ (q_1, y_1, m_1),\ldots,(q_n, y_n, m_n) \}$$

é uma *transição* da máquina;

d q_0 é um elemento distinguido de **Q**, denominado *estado inicial*;

e **F** é um subconjunto de **Q**, denominado *conjunto de estados finais*;

f **V** é um *alfabeto auxiliar* (pode ser vazio);

g ✪ é o *símbolo de início* ou *marcador de início* da fita;

h † é o *símbolo de fim* ou *marcador de fim* da fita. ❑

As definições de *linguagem aceita* (ou *linguagem reconhecida*), *linguagem rejeitada* e *linguagem loop* são análogas às da máquina de Turing.

exemplo 8.6 – Máquina de Turing com fita limitada: palavra duplicada

Considere a linguagem:

$$L = \{ ww \mid w \text{ é palavra de } \{ a, b \}^* \}$$

A máquina de Turing com fita limitada:

$$M = (\{ a, b \}, \{ q_0, q_1,\ldots, q_9, q_f \}, \delta, q_0, \{ q_f \}, \{ X, Y \}, b, =)$$

ilustrada na figura 8.10 é tal que:

ACEITA(M) = L
REJEITA(M) = ~L

e, portanto, **LOOP(M) =** ∅. A partir de $\mathbf{q_1}$, o início do primeiro **w** é marcado com um **X**. Os estados $\mathbf{q_2}$ e $\mathbf{q_3}$ definem não determinismos, com o objetivo de marcar com um **Y** o início do segundo **w**. De $\mathbf{q_5}$ a $\mathbf{q_{11}}$ é verificada a igualdade da primeira com a segunda metade da palavra. ❑

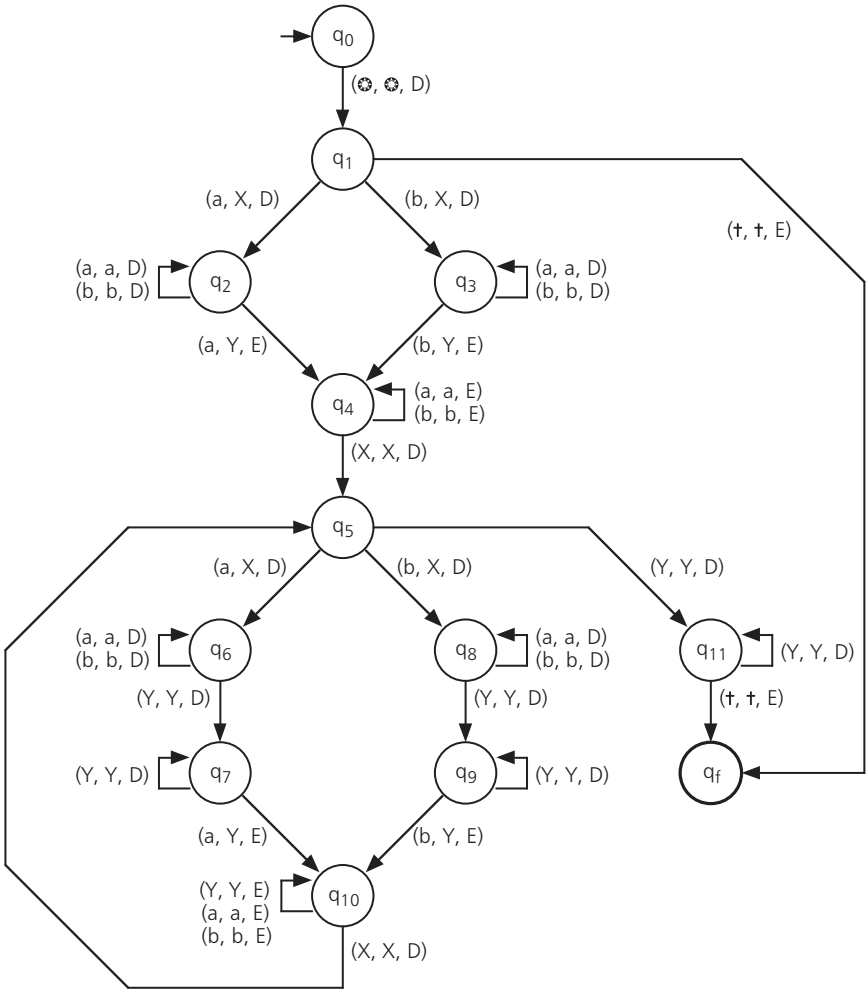


figura 8.10 Máquina de Turing com fita limitada.

O seguinte teorema não será demonstrado.

teorema 8.11 – Linguagem sensível ao contexto × máquina de Turing com fita limitada

L é uma linguagem sensível ao contexto se, e somente se, **L** é reconhecida por uma máquina de Turing com fita limitada. ❑

8.7 propriedades das linguagens recursivamente enumeráveis e recursivas

A seguir, são apresentadas algumas das principais propriedades da classe das linguagens recursivamente enumeráveis e da classe das linguagens recursivas, a saber:

a O complemento de uma linguagem recursiva é uma linguagem recursiva;

b Uma linguagem é recursiva se e somente se a linguagem e seu complemento são linguagens recursivamente enumeráveis.

c A classe das linguagens recursivas está contida propriamente na classe das linguagens recursivamente enumeráveis.

teorema 8.12 – Complemento de uma linguagem recursiva é recursiva

Se uma linguagem **L** sobre um alfabeto Σ qualquer é recursiva, então o seu complemento **~L** também é uma linguagem recursiva.

➥ **Prova: (*direta*)**
Suponha **L** uma linguagem recursiva sobre Σ. Então existe **M**, máquina de Turing, que aceita a linguagem e sempre para para qualquer entrada. Ou seja:

ACEITA(M) = L
REJEITA(M) = ~L
LOOP(M) = ∅

Seja **Inverte** uma máquina de Turing que inverte as condições de **ACEITA** por **REJEITA** e vice-versa, como ilustrado na figura 8.11 (esquerda).

Seja **M'** uma máquina de Turing resultante da composição das máquinas **Inverte** e **M**, conforme o esquema ilustrado na figura 8.11 (direita). Claramente, **M'** aceita a linguagem **~L** e sempre para para qualquer entrada, ou seja:

ACEITA(M') = ~L
REJEITA(M') = L
LOOP(M') = ∅

Portanto, o complemento de uma linguagem recursiva é uma linguagem recursiva. ❑

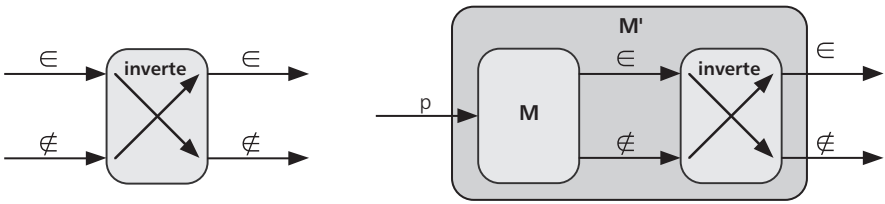


figura 8.11 Complemento de uma linguagem recursiva é recursiva.

teorema 8.13 – Linguagem recursiva × recursivamente enumerável

Uma linguagem **L** sobre um alfabeto Σ qualquer é recursiva se e somente se **L** e **~L** são recursivamente enumeráveis.

➥ **Prova:**

(⇒ *direta*).

Suponha **L** uma linguagem recursiva sobre Σ. Então, como foi mostrado no teorema 8.12 – Complemento de uma linguagem recursiva é recursiva, **~L** é recursiva. Como toda linguagem recursiva também é recursivamente enumerável, então **L** e **~L** são recursivamente enumeráveis;

(⇐ *direta*).

Suponha **L** uma linguagem sobre Σ tal que **L** e **~L** são recursivamente enumeráveis. Então existem $\mathbf{M_1}$ e $\mathbf{M_2}$, máquinas de Turing, tais que:

$$\mathbf{ACEITA(M_1) = L}$$
$$\mathbf{ACEITA(M_2) = \sim L}$$

Seja **Inverte** uma máquina de Turing que inverte as condições de **ACEITA** por **REJEITA** e vice-versa, como ilustrado na figura 8.11 (esquerda).

Seja **M** uma máquina de Turing resultante da composição das máquinas $\mathbf{M_1}$, $\mathbf{M_2}$ e **Inverte** conforme o esquema ilustrado na figura 8.12, ou seja:

- composição não determinista de $\mathbf{M_1}$ com $\mathbf{M_2}$;
- composição sequencial de $\mathbf{M_1}$ com **Inverte**.

Para qualquer palavra de entrada, **M** aceita se $\mathbf{M_1}$ aceita, e **M** rejeita se $\mathbf{M_2}$ aceita. Portanto, claramente, **M** sempre para. Logo, **L** é recursiva. ❑

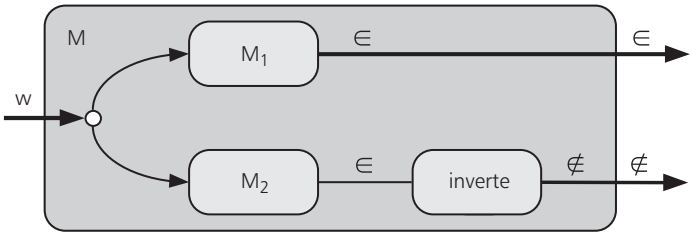


figura 8.12 Máquina de Turing não determinística.

teorema 8.14 – Linguagens recursivas ⊂ recursivamente enumeráveis

A classe das linguagens recursivas está contida propriamente na classe das linguagens recursivamente enumeráveis.

➥ **Prova: (*direta*)**
Para mostrar que a inclusão é própria, basta mostrar que existe pelos menos uma linguagem recursivamente enumerável que não é recursiva. A linguagem recursivamente enumerável que segue é não recursiva, e X_i e T_i são como definidas no teorema 8.4 – Linguagem não recursivamente enumerável (cuidado para não confundir com a linguagem $\{ X_i \mid X_i$ *não* é aceita por $T_i \}$):

$$L = \{ X_i \mid X_i \text{ é aceita por } T_i \}$$

L é recursivamente enumerável. Segue um esboço da construção de uma máquina de Turing **M** que aceita uma palavra **w** qualquer pertencente a **L**:

a **M** gera as palavras $X_1, X_2, \ldots$ em ordem lexicográfica, comparando-as com **w**. Quando $X_i = w$, **M** sabe que **w** é a **i**-ésima palavra na enumeração;

b **M** gera T_i, a **i**-ésima máquina de Turing (conforme algoritmo proposto nos exercícios);

c **M** simula T_i para a entrada $w = X_i$ e, se **w** pertence a **ACEITA**(T_i), então **w** pertence a **ACEITA(M)** (o algoritmo do simulador também é proposto como exercício);

Portanto, **M** aceita **w** se, e somente se, $X_i = w$ é aceita por T_i. Logo, **L** é recursivamente enumerável.

L não é recursiva. Conforme o teorema 8.13 – Linguagem recursiva × recursivamente enumerável, **L** é recursiva se, e somente se, **L** e seu complemento são recursivamente enumeráveis. Como o complemento de **L**, conforme o teorema 8.4 – Linguagem não recursivamente enumerável, não é recursivamente enumerável, então **L** é não recursiva. ❑

8.8 → exercícios

exercício 8.1 Qual a importância do estudo da máquina de Turing para a ciência da computação em geral e para as linguagens formais, em particular?

exercício 8.2 Para cada uma das linguagens abaixo, desenvolva uma máquina de Turing que a aceite e que sempre pare para qualquer entrada:

a $\varnothing$

b $\{ \varepsilon \}$

c $\{ \varepsilon, a, b \}$

d $\{ a, b \}^*$

e $\{ wcw \mid w$ é palavra de $\{ a, b \}^* \}$

f $\{ a^n b^n c^n \mid n \geq 0 \}$

g $\{ a^i b^j c^k \mid i = j$ ou $j = k \}$

h $\{ (awwa)^n \mid w$ é palavra de $\{ a, b \}^*$ e $n \geq 0 \}$

i $\{ w \mid w = a^1b^2a^3b^4 \ldots a^{n-1}b^n$ e n é par $\}$

exercício 8.3 Estenda a função programa da máquina de Turing, usando como argumento um estado e uma palavra, de forma similar à realizada para os autômatos finitos.

exercício 8.4 Sobre autômato com pilha e máquina de Turing:

a Modifique a definição do autômato com pilha, prevendo duas pilhas, definindo o formalismo *autômato com duas pilhas*;

b Analogamente para múltiplas pilhas, definindo o formalismo *autômato com múltiplas pilhas*;

c Demonstre que a classe linguagens recursivamente enumeráveis é composta pelas linguagens que podem ser aceitas por um autômato com duas pilhas.

Dica: mostre que os formalismos autômato com duas pilhas e máquina de Turing possuem o mesmo poder computacional;

d Demonstre que o poder computacional do autômato com múltiplas pilhas não é superior ao do autômato com duas pilhas.

exercício 8.5 Sobre autômato e máquina de Turing:

a Modifique a definição do autômato finito, prevendo a possibilidade do conjunto de estados ser infinito, definindo o formalismo *autômato* (eventualmente infinito);

b Demonstre que a classe das linguagens recursivamente enumeráveis é composta pelas linguagens que podem ser aceitas por um autômato (eventualmente infinito).

Dica: mostre que os formalismos autômato e máquina de Turing possuem o mesmo poder computacional.

exercício 8.6 Esboce um algoritmo que transforme qualquer máquina de Turing em uma máquina de Turing equivalente com somente dois estados.

Dica: a redução do número de estados pode ser compensada com o aumento do número de símbolos do alfabeto auxiliar.

Repare que este resultado mostra que a fita é suficiente como única "memória", não sendo necessário usar os estados para "memorizar" informações passadas.

exercício 8.7 Sobre a hipótese de Church:

a Por que não é demonstrável?

b Qual o seu significado e qual a sua importância?

exercício 8.8 Conforme proposto no teorema 8.4 – Linguagem não recursivamente enumerável, descreva um algoritmo para nomear (codificar) uma máquina de Turing qualquer sobre $\Sigma = \{ a, b \}$ como uma palavra em binário (do mesmo alfabeto Σ).

Sugestão: expresse cada transição do programa como uma **5**-upla em binário:

(estado atual, símbolo lido, novo estado, símbolo gravado, movimento da cabeça)

Então:

- supondo que o programa possua **n** transições (uma função programa sempre é finita), **5n** componentes definem toda a função programa;
- codifique as demais informações como estado inicial e o conjunto de estados finais;
- pelo *teorema fundamental da aritmética*, sabe-se que cada número natural é univocamente decomposto em seus fatores primos;
- assim, as **5n** componentes do programa e as informações complementares podem ser codificadas univocamente como um número natural.

exercício 8.9 Relativamente à classe das linguagens recursivas e à classe das linguagens recursivamente enumeráveis:

a Qual a diferença fundamental entre essas classes?
b Qual a importância de se distinguir essas duas classes?

exercício 8.10 Para cada uma das linguagens abaixo, desenvolva uma máquina de Turing com fita limitada que a aceite e que sempre pare para qualquer entrada:

a $\{ w \in \Sigma* \mid$ o décimo símbolo da direita para a esquerda de **w** é **a** $\}$
b $\{ w \mid w$ possui o mesmo número de símbolos **a**, **b** e **c** $\}$
c $\{ a^n b^m a^{n+m} \mid n \geq 0$ e $m \geq 0 \}$

exercício 8.11 Para cada uma das linguagens abaixo, desenvolva uma gramática que a gere:

a $\{ wcw \mid w$ é palavra de $\{ a, b \}^* \}$
b $\{ www \mid w$ é palavra de $\{ a, b \}^* \}$
c $\{ a^n b^m a^{n+m} \mid n \geq 0$ e $m \geq 0 \}$
d $\{ (a^2)^n \mid n \geq 0 \}$

exercício 8.12 Para cada uma das linguagens abaixo, desenvolva uma gramática sensível ao contexto:

a $\{ w \in \Sigma^* \mid$ o décimo símbolo da direita para a esquerda de **w** é **a** $\}$
b $\{ www \mid w$ é palavra de $\{ a, b \}^* \}$
c $\{ a^n b^m a^{n+m} \mid n \geq 0$ e $m \geq 0 \}$

exercício 8.13 Por que nem toda gramática livre do contexto é uma gramática sensível ao contexto?

exercício 8.14 Uma máquina de Turing com fita limitada possui uma fita finita, um conjunto de estados finitos e alfabetos finitos. Portanto, pode assumir um conjunto finito de estados. Então por que o seu poder computacional não é equivalente ao de um autômato finito? Fundamente a sua resposta.

Dica: é certo que os dois formalismos estão definidos sobre um número finito de estados. Portanto, a solução encontra-se em alguma outra condição a qual é central no correto entendimento do formalismo autômato finito.

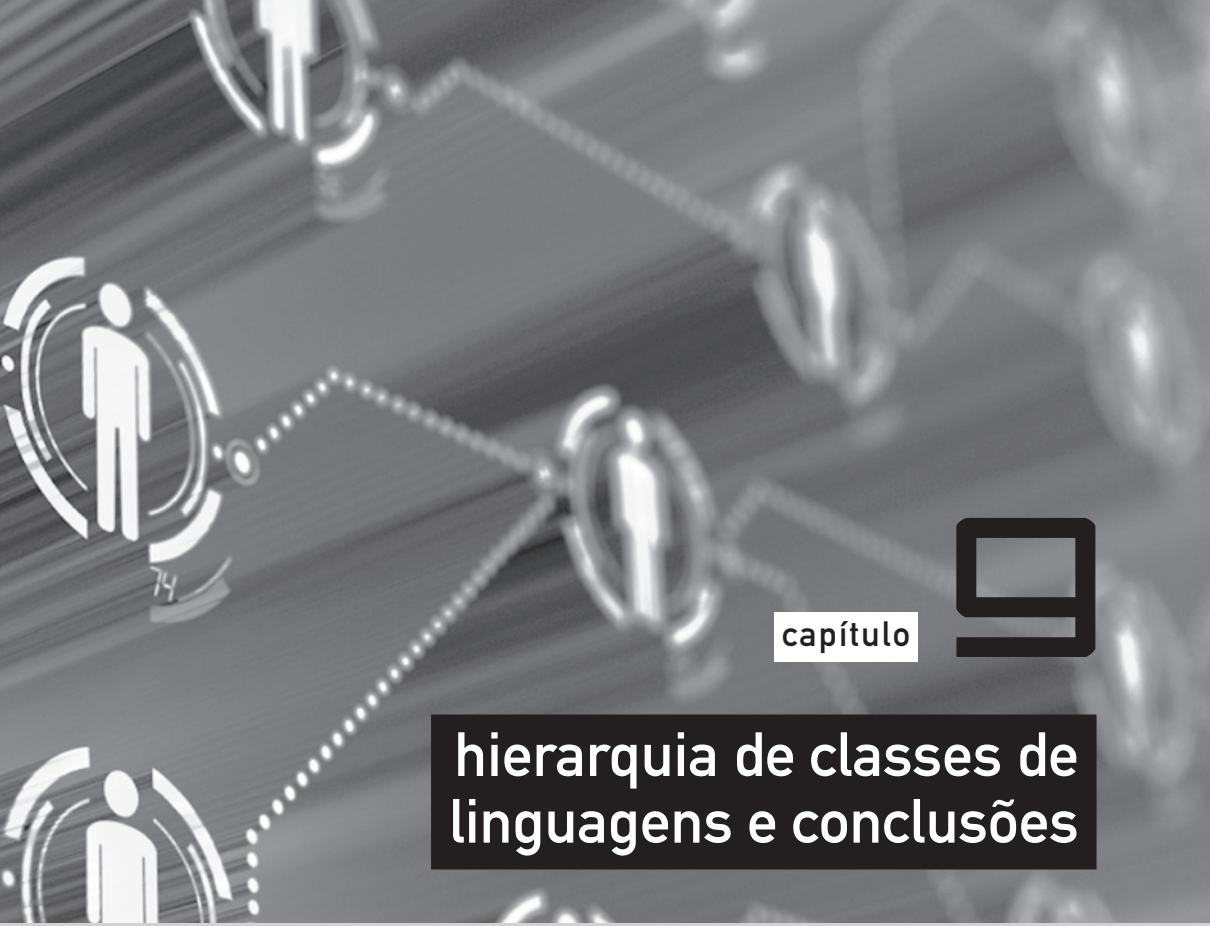
exercício 8.15 Demonstre que a classe das linguagens recursivas é fechada para as seguintes operações:

a União;
b Intersecção;
c Diferença;
d Concatenação sucessiva (**L***, supondo **L** recursiva).

exercício 8.16 Com relação à classe das linguagens recursivamente enumeráveis, o que se pode afirmar para as seguintes operações:

a União?
b Intersecção?
c Diferença?

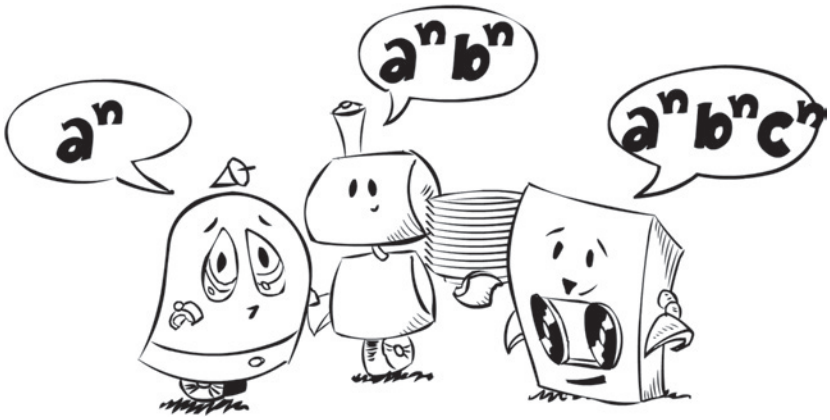
exercício 8.17 Desenvolva um programa em computador que simule qualquer máquina de Turing. A entrada para o simulador deve ser a função programa, e a saída, o estado final da máquina simulada, o conteúdo da fita e o número de movimentos da cabeça da fita.



capítulo 9

hierarquia de classes de linguagens e conclusões

As classes das linguagens regulares, livres do contexto, sensíveis ao contexto e recursivamente enumeráveis e suas inclusões próprias constituem a hierarquia de Chomsky, assunto discutido neste capítulo. O texto também apresenta uma série de problemas em aberto, e, na forma de leitura complementar, as gramáticas de grafos, mostrando que a generalização das gramáticas de Chomsky tem um grande potencial de aplicações na computação e informática.



9.1 hierarquia de Chomsky

As seguintes classes de linguagens estudadas:

- *Regulares* ou *tipo 3*;
- *Livres do contexto* ou *tipo 2*;
- *Sensíveis ao contexto* ou *tipo 1*;
- *Recursivamente enumeráveis* ou *tipo 0*;

e as suas inclusões próprias, como ilustrado na figura 9.1, constituem o que normalmente é conhecido como a *hierarquia de Chomsky*. Noam Chomsky definiu estas classes como (potenciais) modelos para linguagens naturais.

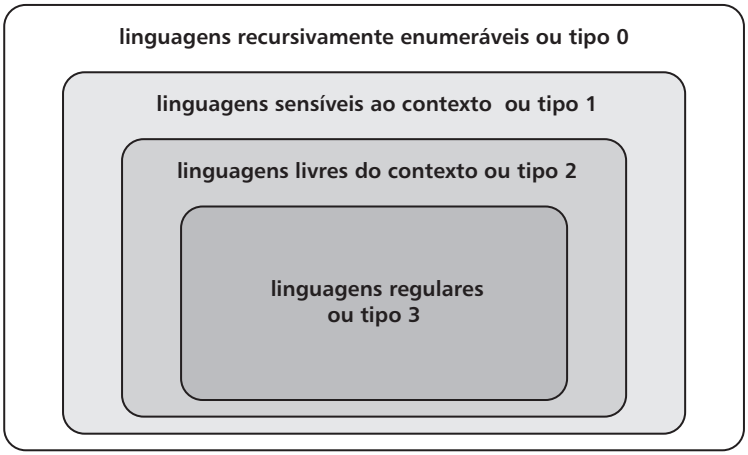


figura 9.1 Hierarquia de Chomsky.

Entretanto, nem sempre as linguagens de programação são tratadas adequadamente na hierarquia de Chomsky. Existem linguagens que não são livres do contexto, para as quais o poder dos formalismos sensíveis ao contexto é excessivo, sendo inadequados principalmente no que se refere à complexidade (quantidade de recursos computacionais como tempo e espaço). Adicionalmente, o conhecimento das linguagens sensíveis ao contexto é relativamente limitado, o que dificulta o seu tratamento. Alguns exemplos de problemas que não podem ser tratados na classe de linguagens livres do contexto são os seguintes:

a Múltiplas ocorrências de um mesmo trecho de programa, como a declaração de um identificador e suas referências de uso (problema análogo ao da linguagem **{ wcw | w** é palavra de **{a, b}* }**, a qual não é livre do contexto);

b Alguns casos de validação de expressões com variáveis de tipos diferentes;

c A associação de um significado (semântica) de um trecho de programa, que dependeria da análise de um conjunto de informações (dependentes de contextos) como identificadores, ambientes, tipos de dados, localização, sequências de operações, etc.

Por outro lado, para algumas linguagens de programação, a classe das linguagens livres do contexto é excessiva, e a classe das linguagens regulares, insuficiente. Por exemplo, o formalismo autômato com pilha possui a facilidade de não determinismo. Entretanto, se a linguagem pode ser denotada por um *autômato com pilha determinístico* (ou seja, se é uma *linguagem livre do contexto determinística*), então é possível implementar (facilmente) um reconhecedor com tempo de processamento proporcional a **2n** (**n** é o tamanho da entrada), o que é muito mais eficiente que o melhor algoritmo conhecido para as linguagens livres do contexto.

De qualquer forma, tanto para linguagens artificiais como para linguagens naturais, o estudo da classe das linguagens livres do contexto tem sido de especial interesse, pois elas permitem uma representação simples da sintaxe, adequada tanto para a estruturação formal, como para a análise computacional. Entretanto, o estudo dessa classe tem mostrado *problemas não solucionáveis*, como, por exemplo:

a Determinar se uma gramática livre do contexto é ambígua (ou seja, se existem duas ou mais árvores de derivação distintas para uma mesma palavra);

b Não existe um algoritmo que verifique a igualdade de duas linguagens livres do contexto, o que dificulta a otimização e o teste de processadores de linguagens;

Portanto, dependendo da linguagem e dos objetivos do trabalho, alguns estudos específicos, eventualmente fora da hierarquia de Chomsky, são recomendados ou necessários (neste sentido, alguns estudos foram apresentados, como a classe de linguagens recursivas). Estudos mais avançados extrapolam o objetivo deste livro, e algumas referências sobre o assunto podem ser encontradas na seção leituras sugeridas.

9.2 conclusões

As linguagens formais oferecem meios para modelar e desenvolver ferramentas que especificam linguagens e seus processos de análise, bem como suas propriedades e limitações algorítmicas.

Alguns problemas referentes às linguagens formais, embora atuais, possuem questões em aberto, como:

a A tradução de linguagens, principalmente as naturais;

b No caso específico dos autômatos finitos, o desenvolvimento de soluções (possivelmente) complexas frequentemente exige um número excessivo de estados, resultando em uma *explosão de estados*. O tratamento da explosão de estados é um importante tema de pesquisa, tanto para a interface humano × máquina como para a implementação computacional;

c O tratamento de *linguagens n-dimensionais*, com destaque para as bi e as tridimensionais, com importantes aplicações como:

- processamento de imagens;
- animações;
- *sistemas biológicos*: simulação do desenvolvimento de sistemas vivos), etc., tanto no plano, quanto no espaço;
- *sistemas concorrentes* (eventualmente *distribuídos* e/ou *comunicantes*): especificação formal e prova de propriedades.

Uma limitação do trabalho desenvolvido ao longo deste livro é o fato de que os formalismos desenvolvidos não são adequados para o tratamento de problemas complexos, pois não possuem construções composicionais em suas definições. De fato, todos os formalismos desenvolvidos são do tipo "monolítico", sem qualquer estruturação modular ou hierárquica. A existência ou não de algumas construções composicionais foi explorada ao longo do texto como união, intersecção, complemento, etc. Entretanto, estas são limitadas em termos de expressividade. Construções mais ricas são, em geral, inspiradas em *teoria das categorias* e constituem uma álgebra sobre os formalismos, com operações expressivas, permitindo o desenvolvimento de soluções potencialmente complexas de forma simples e (em grande parte dos casos) corretas por construção. Essa abordagem transcende o objetivo deste livro, mas trata de uma importante linha de pesquisa para quem deseja dar continuidade aos estudos na área.

9.3 leitura complementar: gramática de grafos

Como ilustração de uma abordagem das linguagens formais às linguagens **n**-dimensionais, no que segue é apresentada a noção de *gramática de grafos*.

A ideia básica das gramáticas de grafos é análoga à das gramáticas de Chomsky, ou seja:

- regras de produção são pares (mas de grafos);
- uma derivação é a substituição de um subgrafo de acordo com uma regra de produção.

As gramáticas de grafos constituem um caso particular das gramáticas categoriais (nenhum conceito de *teoria das categorias* é formalmente introduzido nesta publicação). A ideia básica é substituir palavras por grafos no conceito de gramática de Chomsky. Mas, observe que, na realidade, a noção categorial de gramática é bem mais expressiva, pois:

- gramáticas categoriais podem ser usadas sobre palavras, grafos, conjuntos parcialmente ordenados, redes, autômatos, máquinas, linguagens de programação e outros tipos de objetos, desde que sejam satisfeitas determinadas condições;
- as derivações (aplicações de regras de uma gramática) são generalizadas.

Informalmente, alguns conceitos podem ser visualizados no seguinte exemplo, o qual ilustra o jogo de videogame PacMan.

exemplo 9.1 – Gramática de grafos: PacMan

O jogo PacMan (simplificado) possui um tabuleiro juntamente com algumas entidades especiais como um **PacMan** e dois conjuntos, um de **fantasmas** e outro de **maçãs**. A figura 9.2 ilustra um grafo representando um estado do sistema no qual, para facilitar a visualização, as entidades **PacMan**, **fantasmas** e **maçãs** são representadas por nodos com simbologia própria. Na figura, vê-se que:

- os nodos **pretos** representam os lugares do tabuleiro, e as correspondentes arestas, os caminhos possíveis entre dois lugares;
- para as entidades **PacMan**, **fantasmas** e **maçãs**, os correspondentes arcos denotam o seu posicionamento no tabuleiro;
- uma **maçã** cujo nodo destino é o nodo branco denota uma maçã já comida;
- a aresta numerada com origem e destino no nodo **branco** identifica a **fase** em que se encontra o jogo (no caso, a segunda fase).

A figura 9.3 ilustra as seguintes regras de produção:

- **move**: o **PacMan** move-se para uma casa adjacente;
- **come**: o **PacMan** come a **maçã**. A **maçã** comida é posicionada no nodo **branco**;
- **mata**: o **fantasma** mata o **PacMan**. O **PacMan** é excluído.

Por fim, a figura 9.4 ilustra um possível grafo resultante da aplicação da regra **move** ao grafo ilustrado na figura 9.2. Observe que, no grafo transformado, o **PacMan** encontra-se em um lugar onde existe um fantasma. Assim, a próxima produção a ser aplicada pode ser **morre** ou **move**. ❑

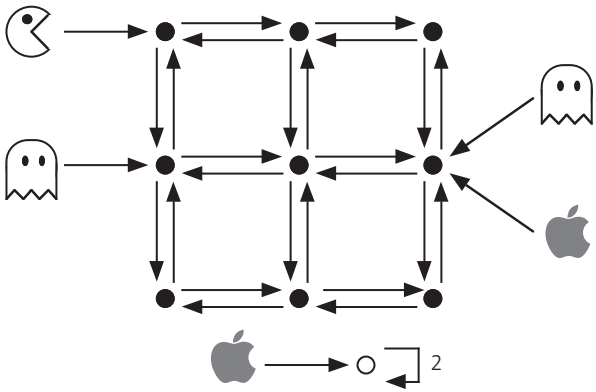


figura 9.2 Gramática de grafos: grafo do jogo PacMan.

Comparativamente com as gramáticas de Chomsky, as gramáticas de grafos:

- em geral, não distinguem entre variáveis e terminais (todos os símbolos – no caso, grafos – são tratados como terminais);
- possui um símbolo inicial (no caso, grafo inicial);
- a linguagem gerada é o conjunto de grafos que podem ser gerados, via derivações, a partir do grafo inicial.

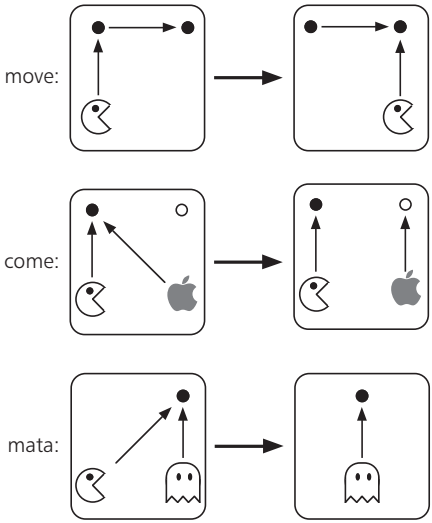


figura 9.3 Gramática de grafos: regras de produção do PacMan.

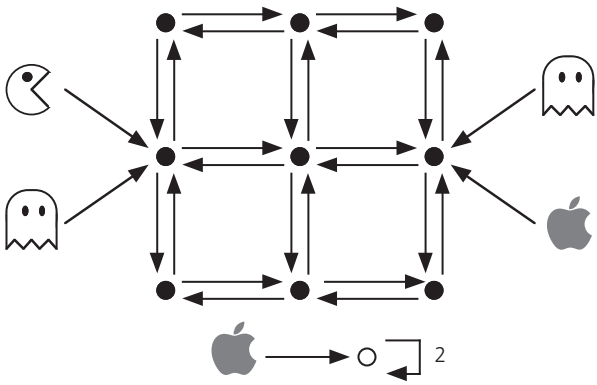


figura 9.4 Grafo resultante da aplicação da regra **move**.

9.4 → exercícios

exercício 9.1 Qual a importância da hipótese de Church na hierarquia de Chomsky?

exercício 9.1 Para quais classes de linguagens da hierarquia de Chomsky pode-se afirmar que, para qualquer linguagem da classe, sempre existe um algoritmo que aceita a linguagem e que sempre para para qualquer entrada? Justifique a sua resposta para cada classe.

exercício 9.1 Considere o exemplo 9.1 – Gramática de grafos: PacMan. No estado do jogo representado na figura 9.4, é possível aplicar tanto a regra **move** como a regra **morre**. Modifique a gramática de forma a permitir que, nessa situação, somente a regra **morre** possa ser aplicada.

exercício 9.2 Desenvolva uma gramática de grafos para o **jogo da velha** de tal forma que:

- considere jogadas alternadas de dois jogadores;
- apresente condição de parada quando um dos jogadores completa uma linha de três casas (horizontal, vertical ou oblíqua).

Como ilustração, na figura 9.5, é mostrada uma possível configuração do jogo, vencido pelo jogador com as pedras em formato de ×.

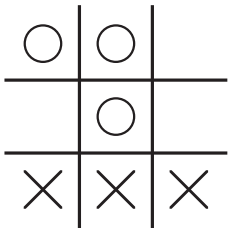


figura 9.5 Jogo da velha: jogador com as pedras identificadas por × ganhou.

exercício 9.3 Desenvolva uma gramática de grafos para o **jogo de damas**, prevendo que o jogador com as pedras brancas inicia o jogo.

Dica: na definição da regra "comer uma pedra", lembre-se de que o movimento é sempre em "linha reta" (não pode fazer uma "curva" de 90° no tabuleiro).

exercício 9.4 Justifique a seguinte afirmação:

> *gramáticas de grafos são gramáticas do tipo irrestritas, no que se refere às produções.*

leituras sugeridas

ACCORSI, F. *Anima ‹o bidimensional para World Wide Web baseada em autômatos finitos*. 2002. 113 f. Dissertação (Mestrado em Computação) – Universidade Federal do Rio Grande do Sul, Porto Alegre, 2002.

ACCORSI, F.; MENEZES, P. F. B.; NEDEL, L. P . Animacão gráfica baseada em autômatos temporizados sincronizados. In: WORKSHOP DE MÉTODOS FORMAIS, 4., 2001, Rio de Janeiro. *Anais*... Rio de Janeiro: SBC, 2001. p. 75-80. v. 1.

AHO, A. V. *Currents in the theory of computing*. Englewood Cliffs: Prentice-Hall, 1973.

AHO, A. V.; ULLMAN, J. D. *The theory of parsing*: translation and compiling. Englewood Cliffs: Prentice-Hall, 1972.

APPLE COMPUTER INC. *QuickTime File Format*. [S.l.]: Apple Computer, 2000. Disponível em: <http://developer.apple.com/techpubs/quicktime/qtdevdocs/RM/pdfframe.htm>. Acesso em: 19 ago. 2010.

BARR, M.; WELLS, C. *Category theory for computing science*. 2nd ed. Englewood Cliffs: Prentice Hall, 1995.

BECKMAN, F. S. *Mathematical foundations of programming*. Reading: Addison-Wesley, 1980.

BROOKSHEAR, J. G. *Theory of computation*: formal languages, automata, and complexity. Redwood City: Benjamin/Cummings, 1989.

BUSH, Vannevar. As we may think. *The Atlantic Monthly*, v. 176, n. 1, p.101-108, 1945. Disponível em: <http://web.mit.edu/STS.035/www/PDFs/think.pdf>. Acesso em: 19 ago. 2010.

CLEAVELAND, J. C. *Grammars for programming languages*. New York: Elsevier, 1977.

COMPUSERVE INCORPORATED. *Graphics Interchange Format*: programming reference version 89a. [S.l.]: Compuserve Incorporated, 1990.

CONKLIN, J. Hypertext: an introduction and survey. *IEEE Computer*, v. 20, n. 9, p. 1741, 1987.

DEGANO, P.; GORRIERI, R.; MARCHETTI-SPACCAMELA, A. Lecture notes in computer science. In: INTERNATIONAL COLLOQUIUM ON AUTOMATA, LANGUAGE AND PROGRAMMING, 24., 1997. *Proceedings*… Berlin: Springer-Verlag, 1997. v. 1256.

DIVERIO, T. A.; MENEZES, P. B. *Teoria da computação*: máquinas universais e computabilidade. 2. ed. Porto Alegre: Sagra Luzzatto, 2000.

ENGELBART, D. C.; ENGLISH, W. K. A research center for augmenting human intellect. In: FALL JOINT COMPUTER CONFERENCE, 1994, San Francisco. *Proceedings*… Washigton: Thompson Books, 1968. p. 395-410.

FERREIRA, A. B. H. *Novo dicionário da língua portuguesa*. Rio de Janeiro: Nova Fronteira, 1999.

GOUGH, K. J. *Syntax analysis and software tools*. Sydney: Addison-Wesley, 1988.

HARRISON, M. A. *Introduction to formal language theory*. Sydney: Addison-Wesley, 1978.

HOPCROFT, J. E.; MOTWANI, R.; ULLMAN, J. D. *Introduction to automata theory, languages and computation*. 2nd ed. Sydney: Addison-Wesley, 2000.

HOPCROFT, J. E.; ULLMAN, J. D. *Formal languages and their relation to automata*. Sydney: Addison-Wesley, 1969.

HOPCROFT, J. E.; ULLMAN, J. D. *Introduction to automata theory, languages, and computation*. Sydney: Addison-Wesley, 1979.

INTERNATIONAL COLLOQUIUM ON AUTOMATA, LANGUAGE AND PROGRAMMING, 22., 1995. *Proceedings*… Berlin: Springer-Verlag, 1997.

INTERNATIONAL ORGANISATION FOR STANDARDISATION. Short MPEG-1 description. [S.l.]: ISO, 1996. Dispon'vel em: <http://mpeg.chiariglione.org/standards/ mpeg-1/mpeg-1.htm>. Acesso em: 24 ago. 2010.

LÖWE, M. Algebraic approach to single-pushout graph transformation. *Theoretical Computer Science*, v. 109, n. 1-2, p. 181-224, 1993.

MACHADO, J. et al. Autômatos finitos: um formalismo para cursos na web. In: SIMPÓSIO BRASILEIRO DE ENGENHARIA DE SOFTWARE, 13., 1999, Florianópolis. *Anais*... Florianópolis: UFSC, 1999. p. 213-223.

MACHADO, J. et al. Structuring Web Course Pages as Automata: revising concepts. In: RECHERCHE D'INFORMATIONS ASSISTÉE PAR ORDINATEUR, 2000, Paris. *Proceedings*… Paris: C.I.D, 2000. p. 150-159. v. 1.

MANNA, Z. *Mathematical theory of computation*. New York: McGraw-Hill, 1974.

MAURER, H. A. *On non context-free grammar forms*. Karlsruhe: Universität Karlsruhe, 1977.

MENEZES, P. B. A categorial framework for concurrent, anticipatory systems. In: AIP CONFERENCE, 1999, Woodbury. *Proceedings*... Woodbury: American Institute of Physics, 1999. p. 185-199. v. 465.

MENEZES, P. B. *Matemática discreta para computação e informática*. Porto Alegre: Sagra Luzzatto, 2004.

MENEZES, P. B.; COSTA, J. F.; SERNADAS, A. S. C. Refinement mapping for general (discrete event) system theory. *Lecture Notes in Computer Science*, v. 1030, p. 103-116, 1996.

MENEZES, P. B.; HAEUSLER, E. H. *Teoria das categorias para ciência da computação*. Porto Alegre: Sagra Luzzatto, 2001.

MENEZES, P. B.; MACHADO, J. Adaptive web courses: a categorial framework. *International Journal of Computing Anticipatory Systems*, v. 9, p. 318-336, 2001.

MENEZES, P. B.; RIBEIRO, L. A graph transformation framework for concurrent systems. In: INTERNATIONAL CONFERENCE ON SYSTEMS RESEARCH, INFORMATICS AND CYBERNETICS, 10., 1998, Baden-Baden. *Proceedings*... Baden-Baden: IIAS, 1998. p. 85-89. v. 4.

MENEZES, P. B.; SERNADAS, A.; COSTA, J. F. Nonsequential automata semantics for concurrent, object-based language. *Electronic Notes in Theoretical Computer Science*, v. 14, p. 245-273, 1998.

MICROSOFT CORPORATION. *AVI RIFF file reference*. [S.l.]: Microdoft Corporation, c2005. Disponível em: <http://msdn.microsoft.com/en-us/library/ms899421.aspx>. Acesso em: 23 ago. 2010.

MOLL, R. N.; ARBIB, M. A.; KFOURY, A. J. *An introduction to formal language theory*. Berlin: Springer-Verlag, 1988.

MORAIS, C. T. et al. A web teaching system based on formal methods. In: IFIP WORLD COMPUTER CONGRESS, 16., 2000, Beijing. *Proceedings*… Beijing: PHEI, 2000. p. 221-224. v. 1.

NELSON, T. H. Getting it out of our system. In: SCHECHTER, G. (Ed.). *Information retrieval*: a critical review. Washington: Thompson Books, 1967.

NIJHOLT, A. *Context-free grammars*: covers, normal forms, and parsing. Berlin: Springer, 1980.

PAUN, G.; SALOMAA, A. (Ed.). *New trends in formal languages*: control, cooperation, and combinatorics. Berlin: Springer, 1997.

RIBEIRO, L. *Parallel composition and unfolding semantics of graph grammars*. Berlin: Technical University of Berlin, 1996.

ROZENBERG, G.; SALOMAA, A. (Ed.). *Handbook of formal languages*. Berlin: Springer, 1997.

SALOMAA, A. *Formal languages*. New York: Academic Press, 1973.

SERNADAS, C. *Introduão à teoria da computação*. Portugal: Presença, 1992.

SIPPU, S.; SOISALON-SOININEN, E. *Parsing theory*. Berlin: Springer-Verlag, 1988. v. I e II.

STOY, F. E. *Denotational semantics*: the Scott-Strachey approach to programming language theory. Cambridge: MIT Press, 1977.

VERSON, J. A.; NOONAN, R. *Heuristic approach to context sensitive parsing*. Maryland: University of Maryland, 1974.

índice

A

absurdo (prova), 38,40
ação atômica, 68
aceita
  AFD, 74, 75
  AFN, 82
  AFNε, 90
  AP, 178, 179
  MT, 217
  MTFL, 228
AFD (autômato finito determinístico), 71
AFN (autômato finito não determinístico), 80
AFNε (autômato finito c/ movimentos vazios), 87
alfabeto, 54
  auxiliar (AP), 177
  auxiliar (MT), 216
  auxiliar (MTFL), 228
  da pilha (AP), 177
  de entrada (AFD), 71
  de entrada (AFN), 80
  de entrada (AFNε), 87
  de entrada (AP), 177
  de entrada (Mealy), 135
  de entrada (Moore), 136
  de entrada (MT), 216
  de entrada (MTFL), 228
  de saída (Mealy), 135
  de saída (Moore), 136
  de símbolos de entrada (AFD), 71
  de símbolos de entrada (AFN), 80
  de símbolos de entrada (AFNε), 87
  de símbolos de entrada (AP), 177
  de símbolos de entrada (Mealy), 135
  de símbolos de entrada (Moore), 136
  de símbolos de entrada (MT), 216
  de símbolos de entrada (MTFL), 228
  de símbolos de saída, 134
  de símbolos de saída (Mealy), 135
  de símbolos de saída (Moore), 136
algoritmo, 212, 220
  para exclusão dos símbolos inúteis, 162
  autômato com pilha descendente, 195, 201
  Cocke-Younger-Kasami, 195, 203
  CYK, 203
  de minimização (AFD), 124
  Early, 195, 205
  forma normal de Chomsky, 168
  forma normal de Greibach, 171
  produções que substituem variáveis, 166
  produções vazias, 164
  recursões à esquerda, 174
  símbolos inúteis, 162
analisador
  léxico, 68
  sintático, 138, 154
análise
  léxica, 18, 19, 67
  sintática, 18, 19, 154
animação, 240
  sistema, 18, 67, 145
  em tempo real, 145
  quadro a quadro, 134, 145
antissimétrica (relação), 25
AP (autômato com pilha), 177
aplicação, 30
arco, 27
aresta, 27
árvore, 155, 157
árvore de derivação, 155, 158
associativa
  concatenação, 56
  conjunção, 49
  disjunção, 49
  intersecção, 24, 46
  união, 24, 146
atômica (proposição), 34
átomo, 34
audio video interleave (AVI), 145
autômato
  eventualmente infinito, 233
  com duas pilhas, 185, 220, 233
  com múltiplas pilhas, 185, 220, 233
  com pilha, 20, 154, 175-178
  com pilha descendente, 201
  com pilha descendente (algoritmo), 195, 201

com pilha determinístico, 185, 239
com pilha não determinístico, 175-177, 185
finito, 20, 66, 69-72
finito com movimentos vazios, 69, 87-89
finito com saída, 134
finito determinístico, 69-71, 115, 122, 123
finito determinístico mínimo, 115, 122, 123
finito mínimo, 67
finito não determinístico, 69, 80, 81
finito não determinístico e com movimentos vazios, 87
finito equivalente, 76
autorrelação, 25
AVI (*audio video interleave*), 145
axiomático (formalismo), 20

B

backus naur form, 157
base (pilha), 175
base de indução, 42
bicondição (conetivo), 35
bijeção, 31
bijetora (função), 31
bit, 55
bloco da partição, 28
bloco-estruturada (linguagem), 154
BNF (*backus naur form*), 157
boa ordem (relação), 41
bombeamento
linguagens livres do contexto, 196
linguagens regulares, 115
Boole, George, 33
Boole (lógica), 33
booleana (lógica), 33
branco (MT), 216
Bush, Vannevar, 142

C

c (linguagem), 67, 110, 154, 156
cabeça da fita
ap, 176
autômato finito, 70
MT, 216
de saída, 134
cabeça da pilha (AP), 176
cadeia, 27
caracteres, 54, 55
vazia, 55
cálculo lambda, 222
caminho (grafo), 78
caractere, 54
cardinalidade, 32
finita, 32
infinita, 32
categorias (teoria), 240
Chomsky (gramáticas), 19
Chomsky, N., 238
Church, A., 212
Church (hipótese), 212, 222
Church (tese), 212, 222
ciência da computação, 212
circuito lógico, 18, 122
classe de equivalência, 28
Cocke, J., 203
Cocke-Younger-Kasami (algoritmo), 195, 203
codomínio (relação), 25
compilador, 69, 138, 157
complemento, 23
complexidade de algoritmos, 67, 115, 122
composição
funções parciais, 30
concorrente (sistemas), 68
de funções, 31
de sistemas, 67
não determinista (sistemas), 68
sequencial (sistemas), 68
composicionalidade (horizontal), 20
compreensão (denotação por), 22
comprimento (palavra), 57
computabilidade, 68
computação, 212, 217
AFD, 75
AFN, 81
AFNε, 88-90
AP, 178
Mealy, 135
Moore, 137
vazia, 89
computador, 68
comunicantes (sistemas), 240
comutativa
conjunção, 49
disjunção, 49
intersecção, 24, 46
união, 24, 46
concatenação
palavras, 56
sucessiva, 56
conclusão (condição), 35
concorrência
modelos, 68, 69
falsa, 69
concorrente
composição de sistemas, 68
sistema, 240
condição (conetivo), 35
conetivo
lógico, 34
e, 34
não, 34

ou, 35
se-então, 35
se-somente-se, 35
conexa (relação), 25
conjunção (conetivo), 34
conjunto, 20
contável, 33
das partes, 23
de chegada (relação), 25
de estados
AFD 71
AFN, 80
AFNε, 87
AP, 177
Mealy, 135
Moore, 136
MT, 216
MTFL, 228
de estados finais
AFD, 71
AFN, 81
AFNε, 88
AP, 177
Mealy, 135
Moore, 136
MT, 216
MTFL, 228
conjunto
de partida (relação), 25
de todas as palavras, 57
dos números inteiros, 23
dos números irracionais, 23
dos números naturais, 23
dos números racionais, 23
dos números reais, 23
enumerável, 33
falsidade, 34
finito, 22, 32
imagem, 29
infinitamente contável, 33
infinito, 22, 32
não contável, 33
universo, 23
vazio, 21
verdade, 34
disjunto, 23
igual, 21
independente, 23
isomorfo, 31
mutuamente exclusivo, 23
contável, 33
contém, 21
contém propriamente, 21
contido, 21
contido propriamente, 21
contradição, 34
contradomínio (relação), 25
contraposição, 36
contraposição (prova), 38, 40
controle finito (autômato finito), 70, 176
corolário, 37
CYK (algoritmo), 203

D

definição
indutiva, 27, 42, 44
recursiva, 27, 42, 44
definida (função parcial), 29
demonstração
direta, 39
por absurdo, 38, 40
por contraexemplo, 40
por contraposição, 38, 40
por indução, 38
por redução ao absurdo, 38, 40
DeMorgan
conjuntos, 24, 46, 47
lógica, 49
denotação
compreensão, 22
por extensão, 21
denotacional (formalismo), 20
derivação
gramática, 59
mais à direita, 159
mais à esquerda, 159
destino (relação), 25
diferença (conjuntos), 23
direta (prova), 38, 39
disjunção (conetivo), 35
distribuído (sistema), 240
distributiva
conjunção sobre a disjunção, 49
disjunção sobre a conjunção, 49
intersecção sobre a união, 24, 46
união sobre a intersecção, 24, 46
domínio (relação), 25
dupla negação, 49
duplo complemento (conjuntos), 24, 46

E

e (conetivo), 34
Early (algoritmo), 195, 205
elemento, 20
absorvente, 63
inverso, 63
neutro
concatenação, 56
intersecção, 46
união, 46

encapsulação
orientação a objetos, 70
transição, 70, 87
endorrelação, 25
endotransição, 78
enumeração, 33
enumerável (conjunto), 33
epimorfismos, 31
equivalência (relação), 28, 36
equivalentes (autômatos finitos), 76
ER (expressão regular), 20, 93
estado
MT, 216
MTFL, 228
alcançável (AFD), 123
estado final
AFD, 71
AFN, 81
AFNε, 88
AP, 177
Mealy, 135
Moore, 136
MT, 216
MTFL, 228
estado inacessível, 130
estado inacessível (AFD), 123
estado inicial
AFD, 71
AFN, 80
AFNε, 88
AP, 177
Mealy, 135
Moore, 136
MT, 216
MTFL, 228
estado inútil (AFD), 124
estados
AFD, 71
AFN, 80
AFNε, 88
equivalentes (AFD), 123
finitos (sistema), 66, 67, 69
explosão de estados
autômato finito, 240
sistema de estados finitos, 68
expressão regular, 20, 66, 93, 94
extensão (denotação por), 21

F

falso (valor verdade), 33
faz nada, 78
fecho
relação, 26
transitivo e reflexivo, 27
transitivo, 27, 45
vazio (função), 89
finito (conjunto), 22, 32
fita, 221
AP, 176
autômato finito, 70
MT, 215
de saída, 134
folha (árvore), 155, 157, 158
forma
de Backus Naur, 157
normal de Chomsky, 167, 168
normal de Greibach, 168, 170
formalismo
axiomático, 20
denotacional, 20
funcional, 20
gerador, 20
operacional, 20
reconhecedor, 20
fórmula, 34
fórmula (lógica), 34
função, 30
bijetora, 31
computável, 212
de saída (moore), 136
de transição
AFD, 71
AFN, 80
AFNε, 87
AP, 176, 177
autômato finito, 70
Mealy, 135
Moore, 136
MT, 215, 216
MTFL, 228
fecho vazio, 89
identidade, 31
inclusão, 32
injetora, 31
inversa à direita, 49
inversa à esquerda, 49
parcial, 29
programa
AFD, 71
AFN, 80
AFNε, 87
AP, 176, 177
autômato finito, 70
Mealy, 135
Moore, 136
MT, 215, 216
MTFL, 228
estendida
AFD, 75
AFN, 81
AFNε, 90
sobrejetora, 31
total, 30

funcional (formalismo), 20
funções recursivas (Kleene), 222

G

gerador (formalismo), 20
GIF (*graphics interchange format*), 145
GLD (gramática linear à direita), 100
GLE (gramática linear à esquerda), 100
GLUD (gramática linear unitária à direita), 100
GLUE (gramática linear unitária à esquerda), 100
GR (gramática regular), 100, 101
grafo
  direto, 27
  dirigido, 27
  finito direto, 74
grafos (gramática), 240
gramática, 20, 58
  ambígua, 155, 160, 161
  de Chomsky, 19, 58
  de grafos, 240
  irrestrita, 20, 58, 213, 225
  linear, 100
    à direita, 100
    à esquerda, 100
    unitária à direita, 100
    unitária à esquerda, 100
  livre do contexto, 20, 154, 155
  livre do contexto ambígua, 160
  regular, 20, 66, 100, 101
  sensível ao contexto, 20, 213, 216
  Equivalente, 62
graphics interchange format (GIF), 145

H

hierarquia de Chomsky, 66, 154, 238
hipermídia, 18, 67, 134, 142
hipertexto, 18, 67, 134, 141
hipótese, 37
  de Church, 212, 222
  de indução, 42
  de Turing-Church, 222

I

idempotência
  conjunção, 49
  disjunção, 49
  intersecção, 24, 46
  união, 24, 46
identidade (função), 31
igualdade (conjuntos), 21
imagem, 29
implicação (relação), 36
inclusão (função), 32
independência (concorrência), 68
indução
  base, 42
  hipótese, 42
  passo, 42
  prova, 38, 42
  em estrutura, 44
  estruturada, 44
  estrutural, 44
  matemática
    primeiro princípio, 42
      princípio, 41, 42
      segundo princípio, 43
indutiva
  definição, 27, 42
  prova, 42
indutivamente definido, 44
infinitamente contável (conjunto), 33
infinito (conjunto), 22, 32
injeção, 31
injetora (função), 31
inteiros (conjunto), 23
intersecção, 23
inversa
  à direita (função), 49
  à esquerda (função), 49
irracionais (conjunto), 23
irrestrita (gramática), 20, 213, 225
isomorfismo, 31
isomorfismo de autômatos finitos, 128

J

Java, 67 ,110, 154, 156

K

Kasami, T., 203
Kleene, S. C., 222

L

lambda, cálculo, 222
lema, 37
lema do bombeamento
  linguagens livres do contexto, 196
  linguagens regulares, 115
léxica (análise), 18, 19
linguagem, 54, 57
  aceita
    AFD, 75
      AFN, 82
      AFNε, 90
      AP, 179
      MT, 217
      MTFL, 228

artificial, 18, 154
de programação, 18, 154
espacial, 18
formal, 57
gerada
ER, 94, 95
GR, 101, 155
gramática, 60
inerentemente ambígua, 161
livre do contexto, 154, 155, 238
livre do contexto determinística, 185, 191, 239
loop
AP, 179
MT, 217, 218
MTFL, 228
não linear, 18
natural, 18, 238
n-dimensional, 18, 240
orientada a objetos, 70
planar, 18
reconhecida
AFD, 75
AFN, 82
AFNε, 90
AP, 179
MT, 217
MTFL, 228
recursiva, 213, 225
recursivamente enumerável, 213, 223, 238
regular, 66, 76, 86, 92, 238
rejeitada
AFD, 76
AFN, 82
AFNε, 90
AP, 179
MT, 217, 218
MTFL, 228
sensível ao contexto, 213, 226, 238
tipo 0, 213, 223, 238
tipo 1, 213, 226, 238
tipo 2, 154, 155, 238
tipo 3, 66, 76, 86, 92, 238
linguagens formais, 18, 19, 69, 79, 240
link (hipertexto), 141
livre do contexto
gramática, 20, 154
linguagem, 154, 238
determinística (linguagem), 239
lógica
booleana, 33
de Boole, 33
matemática, 33
loop
AP, 179
MT, 217, 218
MTFL, 228

M

máquina
de Mealy, 134, 135
de Moore, 134, 136, 137
de Turing, 20, 68, 185, 212, 214, 216, 220, 222
com fita infinita à esquerda e à direita, 221
com fita limitada, 213, 227, 228
com múltiplas cabeças, 221
com múltiplas fitas, 221
multidimensional, 221
não derminística, 221
marcador
de fim de fita (MTFL), 228
de início de fita (MT), 216
de início de fita (MTFL), 228
matemática (lógica), 33
modelos para concorrência, 69, 79
monomorfismo, 31
movimento vazio, 70, 87, 88
movimento vazio (ap), 177
moving picture expert group (MPEG), 145
MPEG (*moving picture expert group*), 145
MT (máquina de Turing), 216
MTFL (máquina de Turing com fita limitada), 228
multimídia (recursos), 142

N

não
conetivo, 34
contável (conjunto), 33
determinismo, 69, 79, 80
determinismo externo, 68
determinismo interno, 68, 87
determinista (composição de sistemas), 68
terminal (gramática), 59
naturais (conjunto), 23
negação (conetivo), 34
negação (proposição), 34
n-dimensional (linguagem), 240
no operation, 78
nodo, 27
nop (*no operation*), 78
números
inteiros (conjunto), 23
irracionais (conjunto), 23
naturais (conjunto), 23
racionais (conjunto), 23
reais (conjunto), 23

O

operacional (formalismo), 19
operador lógico, 34
ordem conexa (relação), 27

ordem parcial (relação), 27
orientação a objetos, 70
origem (relação), 25
ou (conetivo), 35

P

palavra, 54, 55
palavra vazia, 55
palíndromo, 58
par ordenado, 23
partição (conjunto), 28
Pascal, 67, 110, 154, 156, 206
  alfabeto, 55
  linguagem, 58
  programa, 56
passo de indução, 42
pertence, 21
pilha, 154, 175, 220, 221
pilha (ap), 176
prefixo, 55
premissa (condição), 35
primeiro princípio da indução matemática, 42
princípio da indução matemática, 38, 41, 42
problema
  computável, 213
  não computável, 213
  não solucionável, 32, 239
  parcialmente solucionável, 213
  solucionável, 32, 213
procedimento efetivo, 212
processador de texto, 68, 154, 157
processamento de imagens, 240
produção (gramática), 59
  que substitui variáveis, 161
  que substitui variáveis (algoritmo), 166
  vazia, 161, 164
  vazia (algoritmo), 164
produções (relação), 59
produto cartesiano, 23
programa
  AFD, 71
  AFN, 80
  AFNε, 87
  AP, 176, 177
  autômato finito, 70
  Mealy, 135
  Moore, 136
  MT, 215, 216
  MTFL, 228
proposição, 33
  atômica, 34
  sobre um conjunto universo, 33
prova
  direta, 38, 39
  indutiva, 42
  por absurdo, 38, 40
  por contraexemplo, 40
  por contraposição, 38, 40
  por indução, 38, 42
  por redução ao absurdo, 38, 40

Q

quadro a quadro (animação), 134, 145
QuickTime, 145

R

racionais (conjunto), 23
raiz (árvore), 155, 157, 158
reais (conjunto), 23
reconhecedor (formalismo), 20
recursão à esquerda, 174
recursão à esquerda (algoritmo), 174
recursiva (definição), 27, 42
recursiva (linguagem), 213, 225
recursivamente definido, 42
recursivamente enumerável (linguagem), 213, 223, 238
rede lógica, 18, 122
redução ao absurdo, 36
redução ao absurdo (prova), 38, 40
reflexiva (relação), 25
regra de produção (gramática), 59
regular
  expressão, 20, 66
  gramática, 20, 66
  linguagem, 66, 76, 86, 92, 238
rejeita
  AFD, 74, 75
  AFN, 82
  AFNε, 90
  AP, 178, 179
  MT, 217, 218
  MTFL, 228
relação, 25
  antissimétrica, 25
  conexa, 25
  de boa ordem, 41
  de derivação (gramática), 59
  de equivalência, 28, 36
  de implicação, 36
  de ordem conexa, 27
  de ordem parcial, 27
  de produções (gramática), 59
  em um conjunto, 25
  reflexiva, 25
  simétrica, 25
  transitiva, 25

S

se-então (conetivo), 35
segundo princípio da indução matemática, 43
semântica, 19, 58
semântica formal, 69
sensível ao contexto (gramática), 20, 213, 226
sensível ao contexto (linguagem), 213, 226, 238
sentença, 55
sequencial (composição de sistemas), 68
se-somente-se (conetivo), 35
seta, 27
símbolo, 54
  branco (MT), 216
  de fim de fita (MTFL), 228
  de início de fita (MT), 216
  de início de fita (MTFL), 228
  inicial (gramática), 59
  inútil, 161, 162
  inútil (algoritmo), 162
  não terminal (gramática), 59
  terminal (gramática), 59
  variável (gramática), 59
simétrica (relação), 25
sintática (análise), 18, 19, 154
sintaxe, 19
sistema
  biológico, 18, 240
  concorrente, 240
  de animação, 18, 67, 145
  de estados finitos, 66, 67, 69
  distribuído, 240
  operacional, 69
  comunicante, 240
site, 142
sítio, 142
sobrejeção, 31
sobrejetora (função), 31
solucionabilidade de problemas, 68
subconjunto, 21
subconjunto próprio, 21
subpalavra, 55
sufixo, 55

T

tabela-verdade, 34
tamanho (palavra), 57
tautologia, 34
técnicas de demonstração, 38
tempo real (animação), 145
teorema, 36, 37
teoria
  da computação, 68, 69, 79, 195
  das categorias, 240, 241
  das linguagens formais, 18
  dos autômatos, 74
  dos conjuntos, 20
  dos grafos, 74
terminal (gramática), 59
tese, 37
  de Church, 212, 222
  de Turing-Church, 222
tipo 0 (linguagem), 213, 223, 238
tipo 1 (linguagem), 213, 226, 238
tipo 2 (linguagem), 154, 155, 238
tipo 3 (linguagem), 66, 76, 86, 92, 238
topo (pilha), 175
tradutor de linguagem, 138, 154
transição
  AFD, 71
  AFN, 80
  AFNε, 88
  AP, 177
  MT, 216
  MTFL, 228
  encapsulada, 70, 87
  estendida (AFN), 82
  vazia, 88
  paralela, 71
transitiva (relação), 25
Turing, A., 212, 214, 222
Turing-Church
  hipótese, 222
  tese, 222

U

união, 23
unidade de controle
  AP, 176
  autômato finito, 70
  MT, 215
universo (conjunto), 23

V

valor verdade, 33
variável (gramática), 59
variável inicial (gramática), 59
vazio (conjunto), 21
verdadeiro (valor verdade), 33
vértice, 27
  folha (árvore), 158
  interior (árvore), 158

W

world wide web, 134, 142

Y

Younger, D. H., 203

linguagens formais e autômatos

3

paulo blauth menezes

linguagens formais e autômatos, 6.ed., apresenta os principais conceitos e resultados de linguagens formais e autômatos, de uma forma simples e acessível, sem descuidar do desenvolvimento do raciocínio nem dos aspectos matemático-formais. Sempre que possível, as construções apresentadas são instanciadas em casos aplicados a computação e informática. Este é um livro-texto para disciplinas dos cursos de graduação em computação e informática, de acordo com as Diretrizes Curriculares do MEC, bem como um livro de referência para diversos aspectos da computação. O livro é autocontido e possui uma apresentação que facilita a adequação do texto aos objetivos propostos, à carga horária e aos pré-requisitos da disciplina. É ilustrado com cerca de 125 figuras, 90 exemplos detalhados, 250 exercícios em níveis crescentes de raciocínio e um completo índice com cerca de 720 entradas.

Material didático para professores
Visite www.bookman.com.br

COMPUTAÇÃO/FUNDAMENTOS

ISBN 978-85-7780-799-4

9 788577 807994

grupo a
Conhecimento que transforma.

.inf INSTITUTO DE INFORMÁTICA UFRGS

bookman